基于两级线性规划与滚动调整的微网购电及储能调度

摘　要

在微网运行中，光伏出力与用电负荷在时间分布上存在差异，需通过外网购电和储能调节维持供需平衡。本文针对供需变化与电价波动下的购电决策，构建结合**两级线性规划**、**风险裕量与实时反馈**的调度框架，分别研究典型日优化、日初固定采购、日内滚动调整及波动电价下的运行安排。问题一采用典型日数据，问题二至四的主方案在**统一风险参数 $\alpha = 0.8$ 下**比较，以 2025 年 2—12 月为评价期。

针对问题一，建立以购电费用最小为目标的**购电与储能联合调度模型**，在满足电量平衡、容量和功率约束的基础上，采用**两级线性规划**减少不必要的充放电。典型日总购电费用为 **35,126.95 元**，较无储能方案降低 **26.90%**，日末储电量恢复至日初水平。储能吸收富余光伏电量，并将部分高价购电需求转移至低价时段。

针对问题二，考虑合同当天不能调整且供需预测存在偏差，采用**指数加权预测与经验分位数风险裕量**确定日初购电量，再通过**储能实时反馈**弥补实际偏差，连续传递日末储电量。评价期总购电费用为 **13,923,903.39 元**，较相同预测和控制规则下的零裕量方案降低 **9.18%**，紧急购电量减少 **79.45%**。分月比较中，主方案在十个月费用较低，但并非每月均占优。

针对问题三，利用分时更新的光伏预报，建立**逐次结算的滚动购电模型**，每次优化当天剩余时段，并计入相对上次合同的增减费用。主方案总购电费用为 **13,833,843.32 元**，较仅使用日初预报且不改约的方案降低 **1.96%**。对照结果显示，官方预报减少了平均预测误差与紧急购电量，但其总购电费用高于历史光伏预测方案，说明预测误差较小并不必然带来更低购电费用。

针对问题四，在当天各时段电价已知的条件下，建立**固定合同与滚动调整两类波动电价调度模型**，保留原有预测、风险裕量和控制规则重新求解。两类主方案的总购电费用分别为 **14,571,629.31 元**和 **14,487,237.24 元**，滚动调整方案降费 **0.58%**。各策略内部统一期初储电量后，风险裕量方案相对零裕量方案的降费方向保持一致。

关键词：微网；储能调度；两级线性规划；风险裕量；滚动优化

一、问题重述

1.1 问题背景

微网是由分布式电源和用电负荷等组成的小型电力系统，可实现光伏等分布式能源的就地利用。非孤岛式微网与外部电网（以下简称外网）相连，可在电价较低或发电富余时储存电能，在电价较高时释放电能。[1,2]小区用电需求、光伏出力和外网电价均随时间变化，而储能的调节能力受到容量、功率及充放电效率的限制。因此，需要结合供需变化与实际储电量，合理安排外网购电和储能充放电，在满足用电需求的同时尽可能降低购电费用。

1.2 问题提出

根据给定数据与储能设备参数，建立数学模型，解决以下四个问题。

问题一：在各日分时电价与小区负荷相同、当天光伏功率预测已给定的条件下，利用典型日数据在每天 0 时制定购电策略，使各时段供电量满足用电需求，且 0 时与 24 时储电量相等。给出指定时段购电量、全天购电量与购电费，以及各四小时时段的充放电量和首末储电量，形成完整的日内购电与储能调度计划。

问题二：在各日分时电价相同、负荷与光伏出力随时间变化的条件下，利用固定分时电价及此前已获得的负荷、光伏实测数据，在每天 0 时制定当天购电策略。供电不足时按交易时刻电价的 5 倍紧急补购，其余购电费用按计划购电量计算。给出指定日期的购电量、购电费、充放电量、首末储电量及紧急购电时段与电量，形成完整的逐日购电与储能运行结果。

问题三：利用每日 0、6、12、18 时已发布的未来 24 小时整点光伏预报，结合固定分时电价与当时已获得的负荷、光伏实测数据，制定日初购电计划，并在后续预报发布时调整。减购部分按交易电价的 50% 计收违约费，增购部分按 1.5 倍电价计费，总购电费用包含计划购电、紧急购电及合同调整费用。给出指定日期的相应结果，形成包含日内合同调整的逐日购电策略，并分析是否需要利用其他时刻的预报调整购电计划。

问题四：根据负荷、光伏及波动电价数据，建立波动电价下的购电策略模型，重新计算问题二和问题三，给出相应的购电、充放电、储电量、紧急购电及费用结果，分别形成两类完整的逐日调度结果。

二、问题分析

2.1 问题一的分析

问题一已知典型日各时段的负荷、光伏预测与分时电价，可在日初统一安排全天购电量和充放电量，并以储电量连接相邻时段。模型以电量平衡、储能容量、充放电功率及互斥条件为约束，要求日末储电量恢复至日初水平。在此结构下，先最小化全天购电费用，再减少费用基本相同方案中的充放电总量，得到满足供电要求的确定性调度计划。

2.2 问题二的分析

问题二日初可利用历史负荷、光伏实测和固定分时电价，需先预测当天净负荷。合同购电量在日初确定并全天固定，充放电量与紧急购电量则随实际供需变化。规划阶段将历史误差形成的风险裕量加入预测需求，沿用电量平衡和储能边界约束；执行阶段按实际净缺口调节储能，剩余缺口由紧急购电满足。各日通过实际储电量连续衔接，以合同购电与高价补购的合计费用评价策略。

2.3 问题三的分析

问题三在历史数据基础上增加日内分时发布的光伏预报，可据此调整尚未开始时段的合同购电量。决策变量包括调整后的合同量、增减购电量及规划充放电量；除供需与储能约束外，还需用合同增减关系连接调整前后的购电量。每次以实际储电量为起点优化当天剩余时段，将增购支出、减购退款和预测补购费用纳入目标，形成滚动调整与实时储能控制相结合的策略。

2.4 问题四的分析

问题四增加逐日变化的电价，以给定的目标日完整价格序列作为规划信息。分别保留问题二固定合同与问题三滚动调整的决策权限、预测规则和储能约束，只将各交付时段的价格代入相应目标及结算公式。前

者在日初确定全天采购量，后者还可在预报更新时调整未来合同，由此比较波动电价对购电时序、储能安排和总购电费用的影响。

四问共用电量平衡和储能运行约束，并随可用信息和合同调整权限的变化扩展决策变量，整体建模框架如图 1 所示。问题四分别承接固定合同与滚动调整两类策略，考察其在波动电价下的运行结果。

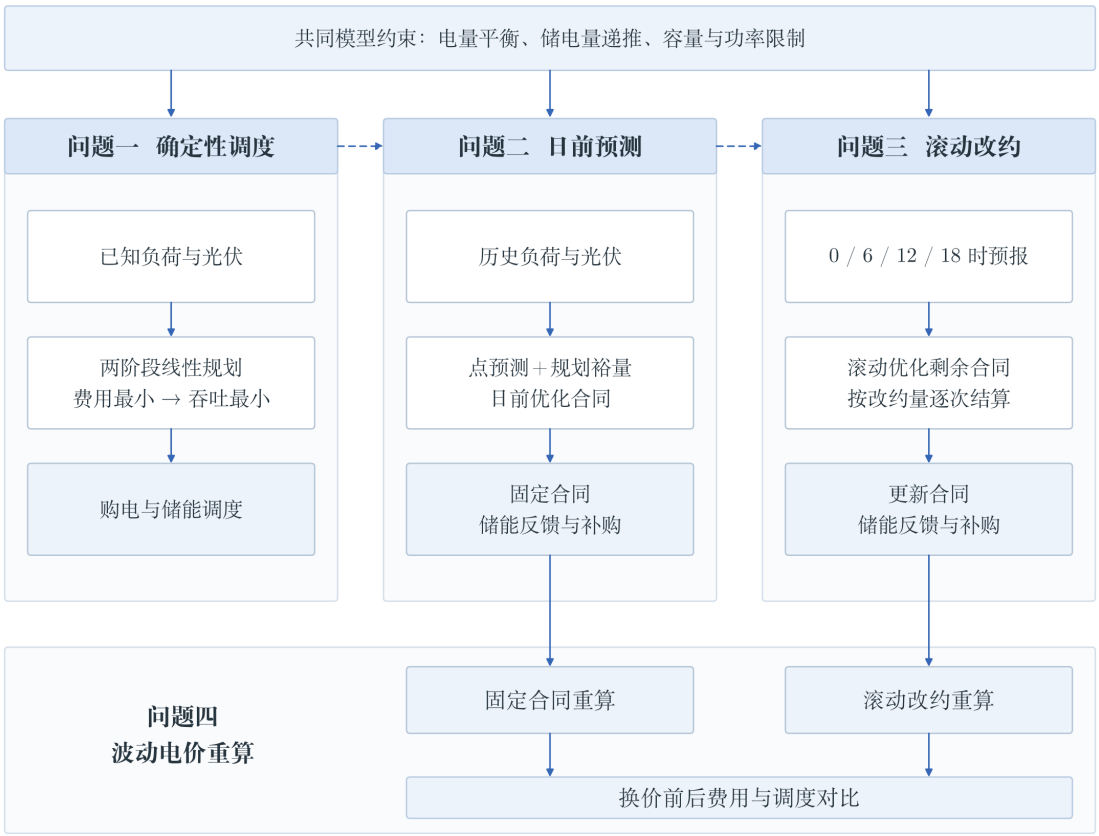


图 1　微网电力调控的总体建模框架

三、模型假设

为简化问题，结合微网运行与购电结算方式，作如下假设：

假设 1：假设每个 10 分钟时段内的负荷、光伏及储能充放电功率保持恒定，并以该时段的代表功率近似计算电量。问题二至四以当期实测功率确定该时段的充放电量。

假设 2：假设储能充、放电效率在研究期间均保持为 90%，充电功率按交流侧输入计量，放电功率按交流侧输出计量。

假设 3：假设外网能够满足合同购电及紧急补购需求；微网不向外网售电，未被利用的富余电量允许无成本弃置，且不另设弃置量上限。

假设 4：假设每次合同调整均以上一次确定的购电量为依据，以对应交付时段的正常电价为计价基准：增购支付 1.5 倍电价，减购净退 0.5 倍正常购电款；已交付但未利用的电量不予退款。

假设 5：假设问题四在制定计划时已知目标日各时段电价，并据此重新计算调度方案。

四、符号说明

本文主要符号及其含义见表 4-1，局部符号在相应模型中定义。

表 4-1　主要符号说明

符号	含义	单位
D, t	日期索引、日内 10 分钟时段索引，$t = 1, \ldots, 144$	—
k	光伏预报发布或合同规划时点对应的小时数，$k \in \{0, 6, 12, 18\}$	h

续表 4-1

符号	含义	单位
Δt	单个时段长度，取 1/6	h
$L_{D,t}, G_{D,t}$	第 D 日第 t 时段的负荷功率、实际光伏功率	kW
$p_t, p_{D,t}$	固定分时电价、逐日波动电价	元/kWh
$n_{D,t}$	实际净负荷电量，$(L_{D,t}-G_{D,t})\Delta t$	kWh
$q_{D,t}$	日初确定的合同购电量	kWh
$y_{D,t}$	各时段交付前最后确定的合同购电量	kWh
$c_{D,t}, r_{D,t}$	交流侧充电输入量、放电输出量	kWh
$S_{D,t}$	第 t 时段结束时电池内部储电量，$S_{D,0}$ 为日初储电量	kWh
$e_{D,t}, w_{D,t}$	实际紧急购电量、未利用电量	kWh
η_c, η_d	充电效率、放电效率	—
U	单时段交流侧充、放电量上限，取 5000/6	kWh
$b_{D,t}$	根据净负荷预测误差设置的风险裕量	kWh
α	构造风险裕量所用的经验分位数水平	—
$a^{+}_{D,k,t}, a^{-}_{D,k,t}$	k 时相对上一次约定购电量的增购量、减购量	kWh
v_D, λ_D	日末储电量价值系数，两者按相同规则取值	元/kWh
$J_D, J_{D,k}$	日初及日内更新时的规划目标值，包含日末价值项	元
$C_1, C_2, C_3, C_{4-2}, C_{4-3}$	各问题或对应分支的总购电费用	元

五、问题一的模型建立与求解

5.1 数据处理与决策变量

在典型日供需与电价给定的条件下，充放电会改变后续可用储电量，因此需要在全天范围内联合确定购电与储能计划。典型日数据包含 144 个时段的负荷功率、光伏预测和固定分时电价，数值完整，负荷与光伏功率非负，电价为正，保留原始数值。

将每日划分为 144 个 10 分钟时段，记 $\mathcal{T}=\{1,2,\ldots,144\}$，时段长度 $\Delta t = 1/6$ h。原始时刻标签对应此前 10 分钟区间的结束时刻，如 00:10 对应 00:00—00:10，日末标签对应 23:50—24:00。每日 144 个区间对应日初及各区间结束时的 145 个储电状态。将以 kW 计量的区间代表功率乘以 Δt，得到 kWh 电量。以给定负荷 L_t、光伏预测 $\widehat{G}_t$ 和电价 p_t 为输入，净负荷电量为

$$n_t = (L_t - \widehat{G}_t)\Delta t \tag{1}$$

其中，$n_t > 0$ 表示光伏发电不足以满足用电需求，$n_t < 0$ 表示存在富余光伏电量，均予以保留。记 q_t 为计划购电量，c_t 和 r_t 为交流侧充电输入量与放电输出量，w_t 为未利用电量，S_t 为时段结束时的内部储电量，单位均为 kWh。电价以元/kWh 计量，乘以相应购电量得到区间费用。四小时充电量和放电量分别由 24 个时段累计；计算保留原始精度，结果通常保留两位小数，电价和数值误差保留必要精度。

5.2 购电与储能联合调度模型

外网购电、光伏发电与储能放电共同满足用电和充电需求。[3]微网不向外网售电，未被负荷和储能吸收的富余电量允许无成本弃置，并以非负变量 w_t 表示，使供需关系能够写为电量平衡等式。充电和放电效率均取 90%，即 $\eta_c = \eta_d = 0.9$；充电输入 c_t 使内部储电量增加 $\eta_c c_t$，向交流侧放出 r_t 则消耗内部电量 r_t/η_d。

储能额定容量为 12,000 kWh，允许的储电量范围为 1,200—10,800 kWh。最大充电输入功率和放电输出功率均按交流侧 5,000 kW 计，单时段充、放电量上限为 $U = 5000\Delta t = 5000/6$ kWh。日初储电量取 6,000 kWh，日末恢复至相同水平。以全天购电费用最小为目标，建立确定性调度模型：

$$\min_{q,c,r,w,S} \quad C_1 = \sum_{t=1}^{144} p_t q_t \tag{2}$$

$$\text{s.t.}\quad \begin{cases} q_t + r_t = n_t + c_t + w_t & (3)\\ S_t = S_{t-1} + \eta_c c_t - \dfrac{r_t}{\eta_d} & (4)\\ 1200 \le S_t \le 10800, \quad 0 \le c_t, r_t \le U & (5)\\ q_t, w_t \ge 0, \quad c_t r_t = 0, \quad S_0 = S_{144} = 6000 & (6) \end{cases}$$

上述时段约束对所有 $t \in \mathcal{T}$ 成立。电量平衡保证用电需求得到满足，储电量递推式连接相邻时段的充放电决策，容量与功率约束限定设备运行范围，$c_t r_t = 0$ 保证同一时段不同时充放电。首末储电量约束使日末恢复至初始水平。全天购电量 $Q_1 = \sum_{t=1}^{144} q_t$ 与购电费用一并报告。

5.3 基于两级线性规划的模型求解

充放电互斥约束含有乘积项。为保持模型的线性结构，先暂时去除该约束，将其余约束构成的可行域记为 $\mathcal{F}$，求得最低购电费用 C_1^*。随后，限定购电费用不超过该最优值与给定容差之和，以充放电总量最小为第二级目标：

$$\min_{(q,c,r,w,S)\in\mathcal{F}} \quad \sum_{t=1}^{144}(c_t + r_t) \tag{7}$$

$$\text{s.t.}\quad \left\{ \sum_{t=1}^{144} p_t q_t \le C_1^* + \varepsilon_C \right. \tag{8}$$

其中，$\varepsilon_C = 10^{-6}$ 元为数值求解容差。第二级优化在费用基本相同的方案中减少充放电总量。[4]

这一处理能够保证充放电互斥。若某时段 c_t, r_t 同时为正，令 $a = \min\{c_t, r_t/(\eta_c\eta_d)\}$，并作变换

$$c_t' = c_t - a, \quad r_t' = r_t - \eta_c\eta_d a, \quad w_t' = w_t + (1 - \eta_c\eta_d)a \tag{9}$$

变换不改变储电量递推关系、电量平衡及购电费用，且至少使充电量或放电量之一归零。由于未利用电量允许无成本弃置，且不另设弃置上限，变换后的方案仍满足所有约束，而充放电总量严格下降。因此，第二级最优解不会保留同时充放电的时段，无需额外引入整数变量。

采用 Python 中 SciPy 的 `linprog` 接口，设置 `method="highs"`，由 HiGHS 完成两级线性规划求解。[5]首先，按购电量、充电量、放电量、未利用电量和储电量的顺序排列决策变量，将电量平衡、储电量递推和日末恢复条件写为等式约束，将容量、功率及非负性要求写为变量上下界。日初储电量作为已知量代入首个时段，形成包含 720 个变量和 289 个等式约束的线性模型。

随后，以电价作为购电变量的目标系数，求得第一级最优值 C_1^*；在原约束上加入式 (8)，将充、放电变量的目标系数均设为 1，其余设为 0，求解第二级。两次求解均检查返回状态，以第二级解作为最终调度方案。将结果还原为 144 个时段的购电与充放电序列，核对电量平衡、储电量及设备边界后，分别汇总指定时段购电量、四小时充放电量和全天费用。

5.4 调度结果与分析

题目指定时段的购电量及全天购电结果见表 5-1。

表 5-1　指定时段购电量及全天购电结果（电量：kWh；费用：元）

时间段或指标	数值	时间段或指标	数值	时间段或指标	数值
10:00—10:10	0.00	12:00—12:10	480.41	14:00—14:10	0.00
16:00—16:10	445.43	18:00—18:10	531.89	20:00—20:10	0.00
全天总购电量	**59,482.70**	**全天总购电费用**	**35,126.95**	—	—

表 5-1 中，午间仍安排外网购电，而所列部分时段购电量为零。相应的充放电计划见表 5-2。

表 5-2　储能充放电量及首末储电量（单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00—04:00	4,500.00	0.00	04:00—08:00	833.33	6,365.84
08:00—12:00	4,787.96	1,703.00	12:00—16:00	5,286.04	91.10
16:00—20:00	0.00	5,780.13	20:00—24:00	5,333.33	2,859.87
00:00 储电量	6,000.00	—	24:00 储电量	6,000.00	—

表 5-2 显示，夜间和日间均有充电，晚间通过放电减少外网购电，日末储电量恢复至日初水平。将这些安排与分时电价对照，得到图 2 所示的典型日调度过程。

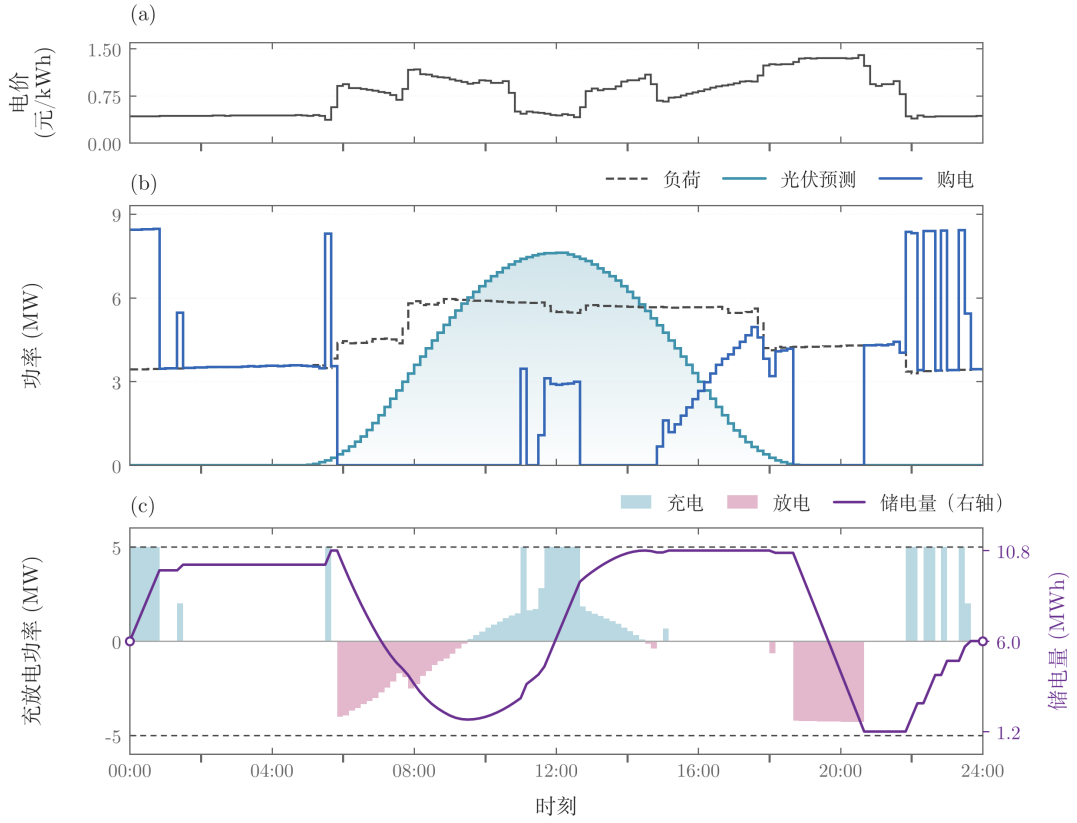


图 2　典型日确定性调度下的电价、购电与储能运行

图 2 中，低价时段的购电高峰与储电量上升相对应，早间和晚间则通过放电减少外网购电；日间的富余光伏电量与低价购电共同补充储能。图中充电为正、放电为负，储电量对应右轴，功率与储电量分别以 MW 和 MWh 表示。

以 12:00—12:10 为例，光伏出力虽已高于负荷，方案仍以 0.4411 元/kWh 购入 480.41 kWh，与富余光伏电量一同用于充电。到 20:00—20:10，电价升至 1.3513 元/kWh，储能放电满足当期用电需求，外网购电量降为零。这说明购电安排不仅取决于当前净负荷，还需考虑所购电量在后续时段的使用价值。

为衡量储能调度的经济效果，在相同负荷、光伏和电价条件下设置无储能基准方案。令 $c_t = r_t = 0$，则各时段购电量和未利用电量分别为 $q_t^{(0)} = \max\{n_t, 0\}$ 和 $w_t^{(0)} = \max\{-n_t, 0\}$，比较结果见表 5-3。

表 5-3　储能调度与无储能基准比较

指标	无储能基准	储能调度方案
全天总购电量（kWh）	**61,789.94**	**59,482.70**
全天总购电费用（元）	**48,052.05**	**35,126.95**
未利用电量（kWh）	6,247.96	0.00

相较无储能基准方案，储能方案减少购电费用 12,925.10 元，降幅为 26.90%，而购电量仅减少 3.73%。费用降幅大于购电量降幅，表明储能收益既来自富余光伏电量的利用，也来自将部分高价时段的购电需求转移至低价时段。

全天充电量为 20,740.67 kWh，放电量为 16,799.94 kWh。在首末储电量相等的条件下，满足 $\sum_t r_t = 0.81 \sum_t c_t$，对应充放电损耗 3,940.73 kWh。储能方案吸收了无储能基准方案中未被利用的 6,247.96 kWh 富余电量，扣除充放电损耗后，净减少外网购电 2,307.24 kWh。

5.5 模型检验

将 144 个时段的购电与充放电结果代入式 (3)—(6)，电量平衡与储电量递推的最大残差分别为 2.27×10^{-13} kWh 和 8.31×10^{-13} kWh，储电量、充放电功率及首末状态均满足约束，未出现同一时段同时充放电。按 10 分钟结果重新汇总四小时充放电量，与表 5-2 的差异小于 4×10^{-12} kWh，说明逐时段计算与汇总结果一致。

第二级解较第一级解的购电费用仅增加约 10^{-6} 元，与预设容差一致。结合第 5.3 节的互斥性分析，所得方案在保持一级费用目标的同时满足充放电互斥要求，支持典型日调度结果的可行性。

六、问题二的模型建立与求解

6.1 数据处理与负荷、光伏预测

问题二中，合同购电量须在日初确定，而实际负荷与光伏出力可能偏离预测。合同无法随偏差调整，储能却可根据实际供需充放电，因此将决策分为日初计划与实时调节：先依据历史预测和风险裕量确定采购量，再通过储能和紧急购电满足实际需求。[6,7]

全年负荷与光伏实测覆盖 2025 年 1 月 1 日至 12 月 31 日，各形成 365×144 的矩阵，日期和时段一一对应，数值完整且非负。沿用典型日的时段映射与单位换算，固定分时电价在各日相同时段重复使用。记 D 为日期，实际净负荷电量为

$$n_{D,t} = (L_{D,t} - G_{D,t})\Delta t \tag{10}$$

光伏零出力与负净负荷均保留。为反映不同星期几的负荷差异，并兼顾近期变化，从目标日前 28 天中选取相同星期几的日期，构成集合 $\mathcal{H}_D$；光伏预测采用此前最近 7 天的数据，记其日期集合为 $\mathcal{P}_D$。对历史日期 j 赋予权重 $\omega_{D,j} = \exp[-(D-j)/7]$，其中 $D-j$ 以天计，得到

$$\widehat{L}_{D,t} = \frac{\sum_{j \in \mathcal{H}_D} \omega_{D,j} L_{j,t}}{\sum_{j \in \mathcal{H}_D} \omega_{D,j}} \tag{11}$$

$$\widehat{G}_{D,t} = \max\left\{0, \frac{\sum_{j \in \mathcal{P}_D} \omega_{D,j} G_{j,t}}{\sum_{j \in \mathcal{P}_D} \omega_{D,j}}\right\} \tag{12}$$

若此前尚无相同星期几的历史记录，则依次选取前 28 天内与目标日同为工作日或周末的日期、此前最近 7 天的可用日期进行负荷预测。预测净负荷电量为 $\widehat{n}_{D,t} = (\widehat{L}_{D,t} - \widehat{G}_{D,t})\Delta t$。

6.2 考虑风险裕量的日初购电模型

低估净负荷可能导致按 5 倍正常电价结算的紧急购电，适当增加正常购电则可减少后续补购。因此，利用历史预测误差设置风险裕量，在预测需求之外预留购电余量。分位数可用于描述电力负荷预测的不确定性[8]，概率预测与偏差成本的结合可为电力交易决策提供依据[9]。记 $\epsilon_{j,t} = n_{j,t} - \widehat{n}_{j,t}$，其中 $n_{j,t} = (L_{j,t} - G_{j,t})\Delta t$，$\widehat{n}_{j,t}$ 为当日 0 时形成的预测。将每日划分为六个四小时组，每组包含 24 个时段，记 t 所属组为 $g(t)$。汇集目标日前最多 28 天中同组的有效历史误差，形成样本 $\mathcal{E}_{D,g}$，据此确定风险裕量及加入裕量后的规划净需求：

$$b_{D,t} = \max\left\{0, Q_{0.8}\left(\mathcal{E}_{D,g(t)}\right)\right\}, \quad \widetilde{n}_{D,t} = \widehat{n}_{D,t} + b_{D,t} \tag{13}$$

其中，$Q_{0.8}$ 为采用线性插值计算的 80% 经验分位数；有效误差所覆盖的历史日期不足 7 天时，取 $b_{D,t}=0$。同一四小时组内的各 10 分钟时段采用相同裕量，单位为 kWh。

以 $q_{D,t}$ 表示日初确定的合同购电量，用 $\widetilde{c}_{D,t}$、$\widetilde{r}_{D,t}$、$\widetilde{w}_{D,t}$ 和 $\widetilde{S}_{D,t}$ 分别表示规划中的充电量、放电量、未利用电量与储电量。储能效率、容量及功率约束沿用问题一，但不再要求每日首末储电量相等。考虑到日末剩余电量可用于次日调度，引入日末储电量价值系数 v_D，建立日初购电模型：

$$\min_{q,\widetilde{c},\widetilde{r},\widetilde{w},\widetilde{S}} \quad J_D=\sum_{t=1}^{144}p_t q_{D,t}-v_D\widetilde{S}_{D,144} \tag{14}$$

$$\text{s.t.}\quad \begin{cases} q_{D,t}+\widetilde{r}_{D,t}=\widetilde{n}_{D,t}+\widetilde{c}_{D,t}+\widetilde{w}_{D,t} & (15)\\ \widetilde{S}_{D,t}=\widetilde{S}_{D,t-1}+\eta_c\widetilde{c}_{D,t}-\dfrac{\widetilde{r}_{D,t}}{\eta_d} & (16)\\ 1200\le\widetilde{S}_{D,t}\le 10800,\quad 0\le\widetilde{c}_{D,t},\widetilde{r}_{D,t}\le U & (17)\\ q_{D,t},\widetilde{w}_{D,t}\ge 0,\quad \widetilde{S}_{D,0}=S_{D,0} & (18)\end{cases}$$

各时段约束对 $t=1,\dots,144$ 成立，$S_{D,0}$ 取上一日实际运行后的储电量，$U=5000/6$ kWh。日末价值系数设为 $v_D=\gamma\eta_d\,\mathrm{median}_t(p_t)$，其中 $\gamma=0.5$ 为日末电量价值的折减因子，12 月 31 日取零，用于衡量剩余电量的后续价值。沿用第五章的两级线性规划，先求得一级最优值 J_D^{*}，再在 $J_D\le J_D^{*}+10^{-6}$ 元的条件下最小化 $\sum_t(\widetilde{c}_{D,t}+\widetilde{r}_{D,t})$。充放电互斥仍按第五章的方法处理。

6.3 储能实时调节与逐日求解

日初确定合同购电量后，充放电根据实际供需调节。设第 t 个时段的实际净缺口为 $z_{D,t}=n_{D,t}-q_{D,t}$，以 $[x]_+=\max\{x,0\}$ 表示正部。供电富余时优先充电，供电不足时先在设备允许范围内放电，再通过紧急购电补足剩余缺口，实时调节规则为

$$c_{D,t}=\min\left\{[-z_{D,t}]_+,\ U,\ \frac{10800-S_{D,t-1}}{\eta_c}\right\} \tag{19}$$

$$r_{D,t}=\min\left\{[z_{D,t}]_+,\ U,\eta_d(S_{D,t-1}-1200)\right\} \tag{20}$$

$$e_{D,t}=[z_{D,t}-r_{D,t}]_+,\quad w_{D,t}=[-z_{D,t}-c_{D,t}]_+ \tag{21}$$

$$S_{D,t}=S_{D,t-1}+\eta_c c_{D,t}-\frac{r_{D,t}}{\eta_d} \tag{22}$$

该调节规则满足 $q_{D,t}+r_{D,t}+e_{D,t}=n_{D,t}+c_{D,t}+w_{D,t}$。储能无法吸收的富余电量计入 $w_{D,t}$，未覆盖的缺口由 $e_{D,t}$ 补足。实际日末储电量作为次日初始储电量，即 $S_{D+1,0}=S_{D,144}$。

计算按日期和日内时段设置两层循环。以 6,000 kWh 为 2025 年 1 月 1 日 0 时的初始储电量，首日不安排合同购电和储能充放电，光伏发电不足以覆盖的需求由紧急购电满足。自 1 月 2 日起，外层循环依次更新当天预测、历史误差与风险裕量，以上一日实际日末储电量为起点，构造式 (14)—(18) 的日初规划。

每日模型包含 720 个变量和 288 个等式约束。电量平衡与储电量递推的系数采用稀疏矩阵保存，各日重复使用相同结构，更新规划净需求、日初储电量及日末价值系数，再按第 5.3 节的顺序完成两级求解。合同购电量确定后，内层循环依次执行 144 个时段，按式 (19)—(22) 计算充放电、紧急购电与未利用电量，并更新实际储电量；日内运行结束后，将实际日末储电量传入次日。

1 月用于积累历史信息、校准误差并形成储电状态，正式评价 2025 年 2 月 1 日至 12 月 31 日的 334 天，共 48,096 个时段，费用不含 1 月。主方案在 2 月 1 日的实际储电量约为 2,475.00 kWh。

记正式评价日期集合为 $\mathcal{D}$，总购电费用按合同支出和紧急购电支出计算：

$$C_2=\sum_{D\in\mathcal{D}}\sum_{t=1}^{144}\left(p_t q_{D,t}+5p_t e_{D,t}\right) \tag{23}$$

总购电费用为合同购电费与紧急购电费之和，总购电量为合同购电量与紧急购电量之和，日末价值只用于规划。按评价日期汇总全年运行结果，并提取 3 月 20 日、6 月 21 日、9 月 23 日和 12 月 21 日的购电与储能安排；相邻补购时段合并为连续事件。

6.4 结果与分析

6.4.1 评价期费用与基准方案比较

在预测方法、储能控制及日末价值设置相同的条件下，令 $b_{D,t}=0$，构造不设置风险裕量的基准方案，以下简称零裕量方案。两方案均自 1 月 1 日连续运行，2 月 1 日各自使用前一日实际运行后的储电量进入评价期。2—12 月的比较结果见表 6-1。

表 6-1　风险裕量策略与零裕量方案的购电费用及电量比较

类别	指标	零裕量方案	风险裕量策略
费用	**总购电费用（元）**	**15,331,204.79**	**13,923,903.39**
费用	合同购电费（元）	12,182,842.35	13,294,871.02
费用	紧急购电费（元）	3,148,362.44	629,032.36
电量	**总购电量（kWh）**	**20,600,871.01**	**21,717,677.82**
电量	合同购电量（kWh）	20,056,095.52	21,605,749.45
电量	紧急购电量（kWh）	544,775.49	111,928.38
利用情况	未利用电量（kWh）	1,440,829.89	2,589,736.79

表 6-1 显示，风险裕量策略的总购电费用降低 9.18%，紧急购电量下降 79.45%。合同购电支出增加 1,112,028.68 元，而紧急购电支出减少 2,519,330.08 元，净节省 1,407,301.40 元。两方案的净负荷预测平均绝对误差均为 43.97 kWh/时段，因此节费体现的是采购安排及后续储能运行的改变，而非预测精度提高。

主方案的总购电量和未利用电量均高于零裕量方案。因此，费用降低并非来自各项电量同步减少，而是以更多正常采购换取更少的高价补购。评价期内有 880 个时段发生紧急购电，合并为 201 次连续补购事件。

为考察节费是否持续出现在不同月份，进一步比较两方案的分月总购电费用，如图 3 所示。

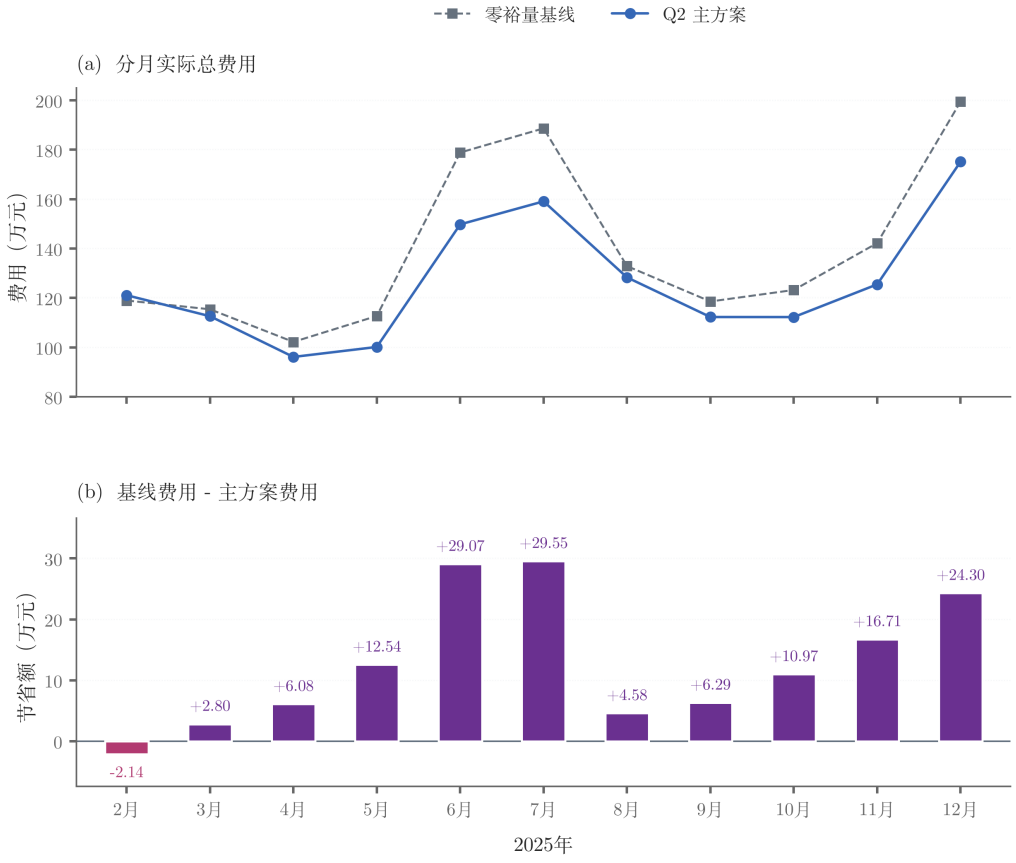


图 3　问题二主方案与零裕量方案的分月购电费用及节省额

图 3 显示，评价期 11 个月中，主方案有 10 个月的总购电费用低于零裕量方案，仅 2 月多支出 2.14 万元。6 月、7 月和 12 月分别节省 29.07 万元、29.55 万元和 24.30 万元，节费较为集中。因此，风险裕量的

费用改善出现在多数月份，但并非各月均占优。上图比较分月总购电费用，下图以零裕量方案费用减去主方案费用表示节省额，单位均为万元。

6.4.2 指定日期的购电与储能运行

四个指定日期的实际净负荷、预测值与风险裕量如图 4 所示。各子图上部展示净负荷曲线，下部展示实际值减预测值所得的误差，以及相应的风险裕量。图中将各 10 分钟时段的电量换算为等效平均功率，以 MW 表示。阴影表示规划中预留的风险裕量。

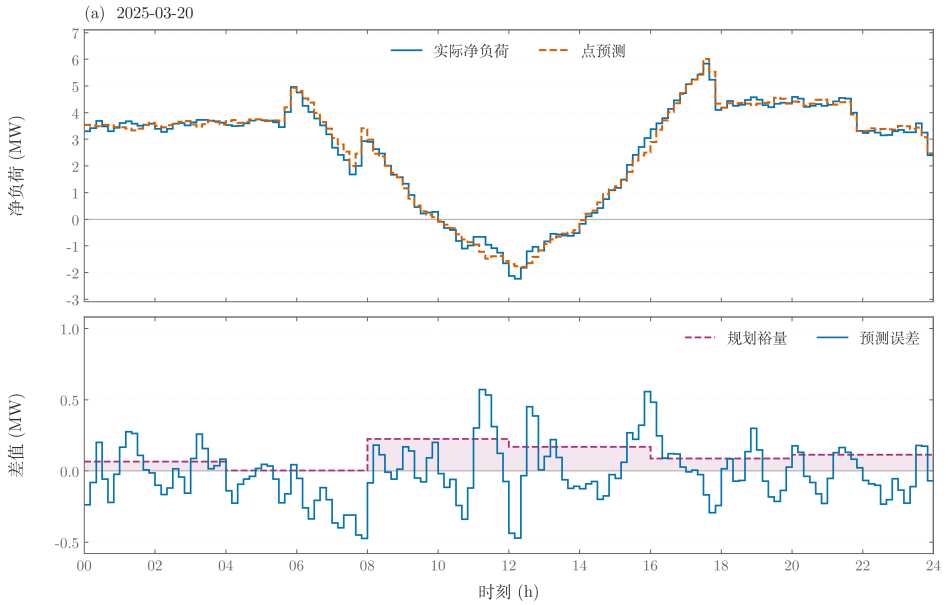
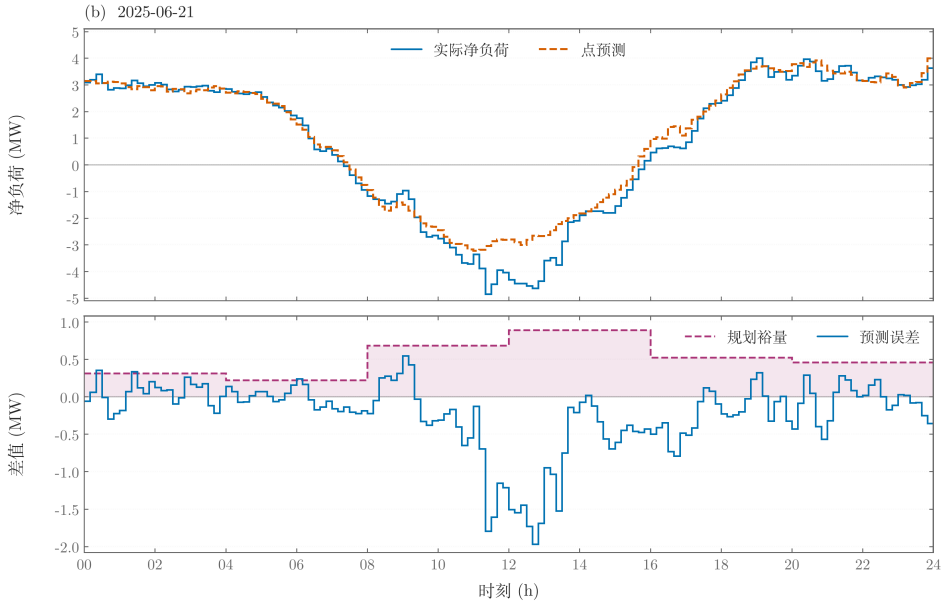


图 4　（a）3 月 20 日净负荷预测误差与风险裕量

3 月 20 日预测净负荷与实际值的日内变化趋势整体接近，午间均为负值。下部误差正负交替，部分时段的实际净负荷超过预测值与裕量之和，因此仍需结合实际储电量调节充放电。

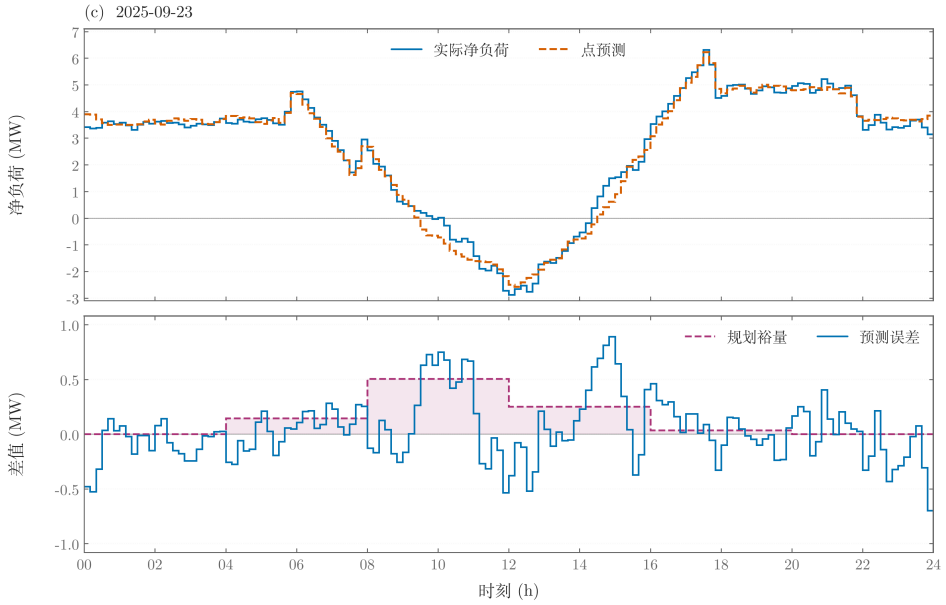
6 月 21 日的实际净负荷、预测与风险裕量见图 4（b）。



续图 4　（b）6 月 21 日净负荷预测误差与风险裕量

6 月 21 日午间实际净负荷明显低于预测值，表明实际富余电量多于预测。相比 3 月 20 日，该日午间负误差更集中、幅度更大，而根据历史误差设置的各组裕量仍为正，说明按历史数据预留的购电余量不一定与当天实际误差方向一致。

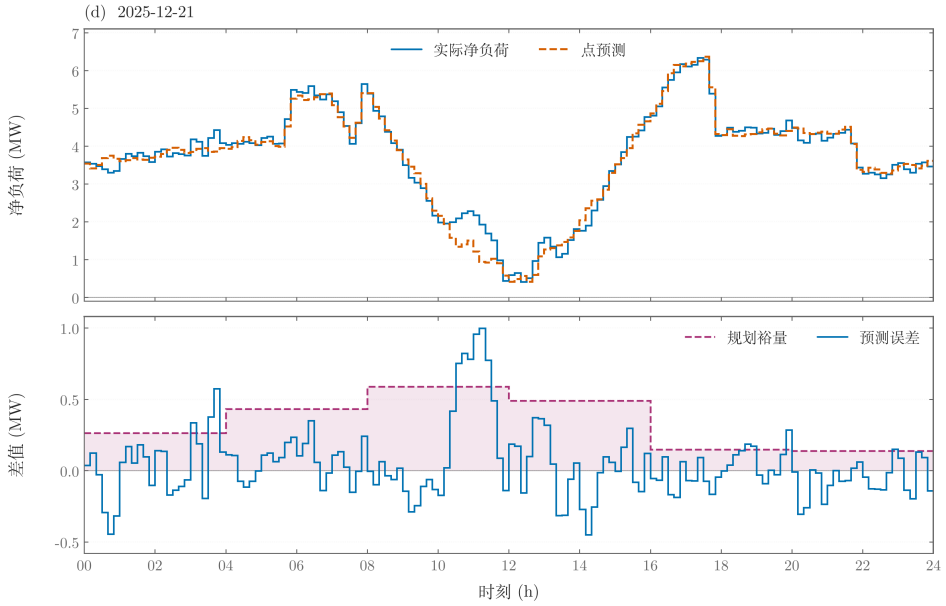
9 月 23 日的实际净负荷、预测与风险裕量见图 4（c）。



续图 4　（c）9 月 23 日净负荷预测误差与风险裕量

9 月 23 日上午和下午部分时段的正误差较集中，晚间也有误差超过相应风险裕量的情况。这些时段的补购需求还取决于剩余储电量和放电功率限制。

12 月 21 日的实际净负荷、预测与风险裕量见图 4（d）。



续图 4　（d）12 月 21 日净负荷预测误差与风险裕量

12 月 21 日净负荷全天为正，午间虽降至低谷，光伏发电仍不足以满足全部用电需求。约 11 时实际净负荷高于预测值，正误差超过该组裕量；晚间两条净负荷曲线较接近，但所需供电量仍高于午间。

四个指定日期各时段的合同购电量见表 6-2。

表 6-2　指定日期各时段的合同购电量（单位：kWh）

时段	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00—10:10	0.00	0.00	0.00	0.00
12:00—12:10	580.62	0.00	459.48	983.20
14:00—14:10	0.00	0.00	0.00	0.00
16:00—16:10	496.08	248.77	518.12	834.89
18:00—18:10	709.87	187.84	786.65	740.27
20:00—20:10	0.00	0.00	0.00	0.00

表 6-2 中，四个指定日在 10 时、14 时和 20 时所列区间均未安排合同购电，其余区间随日内供需预测变化。全天合同购电量与费用见表 6-3。

表 6-3　指定日期的全天购电量与费用

指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
全天合同购电量（kWh）	68,389.28	36,888.64	68,767.82	100,208.01
全天合同购电费（元）	41,732.18	22,463.51	42,673.47	64,927.52
全天紧急购电费（元）	0.00	0.00	3,543.85	0.00
全天总购电费用（元）	**41,732.18**	**22,463.51**	**46,217.32**	**64,927.52**

表 6-3 中，6 月 21 日的合同购电量与合同购电费最低，12 月 21 日最高；仅 9 月 23 日发生紧急购电支出。表 6-4 进一步给出对应的实际充放电安排。

各四小时时段按“充电量 / 放电量”列示，由对应 10 分钟时段的实际充电量、放电量分别累计；末两行为日初、日末储电量。

表 6-4　指定日期的储能运行结果（单位：kWh）

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
00:00—04:00	8,120.02 / 78.30	1,086.50 / 11.28	8,966.62 / 32.45	8,618.55 / 82.69
04:00—08:00	1,936.62 / 5,166.20	654.96 / 1,811.94	1,646.87 / 6,012.29	2,012.08 / 1,422.32
08:00—12:00	6,154.58 / 2,777.38	8,198.21 / 0.00	5,563.98 / 1,896.62	5,990.01 / 6,609.03
12:00—16:00	3,715.84 / 369.62	0.00 / 0.00	4,505.29 / 551.62	5,883.55 / 2,206.78
16:00—20:00	270.55 / 5,655.84	264.84 / 4,969.88	114.27 / 5,917.57	0.00 / 5,516.21
20:00—24:00	553.33 / 2,965.26	1,732.39 / 2,485.90	500.41 / 2,617.01	610.35 / 2,844.36
00:00 储电量	2,061.86	3,880.09	1,338.15	2,013.81
24:00 储电量	1,834.84	4,313.30	1,586.33	2,059.79

表 6-4 中，6 月 21 日 08:00—12:00 累计充入 8,198.21 kWh，12:00—16:00 未发生充放电，傍晚和夜间再释放储存电量。四个指定日的日末储电量均未回到日初水平，余量由后续日期继续使用。

指定日期的紧急购电结果见表 6-5。

表 6-5　指定日期的紧急购电结果

日期	紧急购电时段	购电量（kWh）
3 月 20 日	无	0.00
6 月 21 日	无	0.00
9 月 23 日	20:20—20:40	413.39
9 月 23 日	20:50—21:10	81.94
9 月 23 日	21:20—22:00	63.20
12 月 21 日	无	0.00

四个指定日中，仅 9 月 23 日发生紧急购电，合计 558.53 kWh，费用为 3,543.85 元。当日 20:20—20:30 的净缺口为 839.71 kWh，超过单时段最大放电量 833.33 kWh，因此补购 6.38 kWh。下一时段受剩余储电量限制，仅能放出 377.88 kWh，放电后达到储电量下限，补购量增加至 407.01 kWh。其后补购主要发生在储电量接近下限的时段，说明储能弥补预测误差的能力同时受放电功率和可用电量限制。

为观察整个评价期的补购情况，按日期和日内时段汇总紧急购电量，其分布与累计结果如图 5 所示。

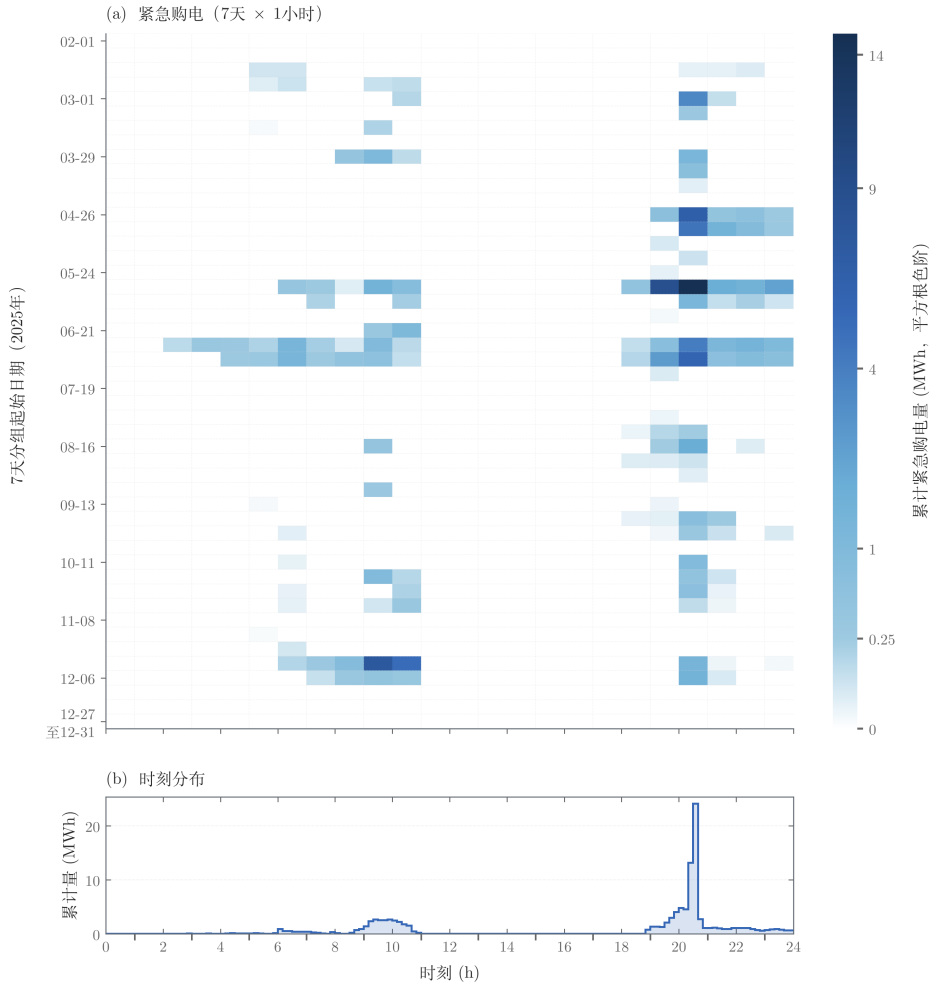


图 5　评价期紧急购电量的时段分布与累计特征

图 5 显示，紧急购电主要集中于部分日内时段，其中 20:00—21:00 占评价期总量的 45.18%，20:30—20:40 的累计量最大，上午部分时段也较为集中。上部每格累计连续 7 天内同一小时的补购量，最后一组为 5 天；下部按日内 10 分钟时段累计，单位均为 MWh。白色表示零值，色阶采用平方根映射。

6.5　模型检验

6.5.1　预测精度与基准比较

为检验指数加权预测的适用性，设置两种基准：分别采用上周同一时段负荷与前一日同一时段光伏出力，以及相同历史窗口内的等权均值。各方案依据自身历史误差，按相同规则设置 $\alpha = 0.8$ 的风险裕量，保持购电模型、日末价值系数与储能控制不变，从 1 月 1 日相同储电量开始连续运行，按正式评价区间比较预测误差与实际费用，结果见表 6-6。

表 6-6　不同预测方法的精度与调度费用比较

预测方案	负荷 MAE/kW	光伏 MAE/kW	净负荷 MAE/(kWh/时段)	总购电费用（元）
上周同期负荷＋昨日同期光伏	176.80	185.35	50.64	14,101,205.81
同窗口等权均值	188.88	153.40	47.48	13,913,353.02
本文指数加权预测	166.03	150.61	43.97	13,923,903.39

表 6-6 显示，本文方法的净负荷平均绝对误差（MAE）为 43.97 kWh/时段，较同期延续基准和同窗口等权均值分别降低 13.16% 和 7.39%。但等权均值方案的总购电费用比本文方案低 10,550.37 元，说明平均预测误差较小并不必然带来最低调度费用。进一步计算，本文负荷预测的均方根误差（RMSE）为 233.28

kW，平均绝对百分比误差（MAPE）为 3.87%；净负荷 RMSE 为 63.41 kWh/时段，绝对误差总和与实测绝对值总和之比（WAPE）为 8.79%。

6.5.2 储能参数敏感性分析

以问题二主方案为基准，分别改变储能容量、充放电功率和效率，其余预测、风险裕量与控制规则保持不变。容量和功率分别上下调整 20%，充、放电效率同时取 85% 或 95%；容量变化时，储电量上下限仍取额定容量的 10% 和 90%。各场景以 1 月 1 日 6,000 kWh 为起点连续运行，按 334 天评价区间统计，结果见表 6-7。

表 6-7　储能物理参数变化下的调度结果

场景	总购电费用（元）	费用变化（元）	变化率	紧急购电量 (kWh)	未利用电量 (kWh)
基准方案	13,923,903.39	—	—	111,928.38	2,589,736.79
充、放电效率 95%	13,461,274.33	-462,629.06	-3.32%	114,870.84	2,598,490.44
充、放电效率 85%	14,402,281.93	+478,378.54	+3.44%	108,231.25	2,580,296.27
容量 14,400 kWh	13,426,837.17	-497,066.21	-3.57%	107,065.48	2,390,168.93
容量 9,600 kWh	14,550,231.24	+626,327.86	+4.50%	117,928.88	2,774,509.51
功率 6,000 kW	13,878,859.58	-45,043.81	-0.32%	112,605.05	2,591,728.29
功率 4,000 kW	14,048,037.89	+124,134.51	+0.89%	112,217.99	2,546,158.97

表 6-7 显示，在本次扰动范围内，容量和效率变化对费用的影响大于功率变化。容量减少 20% 使总购电费用增加 4.50%，增加 20% 则降低 3.57%；效率降至 85% 和升至 95% 时，费用分别增加 3.44% 和降低 3.32%。相应地，功率减少和增加 20% 仅使费用变化 0.89% 和 −0.32%。各场景的费用与紧急购电量变化如图 6 所示。

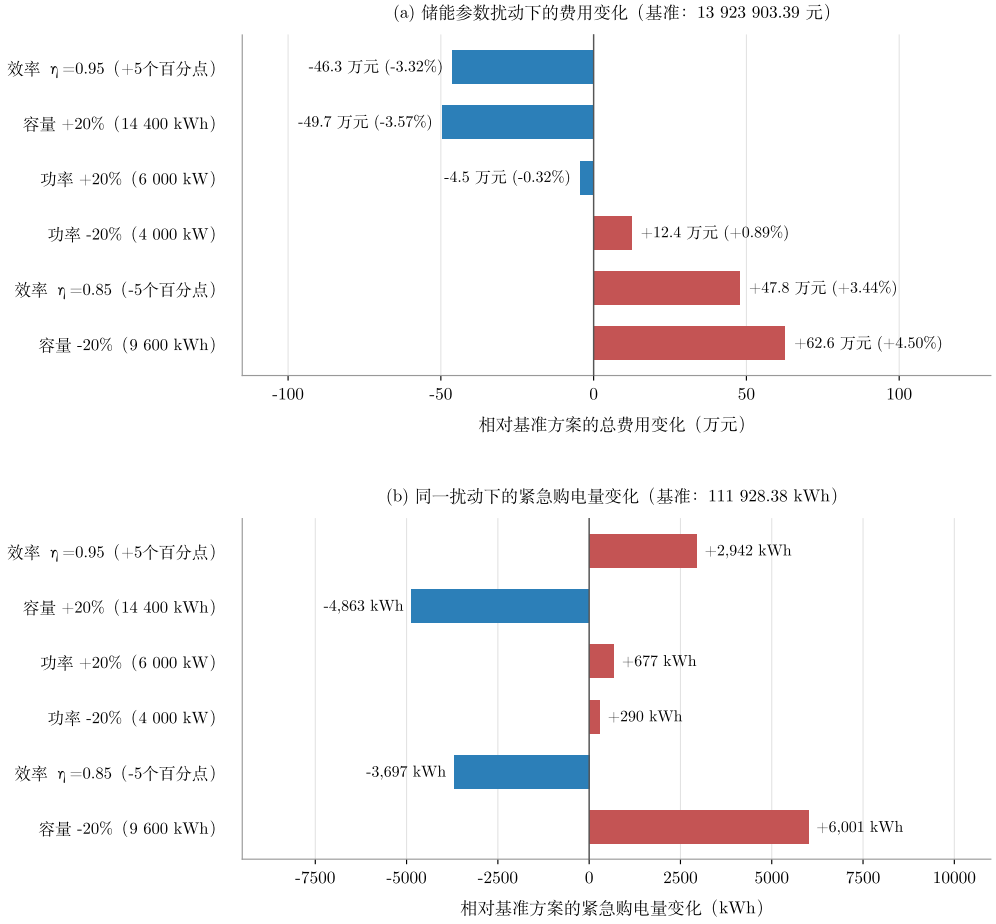


图 6　储能物理参数变化下的费用与紧急购电量

图 6 显示，费用与紧急购电量并非始终同向变化：提高效率降低了总购电费用，却略增紧急购电量；增加容量则同时降低两项指标。容量减少 20% 时，累计充电量和放电量分别减少 1,019,464.99 kWh 和 825,740.65 kWh，未利用电量增加 184,772.71 kWh，表明可用容量缩小削弱了电量跨时段配置的作用。七组场景均满足物理约束，所测总购电费用变化介于 −3.57% 与 4.50% 之间。

6.5.3 相同初始储电量检验

主方案与零裕量方案经过 1 月运行后，进入评价期的储电量不同。分别以 6,000 kWh 和主方案 2 月 1 日的实际储电量为共同起点，在其余设置相同的条件下重新运行，结果见表 6-8。

表 6-8 相同评价期初始储电量下的费用比较

共同初始储电量（kWh）	主方案总购电费用（元）	零裕量方案总购电费用（元）	节省金额（元）	节省率
6,000.00	13,922,221.76	15,329,019.37	1,406,797.61	9.18%
2,475.00	13,923,903.39	15,330,700.99	1,406,797.61	9.18%

两个共同起点下，风险裕量策略均节省约 140.68 万元，降幅仍为 9.18%。与原连续运行结果相比，节省额变化均不足 504 元，占原节省额的 0.04%，说明表 6-1 的费用改善并非主要来自评价期初始储电量的差异。

6.5.4 运行一致性检验

对包含 1 月初始化阶段在内的全年运行记录进行检查，规划阶段电量平衡与储电量递推的最大残差均小于 5×10^{-12} kWh，实际运行阶段均小于 10^{-12} kWh，未出现储电量越界、功率超限或同时充放电。各日实际日末储电量与次日规划起点一致，逐日费用分项与合计的最大差异小于 2×10^{-11} 元。

相同配置重复计算时，逐时段结果的最大数值差异约为 5.00×10^{-11}，按公式独立计算的同星期负荷预测与程序结果一致。

七、问题三的模型建立与求解

7.1 光伏预报处理与风险裕量

问题三中，新光伏预报改变了对剩余时段供需的判断，合同调整又涉及增购支出与减购退款。因此，需要权衡提前调整合同与保留原计划的费用。在问题二的负荷预测和储能控制基础上，采用滚动优化思路[10]，每次预报更新后重新规划当天剩余时段，并将历次增减购电费用纳入结算。

光伏预报每日发布四次，每次给出未来 24 小时的整点预测。将同日连续记录中省略的日期按上一有效日期补齐，按日期、发布时间和提前小时数整理，共得到 1,460 条完整发布记录，每条包含 24 个非负预报值，未发现缺失或重复。

以 $k\in\mathcal{K}=\{0,6,12,18\}$ 表示预报发布的小时数，截至发布时间已完成 $m_k=6k$ 个时段，剩余规划范围为 $\mathcal{R}_k=\{m_k+1,\dots,144\}$。负荷预测沿用式 (11)，当天各次规划使用相同预测值；光伏预测则采用各时点新发布的预报，只使用其中针对当天剩余时段的部分。

记 $F_{D,k,\ell}$ 为日期 D 的 k 时发布、提前 ℓ 小时的光伏功率预报，$F_{D,k,0}$ 取截至发布时间最近一个已结束区间的实测值。对目标时段 $t\in\mathcal{R}_k$，令 $\ell=\min\{\lfloor t\Delta t-k\rfloor,23\}$、$\theta=t\Delta t-k-\ell$，按目标区间的结束时刻进行线性插值，得到

$$\widehat{G}_{D,t|k}=\max\{0,(1-\theta)F_{D,k,\ell}+\theta F_{D,k,\ell+1}\},\quad \widehat{n}_{D,t|k}=(\widehat{L}_{D,t}-\widehat{G}_{D,t|k})\Delta t \tag{24}$$

6 时更新以 05:50—06:00 的光伏实测出力作为插值起点；0 时取上一日最后一个区间的实测值，首次运行时取零。

净负荷预测误差与目标时段及预测提前时长有关，因此不再像问题二那样汇集同一四小时组内的全部误差。将提前时长划分为 $(0,6]$、$(6,12]$、$(12,18]$ 和 $(18,24]$ 四档。记 j 为历史日期、$v\in\mathcal{K}$ 为该日的发布小时数。对同一目标时段 t，汇集此前最多 28 天中提前时长处于相同档位的有效误差 $\epsilon_{j,t|v}=n_{j,t}-\widehat{n}_{j,t|v}$，构成 $\mathcal{E}_{D,t|k}$。记其覆盖的历史日期数为 $N_{D,t|k}$，则

$$b_{D,t|k} = \max\{0, Q_\alpha(\mathcal{E}_{D,t|k})\}, \quad N_{D,t|k} \geq 7 \tag{25}$$

$$b_{D,t|k} = 0, \quad N_{D,t|k} < 7 \tag{26}$$

$$\widetilde{n}_{D,t|k} = \widehat{n}_{D,t|k} + b_{D,t|k} \tag{27}$$

主方案取 $\alpha = 0.8$，分位数仍采用线性插值计算。历史误差以各时点当时发布的预报为依据，当日已结束时段的误差从次日起才参与校准。

7.2 基于逐次结算的滚动购电模型

合同购电量与储能充放电的调整频率不同，其关系如图 7 所示。新预报发布时，重新确定当天剩余时段的合同购电量；两次发布之间，仍根据当期净负荷与储电量调节充放电。每次调整费用均以上一次约定的合同购电量为计算依据。

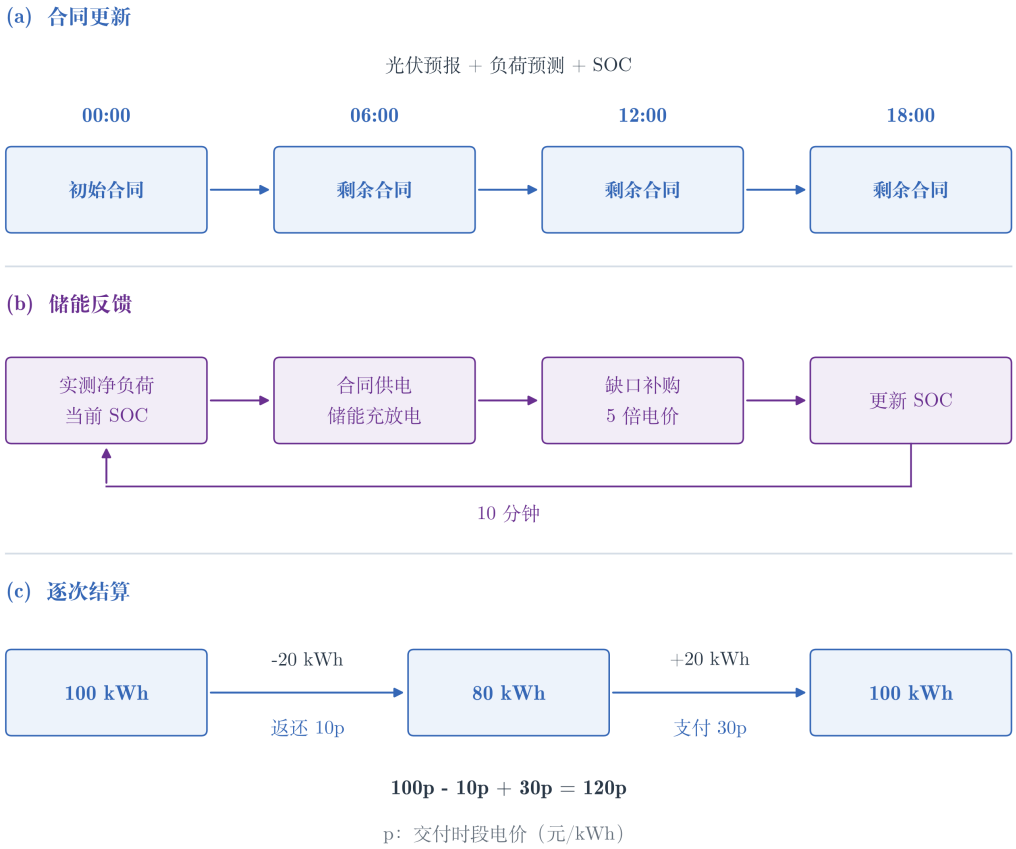


图 7　日内合同滚动调整、储能反馈与逐次结算机制

图中的 100→80→100 kWh 表示同一未来交付时段的合同购电量经过两次调整的过程。示例采用固定单位电价。以下分别建立合同调整费用和剩余时段购电模型。

7.2.1 合同调整的费用计算

记 $q_{D,t}$ 为 0 时确定的合同购电量，$\bar{y}_{D,t|k}$ 为本次调整前约定的购电量，$y_{D,t|k}$ 为调整后的购电量。日初合同按对应交付时段的正常电价计费；日内增购按该电价的 1.5 倍计费，减购按假设 4 净返 0.5 倍正常购电款。以增购支出减去减购退款，得到调整净费用。本次增减量及调整净费用为

$$a^{+}_{D,k,t} = [y_{D,t|k} - \bar{y}_{D,t|k}]_+, \quad a^{-}_{D,k,t} = [\bar{y}_{D,t|k} - y_{D,t|k}]_+ \tag{28}$$

$$h_{D,k,t} = 1.5 p_t a^{+}_{D,k,t} - 0.5 p_t a^{-}_{D,k,t} \tag{29}$$

每次调整成功后，以新的合同购电量作为下一次增减量的计算依据，日初确定的 $q_{D,t}$ 保持不变。总购电费用按历次调整分别累计。例如，电价为 1 元/kWh，合同购电量由 100 kWh 减至 80 kWh，再恢复至 100

kWh，累计合同费用为 $100-0.5\times20+1.5\times20=120$ 元。最终电量虽未改变，中间的减购和增购仍产生费用。

7.2.2 日初计划与剩余时段规划

0 时采用式 (15)—(18) 的约束，将规划净需求替换为 $\widetilde{n}_{D,t|0}$，求解

$$\min J_{D,0}=\sum_{t=1}^{144}p_tq_{D,t}-\lambda_D(\widetilde{S}_{D,144}-1200) \tag{30}$$

其中，$\lambda_D=v_D$，系数取值及 12 月 31 日的处理沿用第六章，当天各次规划使用相同值。以实际日初储电量为起点求得合同购电量，初始约定量为 $\bar{y}_{D,t}=q_{D,t}$。

在 6、12、18 时，以当前实际储电量和最新光伏预报重新规划当天所有尚未开始的时段。以下省略单次规划变量中的日期与发布时点下标，引入非负的增、减购电量 a_t^+,a_t^- 及规划紧急购电量 $\widetilde{e}_t$，建立线性模型：

$$\min_{y,a^+,a^-,\widetilde{c},\widetilde{r},\widetilde{e},\widetilde{w},\widetilde{S}}\quad J_{D,k}=\sum_{t\in\mathcal{R}_k}\left(1.5p_ta_t^+-0.5p_ta_t^-+5p_t\widetilde{e}_t\right)-\lambda_D(\widetilde{S}_{144}-1200) \tag{31}$$

$$\text{s.t.}\quad\begin{cases}y_t+\widetilde{r}_t+\widetilde{e}_t=\widetilde{n}_{D,t|k}+\widetilde{c}_t+\widetilde{w}_t & (32)\\ y_t-a_t^++a_t^-=\bar{y}_t & (33)\\ \widetilde{S}_t=\widetilde{S}_{t-1}+\eta_c\widetilde{c}_t-\dfrac{\widetilde{r}_t}{\eta_d} & (34)\\ 1200\le\widetilde{S}_t\le10800,\quad 0\le\widetilde{c}_t,\widetilde{r}_t\le U & (35)\\ y_t,a_t^+,a_t^-,\widetilde{e}_t,\widetilde{w}_t\ge0,\quad \widetilde{S}_{m_k}=S_{D,m_k} & (36)\end{cases}$$

上述时段约束对 $t\in\mathcal{R}_k$ 成立。日初合同费及此前调整费用已支付，不随本次决策改变，因此不重复计入当前目标。增购量与减购量采用线性约束表示。若 a_t^+ 与 a_t^- 同时减少相同正量，合同交付量不变而费用下降，故一级最优解不会同时增购和减购。

由于合同增购不受额外容量限制，且价格低于紧急购电，主方案一级最优解中的规划补购量 $\widetilde{e}_t=0$。

7.3 滚动求解与实时执行

问题三采用以预报发布时间为更新节点的滚动求解。每天 0 时，以全天 144 个时段的规划净需求和实际日初储电量为输入，求解日初模型并确定原始合同。到 6、12、18 时，分别截取当天剩余 108、72 和 36 个时段，更新光伏预报与风险裕量，以上一次约定的剩余合同购电量和当前实际储电量组装式 (31)—(36) 的目标与约束。

日初模型包含 720 个变量和 288 个等式约束。日内模型增加合同增购量、减购量及规划紧急购电量，共有八组时段变量，每个时段对应电量平衡、储电量递推和合同增减分解三类等式约束。因此，剩余 m 个时段对应 $8m$ 个变量和 $3m$ 个等式约束，6 时规划包含 864 个变量和 324 个等式约束。程序按剩余时段长度保存稀疏系数矩阵，各次求解更新约束右端值与日末价值项。

各次规划均由 HiGHS 依次完成两级求解：先最小化主目标，再将一级目标值限制在最优值与 10^{-6} 元容差之和以内，最小化规划充放电总量。以第二级解确定新的合同购电量，计算相对上次约定量的增减量，按式 (28)—(29) 累计本次调整费用，并更新尚未开始时段的合同。

储能仍按式 (19)—(22) 每 10 分钟调节充放电，只需将其中的固定合同购电量 $q_{D,t}$ 替换为当前约定的合同购电量 $y_{D,t}$。两次预报发布之间，合同购电量保持不变，充放电随实际供需调节。实际日末储电量作为次日调度起点，首次运行规则与第六章相同；主方案在 2 月 1 日的初始储电量约为 2,492.63 kWh。

对 2 月 1 日至 12 月 31 日的 334 天，总购电费用为

$$C_3=\sum_{D\in\mathcal{D}}\left[\sum_{t=1}^{144}p_tq_{D,t}+\sum_{k\in\{6,12,18\}}\sum_{t\in\mathcal{R}_k}h_{D,k,t}+\sum_{t=1}^{144}5p_te_{D,t}\right] \tag{37}$$

统一汇总每日的日初合同购电量、最终合同购电量、实际充放电量与储电量，并将相邻补购时段合并为连续紧急购电事件。其中，最终合同购电量为各时段交付前最后约定的电量。

7.4 结果与分析

7.4.1 评价期费用与基准方案比较

以预测、合同调整和储能控制规则相同的零裕量方案为基准，两方案均由 1 月连续运行形成评价期初始储电量。总购电费用为日初合同购电费、累计调整净费用与紧急购电费之和；总购电量为最终合同购电量与紧急购电量之和。334 天的比较结果见表 7-1。

表 7-1　滚动调整策略与零裕量方案的总购电费用及电量比较

类别	指标	零裕量方案	主方案（风险裕量策略）
费用	**总购电费用（元）**	**13,931,811.69**	**13,833,843.32**
费用	日初合同购电费（元）	12,314,073.47	13,456,429.66
费用	累计调整净费用（元）	683,379.84	130,188.78
费用	紧急购电费（元）	934,358.38	247,224.89
电量	**总购电量（kWh）**	**20,483,082.87**	**21,109,375.60**
电量	最终合同购电量（kWh）	20,290,981.25	21,057,849.90
电量	紧急购电量（kWh）	192,101.62	51,525.70
调整前计划	日初合同购电量（kWh）	20,262,429.61	21,848,677.89

表 7-1 显示，主方案相对零裕量方案的总购电费用降低 0.70%，紧急购电量下降 73.18%。日初合同购电支出增加 1,142,356.18 元，调整净费用与紧急购电费分别减少 553,191.06 元和 687,133.49 元，合计节省 97,968.37 元。增加初始采购降低了后续调整与补购支出，但总购电量仍高于零裕量方案。

主方案累计增购 502,453.27 kWh、减购 1,293,281.26 kWh，最终合同购电量比日初约定总量少 790,827.99 kWh，但调整净费用仍为正。这是因为增减购的计费标准不同，各交付时段电价也存在差异，且每次调整都单独结算。合同购电与调整费用合计 13,586,618.43 元，加上紧急购电费后，总购电费用为 13,833,843.32 元。

7.4.2 指定日期的合同与储能运行

四个指定日期的购电结果见表 7-2。所列时段电量均为交付前最后确定的合同购电量，不包含紧急购电；全天总购电费用按式 (37) 计算。

表 7-2　指定日期的合同购电量与费用（电量：kWh；费用：元）

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00—10:10	0.00	0.00	0.00	0.00
12:00—12:10	539.45	0.00	305.21	943.92
14:00—14:10	0.00	0.00	0.00	0.00
16:00—16:10	527.90	196.33	537.86	871.33
18:00—18:10	709.09	31.40	786.76	737.52
20:00—20:10	0.00	0.00	0.00	0.00
日初合同购电量	71,087.52	36,147.79	72,693.83	100,303.17
全天最终合同购电量	66,819.56	36,041.52	67,831.73	98,173.39
累计调整净费用	645.29	84.18	-1,405.03	-295.00
全天总购电费用	**44,686.40**	**21,832.88**	**44,246.39**	**64,454.85**

3 月 20 日最终合同购电量低于日初约定量，但调整净费用为 645.29 元；9 月 23 日则净退款 1,405.03 元。两日均为净减购，却分别产生净支出和净退款，说明结算必须考虑各交付时段的价格，以及每次调整前后的购电量。

各四小时时段按“充电量 / 放电量”列示，由对应 10 分钟时段的实际充电量、放电量分别累计；末两行为日初、日末储电量。

表 7-3　指定日期的实际储能运行结果（单位：kWh）

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
00:00—04:00	8,118.92 / 106.60	1,144.79 / 0.00	8,938.64 / 30.58	8,757.99 / 63.02
04:00—08:00	1,821.58 / 6,003.43	617.96 / 1,829.67	1,671.30 / 6,591.76	1,946.45 / 1,441.19
08:00—12:00	4,304.48 / 2,708.76	8,419.24 / 0.00	5,848.81 / 1,896.62	5,491.49 / 7,217.29
12:00—16:00	6,508.79 / 290.08	0.00 / 0.00	5,016.84 / 461.71	7,229.26 / 2,270.99
16:00—20:00	263.53 / 5,606.06	339.35 / 5,126.32	195.79 / 5,724.42	0.00 / 5,557.94
20:00—24:00	570.83 / 2,970.23	1,723.96 / 2,485.90	533.68 / 3,089.76	615.16 / 2,844.63
00:00 储电量	2,052.21	3,669.18	1,391.90	1,931.21
24:00 储电量	1,831.34	4,198.96	1,604.40	2,017.46

6 月 21 日 08:00—12:00 累计充电 8,419.24 kWh，16:00—20:00 放电 5,126.32 kWh，日间储存的电量用于傍晚供电。

四个指定日期的紧急购电情况见表 7-4。

表 7-4　指定日期的紧急购电结果

日期	紧急购电时段	购电量（kWh）
3 月 20 日	09:40—10:00	68.61
6 月 21 日	无	0.00
9 月 23 日	20:20—20:30	6.38
9 月 23 日	20:50—21:10	40.68
9 月 23 日	21:20—21:40	41.81
9 月 23 日	21:50—22:00	0.32
12 月 21 日	无	0.00

表 7-4 中的紧急购电集中于 3 月 20 日上午和 9 月 23 日晚间。3 月 20 日 09:40—09:50 的净缺口为 41.26 kWh，而当时剩余储电量仅能支持 17.85 kWh 放电，两个连续时段合计补购 68.61 kWh。9 月 23 日补购 89.18 kWh，其中 20:20—20:30 的缺口超过单时段放电上限，其余补购主要受剩余储电量限制。

以 3 月 20 日为例，图 8 对应展示各次确定的合同购电量、每次调整量、实际储电量和紧急购电量，分析日内计划调整与实际运行之间的联系。

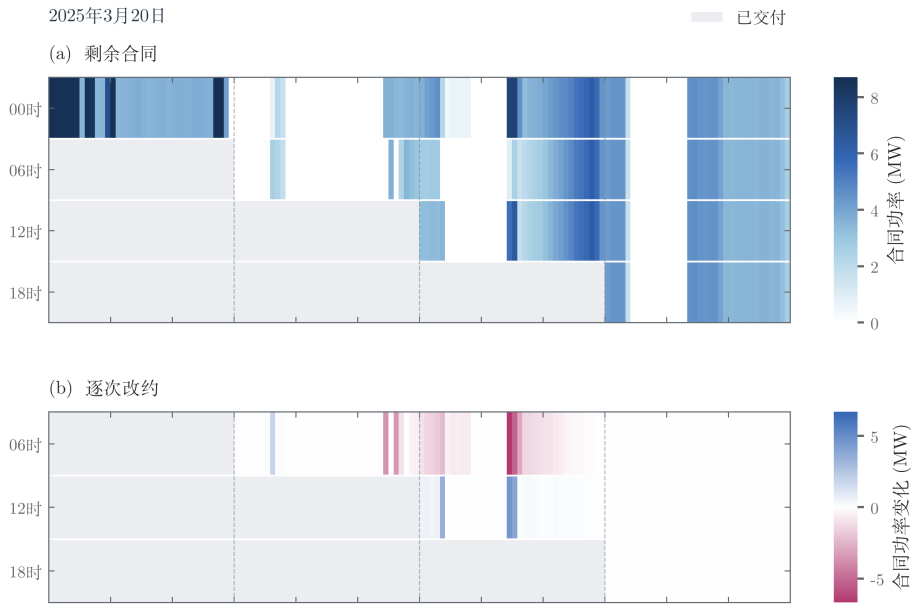
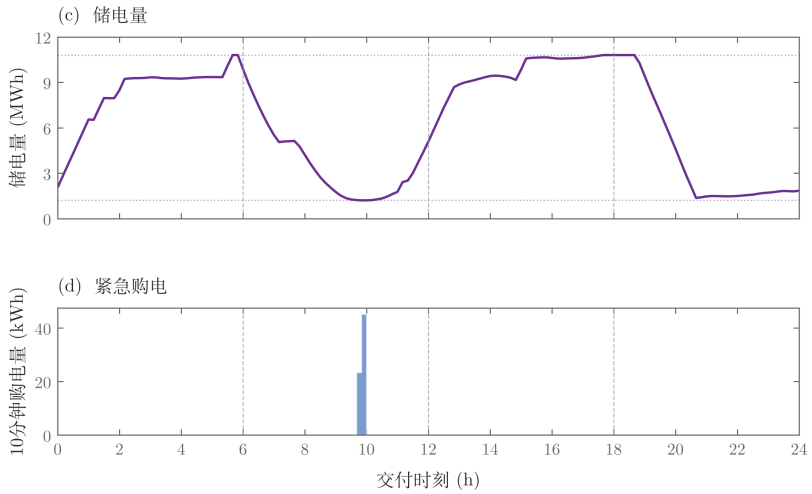


图 8　3 月 20 日滚动合同调整与储能运行过程：(a) 剩余合同；(b) 逐次改约



续图 8　3 月 20 日滚动合同调整与储能运行过程：(c) 储电量；(d) 紧急购电

图 8 中，6 时和 12 时对部分未来时段的购电量作了调整，18 时变化接近零；上午的紧急购电与储电量低谷相对应。前两部分分别展示各次约定的剩余时段购电量及相对上次的变化，均换算为 MW 表示的等效平均功率。灰色区域为已交付时段，蓝色与紫红色分别表示增购和减购。

7.4.3　预报发布时间组合比较

为考察日内调整的经济价值，依次比较仅采用 0 时、采用 0 和 6 时、采用 0、6 和 12 时，以及采用全部四个时点预报的方案。各方案使用相同的预测方法和历史误差整理规则，仅在选定时点根据新预报调整合同购电量，储能均每 10 分钟根据实际供需调节，费用比较如图 9 所示。

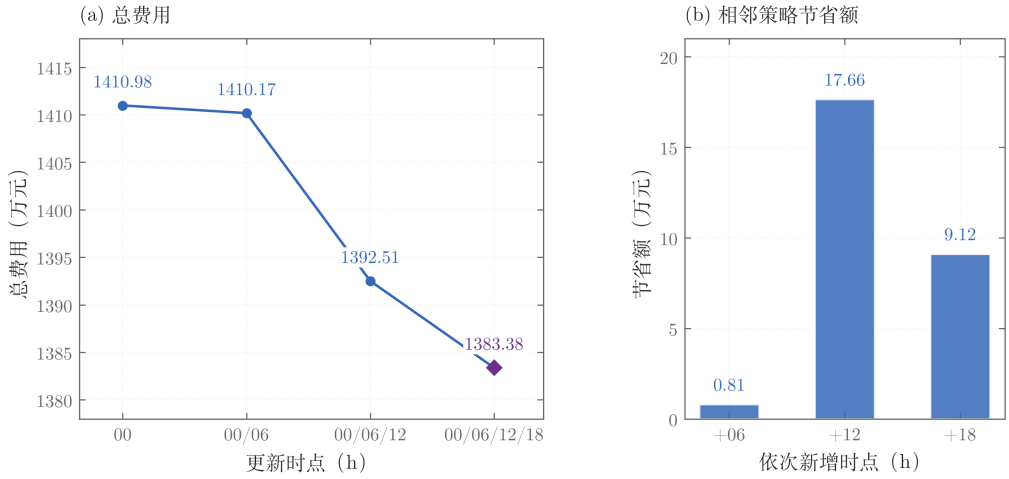


图 9　不同预报更新时点组合下的总购电费用比较

图 9 显示，按给定顺序增加调整时点后，总购电费用逐步下降，新增 12 时调整机会的节费幅度最大，18 时次之，6 时最小。四时点方案比仅 0 时方案节省 275,921.60 元，降幅为 1.96%，说明本次预报更新带来的采购改善能够覆盖合同调整成本。对应的紧急购电量依次为 114,500.40、89,004.76、72,224.73 和 51,525.70 kWh，也逐步减少。图中金额以万元表示，右图各柱表示相邻两种更新时间组合之间的节省额。

7.4.4　风险参数与机制分析

先固定官方光伏预报与实时储能控制，仅改变风险分位数，比较购电费用与紧急购电量，如图 10 所示。

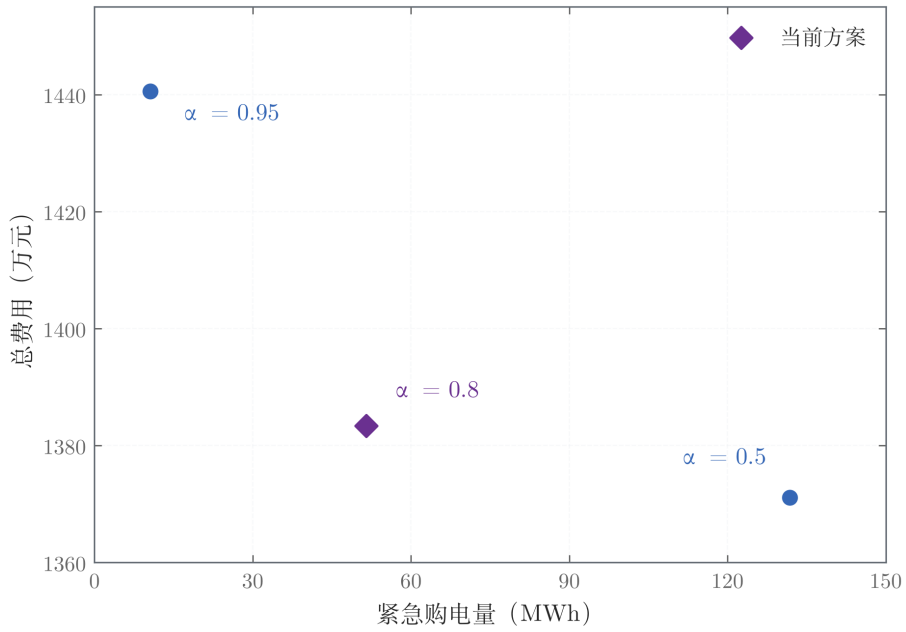


图 10　不同风险分位数水平下的购电费用与紧急购电量

图 10 显示，提高风险分位数减少了紧急购电量，但增加了总购电费用。紧急购电已按 5 倍正常电价计费，以总购电费用为标准，三个已测试参数中 $\alpha = 0.50$ 的费用最低，为 13,711,168.87 元。$\alpha = 0.80$ 比其多支出 122,674.45 元，减少紧急购电 80,319.58 kWh；$\alpha = 0.95$ 的总购电费用进一步升至 14,405,508.82 元，紧急购电量降至 10,535.05 kWh。

本文保留 $\alpha = 0.80$ 主方案，以比较相同风险参数下引入预报更新与合同调整的效果；$\alpha = 0.50$ 为本次参数分析中的最低费用方案。图中的菱形标记主方案，横、纵轴分别以 MWh 和万元表示紧急购电量与总购电费用，$\gamma = 0.5$ 为日末价值系数的折减因子。

为检验日末电量估值的影响，固定风险参数 $\alpha = 0.8$，分别取 $\gamma = 0$、0.5 和 1 进行连续调度比较。$\gamma = 0$ 与 0.5 对应的总购电费用基本一致；相较于 $\gamma = 0.5$，$\gamma = 1$ 时问题二和问题三的费用分别增加 178,663.00 元和 171,891.71 元，并出现更多电量跨日留存。提高日末电量估值未能改善本次调度的经济性，因此保留 $\gamma = 0.5$ 作为折减估值设定，不将其视为唯一最优取值。

进一步保持合同调整时点、风险分位数及储能控制规则相同，将官方光伏预报替换为历史光伏预测。图 11 并列展示两方案的合同购电与调整费用、紧急购电费及紧急购电量。

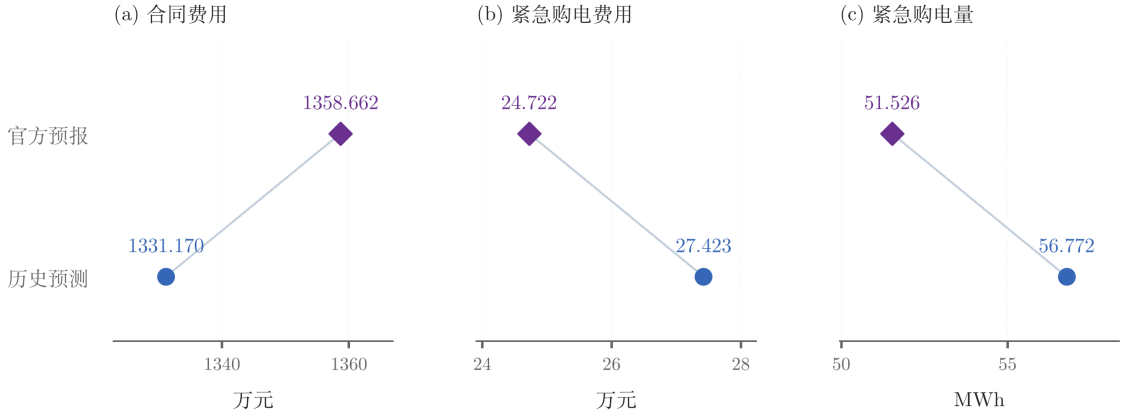


图 11　问题三两类光伏预测下的费用构成与紧急购电量

图 11 显示，官方预报方案的合同购电与调整费用高 27.49 万元，紧急购电费低 2.70 万元，因而总购电费用仍高 24.79 万元。历史光伏预测方案的总购电费用为 13,585,929.92 元，低于主方案的 13,833,843.32 元，但紧急购电量由 51.53 MWh 增至 56.77 MWh。图中“合同费用”包含日初合同购电费与累计调整净费用，与紧急购电费相加即为总购电费用。预测来源同时改变误差分位数、采购安排和后续储电量，因此预测更准确并不必然获得更低费用。

固定执行最近一次储能计划时，总购电费用为 14,822,066.74 元，紧急购电量为 274,333.22 kWh，较实时反馈主方案多支出 988,223.42 元，反映了根据实际供需调整充放电的作用。该比较也包含储能控制变化引起的后续购电调整，给定相同合同安排的储能预留对照见第 7.5.3 节。

7.5 模型检验

7.5.1 预测精度与误差分布

为检验光伏预报更新对净负荷预测的影响，在相同的 48,096 个评价时段比较官方预报与历史光伏预测。两类方案采用相同的负荷预测；历史光伏预测在日初确定，官方预报取各时段最近一次已发布的结果，每个时段仅统计一次。采用 MAE、RMSE 和 WAPE 衡量误差，另以实测光伏出力大于 50 kW 的 24,207 个时段计算光伏 MAPE，以实际净负荷绝对值不低于 100 kWh 的 44,390 个时段计算净负荷 MAPE。表 7-5 中的平均偏差均取预测值减实测值。

表 7-5　评价期内两类预测方法的精度比较

指标	官方光伏预报	历史光伏预测
负荷 MAE（kW）	166.03	166.03
负荷 RMSE（kW）	233.28	233.28
负荷 MAPE（%）	3.87	3.87
负荷平均偏差（kW）	+2.92	+2.92
光伏 MAE（kW）	86.41	150.61
光伏 RMSE（kW）	168.53	300.61
光伏平均偏差（kW）	-1.39	+2.40
光伏 MAPE（实测出力 >50 kW，%）	9.24	7.82
光伏全期电量相对偏差（%）	-0.06	+0.10
净负荷 MAE（kWh/时段）	34.54	43.97
净负荷 RMSE（kWh/时段）	47.65	63.41
净负荷 WAPE（%）	6.91	8.79
净负荷 MAPE（实测绝对值 ≥100 kWh，%）	8.61	11.65
净负荷平均偏差（kWh/时段）	+0.72	+0.09

表 7-5 显示，采用官方预报后，光伏 MAE 与 RMSE 均降低，净负荷 MAE 由 43.97 降至 34.54 kWh/时段，WAPE 由 8.79% 降至 6.91%。但在所选正出力时段，官方光伏预报的 MAPE 高于历史预测，说明按实测出力归一化后的相对误差与全期绝对误差并不具有相同排序。典型周预测曲线及分时段误差分布见图 12。

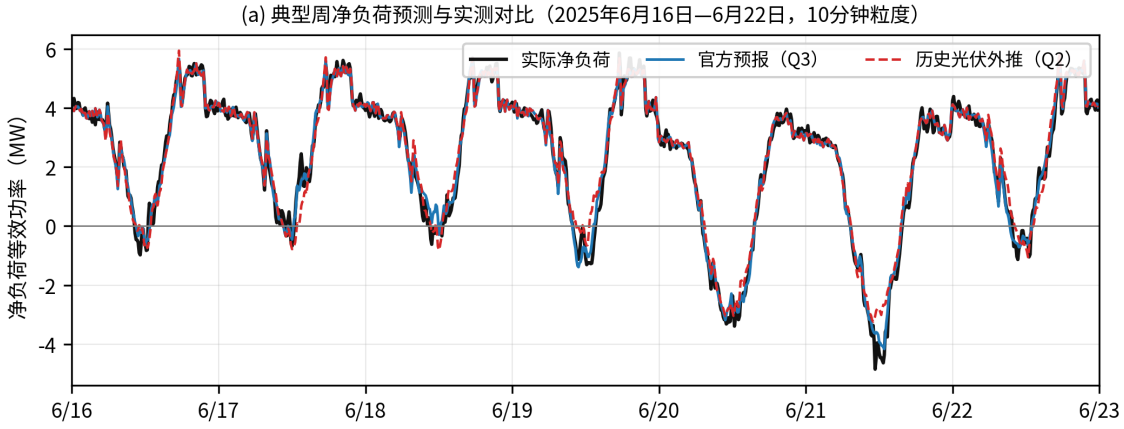
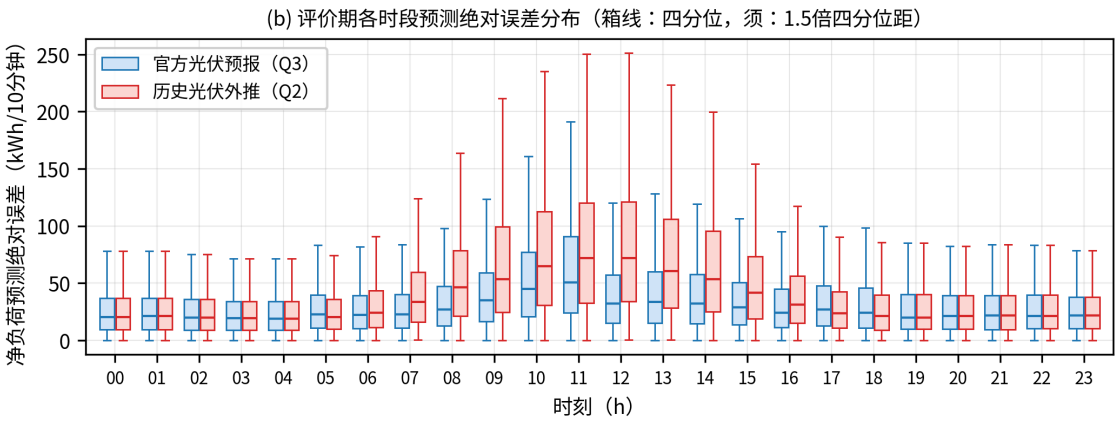


图 12　净负荷预测与实测对比及分时段误差分布：（a）典型周预测与实测对比



续图 12　净负荷预测与实测对比及分时段误差分布：（b）评价期分时段预测绝对误差

图 12 中，两类预测均反映了典型周净负荷的日内变化，差异主要出现在光伏发电时段。6 月 21 日午间，历史预测高于实际净负荷，与第 6.4.2 节中实际富余电量多于预测的结果一致。分时段箱线图显示，白天误差的离散程度高于夜间，官方预报在午间多数时段的误差分布更集中。各小时均汇集对应六个 10 分钟时段的评价期记录。

结合图 11 可见，官方预报的全期绝对误差较小，但合同购电与调整费用的增加超过了紧急购电支出的减少，因而没有取得更低的总购电费用。

7.5.2　风险裕量覆盖检验

为检验历史误差分位数对后续实际需求的覆盖情况，定义经验覆盖率为

$$\widehat{C} = \frac{1}{N} \sum_{(D,t)} \mathbf{1}\{n_{D,t} \le \widehat{n}_{D,t} + b_{D,t}\} \tag{38}$$

其中，N 为评价时段数。问题二采用日初预测与裕量，问题三采用各时段最近一次已发布的预测与裕量，结果见表 7-6。

表 7-6　评价期风险裕量的实际覆盖情况

策略	样本时段数	裕量覆盖率	超越比例
Q2 日初预测与裕量	48,096	78.82%	21.18%
Q3 各时段最新已发布预测与裕量	48,096	77.35%	22.65%

两类策略的覆盖率分别为 78.82% 和 77.35%，均略低于 80% 的经验分位数水平。按月统计，问题二覆盖率为 76.03%—82.56%，问题三为 75.60%—81.81%，说明历史裕量对实际偏差的覆盖程度随月份变化。该指标反映预测值加裕量对净负荷的覆盖情况，实际紧急购电仍由合同安排、可用储电量及放电功率共同决定。

7.5.3　给定合同安排下的储能预留对照

为检验考虑后续电价与供需缺口能否改善储能调度，保持问题三主方案各次合同购电量、调整费用及评价期初始储电量相同，设置储能预留策略。

该策略每 10 分钟结合分时电价、已发布的光伏预报、原风险裕量和当前合同，从实际储电量出发优化当前时段至 24 时的充放电，以预测紧急购电费用减去日末储电量价值为目标，每次仅执行当前动作。当前时段采用实际净负荷，充电仅吸收当期富余电量，结果见表 7-7。

表 7-7　给定合同安排下的储能控制策略比较

指标	原实时反馈策略	价格感知储能预留策略
合同购电与调整费用（元）	13,586,618.43	13,586,618.43
紧急购电费（元）	247,224.89	351,752.97
总购电费用（元）	**13,833,843.32**	**13,938,371.41**
紧急购电量/kWh	51,525.70	88,689.61
未利用电量/kWh	1,961,159.10	2,007,040.48
评价期初储电量/kWh	2,492.63	2,492.63
评价期末储电量/kWh	1,661.10	1,661.10

表 7-7 显示，储能预留策略的总购电费用增加 104,528.08 元，增幅为 0.76%，紧急购电量和未利用电量分别增加 37,163.91 kWh 和 45,881.38 kWh。两方案期末储电量相同，费用比较不受期末电量结余差异影响。在本次预测、风险参数与既定合同安排下，所测试的预留策略未能降低费用，原实时反馈策略的费用和紧急购电量均较低，为沿用式 (19)—(22) 提供了补充依据。

7.5.4　数值与重复运行检验

对评价期 48,096 个时段及 72,144 条合同调整记录进行检查，运行记录重算的电量平衡与储电量递推残差均小于 10^{-9} kWh，未出现容量越界、功率超限或同时充放电。按时间顺序累计各次合同增减量，所得最终购电量与实际交付记录一致；累计调整费用及总购电费用的复算差异均小于 10^{-8} 元。相同配置重复运行后，合同购电、紧急购电、费用、充放电量及期末储电量等 12 项指标的差异均为零，支持所测配置下计算结果的一致性。

八、问题四的模型建立与求解

8.1　波动电价处理与信息条件

波动电价改变了日内各时段购电的相对成本，需要重新安排采购与充放电。为比较原有策略在新电价条件下的表现，分别保留问题二的固定合同策略和问题三的滚动调整策略，记为 Q4-2 和 Q4-3。以日期 D 的电价序列 $\boldsymbol{p}_D = (p_{D,1}, \ldots, p_{D,144})$ 替换固定分时电价，按各自原有的预测、风险裕量、合同调整和储能控制规则重新求解。

波动电价覆盖 2025 年全年，形成 365×144 的价格矩阵，与负荷、光伏实测的日期及区间结束时刻逐一匹配，价格均为正且无缺失。日初合同购电、日内调整与紧急购电均以交付时段的 $p_{D,t}$ 计价，规划采用给定的目标日完整价格序列。沿用连续初始化方式，以 2 月 1 日至 12 月 31 日为评价区间。

8.2　波动电价下的购电与储能调度模型

Q4-2 沿用式 (11)—(18)：根据相同星期几的历史负荷和近 7 天光伏数据进行预测，按四小时组设置风险裕量，每天 0 时确定合同购电量后不再修改。Q4-3 沿用式 (24)—(36)：采用官方光伏预报，按目标时段和预测提前时长分组设置风险裕量，并在每次调整时结算增减购电费用；0 时确定日初合同购电量，6、12、18 时调整当天尚未开始时段的合同。两类策略的主方案均保留 $\alpha = 0.8$。

在各自的日初规划约束下，两类策略分别最小化

$$J_{D,0}^{(4-2)} = \sum_{t=1}^{144} p_{D,t} q_{D,t} - v_D \widetilde{S}_{D,144} \tag{39}$$

$$J_{D,0}^{(4-3)} = \sum_{t=1}^{144} p_{D,t} q_{D,t} - \lambda_D (\widetilde{S}_{D,144} - 1200) \tag{40}$$

其中，$v_D = \lambda_D = 0.5\eta_d \,\mathrm{median}_t(p_{D,t})$，在当天各次规划中保持不变，12 月 31 日取零。Q4-3 在日内发布时点 $k \in \{6, 12, 18\}$，以实际储电量为起点，在式 (32)—(36) 的剩余时段约束下最小化

$$J_{D,k}^{(4-3)} = \sum_{t \in \mathcal{R}_k} p_{D,t} \left(1.5 a_{D,k,t}^{+} - 0.5 a_{D,k,t}^{-} + 5 \widetilde{e}_{D,t|k} \right) - \lambda_D (\widetilde{S}_{D,144} - 1200) \tag{41}$$

各次增减量仍以上一次约定的合同购电量为依据，已支付费用不重复计入当前目标。两类策略分别按日期循环求解，每日先读取对应的 144 个电价，更新正常购电、合同调整和紧急购电的目标系数，并计算当天的日末价值系数。电量平衡、储电量递推、容量及功率约束保持不变，由 HiGHS 先求解一级目标，再在 10^{-6} 元目标容差内最小化规划充放电总量。

Q4-2 沿用第 6.3 节的日初规划与逐时段执行流程，每天 0 时确定合同后不再修改。Q4-3 沿用第 7.3 节的滚动求解流程，在 6、12、18 时分别提取剩余时段的电价、最新预报和上次合同，以当前实际储电量为起点重新求解，并累计各次调整费用。两类策略分别维护自己的实际储电量，按式 (19)—(22) 逐时段计算充放电和紧急购电，再将实际日末状态传入次日。

按前述首次运行规则，以 2025 年 1 月 1 日的 6,000 kWh 为初始储电量连续运行，正式统计 2 月 1 日至 12 月 31 日。对评价日期集合 $\mathcal{D}$，总购电费用为

$$C_{4-2} = \sum_{D\in\mathcal{D}} \sum_{t=1}^{144} \left(p_{D,t} q_{D,t} + 5 p_{D,t} e_{D,t} \right) \tag{42}$$

$$C_{4-3} = \sum_{D\in\mathcal{D}} \left[\sum_{t=1}^{144} p_{D,t} \left(q_{D,t} + 5 e_{D,t} \right) + \sum_{k\in\{6,12,18\}} \sum_{t\in\mathcal{R}_k} p_{D,t} \left(1.5 a^{+}_{D,k,t} - 0.5 a^{-}_{D,k,t} \right) \right] \tag{43}$$

分别汇总两类策略的逐日合同购电、实际充放电、储电量和紧急购电结果；Q4-3 同时记录日初与最终合同购电量。

8.3 结果与分析

8.3.1 评价期费用与策略比较

分别以对应预测、价格、储能控制和结算规则下的零裕量方案为基准。四个方案均以 1 月 1 日相同的储电量开始运行，2 月 1 日各自使用此前实际运行形成的储电量进入 334 天评价期，结果见表 8-1。

表 8-1　波动电价下两类策略的总购电费用及电量比较

类别	指标	Q4-2 零裕量方案	Q4-2 主方案	Q4-3 零裕量方案	Q4-3 主方案
费用	**总购电费用（元）**	**16,126,251.22**	**14,571,629.31**	**14,630,561.99**	**14,487,237.24**
费用	日初合同购电费（元）	12,720,978.74	13,897,896.14	12,869,294.82	14,075,392.47
费用	累计调整净费用（元）	—	—	713,533.18	130,317.23
费用	紧急购电费（元）	3,405,272.47	673,733.17	1,047,734.00	281,527.53
电量	**总购电量（kWh）**	**20,673,756.38**	**21,778,514.02**	**20,550,086.40**	**21,167,857.56**
电量	最终合同购电量（kWh）	20,110,518.78	21,664,258.79	20,339,005.13	21,111,489.90
电量	紧急购电量（kWh）	563,237.60	114,255.22	211,081.27	56,367.66

相对各自的零裕量方案，Q4-2 节省 1,554,621.91 元，降幅为 9.64%；Q4-3 节省 143,324.76 元，降幅为 0.98%。紧急购电量分别下降 79.71% 和 73.30%。两类策略均通过增加正常合同购电减少高价补购，但总购电量仍高于相应基准方案，体现了增加采购余量的成本与减少紧急购电支出之间的权衡。

比较两个主方案，Q4-3 比 Q4-2 净节省 84,392.08 元，降幅为 0.58%，紧急购电量减少 57,887.57 kWh。其合同购电与调整费用增加 307,813.56 元，紧急购电费却减少 392,205.64 元，后者超过前者形成总购电费用优势。该比较对应预测来源、裕量分组和合同调整规则不同的两套完整策略。

8.3.2 固定电价与波动电价的比较

以第六、七章固定分时电价下的主方案为参照，分别计算更换电价后的各项费用变化，结果见表 8-2。其中，“合同购电与调整费用” 在 Q4-2 中仅包含日初合同购电费，在 Q4-3 中包含日初合同购电费及累计调整净费用。

表 8-2　更换电价条件后的费用变化

比较	合同购电与调整费用增加（元）	紧急购电费增加（元）	总购电费用增加（元）	总购电费用增幅
Q4-2 相对 Q2	603,025.12	44,700.81	647,725.93	4.65%
Q4-3 相对 Q3	619,091.27	34,302.64	653,393.91	4.72%

采用给定波动电价后，两类策略的合同购电支出与紧急购电支出均增加，总购电费用增幅分别为 4.65% 和 4.72%。将新电价代入优化目标后，合同购电与储能安排重新确定，因此费用变化同时反映价格与调度的调整。

以 9 月 23 日为例，固定分时电价下，问题二在 18:00—18:10 购入 786.65 kWh，在 20:00—20:10 不购电。波动电价在这两个时段分别为 1.3273 和 1.2687 元/kWh，Q4-2 相应购入 0 和 799.35 kWh。储能在前一时段放电 764.33 kWh 满足用电需求，在后一时段仅放电 27.83 kWh。可见，购电与充放电安排会随当天各时段的相对电价调整。

以 3 月 20 日为例，比较两类策略在固定分时电价和波动电价下的全天运行结果，如图 13 所示。左右两列分别对应固定合同与滚动调整策略，由上至下展示电价、实际购电功率和储电量，虚线表示固定分时电价，实线表示波动电价。实际购电功率由合同交付电量与紧急购电量之和换算得到。

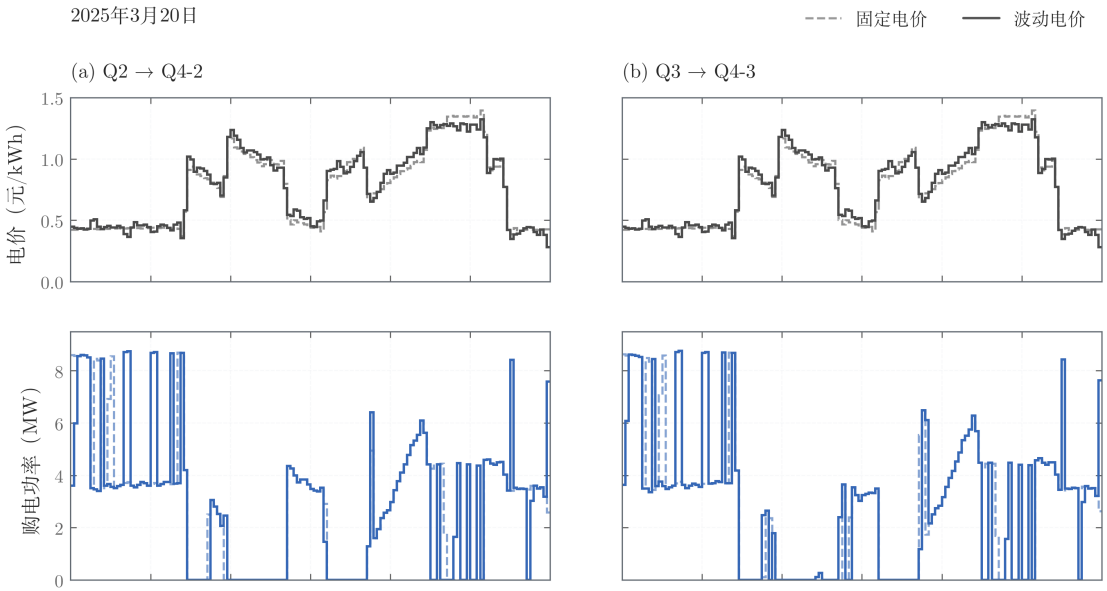
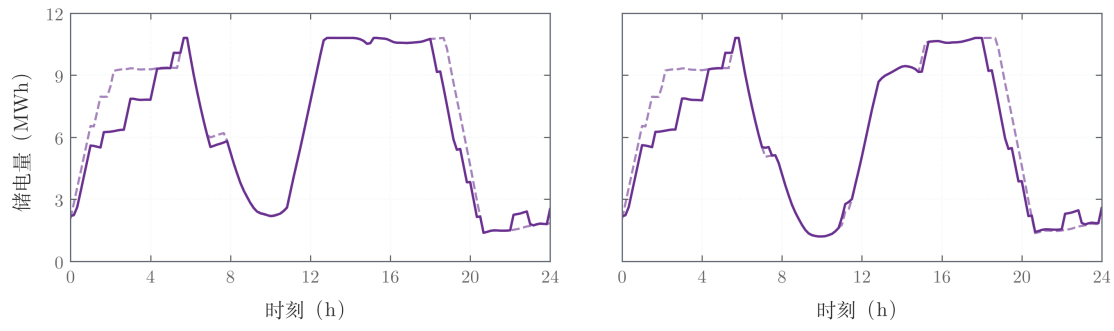


图 13　两类策略在固定与波动电价下的典型日调度对比（3 月 20 日）：电价与购电功率



续图 13　典型日调度对比：储电量。左列为固定合同，右列为滚动调整；虚线为固定电价，实线为波动电价

固定合同策略下，电价变化后夜间购电高峰的出现时点有所调整，储电量的上升过程也随之改变。两种电价条件下，午间储电量均恢复至较高水平，晚间开始下降的时点和下降幅度则不同，说明电价变化既影响购电费用，也改变日内购电和放电安排。

滚动调整策略下，午间部分购电时段随电价变化而调整，储电量达到低谷和高位的时段整体接近。两种电价条件使用同日负荷和光伏数据，日初储电量分别承接各自此前的运行结果。

上述曲线反映了更换电价后购电与储能运行的变化。在每种电价条件下，再比较固定合同策略与滚动调整策略，并将总购电费用差额分为合同购电费用变化、紧急购电费用变化及两者之和，如图 14 所示。

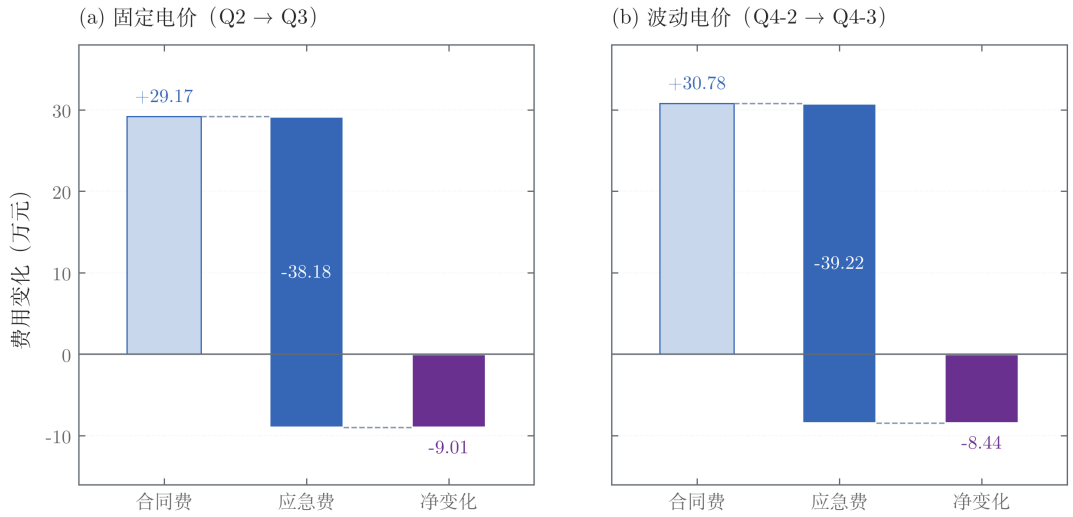


图 14　固定与波动电价下滚动策略的费用变化分解

图 14 显示，两类电价下，滚动调整策略节省的紧急购电费均超过新增合同购电与调整费用，分别净节省 9.01 万元和 8.44 万元。新增合同支出抵消了大部分补购费用节约，因此总购电费用降幅较小。紫色柱表示净费用变化，合同费包含日初合同购电费与累计调整净费用，“应急费”即紧急购电费。

8.3.3　指定日期的购电与储能运行

四个指定日期的合同购电与费用汇总见表 8-3。日初合同购电量和最终合同购电量均不含紧急购电，总购电费用则包含紧急购电支出。指定 10 分钟时段的购电量、各四小时时段的实际充放电量、首末储电量及连续紧急购电事件见附录 C。

表 8-3　指定日期的合同与实际运行汇总（电量：kWh；费用：元）

策略	日期	日初合同购电量	最终合同购电量	紧急购电量	总购电费用
Q4-2	3 月 20 日	69,244.28	69,244.28	0.00	42,829.49
Q4-2	6 月 21 日	37,756.89	37,756.89	0.00	19,283.62
Q4-2	9 月 23 日	68,767.82	68,767.82	573.54	47,542.24
Q4-2	12 月 21 日	100,571.21	100,571.21	0.00	76,380.42
Q4-3	3 月 20 日	71,978.38	67,674.25	64.12	45,600.97
Q4-3	6 月 21 日	36,968.31	36,861.23	0.00	18,828.81
Q4-3	9 月 23 日	72,713.64	67,828.85	88.02	45,613.62
Q4-3	12 月 21 日	100,668.18	98,534.76	0.00	75,834.40

表 8-3 显示，两类策略的费用高低随日期变化：Q4-3 在 3 月 20 日的费用高于 Q4-2，在其余三个指定日则较低。Q4-3 四个指定日的最终合同购电量均低于日初约定量；9 月 23 日的紧急购电量为 88.02 kWh，也低于 Q4-2 的 573.54 kWh。

从储能运行看，6 月 21 日 Q4-2 和 Q4-3 在 08:00—12:00 分别充入 8,423.93 和 8,501.70 kWh，在 16:00—20:00 分别放出 4,967.66 和 5,117.26 kWh，当天均未发生紧急购电。9 月 23 日 20:30—20:40，Q4-2 受剩余储电量限制，仅能放出 366.24 kWh，需要补购 418.66 kWh；Q4-3 可在该时段放出 784.90 kWh，完全覆盖净负荷，无需补购。两者此前的合同安排和实际充放电共同决定了当期可用储电量，进而影响紧急购电需求。

8.4 模型检验

8.4.1 固定电价恢复检验

为检查更换电价后是否仍沿用原有策略，将问题四模型中的价格矩阵恢复为问题二、三采用的固定分时电价。两类策略分别得到与问题二、三一致的总购电费用，绝对差异均小于 10^{-8} 元，合同电量、紧急购电量及评价期首末储电量也在预设容差内一致，说明本问计算保持了前述策略的模型结构。

8.4.2 相同初始储电量检验

主方案与零裕量方案经过 1 月运行后，在 2 月 1 日的储电量有所不同。为比较相同初始储电量下的风险裕量效果，分别以每类策略主方案的 2 月 1 日储电量为起点，重新运行对应的主方案与零裕量方案。历史误差校准、价格、预测和储能控制规则均保持不变，对照结果见表 8-4。

表 8-4　共同期初储电量下的主方案与零裕量方案比较

策略	共同期初储电量（kWh）	主方案总购电费用（元）	基准方案总购电费用（元）	节省金额（元）	节省率
Q4-2	3,941.58	14,571,629.31	16,125,862.38	1,554,233.06	9.64%
Q4-3	3,956.00	14,487,237.24	14,630,234.74	142,997.50	0.98%

统一各类策略内部的初始储电量后，两类主方案的费用降幅分别为 9.64% 和 0.98%，与原连续运行结果接近。这说明，在所测试的起点下，费用改善并非仅由 1 月末储电量差异造成。

8.4.3 运行与重复计算检验

对两类主方案各 48,096 个时段的运行记录进行核验，实际电量平衡与储电量递推的最大残差均小于 10^{-9} kWh，容量、功率及充放电互斥约束均满足，各日初始储电量与前一日实际日末储电量一致。按时间顺序重算 Q4-3 的 72,144 条合同调整记录，所得合同量与原记录相符，累计调整费用及总购电费用的复算差异均小于 10^{-8} 元。

对 Q4-3 进行 35 天重复运行，总购电费用、日初合同购电量、最终合同购电量、紧急购电量和期末储电量五项指标均一致，两次运行均通过物理约束检查。结合第 8.4.1 节的固定电价恢复结果，可见两类策略在本次价格替换及重复计算中保持了模型与结果的一致性。

九、模型评价、改进与推广

9.1 模型评价

9.1.1 模型的优点

（1）购电规划与实际运行相衔接。将合同决策与储能实时控制分开处理，利用预测确定采购量，再按实际供需调节充放电。各日传递实际储电量，使预测偏差对当前运行及后续购电的影响连续体现。

（2）按目标优先级求解，保留线性结构。两级线性规划先优化费用目标，再在给定容差内减少充放电总量。在允许无成本弃置富余电量的条件下，利用互斥性分析消除同时充放电，无需增加充放电状态整数变量。

（3）结算反映每次合同调整的成本。以调整前后的约定电量确定增购与减购，分别累计支出和退款，使同一时段多次改约的费用得到保留，并明确区分合同交付量、紧急购电量及总购电费用。

（4）对照分析区分预测精度与调度收益。从分月费用、预报来源、风险参数、设备参数及储能控制方式开展比较，并通过相同初始储电量检验识别起点差异的影响。对照结果既展示节费效果，也保留部分月份或方案费用更高的情形，便于判断改善的来源和适用条件。

9.1.2 模型的不足

（1）风险参数尚未随费用条件动态选择。裕量由历史误差更新，但分位数水平保持固定，未与当日电价和储电量联合确定。主方案采用相同参数便于跨问比较，却不等同于各问中费用最低的参数选择。

（2）跨日价值与实时控制仍采用近似处理。日末价值项概括了次日用电收益，实际充放电优先弥补当期缺口，尚未在多日范围内联合优化购电与储能控制。现有储能预留对照仅考察当日剩余时段，不能充分反映更长周期的供需变化。

（3）波动电价结果依赖价格已知条件。问题四在规划时使用当天完整电价，未考虑电价预测误差。当未来价格只能预测时，采购时序和储能收益可能变化，现有费用结果需在相应信息条件下重新评价。

（4）设备成本采用简化处理。充放电效率和可用容量在主方案中保持不变，目标未计入电池循环衰减、寿命消耗和新增设备投资。因此，设备参数敏感性反映的是运行费用变化，尚不能直接作为扩容或设备选型的经济依据。

9.2 模型的改进

针对风险参数，可在逐日推进的历史验证窗口内比较不同分位数，以实际购电费用为主要标准，并结合紧急购电需求评价所选参数，再在后续日期检验效果。

针对跨日价值与实时控制，可在现有日内储能预留对照的基础上延长规划范围，将后续日期的可用预测纳入购电和留电决策。由相同初始储电量比较费用与补购变化，评价多日协调的作用。

针对未来电价未知的情形，可保留合同调整和逐次结算规则，以决策时点可获得的信息预测电价，再按实际交付价格计算费用，比较价格预测误差对采购和充放电安排的影响。

针对设备成本，可将循环充放电引起的容量衰减和寿命成本纳入费用评价；在比较扩容或提高功率的方案时，同时考虑新增投资，以区别短期运行节费与长期经济收益。

9.3 模型的推广

该框架可用于研究具备光伏、储能和外网购电条件的园区、校园等微网。应用于其他系统时，需重新确定储能容量、功率和效率，核实预报发布时间、计划制定时可获得的电价信息及合同调整规则，并利用当地历史数据校准预测与风险裕量。若允许余电上网或存在购电容量上限，还需修改相应交易变量和约束，重新检查两级规划的适用条件。

综上，在本题数据和既定结算条件下，储能能够将部分高价购电需求转移至低价时段，日内合同调整能够根据新预报修正后续采购，二者均体现出降低费用的作用。增加正常采购可减少高价补购，但其收益受到充放电损耗、调整费用及储电量限制。微网调度应同时考虑购电时机和实际供需偏差，以总购电费用及紧急购电需求评价策略，而非单独追求更大的采购余量。

AI 使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于简要用途和技术支持，如语言润色、代码调试、文献检索、网页搭建等，详细使用情况见支撑材料。

参考文献

[1] 徐婉迪, 魏来, 罗俊, 等. 储能产业发展带来的能源革命及其关键管理科学问题 [J]. 系统管理学报, 2021, 30(1): 191-197. DOI: 10.3969/j.issn.1005-2542.2021.01.018.

[2] 殷辉, 陆信辉, 周开乐. 分布式与集中式储能并存的微电网负荷优化调度 [J]. 中国管理科学, 2023, 31(2): 118-128. DOI: 10.16381/j.cnki.issn1003-207x.2020.0762.

[3] 崔维伟, 刘新波. 多源供电模式下企业生产运作与能源管理集成优化研究 [J]. 中国管理科学, 2023, 31(8): 173-183. DOI: 10.16381/j.cnki.issn1003-207x.2020.1176.

[4] MARLER R T, ARORA J S. Survey of multi-objective optimization methods for engineering[J]. Structural and Multidisciplinary Optimization, 2004, 26(6): 369-395. DOI: 10.1007/s00158-003-0368-6.

[5] HUANGFU Q, HALL J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119-142. DOI: 10.1007/s12532-017-0130-5.

[6] BACHER P, MADSEN H, NIELSEN H A. Online short-term solar power forecasting[J]. Solar Energy, 2009, 83(10): 1772-1783. DOI: 10.1016/j.solener.2009.05.016.

[7] HONG T, FAN S. Probabilistic electric load forecasting: A tutorial review[J]. International Journal of Forecasting, 2016, 32(3): 914-938. DOI: 10.1016/j.ijforecast.2015.11.011.

[8] 何耀耀, 秦杨, 杨善林. 基于 LASSO 分位数回归的中期电力负荷概率密度预测方法 [J]. 系统工程理论与实践, 2019, 39(7): 1845-1854. DOI: 10.12011/1000-6788-2017-2168-10.

[9] PINSON P, CHEVALLIER C, KARINIOTAKIS G. Trading wind generation from short-term probabilistic forecasts of wind power[J]. IEEE Transactions on Power Systems, 2007, 22(3): 1148-1156. DOI: 10.1109/TPWRS.2007.901117.

[10] PARISIO A, RIKOS E, GLIELMO L. A model predictive control approach to microgrid operation optimization[J]. IEEE Transactions on Control Systems Technology, 2014, 22(5): 1813-1827. DOI: 10.1109/TCST.2013.2295737.

附录 A　文件列表

表 A-1 列出随包提供的文件。未标注目录的 Python 模块位于 `code/`，原始附件位于 `data/`，对照表、检验结果及主方案逐时段记录位于 `comparison/`；五份结果工作簿、统一入口、启动脚本、AI 工具使用说明与交互网页位于根目录。

补充实验随包提供逐日及分月汇总、比较指标、实验设置和原实验核验结果。补充实验逐时段记录与预测、实测及裕量数组由附录 D.1 所述命令重新生成并核验。

表 A-1　辅助材料文件列表

文件名称	功能描述
result1.xlsx	问题一典型日调度结果。
result2.xlsx	问题二固定合同调度结果。
result3.xlsx	问题三滚动调整调度结果。
result4-2.xlsx	波动电价下固定合同调度结果。
result4-3.xlsx	波动电价下滚动调整调度结果。
run.py、start_windows.bat、start_mac.command	统一运行入口及 Windows、Mac 启动程序。
run_models.py	组织四问计算与结果输出。
q1_main.py、q1_common.py、q1_baseline.py	求解问题一及无储能基准。
q2_full_feedback.py、q2_common.py、q2_forecast_experiment.py	实现问题二预测、购电与储能调节。
q3_model.py、q4_model.py	求解问题三、四及相应对照方案。
export_results.py、verify_results.py	导出结果并核对电量、储能状态及费用。
verify_inheritance.py、run_common_soc_robustness.py	执行电价替换一致性与相同初始储电量检验。
附件 1.xlsx、附件 2.xlsx、附件 3.xlsx、附件 4.xlsx	模型所用原始数据。
frequency_comparison.csv、mechanism_comparison.csv、sensitivity_comparison.csv	预报时点、机制与参数对照结果。
inheritance_regression.json、q4_robustness_summary.json	问题四一致性与稳健性检验结果。
q3_adjustment_events.csv.gz、q43_main_adjustment_events.csv.gz	合同调整电量与费用记录。
AI 工具使用详情.pdf	AI 工具使用、输出采纳及人工核验说明。
CUMCM_lab.html	微网调度方案辅助分析交互网页。
run_supplement.py	组织补充实验计算与核验。
q2_main_interval_ledger.csv.gz、q3_main_interval_ledger.csv.gz	主方案逐时段记录。
experiments/forecast/run_validation.py、experiments/forecast/verify_validation.py	预测基准、裕量覆盖与日末估值检验。
experiments/storage/run_experiment.py、experiments/storage/verify_experiment.py	给定合同下的储能预留对照与核验。
experiments/forecast/results/forecast_comparison.csv	预测误差与调度费用。
experiments/forecast/results/gamma_comparison.csv	日末估值参数比较。
experiments/forecast/results/coverage.csv	风险裕量覆盖统计。
experiments/forecast/results/q2_adopted_daily.csv	指数加权预测的逐日汇总。
experiments/forecast/results/q2_seasonal_persistence_daily.csv	同期延续预测的逐日汇总。
experiments/forecast/results/q2_equal_window_mean_daily.csv	等权均值预测的逐日汇总。
experiments/forecast/results/q2_gamma_0.0_daily.csv	问题二 γ=0 的逐日汇总。
experiments/forecast/results/q2_gamma_0.5_daily.csv	问题二 γ=0.5 的逐日汇总。
experiments/forecast/results/q2_gamma_1.0_daily.csv	问题二 γ=1 的逐日汇总。
experiments/forecast/results/q3_gamma_0.0_daily.csv	问题三 γ=0 的逐日汇总。
experiments/forecast/results/q3_gamma_0.5_daily.csv	问题三 γ=0.5 的逐日汇总。
experiments/forecast/results/q3_gamma_1.0_daily.csv	问题三 γ=1 的逐日汇总。
experiments/forecast/results/protocol.json、experiments/forecast/results/summary.json	实验设置与汇总指标。
experiments/forecast/results/engine_verification.json、experiments/forecast/results/independent_verification.json	模型约束与独立核验结果。
experiments/storage/full_run/comparison.csv	储能控制对照结果。
experiments/storage/full_run/greedy_replay_daily.csv	原实时反馈的逐日汇总。

续表 A-1

文件名称	功能描述
experiments/storage/full_run/price_aware_mpc_daily.csv	储能预留的逐日汇总。
experiments/storage/full_run/paired_daily.csv、experiments/storage/full_run/monthly.csv	逐日、分月费用差异。
experiments/storage/full_run/protocol.json、experiments/storage/full_run/summary.json	对照设置与汇总指标。
experiments/storage/full_run/independent_verification.json	运行约束与费用核验结果。

附录 B　微网调度方案辅助分析交互网页

微网调度方案辅助分析交互网页的界面见图 15。



图 15　微网调度方案辅助分析的交互网页截图

网页将典型日调度、日初购电、滚动调整与波动电价比较集中展示。选择策略和日期后，可查看净负荷、合同购电量、充放电及储电量变化，并比较当日购电费用、外网购电量和紧急购电量。

页面设置策略比较、风险参数和结果追溯等分析入口，支持明暗主题切换与逐时段明细导出，为调度结果查看和方案辅助分析提供交互界面。

附录 C　问题四指定日期的运行结果

本附录列出 Q4-2 与 Q4-3 在四个指定日期的合同购电、实际充放电及紧急购电结果。

C.1　固定合同策略（Q4-2）

C.1.1　合同购电量及费用

指定时段按最终合同购电量列示，电量单位为 kWh，费用单位为元。

Q4-2 的指定时段合同购电量与全天费用见表 C-1。

表 C-1　Q4-2 指定日期的合同购电与费用

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00—10:10	0.00	0.00	0.00	0.00
12:00—12:10	580.62	0.00	459.48	983.20
14:00—14:10	0.00	0.00	0.00	0.00
16:00—16:10	496.08	248.77	518.12	834.89
18:00—18:10	0.00	527.31	0.00	740.27
20:00—20:10	0.00	0.00	799.35	0.00
日初合同购电量	69,244.28	37,756.89	68,767.82	100,571.21
全天最终合同购电量	69,244.28	37,756.89	68,767.82	100,571.21
全天紧急购电量	0.00	0.00	573.54	0.00
全天总购电量	**69,244.28**	**37,756.89**	**69,341.36**	**100,571.21**
全天日初合同购电费	42,829.49	19,283.62	43,867.47	76,380.42
累计调整净费用	0.00	0.00	0.00	0.00
全天紧急购电费	0.00	0.00	3,674.77	0.00
全天总购电费用	**42,829.49**	**19,283.62**	**47,542.24**	**76,380.42**

C.1.2　实际充放电与首末储电量

各四小时时段按“充电量 / 放电量”列示，分别累计实际运行电量，单位为 kWh。

Q4-2 的四小时充放电量及首末储电量见表 C-2。

表 C-2　Q4-2 指定日期的实际充放电与首末储电量

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
00:00—04:00	6,423.39 / 139.18	1,715.76 / 2,902.39	6,153.68 / 64.25	8,338.74 / 82.69
04:00—08:00	3,646.43 / 5,242.64	3,255.69 / 1,719.95	4,486.18 / 5,999.41	1,210.86 / 801.78
08:00—12:00	6,154.58 / 2,777.38	8,423.93 / 0.00	5,070.54 / 1,896.62	6,043.49 / 6,662.51
12:00—16:00	3,733.52 / 369.62	0.00 / 0.00	5,060.81 / 616.72	5,912.55 / 2,220.11
16:00—20:00	299.00 / 6,400.43	120.55 / 4,967.66	128.20 / 7,206.26	0.00 / 5,178.44
20:00—24:00	2,115.50 / 2,868.77	2,422.88 / 3,091.92	500.41 / 1,327.24	1,372.45 / 3,156.26
00:00 储电量	2,186.68	3,880.09	1,338.15	2,297.25
24:00 储电量	2,546.28	4,134.01	1,586.33	2,774.43

C.1.3　紧急购电事件

Q4-2 在四个指定日期的紧急购电事件见表 C-3。

表 C-3　Q4-2 指定日期的紧急购电

日期	紧急购电时段	紧急购电量（kWh）
3 月 20 日	无	0.00
6 月 21 日	无	0.00
9 月 23 日	18:20—18:40	3.37
9 月 23 日	20:20—20:40	425.03
9 月 23 日	20:50—21:10	81.94
9 月 23 日	21:20—22:00	63.20
12 月 21 日	无	0.00

9 月 23 日紧急购电合计 573.54 kWh，紧急购电费 3,674.77 元。

C.2　滚动调整策略（Q4-3）

C.2.1　合同购电量及费用

指定时段按最终合同购电量列示，电量单位为 kWh，费用单位为元。

Q4-3 的指定时段合同购电量与全天费用见表 C-4。

表 C-4　Q4-3 指定日期的合同购电与费用

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00—10:10	0.00	0.00	0.00	0.00
12:00—12:10	539.45	0.00	389.98	943.92
14:00—14:10	0.00	0.00	0.00	0.00
16:00—16:10	527.90	196.33	537.86	867.17
18:00—18:10	0.00	507.75	0.00	737.52
20:00—20:10	0.00	0.00	804.81	0.00
日初合同购电量	71,978.38	36,968.31	72,713.64	100,668.18
全天最终合同购电量	67,674.25	36,861.23	67,828.85	98,534.76
全天紧急购电量	64.12	0.00	88.02	0.00
全天总购电量	**67,738.37**	**36,861.23**	**67,916.87**	**98,534.76**
全天日初合同购电费	44,882.82	18,761.53	46,614.12	76,268.80
累计调整净费用	394.87	67.27	-1,455.84	-434.40
全天紧急购电费	323.28	0.00	455.34	0.00
全天总购电费用	**45,600.97**	**18,828.81**	**45,613.62**	**75,834.40**

C.2.2　实际充放电与首末储电量

各四小时时段按“充电量 / 放电量”列示，分别累计实际运行电量，单位为 kWh。

Q4-3 的四小时充放电量及首末储电量见表 C-5。

表 C-5　Q4-3 指定日期的实际充放电与首末储电量

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
00:00—04:00	6,437.99 / 167.39	1,787.53 / 2,710.67	6,171.85 / 62.38	8,426.82 / 63.02
04:00—08:00	3,414.23 / 5,977.59	3,103.80 / 1,719.95	4,470.51 / 6,565.10	1,195.20 / 815.53
08:00—12:00	4,304.48 / 2,713.26	8,501.70 / 0.00	5,310.78 / 1,896.62	5,511.37 / 7,233.81
12:00—16:00	6,508.79 / 290.08	0.00 / 0.00	5,548.62 / 461.71	7,270.46 / 2,303.94
16:00—20:00	356.49 / 6,353.17	115.25 / 5,117.26	199.56 / 7,021.01	0.00 / 5,179.98
20:00—24:00	2,133.69 / 2,880.40	2,410.27 / 3,091.92	533.68 / 1,796.22	1,385.80 / 3,199.53
00:00 储电量	2,175.49	3,669.18	1,374.07	2,210.20
24:00 储电量	2,591.27	3,951.66	1,604.40	2,736.65

C.2.3　紧急购电事件

Q4-3 在四个指定日期的紧急购电事件见表 C-6。

表 C-6　Q4-3 指定日期的紧急购电

日期	紧急购电时段	紧急购电量（kWh）
3 月 20 日	09:40—10:00	64.12
6 月 21 日	无	0.00
9 月 23 日	18:20—18:40	3.13
9 月 23 日	20:20—20:30	6.38
9 月 23 日	20:50—21:10	36.38
9 月 23 日	21:20—21:40	41.81
9 月 23 日	21:50—22:00	0.32
12 月 21 日	无	0.00

3 月 20 日紧急购电合计 64.12 kWh，紧急购电费 323.28 元。

9 月 23 日紧急购电合计 88.02 kWh，紧急购电费 455.34 元。

附录 D　代码

本附录完整列出四问求解、对照检验、结果导出与核对所用的源程序。各代码块按标题标注的相对路径分别保存，与附录 A 的原始输入、结果工作簿及对照数据配套使用。

D.1　运行方式与软件调用

程序使用 64 位 Python 3.11—3.13，依赖版本及安装步骤已写入统一入口 `run.py`。保留辅助材料根目录以及 `code/`、`data/`、`comparison/`、`experiments/`的文件位置，在根目录执行下列命令。重新计算的文件写入 `rerun/`下的新目录，根目录的五份结果工作簿保留。

表 D-1　程序运行命令

操作	Windows 命令	macOS 命令
启动交互菜单	`.\start_windows.bat`	`bash start_mac.command`
安装计算依赖	`.\start_windows.bat --setup`	`bash start_mac.command --setup`
核对现有结果	`.\start_windows.bat --task verify`	`bash start_mac.command --task verify`
重算四问主方案及基准	`.\start_windows.bat --task all`	`bash start_mac.command --task all`
重算并执行完整对照检验	`.\start_windows.bat --task full`	`bash start_mac.command --task full`

两项补充实验共用上述计算环境。安装依赖后，执行 `python run.py --task supplement` 重新计算并独立核验；也可通过 Windows 或 macOS 启动脚本传入相同参数。程序在 `rerun/`下新建运行目录，其中 `forecast/`保存预测实验的逐时段记录及预测证据数组，`storage/`保存储能实验的逐时段记录，两项核验报告写入该运行目录。

随包提供补充实验汇总和原实验核验结果，不包含上述补充实验明细；`--task supplement_verify` 会提示使用重新计算与核验命令，不直接核验随包补充实验明细。五份结果工作簿的核对仍使用 `python run.py --task verify`。

首次安装计算依赖需要联网；五份结果工作簿的核对仅使用 Python 标准库。Excel 数据读取、工作簿生成和逐格核对均由下列 Python 程序完成，不依赖 Excel 或 SPSS 交互式命令；相关完整代码列于 D.4—D.8。

D.2　统一运行与系统启动程序

D.2.1　`run.py`

```
#!/usr/bin/env python3
""" 微网购电与储能调度: 跨平台统一入口 (Python 3.11—3.13, 64 位)。

在 Windows、macOS 中均先解压整个文件夹，并保留内部相对位置。
Windows 可双击 start_windows.bat; macOS 可运行 bash start_mac.command。

命令行:
  python run.py --task verify    核对随包提交的五份结果; 无需第三方依赖。
  python run.py --setup          在包内 .venv 安装计算依赖; 首次需要网络。
  python run.py --task all       重算四问主方案及对应基准，导出五份 Excel。
  python run.py --task full      在 all 基础上复现预报时点、参数、机制和 Q4 检验。
  python run.py --task q2        只运行指定问题 (支持 q1/q2/q3/q4)。

运行提示和依赖固定在本程序中，不另提供 requirements.txt 或 README。
根目录的结果文件作为提交值和格式基准保留，本次计算仅写入新建输出目录。
"""
from __future__ import annotations
import argparse
import importlib.metadata
import os
import platform
from pathlib import Path
import subprocess
import sys
```

```
from datetime import datetime

ROOT=Path(__file__).resolve().parent
# 此组合已用于本包端到端重算。只安装公开 Python 依赖，不使用 Node.js、Office 或专有导表包。
PACKAGES=("numpy==2.3.5","scipy==1.17.0","pandas==2.2.3","openpyxl==3.1.5")


def console_setup():
    for stream in (sys.stdout,sys.stderr):
        if hasattr(stream,'reconfigure'):
            stream.reconfigure(encoding='utf-8',errors='replace')
    sys.dont_write_bytecode=True
    os.environ.setdefault('PYTHONUTF8','1')
    os.environ['PYTHONDONTWRITEBYTECODE']='1'
    # 限制数值库线程，避免不同机器默认线程数量造成不必要的并行开销。
    for name in ('OMP_NUM_THREADS','OPENBLAS_NUM_THREADS','MKL_NUM_THREADS'):
        os.environ[name]='1'


def environment_python():
    return ROOT/'.venv'/('Scripts/python.exe' if os.name=='nt' else 'bin/python')


def setup_environment():
    import venv
    if not ((3,11)<=sys.version_info[:2]<=(3,13)) or sys.maxsize<=2**32:
        raise RuntimeError('请安装 64 位 Python 3.11、3.12 或 3.13（建议 3.12），再启动此程序。')
    exe=environment_python()
    if not exe.exists():
        print('正在本文件夹创建独立 Python 环境 .venv，不修改系统环境...',flush=True)
        venv.EnvBuilder(with_pip=True,clear=False).create(ROOT/'.venv')
    print('正在安装固定版本依赖；需要可连接 Python 软件源的网络...',flush=True)
    try:
        subprocess.run([str(exe),'-m','pip','install','--disable-pip-version-check','--only-binary=:all:',*PACKAG
          ES],check=True)
    except subprocess.CalledProcessError as error:
        raise RuntimeError('依赖安装未完成，请确认网络和Python软件源可用后重新执行 --setup。
          已提交的结果文件不受影响。') from error
    print('独立环境已准备完成。',flush=True)
    return exe


def has_dependencies(executable=None):
    if executable is not None:
        code='import importlib.metadata as m; '+ '; '.join(f"assert
          m.version({s.split('==')[0]!r})=={s.split('==')[1]!r}" for s in PACKAGES)
        return subprocess.run([str(executable),'-c',code],stdout=subprocess.DEVNULL,stderr=subprocess.DEVNULL).re
          turncode==0
    try:
        return all(importlib.metadata.version(s.split('==')[0])==s.split('==')[1] for s in PACKAGES)
    except importlib.metadata.PackageNotFoundError:
        return False


def ensure_runtime(task,output):
    """Return None for current interpreter, otherwise invoke the isolated interpreter."""
    if has_dependencies():return None
    exe=environment_python()
    if not (exe.exists() and has_dependencies(exe)):
        if not sys.stdin.isatty():
            raise RuntimeError('计算依赖尚未准备，请先运行 python run.py --setup，再执行本次命令。')
        choice=input('缺少所需版本的计算依赖。是否在本文件夹创建独立环境并联网安装？[y/N] ').strip().lower()
        if choice not in ('y','yes'):raise RuntimeError('未安装依赖。可先选“核对现有结果”，或以后执行 --setup。
          ')
        exe=setup_environment()
    if Path(sys.prefix).resolve()==(ROOT/'.venv').resolve():
        raise RuntimeError('独立环境依赖仍不完整，请运行 --setup 修复。')
```

```
    command=[str(exe),str(ROOT/'run.py'),'--task',task,'--worker']
    if output:command.extend(['--output',output])
    return subprocess.run(command,cwd=ROOT).returncode


def menu():
    print('\n 微网购电与储能调度 — 辅助材料运行入口')
    print('1  核对现有五份结果（不重新求解，不需安装依赖）')
    print('2  重算四问主方案及对应基准，生成五份 Excel')
    print('3  完整重算，同时运行参数、时点、机制和 Q4 对照检验')
    print('4  安装或修复包内独立计算环境（首次需联网）')
    print('5  核对两项补充实验的保存结果')
    print('6  重算两项补充实验并独立核验')
    print('0  退出')
    value=input('请选择 [0—6]：').strip()
    return {'0':'exit','1':'verify','2':'all','3':'full','4':'setup','5':'supplement_verify','6':'supplement'}.ge
      t(value,'invalid')


def main():
    console_setup()
    parser=argparse.ArgumentParser(description='五份结果核对、四问重算与 Excel 导出；不改写已提交结果。')
    parser.add_argument('--task',choices=('verify','q1','q2','q3','q4','all','full','supplement_verify','suppleme
      nt'))
    parser.add_argument('--setup',action='store_true',help='联网安装固定版本依赖至包内.venv')
    parser.add_argument('--output',help='新建输出目录；相对路径以本程序所在文件夹为起点')
    parser.add_argument('--worker',action='store_true',help=argparse.SUPPRESS)
    args=parser.parse_args()
    if args.setup:
        setup_environment();return 0
    task=args.task
    if task is None:
        if not sys.stdin.isatty():parser.print_help();return 0
        task=menu()
        if task=='exit':return 0
        if task=='invalid':raise RuntimeError('无效选项，请重新启动后选择 0—6。')
        if task=='setup':setup_environment();return 0
    sys.path.insert(0,str(ROOT/'code'))
    if task=='verify':
        from verify_results import verify_package
        result=verify_package(ROOT)
        print(f"\n核对通过：{result['numeric_checks']:,}项数值核对；最大绝对差异
          {result['maximum_absolute_difference']:.3g}。")
        return 0
    if not ((3,11)<=sys.version_info[:2]<=(3,13)) or sys.maxsize<=2**32:
        raise RuntimeError('重算需要 64 位 Python 3.11—3.13（建议 3.12）。')
    if task == 'supplement_verify':
        raise RuntimeError('本包提供补充实验汇总结果。逐时段数据的生成与核验请运行 python run.py --task
          supplement；五份结果核对请运行 python run.py --task verify。')
    runtime=ensure_runtime(task,args.output)
    if runtime is not None:return runtime
    output=Path(args.output).expanduser() if args.output else
      ROOT/'rerun'/datetime.now().strftime('%Y%m%d_%H%M%S_%f')
    if not output.is_absolute():output=ROOT/output
    output=output.resolve()
    if output==ROOT or output.exists():raise RuntimeError('输出目录必须为尚不存在的新目录，避免覆盖现有材料。')
    print('运行环境：',platform.system(),sys.version.split()[0],flush=True)
    print('依赖版本：',', '.join(PACKAGES),flush=True)
    print('本次计算输出：',output,flush=True)
    if task in ('supplement_verify','supplement'):
        from run_supplement import run
    else:
        from run_models import run
    run(task,output)
    return 0


```

```
if __name__=='__main__':
    try:
        raise SystemExit(main())
    except (OSError,ValueError,RuntimeError,subprocess.CalledProcessError) as error:
        print(f'\n 运行未完成: {error}',file=sys.stderr)
        print('已提交的原始结果未被覆盖。',file=sys.stderr)
        raise SystemExit(1)
    except KeyboardInterrupt:
        print('\n 运行已中止, 原始结果未被覆盖。',file=sys.stderr)
        raise SystemExit(130)
```

D.2.2 start_windows.bat

```
@echo off
setlocal
chcp 65001 >nul
set "PYTHONUTF8=1"
set "PYTHONDONTWRITEBYTECODE=1"
pushd "%~dp0"
if errorlevel 1 exit /b 1
if exist ".venv\Scripts\python.exe" (
  ".venv\Scripts\python.exe" "run.py" %*
  goto finish
)
where py >nul 2>nul
if not errorlevel 1 (
  py -3.12 -c "import sys;sys.exit(0 if sys.maxsize.bit_length()==63 else 1)" >nul 2>nul
  if not errorlevel 1 (
    py -3.12 "run.py" %*
    goto finish
  )
  py -3.11 -c "import sys;sys.exit(0 if sys.maxsize.bit_length()==63 else 1)" >nul 2>nul
  if not errorlevel 1 (
    py -3.11 "run.py" %*
    goto finish
  )
  py -3.13 -c "import sys;sys.exit(0 if sys.maxsize.bit_length()==63 else 1)" >nul 2>nul
  if not errorlevel 1 (
    py -3.13 "run.py" %*
    goto finish
  )
)
where python >nul 2>nul
if not errorlevel 1 (
  python "run.py" %*
  goto finish
)
echo Python 3.11-3.13 ^(64-bit^) is required. Install Python 3.12 and retry.
set "STATUS=1"
goto cleanup
:finish
set "STATUS=%ERRORLEVEL%"
:cleanup
popd
if "%~1"=="" pause
exit /b %STATUS%

```

D.2.3 start_mac.command

```
#!/bin/sh
# macOS: bash start_mac.command; or chmod +x start_mac.command and double-click.
cd "$(dirname "$0")" || exit 1
export PYTHONUTF8=1
export PYTHONDONTWRITEBYTECODE=1
```

```
PY=""
if [ -x ".venv/bin/python" ]; then
  PY=".venv/bin/python"
else
  for candidate in python3.12 python3.11 python3.13 python3; do
    if command -v "$candidate" >/dev/null 2>&1; then
      if "$candidate" -c 'import sys;sys.exit(0 if (3,11)<=sys.version_info[:2]<=(3,13) and sys.maxsize>2**32 else 1)' >/dev/null 2>&1; then
        PY="$candidate"
        break
      fi
    fi
  done
fi
if [ -z "$PY" ]; then
  echo "Python 3.11-3.13 (64-bit) is required. Install Python 3.12 and retry."
  exit 1
fi
"$PY" run.py "$@"
status=$?
if [ "$#" -eq 0 ] && [ -t 0 ]; then
  printf '\nPress Enter to close...'
  read answer
fi
exit "$status"
```

D.3 四问计算组织程序

D.3.1 code/run_models.py

```
""" 四问的正式计算入口：模型函数取自现有程序，统一输入、结果导出和核对。

Q2 明确采用 forecast(data, 'weekday')，不调用旧模块中的历史演示 main()。
每次计算写入独立目录；根目录的五份正式结果始终保持不变。
"""
from __future__ import annotations
import json
import time
from dataclasses import replace
from pathlib import Path

from export_results import export_q1,export_run,compare_workbooks

ROOT=Path(__file__).resolve().parents[1]


def save_json(path:Path,value):
    path.parent.mkdir(parents=True,exist_ok=True)
    path.write_text(json.dumps(value,ensure_ascii=False,indent=2),encoding='utf-8')


def save_frame(frame,path:Path):
    path.parent.mkdir(parents=True,exist_ok=True)
    frame.to_csv(path,index=False,encoding='utf-8-sig',float_format='%.10f')


def evaluation(frame):
    return frame[(frame['date']>='2025-02-01')&(frame['date']<='2025-12-31')].copy()


def check_run(run):
    valid=run.validation.get('hard_checks_pass',run.validation.get('all_pass',False))
    if not valid:
        raise RuntimeError(f'{run.config.strategy_id} 的模型约束检查未通过：{run.validation}')
    if run.metrics.get('fallback_event_count',run.metrics.get('fallback_day_count',0)):
```

```
        raise RuntimeError(f'{run.config.strategy_id} 触发求解回退，请检查求解环境；不将其作为正式重算结果')


def final_check(name,out,reports):
    result=compare_workbooks(ROOT/name,out/name)
    reports[name]=result
    print(f"  {name} 已导出；与既有结果最大逐格差异 {result['maximum_absolute_difference']:.3g}",flush=True)
    if result['status']!='PASS':
        save_json(out/'reproduction_check.json',reports)
        raise RuntimeError(f"{name}重算值与既有结果存在差异。新文件已单独保留；未覆盖正式结果。详情：
          {result['examples']}")


def run_q1(out,reports):
    import numpy as np
    import q1_common as c
    import q1_main
    import q1_baseline
    folder=out/'Q1';folder.mkdir(parents=True,exist_ok=True)
    data=c.load_q1_input();solved=q1_main.solve_main(data);main=solved.dispatch;base=q1_baseline.solve_baseline(d
      ata)
    previous=np.r_[c.SOC_INITIAL_KWH,main.soc_end[:-1]]
    balance=main.purchase+main.discharge-(data.load_kwh-data.pv_kwh)-main.charge-main.unused
    recurrence=main.soc_end-previous-.9*main.charge+main.discharge/.9
    maxima={'balance':float(np.max(abs(balance))),'storage':float(np.max(abs(recurrence))),
            'overlap':float(np.max(np.minimum(main.charge,main.discharge))),
            'soc_low':max(0.,float(1200-main.soc_end.min())),
            'soc_high':max(0.,float(main.soc_end.max()-10800)),
            'charge_limit':max(0.,float(main.charge.max()-5000/6)),
            'discharge_limit':max(0.,float(main.discharge.max()-5000/6)),
            'terminal':abs(float(main.soc_end[-1])-6000.)}
    if max(maxima.values())>1e-7:raise RuntimeError(f'Q1 约束检查失败：{maxima}')
    c.write_csv(folder/'q1_interval_ledger.csv',
        ['interval_index','interval_label','price_yuan_per_kwh','actual_load_kw','pv_forecast_kw','purchase_kwh',
          'charge_kwh','discharge_kwh','unused_kwh','soc_start_kwh','soc_end_kwh'],
        [[i+1,data.interval_labels[i],data.price[i],data.load_kw[i],data.pv_kw[i],main.purchase[i],main.charge[i]
          ,main.discharge[i],main.unused[i],previous[i],main.soc_end[i]] for i in range(144)])
    save_json(folder/'main_metrics.json',{'stage1_cost_yuan':solved.stage1_cost,'stage2_cost_yuan':solved.stage2_
      cost,
        'purchase_kwh':float(main.purchase.sum()),'charge_kwh':float(main.charge.sum()),'discharge_kwh':float(mai
          n.discharge.sum()),'checks':maxima})
    save_json(folder/'baseline_metrics.json',{'total_purchase_cost_yuan':base.cost,'purchase_kwh':float(base.purc
      hase.sum()),'unused_kwh':float(base.unused.sum())})
    export_q1(ROOT,out,data,main);final_check('result1.xlsx',out,reports)


def run_q2(out,reports):
    import q2_common as c
    import q2_full_feedback as engine
    from q2_forecast_experiment import forecast,audit
    folder=out/'Q2';data=c.load_q2_data()
    f=forecast(data,'weekday');margins=c.build_risk_margins(f,.8)
    settings=[('main',replace(engine.MAIN_CONFIG,strategy_id='M3_weekday_pv7_alpha08_gamma05',role='main')),
              ('baseline',replace(engine.BASELINE_CONFIG,strategy_id='B3_same_forecast_no_margin'))]
    for role,config in settings:
        print(f'Q2 {role}：计算 365 天（正式统计 334 天）...',flush=True)
        run=engine.run_full_feedback_strategy(data,f,margins,config);check_run(run)
        independent=audit(run.ledger,run.daily,data)
        if not independent['all_pass']:raise RuntimeError(f'Q2 独立检查失败：{independent}')
        save_frame(run.ledger,folder/f'q2_{role}_interval_ledger.csv')
        save_frame(run.daily,folder/f'q2_{role}_daily_summary.csv')
        save_json(folder/f'{role}_metrics.json',run.metrics)
        save_json(folder/f'{role}_validation.json',{'engine':run.validation,'independent':independent})
        print(f"  评价期费用 {run.metrics['actual_total_cost_yuan']:,.2f} 元",flush=True)
        if role=='main':
            export_run(ROOT,out,'result2.xlsx',run,c,is_q2=True)
            final_check('result2.xlsx',out,reports)
```

```
96
97
98 def run_q3(out,reports,full=False):
99     import q3_model as m
100     folder=out/'Q3';data=m.load_data();official=m.build_forecast_archive(data,'official')
101     archives={'official':official}
102     configs=[m.PolicyConfig('M1I_official_S3_alpha0.8_gamma0.5','main',collect_ledger=True),
103              m.PolicyConfig('B1I_official_S3_b0_gamma0.5','usable_baseline',alpha=None)]
104     if full:
105         archives['historical']=m.build_forecast_archive(data,'historical')
106         configs += [m.PolicyConfig('D_I_frequency_S0','diagnostic_frequency',selected_release_hours=(0,)),
107                     m.PolicyConfig('D_I_frequency_S1','diagnostic_frequency',selected_release_hours=(0,6)),
108                     m.PolicyConfig('D_I_frequency_S2','diagnostic_frequency',selected_release_hours=(0,6,12)),
109                     m.PolicyConfig('D_I_no_adjustment','diagnostic_mechanism',allow_adjustment=False),
110                     m.PolicyConfig('D_I_historical_pv','diagnostic_mechanism',forecast_source='historical'),
111                     m.PolicyConfig('D_I_fixed_plan_execution','diagnostic_mechanism',feedback_execution=False),
112                     m.PolicyConfig('S_I_alpha0.5','sensitivity',alpha=.5),
113                     m.PolicyConfig('S_I_alpha0.95','sensitivity',alpha=.95),
114                     m.PolicyConfig('S_I_gamma0','sensitivity',gamma=0.),
115                     m.PolicyConfig('S_I_gamma1','sensitivity',gamma=1.)]
116     cache={};runs=[]
117     for config in configs:
118         key=(config.forecast_source,config.alpha);archive=archives[config.forecast_source]
119         if key not in cache:cache[key]=m.build_risk_margins(archive,config.alpha)
120         print(f'Q3 {config.strategy_id}...',flush=True)
121         run=m.run_policy(data,archive,cache[key],config);check_run(run);runs.append(run)
122         print(f"  评价期费用 {run.metrics['actual_total_cost_yuan']:,.2f} 元",flush=True)
123         if config.role in ('main','usable_baseline'):
124             role='main' if config.role=='main' else 'baseline'
125             save_frame(evaluation(run.daily),folder/f'q3_{role}_daily_summary.csv')
126             save_json(folder/f'{role}_metrics.json',run.metrics);save_json(folder/f'{role}_validation.json',run.v
                alidation)
127         if config.role=='main':
128             save_frame(evaluation(run.ledger),folder/'q3_main_interval_ledger.csv')
129             save_frame(evaluation(run.adjustments),folder/'q3_adjustment_events.csv')
130             export_run(ROOT,out,'result3.xlsx',run,m,rolling=True)
131             final_check('result3.xlsx',out,reports)
132     comparison=m._comparison_frame(runs)
133     save_frame(comparison,folder/'strategy_comparison.csv')
134     if full:
135         for role,name in [('diagnostic_frequency','frequency_comparison.csv'),('diagnostic_mechanism','mechanism_
            comparison.csv'),('sensitivity','sensitivity_comparison.csv')]:
136             part=comparison[comparison['role']==role]
137             save_frame(part,folder/name)
138             _compare_frame(ROOT/'comparison'/name,part,reports)
139
140
141 def _compare_frame(reference,computed,reports):
142     import pandas as pd
143     old=pd.read_csv(reference).set_index('strategy_id');new=computed.set_index('strategy_id')
144     columns=['actual_total_cost_yuan','emergency_purchase_kwh']
145     maximum=0.
146     for idx in old.index:
147         if idx not in new.index:raise RuntimeError(f'缺少对照策略 {idx}')
148         for col in columns:
149             delta=abs(float(old.loc[idx,col])-float(new.loc[idx,col]));maximum=max(maximum,delta)
150             if delta>5e-5:raise RuntimeError(f'{reference.name}/{idx}/{col} 与存档结果不一致')
151     reports[reference.name]={'status':'PASS','maximum_absolute_difference':maximum}
152
153
154 def run_q4(out,reports):
155     import q4_model as m
156     folder=out/'Q4';data=m.load_data();archives={k:m.build_forecast_archive(data,k) for k in
        ('historical','official')}
157     for config in m._configurations('core'):
158         archive=archives[config.forecast_source]
159         margins=m.build_inherited_risk_margins(archive,config.question_variant,config.alpha)
160         print(f'Q4 {config.strategy_id}...',flush=True)
```

```
        run=m.run_policy(data,archive,config,margins=margins);check_run(run)
        prefix=config.question_variant.lower().replace('-','')+('_main' if config.role=='main' else '_baseline')
        save_frame(evaluation(run.daily),folder/f'{prefix}_daily_summary.csv')
        save_json(folder/'metrics'/f'{prefix}_metrics.json',run.metrics)
        save_json(folder/'metrics'/f'{prefix}_validation.json',run.validation)
        print(f"  评价期费用 {run.metrics['actual_total_cost_yuan']:,.2f} 元",flush=True)
        if config.role=='main':
            save_frame(evaluation(run.ledger),folder/f'{prefix}_interval_ledger.csv')
            save_frame(evaluation(run.adjustments),folder/f'{prefix}_adjustment_events.csv')
            name='result4-2.xlsx' if config.question_variant=='Q4-2' else 'result4-3.xlsx'
            export_run(ROOT,out,name,run,m,rolling=config.allow_adjustment)
            final_check(name,out,reports)


def run_extra_q4(out,reports):
    import verify_inheritance as inheritance
    import run_common_soc_robustness as paired
    inheritance.OUTPUT_PATH=out/'Q4'/'inheritance_regression.json'
    print('Q4 恢复固定电价的一致性检验...',flush=True);inheritance.main()
    paired.ROUND_DIR=out/'Q4';paired.OUTPUT_PATH=out/'Q4'/'q4_robustness_summary.json'
    print('Q4 各分支内部相同初始储电量检验...',flush=True);paired.main()
    old=json.loads((ROOT/'comparison'/'q4_robustness_summary.json').read_text('utf-8'))
    new=json.loads(paired.OUTPUT_PATH.read_text('utf-8'));maximum=0.
    for variant in ('Q4-2','Q4-3'):
        for role in ('main','baseline'):
            for field in ('actual_total_cost_yuan','emergency_purchase_kwh'):
                delta=abs(old['variants'][variant][role][field]-new['variants'][variant][role][field]);maximum=ma
                  x(maximum,delta)
                if delta>5e-5:raise RuntimeError(f'{variant}/{role}/{field} 共同起点结果不一致')
    if new['status']!='PASS':raise RuntimeError('共同初始状态检验未通过')
    reports['q4_robustness_summary.json']={'status':'PASS','maximum_absolute_difference':maximum}
    reports['inheritance_regression.json']=json.loads(inheritance.OUTPUT_PATH.read_text('utf-8'))


def run(task:str,out:Path):
    out.mkdir(parents=True,exist_ok=False)
    reports={};start=time.time()
    tasks=['q1','q2','q3','q4'] if task in ('all','full') else [task]
    try:
        for key in tasks:
            print('\n'+key.upper()+' 开始计算',flush=True)
            if key=='q1':run_q1(out,reports)
            elif key=='q2':run_q2(out,reports)
            elif key=='q3':run_q3(out,reports,full=(task=='full'))
            elif key=='q4':run_q4(out,reports)
        if task=='full':run_extra_q4(out,reports)
    finally:
        save_json(out/'reproduction_check.json',{'task':task,'runtime_seconds':time.time()-start,'checks':reports
          })
    print(f'\n 全部所选任务完成；本次输出：{out}',flush=True)
    return reports

```

D.4 问题一程序

D.4.1 code/q1_common.py

```
from __future__ import annotations

import csv
import hashlib
import json
import math
import sys
from dataclasses import dataclass
from datetime import time
```

```
from pathlib import Path
from typing import Any

PROJECT_ROOT = Path(__file__).resolve().parents[1]

import numpy as np
from openpyxl import load_workbook

N_INTERVALS = 144
DELTA_HOURS = 1.0 / 6.0
ETA_CHARGE = 0.9
ETA_DISCHARGE = 0.9
SOC_INITIAL_KWH = 6000.0
SOC_MIN_KWH = 1200.0
SOC_MAX_KWH = 10800.0
POWER_LIMIT_KW = 5000.0
INTERVAL_LIMIT_KWH = POWER_LIMIT_KW * DELTA_HOURS
COST_TOLERANCE_YUAN = 1e-6
CHECK_TOLERANCE = 1e-7

SOURCE_PATH = PROJECT_ROOT / "data" / " 附件 1.xlsx"
TEMPLATE_PATH = PROJECT_ROOT / "result1.xlsx"
ROUND_DIR = PROJECT_ROOT / "results" / "Q1" / "experiments" / "round1"
TABLE_DIR = ROUND_DIR / "tables"
METRIC_DIR = ROUND_DIR / "metrics"


@dataclass(frozen=True)
class Q1Input:
    source_labels: list[str]
    interval_starts: list[str]
    interval_ends: list[str]
    interval_labels: list[str]
    price: np.ndarray
    load_kw: np.ndarray
    pv_kw: np.ndarray
    load_kwh: np.ndarray
    pv_kwh: np.ndarray
    source_sha256: str


@dataclass(frozen=True)
class Dispatch:
    purchase: np.ndarray
    charge: np.ndarray
    discharge: np.ndarray
    unused: np.ndarray
    soc_end: np.ndarray
    cost: float


def _format_clock(total_minutes: int) -> str:
    if total_minutes == 24 * 60:
        return "24:00"
    hour, minute = divmod(total_minutes, 60)
    return f"{hour:02d}:{minute:02d}"


def _source_label(value: Any) -> str:
    if isinstance(value, time):
        return f"{value.hour:02d}:{value.minute:02d}"
    return str(value).strip()


def _expected_source_label(index: int) -> set[str]:
    end_minutes = (index + 1) * 10
    if end_minutes == 24 * 60:
        return {"0:00+1", "00:00+1", "24:00"}
```

```
    return {_format_clock(end_minutes), _format_clock(end_minutes).lstrip("0")}


def load_q1_input(path: Path = SOURCE_PATH) -> Q1Input:
    if not path.is_file():
        raise FileNotFoundError(f"Required Q1 input not found: {path}")
    workbook = load_workbook(path, read_only=True, data_only=True)
    rows = list(workbook.active.values)
    expected_headers = (" 时间", " 电价", " 小区负载", " 光伏发电预测功率")
    if tuple(rows[0]) != expected_headers:
        raise ValueError(f"Unexpected Q1 headers: {rows[0]!r}")
    data_rows = rows[1:]
    if len(data_rows) != N_INTERVALS:
        raise ValueError(f"Expected 144 Q1 intervals, found {len(data_rows)}")

    source_labels = [_source_label(row[0]) for row in data_rows]
    for index, label in enumerate(source_labels):
        if label not in _expected_source_label(index):
            raise ValueError(
                f"Unexpected source time label at row {index + 2}: {label!r}; "
                f"expected one of {_expected_source_label(index)}"
            )

    numeric = np.asarray([row[1:] for row in data_rows], dtype=float)
    if not np.isfinite(numeric).all():
        raise ValueError("Q1 input contains missing or non-finite numeric values")
    price, load_kw, pv_kw = numeric.T
    if np.any(price <= 0.0) or np.any(load_kw < 0.0) or np.any(pv_kw < 0.0):
        raise ValueError("Q1 input violates price/load/PV sign requirements")

    starts = [_format_clock(i * 10) for i in range(N_INTERVALS)]
    ends = [_format_clock((i + 1) * 10) for i in range(N_INTERVALS)]
    labels = [f"{start}-{end}" for start, end in zip(starts, ends)]
    return Q1Input(
        source_labels=source_labels,
        interval_starts=starts,
        interval_ends=ends,
        interval_labels=labels,
        price=price,
        load_kw=load_kw,
        pv_kw=pv_kw,
        load_kwh=load_kw * DELTA_HOURS,
        pv_kwh=pv_kw * DELTA_HOURS,
        source_sha256=hashlib.sha256(path.read_bytes()).hexdigest(),
    )


def baseline_dispatch(data: Q1Input) -> Dispatch:
    purchase = np.maximum(data.load_kwh - data.pv_kwh, 0.0)
    unused = np.maximum(data.pv_kwh - data.load_kwh, 0.0)
    zeros = np.zeros(N_INTERVALS)
    soc_end = np.full(N_INTERVALS, SOC_INITIAL_KWH)
    return Dispatch(
        purchase=purchase,
        charge=zeros.copy(),
        discharge=zeros.copy(),
        unused=unused,
        soc_end=soc_end,
        cost=float(data.price @ purchase),
    )


def ensure_output_dirs() -> None:
    TABLE_DIR.mkdir(parents=True, exist_ok=True)
    METRIC_DIR.mkdir(parents=True, exist_ok=True)


def write_csv(path: Path, header: list[str], rows: list[list[Any]]) -> None:
```

```
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8-sig", newline="") as handle:
        writer = csv.writer(handle)
        writer.writerow(header)
        writer.writerows(rows)


def write_json(path: Path, payload: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")


def finite_float(value: float) -> float:
    value = float(value)
    if not math.isfinite(value):
        raise ValueError(f"Non-finite output: {value}")
    return value

```

D.4.2 code/q1_main.py

```
from __future__ import annotations

from dataclasses import dataclass

import numpy as np

from q1_common import (
    COST_TOLERANCE_YUAN,
    ETA_CHARGE,
    ETA_DISCHARGE,
    INTERVAL_LIMIT_KWH,
    N_INTERVALS,
    SOC_INITIAL_KWH,
    SOC_MAX_KWH,
    SOC_MIN_KWH,
    Dispatch,
    Q1Input,
)
from scipy.optimize import linprog


@dataclass(frozen=True)
class MainSolveResult:
    dispatch: Dispatch
    stage1_cost: float
    stage2_cost: float
    stage2_throughput: float
    stage1_status: str
    stage2_status: str


def _build_linear_system(data: Q1Input) -> tuple[np.ndarray, np.ndarray, list[tuple[float | None, float | None]]]:
    n = N_INTERVALS
    a_eq = np.zeros((2 * n + 1, 5 * n), dtype=float)
    b_eq = np.zeros(2 * n + 1, dtype=float)

    for t in range(n):
        # q + pv + d = load + c + w
        a_eq[t, [t, n + t, 2 * n + t, 3 * n + t]] = [1.0, -1.0, 1.0, -1.0]
        b_eq[t] = data.load_kwh[t] - data.pv_kwh[t]

        # S_t = S_{t-1} + eta_c*c - d/eta_d
        a_eq[n + t, [n + t, 2 * n + t, 4 * n + t]] = [
            -ETA_CHARGE,
            1.0 / ETA_DISCHARGE,
```

```
            1.0,
        ]
        if t == 0:
            b_eq[n + t] = SOC_INITIAL_KWH
        else:
            a_eq[n + t, 4 * n + t - 1] = -1.0

    a_eq[-1, 5 * n - 1] = 1.0
    b_eq[-1] = SOC_INITIAL_KWH

    bounds: list[tuple[float | None, float | None]] = (
        [(0.0, None)] * n
        + [(0.0, INTERVAL_LIMIT_KWH)] * n
        + [(0.0, INTERVAL_LIMIT_KWH)] * n
        + [(0.0, None)] * n
        + [(SOC_MIN_KWH, SOC_MAX_KWH)] * n
    )
    return a_eq, b_eq, bounds


def solve_main(data: Q1Input) -> MainSolveResult:
    n = N_INTERVALS
    a_eq, b_eq, bounds = _build_linear_system(data)
    primary_objective = np.r_[data.price, np.zeros(4 * n)]

    stage1 = linprog(primary_objective, A_eq=a_eq, b_eq=b_eq, bounds=bounds, method="highs")
    if not stage1.success:
        raise RuntimeError(f"Q1 stage-1 optimization failed: {stage1.message}")

    secondary_objective = np.r_[
        np.zeros(n),
        np.ones(n),
        np.ones(n),
        np.zeros(2 * n),
    ]
    stage2 = linprog(
        secondary_objective,
        A_ub=primary_objective.reshape(1, -1),
        b_ub=np.asarray([float(stage1.fun) + COST_TOLERANCE_YUAN]),
        A_eq=a_eq,
        b_eq=b_eq,
        bounds=bounds,
        method="highs",
    )
    if not stage2.success:
        raise RuntimeError(f"Q1 stage-2 optimization failed: {stage2.message}")

    purchase, charge, discharge, unused, soc_end = stage2.x.reshape(5, n)
    stage2_cost = float(data.price @ purchase)
    dispatch = Dispatch(
        purchase=purchase,
        charge=charge,
        discharge=discharge,
        unused=unused,
        soc_end=soc_end,
        cost=stage2_cost,
    )
    return MainSolveResult(
        dispatch=dispatch,
        stage1_cost=float(stage1.fun),
        stage2_cost=stage2_cost,
        stage2_throughput=float(charge.sum() + discharge.sum()),
        stage1_status=str(stage1.message),
        stage2_status=str(stage2.message),
    )

```

D.4.3 code/q1_baseline.py

```python
from __future__ import annotations

from q1_common import Dispatch, Q1Input, baseline_dispatch


def solve_baseline(data: Q1Input) -> Dispatch:
    """Compute the approved no-storage baseline."""
    return baseline_dispatch(data)

```

D.5 问题二程序

D.5.1 code/q2_common.py

```python
from __future__ import annotations

import csv
import hashlib
import json
import math
import platform
import sys
import time as time_module
from dataclasses import dataclass
from datetime import date, datetime, time, timedelta
from pathlib import Path
from typing import Any, Iterable

PROJECT_ROOT = Path(__file__).resolve().parents[1]

import numpy as np
import pandas as pd
import scipy
from openpyxl import load_workbook
from scipy.optimize import linprog
from scipy.sparse import csr_matrix, lil_matrix, vstack


N_DAYS = 365
N_INTERVALS = 144
DELTA_HOURS = 1.0 / 6.0
ETA_CHARGE = 0.9
ETA_DISCHARGE = 0.9
SOC_INITIAL_KWH = 6000.0
SOC_MIN_KWH = 1200.0
SOC_MAX_KWH = 10800.0
POWER_LIMIT_KW = 5000.0
INTERVAL_LIMIT_KWH = POWER_LIMIT_KW * DELTA_HOURS
STAGE2_OBJECTIVE_TOLERANCE = 1e-6
CHECK_TOLERANCE = 2e-5
NONZERO_TOLERANCE = 1e-6
EVALUATION_START_INDEX = 31
FINAL_DAY_INDEX = 364
RANDOM_SEED = 2026

SOURCE_ROOT = PROJECT_ROOT / "data"
PRICE_PATH = SOURCE_ROOT / " 附件 1.xlsx"
ACTUAL_PATH = SOURCE_ROOT / " 附件 2.xlsx"
ROUND_DIR = PROJECT_ROOT / "results" / "Q2" / "experiments" / "round1"
TABLE_DIR = ROUND_DIR / "tables"
METRIC_DIR = ROUND_DIR / "metrics"
FIGURE_DIR = ROUND_DIR / "figures"


```

```
@dataclass(frozen=True)
class Q2Data:
    dates: list[date]
    source_labels: list[str]
    interval_starts: list[str]
    interval_ends: list[str]
    interval_labels: list[str]
    price: np.ndarray
    load_kw: np.ndarray
    pv_kw: np.ndarray
    net_load_kwh: np.ndarray
    price_sha256: str
    actual_sha256: str


@dataclass(frozen=True)
class ForecastBundle:
    load_kw: np.ndarray
    pv_kw: np.ndarray
    net_load_kwh: np.ndarray
    errors_kwh: np.ndarray
    valid: np.ndarray
    weight_mode: str


@dataclass(frozen=True)
class StrategyConfig:
    strategy_id: str
    role: str
    alpha: float | None
    gamma: float
    weight_mode: str
    use_risk_margin: bool


@dataclass(frozen=True)
class DailySolve:
    q: np.ndarray
    charge: np.ndarray
    discharge: np.ndarray
    plan_unused: np.ndarray
    soc_end: np.ndarray
    stage1_objective: float
    stage2_objective: float
    stage2_throughput: float
    stage1_status: str
    stage2_status: str
    fallback_used: bool


@dataclass
class StrategyRun:
    config: StrategyConfig
    runtime_seconds: float
    ledger: pd.DataFrame
    daily: pd.DataFrame
    metrics: dict[str, Any]
    validation: dict[str, Any]


def ensure_output_dirs() -> None:
    for path in (TABLE_DIR, METRIC_DIR, FIGURE_DIR):
        path.mkdir(parents=True, exist_ok=True)


def write_json(path: Path, payload: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
```

```
119
120
121 def write_csv(path: Path, rows: Iterable[Iterable[Any]], header: list[str]) -> None:
122     path.parent.mkdir(parents=True, exist_ok=True)
123     with path.open("w", encoding="utf-8-sig", newline="") as handle:
124         writer = csv.writer(handle)
125         writer.writerow(header)
126         writer.writerows(rows)
127
128
129 def sha256(path: Path) -> str:
130     digest = hashlib.sha256()
131     with path.open("rb") as handle:
132         for chunk in iter(lambda: handle.read(1024 * 1024), b""):
133             digest.update(chunk)
134     return digest.hexdigest()
135
136
137 def _clock_label(value: Any) -> str:
138     if isinstance(value, time):
139         return f"{value.hour:02d}:{value.minute:02d}"
140     return str(value).strip()
141
142
143 def _format_clock(total_minutes: int) -> str:
144     if total_minutes == 24 * 60:
145         return "24:00"
146     hour, minute = divmod(total_minutes, 60)
147     return f"{hour:02d}:{minute:02d}"
148
149
150 def _expected_end_label(index: int) -> set[str]:
151     end_minutes = (index + 1) * 10
152     if end_minutes == 24 * 60:
153         return {"0:00+1", "00:00+1", "24:00"}
154     label = _format_clock(end_minutes)
155     return {label, label.lstrip("0")}
156
157
158 def _as_date(value: Any) -> date:
159     if isinstance(value, datetime):
160         return value.date()
161     if isinstance(value, date):
162         return value
163     raise TypeError(f"Expected date value, got {value!r}")
164
165
166 def load_q2_data() -> Q2Data:
167     for path in (PRICE_PATH, ACTUAL_PATH):
168         if not path.is_file():
169             raise FileNotFoundError(path)
170
171     price_wb = load_workbook(PRICE_PATH, read_only=True, data_only=True)
172     price_rows = list(price_wb.active.values)
173     if tuple(price_rows[0]) != (" 时间", " 电价", " 小区负载", " 光伏发电预测功率"):
174         raise ValueError(f"Unexpected attachment 1 headers: {price_rows[0]!r}")
175     if len(price_rows) != N_INTERVALS + 1:
176         raise ValueError(f"Expected 144 price rows, found {len(price_rows) - 1}")
177     price_labels = [_clock_label(row[0]) for row in price_rows[1:]]
178     prices = np.asarray([row[1] for row in price_rows[1:]], dtype=float)
179     for index, label in enumerate(price_labels):
180         if label not in _expected_end_label(index):
181             raise ValueError(f"Unexpected price time label at interval {index}: {label!r}")
182     if not np.isfinite(prices).all() or np.any(prices <= 0.0):
183         raise ValueError("Prices must be finite and positive")
184
185     actual_wb = load_workbook(ACTUAL_PATH, read_only=True, data_only=True)
186     if actual_wb.sheetnames != [" 小区负载", " 光伏发电实际功率"]:
```

```
        raise ValueError(f"Unexpected attachment 2 sheets: {actual_wb.sheetnames}")

    matrices: dict[str, np.ndarray] = {}
    dates: list[date] | None = None
    source_labels: list[str] | None = None
    for sheet_name in actual_wb.sheetnames:
        sheet = actual_wb[sheet_name]
        rows = list(sheet.values)
        if len(rows) != N_DAYS + 1 or len(rows[0]) != N_INTERVALS + 1:
            raise ValueError(f"Unexpected shape for {sheet_name}: {len(rows)} x {len(rows[0])}")
        current_dates = [_as_date(row[0]) for row in rows[1:]]
        current_labels = [_clock_label(value) for value in rows[0][1:]]
        if dates is None:
            dates = current_dates
            source_labels = current_labels
        elif current_dates != dates or current_labels != source_labels:
            raise ValueError("Attachment 2 sheets do not share identical axes")
        matrices[sheet_name] = np.asarray([row[1:] for row in rows[1:]], dtype=float)

    assert dates is not None and source_labels is not None
    expected_dates = [date(2025, 1, 1) + timedelta(days=i) for i in range(N_DAYS)]
    if dates != expected_dates:
        raise ValueError("Attachment 2 dates are not the complete 2025 calendar")
    for index, label in enumerate(source_labels):
        if label not in _expected_end_label(index):
            raise ValueError(f"Unexpected actual time label at interval {index}: {label!r}")

    load_kw = matrices[" 小区负载"]
    pv_kw = matrices[" 光伏发电实际功率"]
    if not np.isfinite(load_kw).all() or not np.isfinite(pv_kw).all():
        raise ValueError("Attachment 2 contains missing or non-finite values")
    if np.any(load_kw < 0.0) or np.any(pv_kw < 0.0):
        raise ValueError("Load and PV must be nonnegative")

    starts = [_format_clock(i * 10) for i in range(N_INTERVALS)]
    ends = [_format_clock((i + 1) * 10) for i in range(N_INTERVALS)]
    labels = [f"{start}-{end}" for start, end in zip(starts, ends)]
    return Q2Data(
        dates=dates,
        source_labels=source_labels,
        interval_starts=starts,
        interval_ends=ends,
        interval_labels=labels,
        price=prices,
        load_kw=load_kw,
        pv_kw=pv_kw,
        net_load_kwh=(load_kw - pv_kw) * DELTA_HOURS,
        price_sha256=sha256(PRICE_PATH),
        actual_sha256=sha256(ACTUAL_PATH),
    )


def _is_weekend(value: date) -> bool:
    return value.weekday() >= 5


def _history_weights(day_index: int, indices: np.ndarray, weight_mode: str) -> np.ndarray:
    if weight_mode == "exponential":
        gaps = day_index - indices
        return np.exp(-gaps / 7.0)
    if weight_mode == "equal":
        return np.ones(len(indices), dtype=float)
    raise ValueError(f"Unknown weight mode: {weight_mode}")


def build_forecasts(data: Q2Data, weight_mode: str) -> ForecastBundle:
    predicted_load = np.full_like(data.load_kw, np.nan)
    predicted_pv = np.full_like(data.pv_kw, np.nan)
```

```
    predicted_net = np.full_like(data.net_load_kwh, np.nan)
    errors = np.full_like(data.net_load_kwh, np.nan)
    valid = np.zeros(N_DAYS, dtype=bool)

    for day_index in range(1, N_DAYS):
        start_28 = max(0, day_index - 28)
        prior_28 = np.arange(start_28, day_index, dtype=int)
        same_type = np.asarray(
            [idx for idx in prior_28 if _is_weekend(data.dates[idx]) == _is_weekend(data.dates[day_index])],
            dtype=int,
        )
        load_indices = same_type if len(same_type) else np.arange(max(0, day_index - 7), day_index, dtype=int)
        pv_indices = np.arange(max(0, day_index - 7), day_index, dtype=int)
        load_weights = _history_weights(day_index, load_indices, weight_mode)
        pv_weights = _history_weights(day_index, pv_indices, weight_mode)
        predicted_load[day_index] = np.average(data.load_kw[load_indices], axis=0, weights=load_weights)
        predicted_pv[day_index] = np.maximum(
            0.0,
            np.average(data.pv_kw[pv_indices], axis=0, weights=pv_weights),
        )
        predicted_net[day_index] = (predicted_load[day_index] - predicted_pv[day_index]) * DELTA_HOURS
        errors[day_index] = data.net_load_kwh[day_index] - predicted_net[day_index]
        valid[day_index] = True

    return ForecastBundle(
        load_kw=predicted_load,
        pv_kw=predicted_pv,
        net_load_kwh=predicted_net,
        errors_kwh=errors,
        valid=valid,
        weight_mode=weight_mode,
    )


def build_risk_margins(bundle: ForecastBundle, alpha: float) -> np.ndarray:
    margins = np.zeros((N_DAYS, N_INTERVALS), dtype=float)
    for day_index in range(1, N_DAYS):
        prior_indices = np.arange(max(1, day_index - 28), day_index, dtype=int)
        valid_indices = prior_indices[bundle.valid[prior_indices]]
        if len(np.unique(valid_indices)) < 7:
            continue
        for group in range(6):
            section = slice(group * 24, (group + 1) * 24)
            samples = bundle.errors_kwh[valid_indices, section].reshape(-1)
            quantile = float(np.quantile(samples, alpha, method="linear"))
            margins[day_index, section] = max(0.0, quantile)
    return margins


class DailyLP:
    def __init__(self, price: np.ndarray) -> None:
        self.price = np.asarray(price, dtype=float)
        n = N_INTERVALS
        a_eq = lil_matrix((2 * n, 5 * n), dtype=float)
        for t in range(n):
            a_eq[t, [t, n + t, 2 * n + t, 3 * n + t]] = [1.0, -1.0, 1.0, -1.0]
            a_eq[n + t, [n + t, 2 * n + t, 4 * n + t]] = [-ETA_CHARGE, 1.0 / ETA_DISCHARGE, 1.0]
            if t > 0:
                a_eq[n + t, 4 * n + t - 1] = -1.0
        self.a_eq = a_eq.tocsr()
        self.bounds = (
            [(0.0, None)] * n
            + [(0.0, INTERVAL_LIMIT_KWH)] * n
            + [(0.0, INTERVAL_LIMIT_KWH)] * n
            + [(0.0, None)] * n
            + [(SOC_MIN_KWH, SOC_MAX_KWH)] * n
        )
        self.secondary = np.r_[np.zeros(n), np.ones(n), np.ones(n), np.zeros(2 * n)]
```

```
323
324     def solve(self, planned_net: np.ndarray, initial_soc: float, terminal_value: float) -> DailySolve:
325         n = N_INTERVALS
326         primary = np.r_[self.price, np.zeros(4 * n)]
327         primary[5 * n - 1] = -terminal_value
328         b_eq = np.r_[planned_net, np.zeros(n)]
329         b_eq[n] = initial_soc
330
331         stage1 = linprog(primary, A_eq=self.a_eq, b_eq=b_eq, bounds=self.bounds, method="highs")
332         if not stage1.success:
333             raise RuntimeError(f"Stage-1 LP failed: {stage1.message}")
334         a_ub = csr_matrix(primary.reshape(1, -1))
335         stage2 = linprog(
336             self.secondary,
337             A_ub=a_ub,
338             b_ub=np.asarray([float(stage1.fun) + STAGE2_OBJECTIVE_TOLERANCE]),
339             A_eq=self.a_eq,
340             b_eq=b_eq,
341             bounds=self.bounds,
342             method="highs",
343         )
344         if not stage2.success:
345             raise RuntimeError(f"Stage-2 LP failed: {stage2.message}")
346         q, charge, discharge, plan_unused, soc_end = stage2.x.reshape(5, n)
347         return DailySolve(
348             q=q,
349             charge=charge,
350             discharge=discharge,
351             plan_unused=plan_unused,
352             soc_end=soc_end,
353             stage1_objective=float(stage1.fun),
354             stage2_objective=float(primary @ stage2.x),
355             stage2_throughput=float(charge.sum() + discharge.sum()),
356             stage1_status=str(stage1.message),
357             stage2_status=str(stage2.message),
358             fallback_used=False,
359         )
360
361
362 def _cold_start(data: Q2Data) -> DailySolve:
363     zeros = np.zeros(N_INTERVALS, dtype=float)
364     return DailySolve(
365         q=zeros.copy(),
366         charge=zeros.copy(),
367         discharge=zeros.copy(),
368         plan_unused=zeros.copy(),
369         soc_end=np.full(N_INTERVALS, SOC_INITIAL_KWH),
370         stage1_objective=0.0,
371         stage2_objective=0.0,
372         stage2_throughput=0.0,
373         stage1_status="cold_start_no_forecast",
374         stage2_status="cold_start_no_forecast",
375         fallback_used=False,
376     )
377
378
379 def _daily_validation(
380     data: Q2Data,
381     day_index: int,
382     initial_soc: float,
383     planned_net: np.ndarray,
384     point_forecast: np.ndarray,
385     margin: np.ndarray,
386     solve: DailySolve,
387     emergency: np.ndarray,
388     actual_unused: np.ndarray,
389 ) -> dict[str, float]:
390     previous_soc = np.r_[initial_soc, solve.soc_end[:-1]]
```

```
    planning_balance = solve.q + solve.discharge - planned_net - solve.charge - solve.plan_unused
    storage_recurrence = solve.soc_end - previous_soc - ETA_CHARGE * solve.charge + solve.discharge /
      ETA_DISCHARGE
    actual_balance = (
        solve.q
        + emergency
        + data.pv_kw[day_index] * DELTA_HOURS
        + solve.discharge
        - data.load_kw[day_index] * DELTA_HOURS
        - solve.charge
        - actual_unused
    )
    actual_residual = data.net_load_kwh[day_index] + solve.charge - solve.discharge - solve.q
    equation17 = actual_residual - (
        data.net_load_kwh[day_index] - point_forecast - margin - solve.plan_unused
    )
    return {
        "planning_balance_max_abs": float(np.max(np.abs(planning_balance))),
        "storage_recurrence_max_abs": float(np.max(np.abs(storage_recurrence))),
        "actual_balance_max_abs": float(np.max(np.abs(actual_balance))),
        "equation17_max_abs": float(np.max(np.abs(equation17))),
        "soc_lower_violation": float(max(0.0, SOC_MIN_KWH - np.min(solve.soc_end))),
        "soc_upper_violation": float(max(0.0, np.max(solve.soc_end) - SOC_MAX_KWH)),
        "charge_limit_violation": float(max(0.0, np.max(solve.charge) - INTERVAL_LIMIT_KWH)),
        "discharge_limit_violation": float(max(0.0, np.max(solve.discharge) - INTERVAL_LIMIT_KWH)),
        "simultaneous_charge_discharge_max": float(np.max(np.minimum(solve.charge, solve.discharge))),
        "emergency_unused_overlap_max": float(np.max(np.minimum(emergency, actual_unused))),
        "stage2_objective_excess": float(max(0.0, solve.stage2_objective - solve.stage1_objective)),
    }


def run_strategy(
    data: Q2Data,
    forecasts: ForecastBundle,
    margins: np.ndarray,
    config: StrategyConfig,
    start_day_index: int = 0,
    starting_soc_kwh: float = SOC_INITIAL_KWH,
) -> StrategyRun:
    started = time_module.perf_counter()
    solver = DailyLP(data.price)
    ledger_rows: list[dict[str, Any]] = []
    daily_rows: list[dict[str, Any]] = []
    if not 0 <= start_day_index < N_DAYS:
        raise ValueError(f"Invalid start_day_index: {start_day_index}")
    initial_soc = float(starting_soc_kwh)
    max_checks: dict[str, float] = {}
    fallback_days: list[str] = []

    for day_index in range(start_day_index, N_DAYS):
        day = data.dates[day_index]
        if day_index == 0:
            point_forecast = np.zeros(N_INTERVALS, dtype=float)
            margin = np.zeros(N_INTERVALS, dtype=float)
            planned_net = np.zeros(N_INTERVALS, dtype=float)
            solve = _cold_start(data)
        else:
            point_forecast = forecasts.net_load_kwh[day_index]
            margin = margins[day_index] if config.use_risk_margin else np.zeros(N_INTERVALS, dtype=float)
            planned_net = point_forecast + margin
            terminal_value = 0.0 if day_index == FINAL_DAY_INDEX else config.gamma * ETA_DISCHARGE *
              float(np.median(data.price))
            try:
                solve = solver.solve(planned_net, initial_soc, terminal_value)
            except RuntimeError:
                q = np.maximum(planned_net, 0.0)
                zeros = np.zeros(N_INTERVALS, dtype=float)
                solve = DailySolve(
```

```
                    q=q,
                    charge=zeros.copy(),
                    discharge=zeros.copy(),
                    plan_unused=np.maximum(-planned_net, 0.0),
                    soc_end=np.full(N_INTERVALS, initial_soc),
                    stage1_objective=float("nan"),
                    stage2_objective=float("nan"),
                    stage2_throughput=0.0,
                    stage1_status="failed_fallback_activated",
                    stage2_status="failed_fallback_activated",
                    fallback_used=True,
                )
                fallback_days.append(str(day))

        actual_residual = data.net_load_kwh[day_index] + solve.charge - solve.discharge - solve.q
        emergency = np.maximum(actual_residual, 0.0)
        actual_unused = np.maximum(-actual_residual, 0.0)
        contract_cost = float(data.price @ solve.q)
        emergency_cost = float(5.0 * data.price @ emergency)
        actual_cost = contract_cost + emergency_cost
        checks = _daily_validation(
            data,
            day_index,
            initial_soc,
            planned_net,
            point_forecast,
            margin,
            solve,
            emergency,
            actual_unused,
        )
        for name, value in checks.items():
            max_checks[name] = max(max_checks.get(name, 0.0), value)

        daily_rows.append(
            {
                "strategy_id": config.strategy_id,
                "date": day.isoformat(),
                "initial_soc_kwh": initial_soc,
                "terminal_soc_kwh": float(solve.soc_end[-1]),
                "planned_purchase_kwh": float(solve.q.sum()),
                "charge_kwh": float(solve.charge.sum()),
                "discharge_kwh": float(solve.discharge.sum()),
                "planned_unused_kwh": float(solve.plan_unused.sum()),
                "emergency_purchase_kwh": float(emergency.sum()),
                "actual_unused_kwh": float(actual_unused.sum()),
                "contract_cost_yuan": contract_cost,
                "emergency_cost_yuan": emergency_cost,
                "actual_cost_yuan": actual_cost,
                "risk_margin_kwh": float(margin.sum()),
                "forecast_mae_kwh": float(np.mean(np.abs(data.net_load_kwh[day_index] - point_forecast))) if
                  day_index else np.nan,
                "stage1_objective": solve.stage1_objective,
                "stage2_objective": solve.stage2_objective,
                "stage2_throughput_kwh": solve.stage2_throughput,
                "fallback_used": solve.fallback_used,
            }
        )

        previous_soc = np.r_[initial_soc, solve.soc_end[:-1]]
        for interval in range(N_INTERVALS):
            ledger_rows.append(
                {
                    "strategy_id": config.strategy_id,
                    "date": day.isoformat(),
                    "interval_index": interval + 1,
                    "interval_start": data.interval_starts[interval],
                    "interval_end": data.interval_ends[interval],
```

```
                    "interval_label": data.interval_labels[interval],
                    "price_yuan_per_kwh": data.price[interval],
                    "actual_load_kw": data.load_kw[day_index, interval],
                    "actual_pv_kw": data.pv_kw[day_index, interval],
                    "actual_net_load_kwh": data.net_load_kwh[day_index, interval],
                    "forecast_load_kw": forecasts.load_kw[day_index, interval] if day_index else np.nan,
                    "forecast_pv_kw": forecasts.pv_kw[day_index, interval] if day_index else np.nan,
                    "point_forecast_net_kwh": point_forecast[interval] if day_index else np.nan,
                    "risk_margin_kwh": margin[interval],
                    "planned_net_load_kwh": planned_net[interval],
                    "planned_purchase_kwh": solve.q[interval],
                    "actual_charge_kwh": solve.charge[interval],
                    "actual_discharge_kwh": solve.discharge[interval],
                    "planned_unused_kwh": solve.plan_unused[interval],
                    "emergency_purchase_kwh": emergency[interval],
                    "actual_unused_kwh": actual_unused[interval],
                    "soc_start_kwh": previous_soc[interval],
                    "soc_end_kwh": solve.soc_end[interval],
                    "contract_cost_yuan": data.price[interval] * solve.q[interval],
                    "emergency_cost_yuan": 5.0 * data.price[interval] * emergency[interval],
                    "fallback_used": solve.fallback_used,
                }
            )
        initial_soc = float(solve.soc_end[-1])

    ledger = pd.DataFrame(ledger_rows)
    daily = pd.DataFrame(daily_rows)
    eval_ledger = ledger[ledger["date"] >= data.dates[EVALUATION_START_INDEX].isoformat()]
    eval_daily = daily[daily["date"] >= data.dates[EVALUATION_START_INDEX].isoformat()]
    q_profiles = eval_ledger.pivot(index="date", columns="interval_index", values="planned_purchase_kwh")
    unique_profiles = len({tuple(np.round(row, 6)) for row in q_profiles.to_numpy()})
    emergency_values = eval_ledger["emergency_purchase_kwh"].to_numpy()
    sorted_emergency = np.sort(emergency_values)[::-1]
    top_count = max(1, int(math.ceil(0.01 * len(sorted_emergency))))
    total_emergency = float(sorted_emergency.sum())
    top_one_percent_share = float(sorted_emergency[:top_count].sum() / total_emergency) if total_emergency > 0
      else 0.0

    metrics = {
        "strategy_id": config.strategy_id,
        "role": config.role,
        "alpha": config.alpha,
        "gamma": config.gamma,
        "weight_mode": config.weight_mode,
        "evaluation_start": data.dates[EVALUATION_START_INDEX].isoformat(),
        "evaluation_end": data.dates[-1].isoformat(),
        "evaluation_days": int(len(eval_daily)),
        "contract_cost_yuan": float(eval_daily["contract_cost_yuan"].sum()),
        "emergency_cost_yuan": float(eval_daily["emergency_cost_yuan"].sum()),
        "actual_total_cost_yuan": float(eval_daily["actual_cost_yuan"].sum()),
        "planned_purchase_kwh": float(eval_daily["planned_purchase_kwh"].sum()),
        "emergency_purchase_kwh": float(eval_daily["emergency_purchase_kwh"].sum()),
        "total_grid_purchase_kwh": float(eval_daily["planned_purchase_kwh"].sum() +
          eval_daily["emergency_purchase_kwh"].sum()),
        "charge_kwh": float(eval_daily["charge_kwh"].sum()),
        "discharge_kwh": float(eval_daily["discharge_kwh"].sum()),
        "planned_unused_kwh": float(eval_daily["planned_unused_kwh"].sum()),
        "actual_unused_kwh": float(eval_daily["actual_unused_kwh"].sum()),
        "soc_2025_02_01_start_kwh": float(eval_daily.iloc[0]["initial_soc_kwh"]),
        "soc_2025_12_31_end_kwh": float(eval_daily.iloc[-1]["terminal_soc_kwh"]),
        "forecast_mae_kwh": float(eval_daily["forecast_mae_kwh"].mean()),
        "emergency_interval_count": int(np.count_nonzero(emergency_values > NONZERO_TOLERANCE)),
        "nonzero_charge_interval_count": int(np.count_nonzero(eval_ledger["actual_charge_kwh"].to_numpy() >
          NONZERO_TOLERANCE)),
        "nonzero_discharge_interval_count": int(np.count_nonzero(eval_ledger["actual_discharge_kwh"].to_numpy() >
          NONZERO_TOLERANCE)),
        "unique_daily_purchase_profiles_rounded_6dp": int(unique_profiles),
```

```
        "emergency_top_1pct_volume_share": top_one_percent_share,
        "charge_at_limit_interval_count": int(np.count_nonzero(np.isclose(eval_ledger["actual_charge_kwh"],
          INTERVAL_LIMIT_KWH, atol=CHECK_TOLERANCE))),
        "discharge_at_limit_interval_count": int(np.count_nonzero(np.isclose(eval_ledger["actual_discharge_kwh"],
          INTERVAL_LIMIT_KWH, atol=CHECK_TOLERANCE))),
        "soc_at_lower_bound_interval_count": int(np.count_nonzero(np.isclose(eval_ledger["soc_end_kwh"],
          SOC_MIN_KWH, atol=CHECK_TOLERANCE))),
        "soc_at_upper_bound_interval_count": int(np.count_nonzero(np.isclose(eval_ledger["soc_end_kwh"],
          SOC_MAX_KWH, atol=CHECK_TOLERANCE))),
        "fallback_day_count": int(len(fallback_days)),
        "fallback_days": fallback_days,
        "january_actual_cost_yuan": float(
            daily[daily["date"] < data.dates[EVALUATION_START_INDEX].isoformat()]["actual_cost_yuan"].sum()
        ),
        "run_start_date": data.dates[start_day_index].isoformat(),
        "starting_soc_kwh": float(starting_soc_kwh),
    }
    validation = {
        "strategy_id": config.strategy_id,
        "tolerance": CHECK_TOLERANCE,
        "max_residuals_or_violations": max_checks,
        "all_pass": bool(
            not fallback_days
            and all(value <= CHECK_TOLERANCE for value in max_checks.values())
            and len(daily) == N_DAYS - start_day_index
            and len(ledger) == (N_DAYS - start_day_index) * N_INTERVALS
        ),
        "daily_rows": int(len(daily)),
        "interval_rows": int(len(ledger)),
        "cross_day_continuity_max_abs": float(
            np.max(
                np.abs(
                    daily["initial_soc_kwh"].to_numpy()[1:]
                    - daily["terminal_soc_kwh"].to_numpy()[:-1]
                )
            )
        ),
        "cost_reconciliation_max_abs": float(
            np.max(
                np.abs(
                    daily["actual_cost_yuan"].to_numpy()
                    - daily["contract_cost_yuan"].to_numpy()
                    - daily["emergency_cost_yuan"].to_numpy()
                )
            )
        ),
    }
    validation["all_pass"] = bool(
        validation["all_pass"]
        and validation["cross_day_continuity_max_abs"] <= CHECK_TOLERANCE
        and validation["cost_reconciliation_max_abs"] <= CHECK_TOLERANCE
    )
    return StrategyRun(
        config=config,
        runtime_seconds=time_module.perf_counter() - started,
        ledger=ledger,
        daily=daily,
        metrics=metrics,
        validation=validation,
    )


def emergency_events(ledger: pd.DataFrame) -> pd.DataFrame:
    events: list[dict[str, Any]] = []
    for day, group in ledger.groupby("date", sort=True):
        ordered = group.sort_values("interval_index")
        active = ordered["emergency_purchase_kwh"].to_numpy() > NONZERO_TOLERANCE
```

```
        amounts = ordered["emergency_purchase_kwh"].to_numpy()
        starts = ordered["interval_start"].tolist()
        ends = ordered["interval_end"].tolist()
        cursor = 0
        event_index = 0
        while cursor < N_INTERVALS:
            if not active[cursor]:
                cursor += 1
                continue
            event_start = cursor
            while cursor + 1 < N_INTERVALS and active[cursor + 1]:
                cursor += 1
            event_end = cursor
            event_index += 1
            events.append(
                {
                    "date": day,
                    "event_index": event_index,
                    "start_interval_index": event_start + 1,
                    "end_interval_index": event_end + 1,
                    "emergency_time_range": f"{starts[event_start]}-{ends[event_end]}",
                    "emergency_purchase_kwh": float(amounts[event_start : event_end + 1].sum()),
                }
            )
            cursor += 1
    return pd.DataFrame(events)


def storage_four_hour_summary(ledger: pd.DataFrame) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    for day, group in ledger.groupby("date", sort=True):
        ordered = group.sort_values("interval_index")
        initial_soc = float(ordered.iloc[0]["soc_start_kwh"])
        terminal_soc = float(ordered.iloc[-1]["soc_end_kwh"])
        for block in range(6):
            section = ordered.iloc[block * 24 : (block + 1) * 24]
            rows.append(
                {
                    "date": day,
                    "block_index": block + 1,
                    "time_range": f"{block * 4:02d}:00-{(block + 1) * 4:02d}:00" if block < 5 else "20:00-24:00",
                    "charge_kwh": float(section["actual_charge_kwh"].sum()),
                    "discharge_kwh": float(section["actual_discharge_kwh"].sum()),
                    "soc_00_kwh": initial_soc if block == 0 else np.nan,
                    "soc_24_kwh": terminal_soc if block == 1 else np.nan,
                }
            )
    return pd.DataFrame(rows)


def specified_date_tables(run: StrategyRun) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    specified = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
    interval_indices = [61, 73, 85, 97, 109, 121]
    table1_rows: list[dict[str, Any]] = []
    for day in specified:
        ledger_day = run.ledger[run.ledger["date"] == day].set_index("interval_index")
        daily = run.daily[run.daily["date"] == day].iloc[0]
        row: dict[str, Any] = {"date": day}
        for interval_index in interval_indices:
            label = ledger_day.loc[interval_index, "interval_label"]
            row[f"{label}_purchase_kwh"] = float(ledger_day.loc[interval_index, "planned_purchase_kwh"])
        row["daily_planned_purchase_kwh"] = float(daily["planned_purchase_kwh"])
        row["daily_contract_cost_yuan"] = float(daily["contract_cost_yuan"])
        row["daily_emergency_cost_yuan"] = float(daily["emergency_cost_yuan"])
        row["daily_actual_total_cost_yuan"] = float(daily["actual_cost_yuan"])
        table1_rows.append(row)
    table1 = pd.DataFrame(table1_rows)
    table2 = storage_four_hour_summary(run.ledger[run.ledger["date"].isin(specified)])
```

```
    table3 = emergency_events(run.ledger[run.ledger["date"].isin(specified)])
    return table1, table2, table3


def perfect_foresight_lower_bound(data: Q2Data, initial_soc: float) -> dict[str, Any]:
    started = time_module.perf_counter()
    net = data.net_load_kwh[EVALUATION_START_INDEX:].reshape(-1)
    price = np.tile(data.price, N_DAYS - EVALUATION_START_INDEX)
    n = len(net)
    a_eq = lil_matrix((2 * n, 5 * n), dtype=float)
    for t in range(n):
        a_eq[t, [t, n + t, 2 * n + t, 3 * n + t]] = [1.0, -1.0, 1.0, -1.0]
        a_eq[n + t, [n + t, 2 * n + t, 4 * n + t]] = [-ETA_CHARGE, 1.0 / ETA_DISCHARGE, 1.0]
        if t > 0:
            a_eq[n + t, 4 * n + t - 1] = -1.0
    b_eq = np.r_[net, np.zeros(n)]
    b_eq[n] = initial_soc
    bounds = (
        [(0.0, None)] * n
        + [(0.0, INTERVAL_LIMIT_KWH)] * n
        + [(0.0, INTERVAL_LIMIT_KWH)] * n
        + [(0.0, None)] * n
        + [(SOC_MIN_KWH, SOC_MAX_KWH)] * n
    )
    objective = np.r_[price, np.zeros(4 * n)]
    result = linprog(objective, A_eq=a_eq.tocsr(), b_eq=b_eq, bounds=bounds, method="highs")
    if not result.success:
        raise RuntimeError(f"Perfect-foresight lower bound failed: {result.message}")
    q, charge, discharge, unused, soc_end = result.x.reshape(5, n)
    balance_residual = q + discharge - net - charge - unused
    previous_soc = np.r_[initial_soc, soc_end[:-1]]
    storage_residual = soc_end - previous_soc - ETA_CHARGE * charge + discharge / ETA_DISCHARGE
    return {
        "evaluation_start": data.dates[EVALUATION_START_INDEX].isoformat(),
        "evaluation_end": data.dates[-1].isoformat(),
        "initial_soc_kwh": float(initial_soc),
        "lower_bound_cost_yuan": float(result.fun),
        "planned_purchase_kwh": float(q.sum()),
        "terminal_soc_kwh": float(soc_end[-1]),
        "balance_max_abs": float(np.max(np.abs(balance_residual))),
        "storage_recurrence_max_abs": float(np.max(np.abs(storage_residual))),
        "solver_status": str(result.message),
        "runtime_seconds": time_module.perf_counter() - started,
    }


def environment_info() -> dict[str, Any]:
    return {
        "python": sys.version,
        "platform": platform.platform(),
        "numpy": np.__version__,
        "pandas": pd.__version__,
        "scipy": scipy.__version__,
        "solver": "scipy.optimize.linprog(method=highs)",
        "random_seed": RANDOM_SEED,
    }

```

D.5.2 code/q2_forecast_experiment.py

```
"""Forecast-only diagnostic; never overwrites the approved M2 outputs.

Fixed before execution: 28-day load window, exp(-age/7) weights; PV unchanged.
weekday: weighted same-weekday mean. trend: weighted same-weekday linear
regression, extrapolated to today, nonnegative; mean fallback with <3 samples.
No parameter search. All variants rebuild past-error margins with the SAME rule.
```

```
"""
from dataclasses import replace
import json
import time
from pathlib import Path

import q2_common as c
import numpy as np
import pandas as pd
from q2_full_feedback import run_full_feedback_strategy, MAIN_CONFIG, save_frame, write_json

OUT = c.PROJECT_ROOT / 'results/Q2/experiments/round4_forecast_only'
OLD = c.PROJECT_ROOT / 'results/Q2/experiments/round3_full_feedback'
MODES = ('current', 'weekday', 'trend')


def forecast(data, mode):
    base = c.build_forecasts(data, 'exponential')
    if mode == 'current':
        return base
    load = base.load_kw.copy()
    for d in range(1, c.N_DAYS):
        ix = np.array([j for j in range(max(0, d-28), d)
                       if data.dates[j].weekday() == data.dates[d].weekday()], dtype=int)
        if not len(ix):
            continue  # same cold-start fallback as current forecast
        x = (ix-d).astype(float)
        w = np.exp(x/7)
        y = data.load_kw[ix]
        mean = np.average(y, axis=0, weights=w)
        if mode == 'trend' and len(ix) >= 3:
            xm = np.average(x, weights=w)
            slope = np.sum(w[:, None]*(x-xm)[:, None]*(y-mean), axis=0) / np.sum(w*(x-xm)**2)
            mean = mean - xm*slope
        load[d] = np.maximum(mean, 0)
    net = (load-base.pv_kw)*c.DELTA_HOURS
    return replace(base, load_kw=load, net_load_kwh=net,
                   errors_kwh=data.net_load_kwh-net)


def audit(ledger, daily, data):
    """Recompute ledger physics and money independently of run metrics."""
    a = lambda key: ledger[key].to_numpy(float)
    checks = {}
    def eq(name, residual):
        checks[name] = float(np.max(np.abs(residual)))
    q,e,ch,dis,w = [a(k) for k in ('planned_purchase_kwh','emergency_purchase_kwh',
                       'actual_charge_kwh','actual_discharge_kwh','actual_unused_kwh')]
    s,t,p = a('soc_start_kwh'),a('soc_end_kwh'),a('price_yuan_per_kwh')
    eq('raw_net_mapping', a('actual_net_load_kwh')-data.net_load_kwh.ravel())
    eq('raw_price_mapping', p-np.tile(data.price,c.N_DAYS))
    eq('balance', q+e+dis-a('actual_net_load_kwh')-ch-w)
    eq('soc_recurrence', t-s-.9*ch+dis/.9)
    eq('soc_chain', s[1:]-t[:-1])
    eq('initial_soc', s[:1]-6000)
    eq('contract_cost', a('contract_cost_yuan')-p*q)
    eq('emergency_cost', a('emergency_cost_yuan')-5*p*e)
    eq('charge_discharge_overlap', np.minimum(ch,dis))
    eq('emergency_unused_overlap', np.minimum(e,w))
    checks['bounds'] = float(max(0,1200-t.min(),t.max()-10800,ch.max()-5000/6,
                                 dis.max()-5000/6,-min(q.min(),e.min(),ch.min(),dis.min(),w.min())))
    for k in ('contract_cost_yuan','emergency_cost_yuan'):
        eq('daily_'+k, ledger.groupby('date',sort=True)[k].sum().to_numpy()-daily[k].to_numpy())
    eq('daily_total', daily.actual_cost_yuan-daily.contract_cost_yuan-daily.emergency_cost_yuan)
    eq('planner_actual_initial', daily.planner_initial_soc_kwh-daily.initial_soc_kwh)
    expected = [(d.isoformat(),t+1) for d in data.dates for t in range(c.N_INTERVALS)]
    shape_ok = list(zip(ledger.date,ledger.interval_index)) == expected
    return {'checks':checks,'coverage_pass':shape_ok,
```

```
            'all_pass':bool(shape_ok and all(v<=c.CHECK_TOLERANCE for v in checks.values()))}


def main():
    start=time.perf_counter()
    data=c.load_q2_data()
    sources = {str(p):c.sha256(p) for p in (c.PRICE_PATH,c.ACTUAL_PATH,Path(c.__file__),
               Path(__file__),Path(__file__).with_name('q2_full_feedback.py'),
               OLD/'tables/q2_M2_interval_ledger.csv', OLD/'tables/q2_M2_daily_summary.csv')}
    old_l=pd.read_csv(OLD/'tables/q2_M2_interval_ledger.csv')
    old_d=pd.read_csv(OLD/'tables/q2_M2_daily_summary.csv')
    checks={'saved_baseline_audit':audit(old_l,old_d,data)}
    assert checks['saved_baseline_audit']['all_pass'], checks
    old_metrics=json.loads((OLD/'metrics/M2_main_metrics.json').read_text(encoding='utf-8'))
    rows=[]; monthly=[]; validations={}
    for mode in MODES:
        print('Running '+mode,flush=True)
        f=forecast(data,mode)
        margin=c.build_risk_margins(f,.8)
        # Change target-day and all future observations. Forecast and margin
        # at the decision boundary must remain identical.
        causal=[]
        for cut in (7,31,180,300):
            load=data.load_kw.copy(); pv=data.pv_kw.copy()
            load[cut:]+=1234; pv[cut:]*=.3
            alt=replace(data,load_kw=load,pv_kw=pv,net_load_kwh=(load-pv)*c.DELTA_HOURS)
            fa=forecast(alt,mode); ma=c.build_risk_margins(fa,.8)
            causal.append(float(max(np.nanmax(abs(f.net_load_kwh[1:cut+1]-fa.net_load_kwh[1:cut+1])),
                                    np.max(abs(margin[:cut+1]-ma[:cut+1])))))
        assert max(causal)==0
        run=run_full_feedback_strategy(data,f,margin,replace(MAIN_CONFIG,strategy_id='E1_'+mode,role='diagnostic'))
        validations[mode]={'engine':run.validation,'independent_ledger':audit(run.ledger,run.daily,data),
                           'future_mutation_max_abs':max(causal)}
        assert run.validation['all_pass'] and validations[mode]['independent_ledger']['all_pass']
        if mode=='current':
            numeric=['planned_purchase_kwh','actual_charge_kwh','actual_discharge_kwh',
                     'emergency_purchase_kwh','soc_end_kwh','contract_cost_yuan','emergency_cost_yuan']
            diff=float(np.max(abs(run.ledger[numeric].to_numpy()-old_l[numeric].to_numpy())))
            checks['baseline_rerun_max_abs']=diff
            checks['baseline_total_difference_yuan']=run.metrics['actual_total_cost_yuan']-old_metrics['actual_total_cost_yuan']
            assert diff<c.CHECK_TOLERANCE and abs(checks['baseline_total_difference_yuan'])<.01
        err=f.errors_kwh[31:]*6
        high=data.price>=np.quantile(data.price,.75)
        m=dict(run.metrics)
        m.update(mode=mode,load_mae_kw=float(np.mean(abs(data.load_kw[31:]-f.load_kw[31:]))),
                 pv_mae_kw=float(np.mean(abs(data.pv_kw[31:]-f.pv_kw[31:]))),
                 net_mae_kw=float(np.mean(abs(err))),net_underestimate_p95_kw=float(np.quantile(err,.95)),
                 high_price_underestimate_mean_kw=float(np.maximum(err[:,high],0).mean()),
                 risk_margin_kwh=float(margin[31:].sum()),
                 savings_yuan=old_metrics['actual_total_cost_yuan']-run.metrics['actual_total_cost_yuan'])
        rows.append(m)
        d=run.daily[run.daily.date>='2025-02-01'].copy()
        d['month']=d.date.str[:7]
        cols=['contract_cost_yuan','emergency_cost_yuan','actual_cost_yuan','planned_purchase_kwh',
              'emergency_purchase_kwh','actual_unused_kwh','risk_margin_kwh']
        month=d.groupby('month')[cols].sum().reset_index();month['mode']=mode;monthly.append(month)
        save_frame(run.daily,OUT/f'tables/{mode}_daily.csv')
        # baseline ledger already exists; avoid duplicating it.
        if mode!='current': save_frame(run.ledger,OUT/f'tables/{mode}_interval.csv')
        write_json(OUT/f'metrics/{mode}.json',m)
        print(json.dumps({k:m[k] for k in
           ('mode','actual_total_cost_yuan','savings_yuan','net_mae_kw')},ensure_ascii=False),flush=True)
    save_frame(pd.DataFrame(rows),OUT/'tables/comparison.csv')
    save_frame(pd.concat(monthly),OUT/'tables/monthly.csv')
    checks['strategies']=validations
    checks['source_hashes_unchanged']=all(c.sha256(Path(p))==h for p,h in sources.items())
```

```
    assert checks['source_hashes_unchanged']
    write_json(OUT/'metrics/verification.json',checks)
    summary={'status':'completed','scope':'diagnostic_only_not_formal_replacement',
             'user_authorization':'那先做第一个实验（基准复核和只替换预测）',
             'parameters':'28-day exp(-age/7); same PV; alpha=.8 gamma=.5; no search',
             'input_and_code_hashes':sources,'environment':c.environment_info(),
             'runtime_seconds':time.perf_counter()-start,'metrics':rows,
             'limitations':['Retrospective structural comparison, not pristine holdout validation.',
                            'Margins recalculated by unchanged rule; includes induced margin/SOC changes.',
                            'No prediction of unseen-year generalization; no automatic main-method adoption.']}
    write_json(OUT/'run_summary.json',summary)
    write_json(c.PROJECT_ROOT/'robustness/Q2/forecast_experiment_summary.json',
               {'status':'CONDITIONAL','claim':'Forecast-only retrospective comparison',
                'evidence':str(OUT/'run_summary.json'),'physical_and_accounting':'PASS',
                'limitation':summary['limitations']})


if __name__=='__main__': main()

```

D.5.3 code/q2_full_feedback.py

```
from __future__ import annotations

import json
import math
import time
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd

from q2_common import (
    CHECK_TOLERANCE,
    DELTA_HOURS,
    ETA_CHARGE,
    ETA_DISCHARGE,
    EVALUATION_START_INDEX,
    FINAL_DAY_INDEX,
    INTERVAL_LIMIT_KWH,
    N_DAYS,
    N_INTERVALS,
    NONZERO_TOLERANCE,
    PRICE_PATH,
    ACTUAL_PATH,
    PROJECT_ROOT,
    RANDOM_SEED,
    SOC_INITIAL_KWH,
    SOC_MAX_KWH,
    SOC_MIN_KWH,
    DailyLP,
    DailySolve,
    ForecastBundle,
    Q2Data,
    StrategyConfig,
    StrategyRun,
    build_forecasts,
    build_risk_margins,
    emergency_events,
    environment_info,
    load_q2_data,
    sha256,
    specified_date_tables,
    storage_four_hour_summary,
)
```

```
45
46
47 ROUND_DIR = PROJECT_ROOT / "results" / "Q2" / "experiments" / "round3_full_feedback"
48 TABLE_DIR = ROUND_DIR / "tables"
49 METRIC_DIR = ROUND_DIR / "metrics"
50
51 MAIN_CONFIG = StrategyConfig(
52     strategy_id="M2_daily_replan_actual_soc_feedback_alpha0.8_gamma0.5_exp",
53     role="main",
54     alpha=0.8,
55     gamma=0.5,
56     weight_mode="exponential",
57     use_risk_margin=True,
58 )
59 BASELINE_CONFIG = StrategyConfig(
60     strategy_id="B2_daily_replan_actual_soc_feedback_b0_gamma0.5_exp",
61     role="usable_baseline",
62     alpha=None,
63     gamma=0.5,
64     weight_mode="exponential",
65     use_risk_margin=False,
66 )
67
68
69 def save_frame(frame: pd.DataFrame, path: Path) -> None:
70     path.parent.mkdir(parents=True, exist_ok=True)
71     frame.to_csv(path, index=False, encoding="utf-8-sig", float_format="%.10f")
72
73
74 def write_json(path: Path, payload: Any) -> None:
75     path.parent.mkdir(parents=True, exist_ok=True)
76     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
77
78
79 def cold_start_plan() -> DailySolve:
80     zeros = np.zeros(N_INTERVALS, dtype=float)
81     return DailySolve(
82         q=zeros.copy(),
83         charge=zeros.copy(),
84         discharge=zeros.copy(),
85         plan_unused=zeros.copy(),
86         soc_end=np.full(N_INTERVALS, SOC_INITIAL_KWH),
87         stage1_objective=0.0,
88         stage2_objective=0.0,
89         stage2_throughput=0.0,
90         stage1_status="cold_start_no_forecast",
91         stage2_status="cold_start_no_forecast",
92         fallback_used=False,
93     )
94
95
96 def fallback_plan(planned_net: np.ndarray, initial_soc: float) -> DailySolve:
97     zeros = np.zeros(N_INTERVALS, dtype=float)
98     return DailySolve(
99         q=np.maximum(planned_net, 0.0),
100         charge=zeros.copy(),
101         discharge=zeros.copy(),
102         plan_unused=np.maximum(-planned_net, 0.0),
103         soc_end=np.full(N_INTERVALS, initial_soc),
104         stage1_objective=float("nan"),
105         stage2_objective=float("nan"),
106         stage2_throughput=0.0,
107         stage1_status="failed_fallback_activated",
108         stage2_status="failed_fallback_activated",
109         fallback_used=True,
110     )
111
112
```

```
def execute_feedback_day(
    actual_net_kwh: np.ndarray,
    frozen_q_kwh: np.ndarray,
    initial_soc_kwh: float,
    enable_feedback: bool,
) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    charge = np.zeros(N_INTERVALS, dtype=float)
    discharge = np.zeros(N_INTERVALS, dtype=float)
    emergency = np.zeros(N_INTERVALS, dtype=float)
    unused = np.zeros(N_INTERVALS, dtype=float)
    soc_start = np.zeros(N_INTERVALS, dtype=float)
    soc_end = np.zeros(N_INTERVALS, dtype=float)
    current_soc = float(initial_soc_kwh)

    for interval in range(N_INTERVALS):
        soc_start[interval] = current_soc
        residual = float(actual_net_kwh[interval] - frozen_q_kwh[interval])
        if enable_feedback:
            charge_max = min(
                INTERVAL_LIMIT_KWH,
                max(0.0, (SOC_MAX_KWH - current_soc) / ETA_CHARGE),
            )
            discharge_max = min(
                INTERVAL_LIMIT_KWH,
                max(0.0, ETA_DISCHARGE * (current_soc - SOC_MIN_KWH)),
            )
            charge[interval] = min(max(-residual, 0.0), charge_max)
            discharge[interval] = min(max(residual, 0.0), discharge_max)
        emergency[interval] = max(residual - discharge[interval], 0.0)
        unused[interval] = max(-residual - charge[interval], 0.0)
        current_soc = (
            current_soc
            + ETA_CHARGE * charge[interval]
            - discharge[interval] / ETA_DISCHARGE
        )
        soc_end[interval] = current_soc

    return charge, discharge, emergency, unused, soc_start, soc_end


def validate_day(
    actual_net: np.ndarray,
    planned_net: np.ndarray,
    initial_actual_soc: float,
    solve: DailySolve,
    actual_charge: np.ndarray,
    actual_discharge: np.ndarray,
    emergency: np.ndarray,
    actual_unused: np.ndarray,
    actual_soc_start: np.ndarray,
    actual_soc_end: np.ndarray,
) -> dict[str, float]:
    planned_soc_start = np.r_[initial_actual_soc, solve.soc_end[:-1]]
    planning_balance = (
        solve.q + solve.discharge - planned_net - solve.charge - solve.plan_unused
    )
    planning_storage = (
        solve.soc_end
        - planned_soc_start
        - ETA_CHARGE * solve.charge
        + solve.discharge / ETA_DISCHARGE
    )
    actual_balance = (
        solve.q
        + emergency
        + actual_discharge
        - actual_net
        - actual_charge
```

```
        - actual_unused
    )
    actual_storage = (
        actual_soc_end
        - actual_soc_start
        - ETA_CHARGE * actual_charge
        + actual_discharge / ETA_DISCHARGE
    )
    return {
        "planning_balance_max_abs": float(np.max(np.abs(planning_balance))),
        "planning_storage_recurrence_max_abs": float(np.max(np.abs(planning_storage))),
        "actual_balance_max_abs": float(np.max(np.abs(actual_balance))),
        "actual_storage_recurrence_max_abs": float(np.max(np.abs(actual_storage))),
        "planner_initial_soc_difference_abs": float(abs(planned_soc_start[0] - initial_actual_soc)),
        "actual_soc_chain_max_abs": float(
            np.max(np.abs(actual_soc_start[1:] - actual_soc_end[:-1]))
        ),
        "actual_soc_lower_violation": float(max(0.0, SOC_MIN_KWH - np.min(actual_soc_end))),
        "actual_soc_upper_violation": float(max(0.0, np.max(actual_soc_end) - SOC_MAX_KWH)),
        "actual_charge_limit_violation": float(
            max(0.0, np.max(actual_charge) - INTERVAL_LIMIT_KWH)
        ),
        "actual_discharge_limit_violation": float(
            max(0.0, np.max(actual_discharge) - INTERVAL_LIMIT_KWH)
        ),
        "actual_simultaneous_charge_discharge_max": float(
            np.max(np.minimum(actual_charge, actual_discharge))
        ),
        "actual_emergency_unused_overlap_max": float(
            np.max(np.minimum(emergency, actual_unused))
        ),
        "planned_simultaneous_charge_discharge_max": float(
            np.max(np.minimum(solve.charge, solve.discharge))
        ),
        "stage2_objective_excess": float(
            0.0
            if solve.fallback_used
            else max(0.0, solve.stage2_objective - solve.stage1_objective)
        ),
    }


def run_full_feedback_strategy(
    data: Q2Data,
    forecasts: ForecastBundle,
    margins: np.ndarray,
    config: StrategyConfig,
) -> StrategyRun:
    started = time.perf_counter()
    solver = DailyLP(data.price)
    actual_initial_soc = SOC_INITIAL_KWH
    ledger_rows: list[dict[str, Any]] = []
    daily_rows: list[dict[str, Any]] = []
    max_checks: dict[str, float] = {}
    fallback_days: list[str] = []

    for day_index in range(N_DAYS):
        day = data.dates[day_index]
        day_initial_soc = actual_initial_soc
        if day_index == 0:
            point_forecast = np.zeros(N_INTERVALS, dtype=float)
            margin = np.zeros(N_INTERVALS, dtype=float)
            planned_net = np.zeros(N_INTERVALS, dtype=float)
            solve = cold_start_plan()
            feedback_enabled = False
        else:
            point_forecast = forecasts.net_load_kwh[day_index]
            margin = (
```

```
                margins[day_index]
                if config.use_risk_margin
                else np.zeros(N_INTERVALS, dtype=float)
            )
            planned_net = point_forecast + margin
            terminal_value = (
                0.0
                if day_index == FINAL_DAY_INDEX
                else config.gamma * ETA_DISCHARGE * float(np.median(data.price))
            )
            try:
                solve = solver.solve(planned_net, day_initial_soc, terminal_value)
            except RuntimeError:
                solve = fallback_plan(planned_net, day_initial_soc)
                fallback_days.append(day.isoformat())
            feedback_enabled = True

        (
            actual_charge,
            actual_discharge,
            emergency,
            actual_unused,
            actual_soc_start,
            actual_soc_end,
        ) = execute_feedback_day(
            data.net_load_kwh[day_index],
            solve.q,
            day_initial_soc,
            feedback_enabled,
        )
        actual_terminal_soc = float(actual_soc_end[-1])
        contract_cost = float(data.price @ solve.q)
        emergency_cost = float(5.0 * data.price @ emergency)
        actual_cost = contract_cost + emergency_cost

        checks = validate_day(
            data.net_load_kwh[day_index],
            planned_net,
            day_initial_soc,
            solve,
            actual_charge,
            actual_discharge,
            emergency,
            actual_unused,
            actual_soc_start,
            actual_soc_end,
        )
        for name, value in checks.items():
            max_checks[name] = max(max_checks.get(name, 0.0), value)

        daily_rows.append(
            {
                "strategy_id": config.strategy_id,
                "date": day.isoformat(),
                "initial_soc_kwh": day_initial_soc,
                "planner_initial_soc_kwh": day_initial_soc,
                "planned_terminal_soc_kwh": float(solve.soc_end[-1]),
                "terminal_soc_kwh": actual_terminal_soc,
                "planned_purchase_kwh": float(solve.q.sum()),
                "planned_charge_kwh": float(solve.charge.sum()),
                "planned_discharge_kwh": float(solve.discharge.sum()),
                "charge_kwh": float(actual_charge.sum()),
                "discharge_kwh": float(actual_discharge.sum()),
                "planned_unused_kwh": float(solve.plan_unused.sum()),
                "emergency_purchase_kwh": float(emergency.sum()),
                "actual_unused_kwh": float(actual_unused.sum()),
                "contract_cost_yuan": contract_cost,
                "emergency_cost_yuan": emergency_cost,
```

```
                "actual_cost_yuan": actual_cost,
                "risk_margin_kwh": float(margin.sum()),
                "forecast_mae_kwh": (
                    float(np.mean(np.abs(data.net_load_kwh[day_index] - point_forecast)))
                    if day_index
                    else np.nan
                ),
                "stage1_objective": solve.stage1_objective,
                "stage2_objective": solve.stage2_objective,
                "stage2_throughput_kwh": solve.stage2_throughput,
                "fallback_used": solve.fallback_used,
            }
        )

        planned_soc_start = np.r_[day_initial_soc, solve.soc_end[:-1]]
        for interval in range(N_INTERVALS):
            ledger_rows.append(
                {
                    "strategy_id": config.strategy_id,
                    "date": day.isoformat(),
                    "interval_index": interval + 1,
                    "interval_start": data.interval_starts[interval],
                    "interval_end": data.interval_ends[interval],
                    "interval_label": data.interval_labels[interval],
                    "price_yuan_per_kwh": data.price[interval],
                    "actual_load_kw": data.load_kw[day_index, interval],
                    "actual_pv_kw": data.pv_kw[day_index, interval],
                    "actual_net_load_kwh": data.net_load_kwh[day_index, interval],
                    "forecast_load_kw": (
                        forecasts.load_kw[day_index, interval] if day_index else np.nan
                    ),
                    "forecast_pv_kw": (
                        forecasts.pv_kw[day_index, interval] if day_index else np.nan
                    ),
                    "point_forecast_net_kwh": (
                        point_forecast[interval] if day_index else np.nan
                    ),
                    "risk_margin_kwh": margin[interval],
                    "planned_net_load_kwh": planned_net[interval],
                    "planned_purchase_kwh": solve.q[interval],
                    "planned_charge_kwh": solve.charge[interval],
                    "planned_discharge_kwh": solve.discharge[interval],
                    "planned_unused_kwh": solve.plan_unused[interval],
                    "planned_soc_start_kwh": planned_soc_start[interval],
                    "planned_soc_end_kwh": solve.soc_end[interval],
                    "actual_charge_kwh": actual_charge[interval],
                    "actual_discharge_kwh": actual_discharge[interval],
                    "emergency_purchase_kwh": emergency[interval],
                    "actual_unused_kwh": actual_unused[interval],
                    "soc_start_kwh": actual_soc_start[interval],
                    "soc_end_kwh": actual_soc_end[interval],
                    "contract_cost_yuan": data.price[interval] * solve.q[interval],
                    "emergency_cost_yuan": 5.0 * data.price[interval] * emergency[interval],
                    "fallback_used": solve.fallback_used,
                }
            )

        actual_initial_soc = actual_terminal_soc

    ledger = pd.DataFrame(ledger_rows)
    daily = pd.DataFrame(daily_rows)
    evaluation_date = data.dates[EVALUATION_START_INDEX].isoformat()
    eval_ledger = ledger[ledger["date"] >= evaluation_date]
    eval_daily = daily[daily["date"] >= evaluation_date]
    emergency_values = eval_ledger["emergency_purchase_kwh"].to_numpy(dtype=float)
    emergency_total = float(emergency_values.sum())
    emergency_weighted_price = (
        float(
```

```
            eval_ledger["price_yuan_per_kwh"].to_numpy(dtype=float)
            @ emergency_values
            / emergency_total
        )
        if emergency_total > 0.0
        else 0.0
    )
    sorted_emergency = np.sort(emergency_values)[::-1]
    top_count = max(1, int(math.ceil(0.01 * len(sorted_emergency))))
    q_profiles = eval_ledger.pivot(
        index="date", columns="interval_index", values="planned_purchase_kwh"
    )
    unique_profiles = len({tuple(np.round(row, 6)) for row in q_profiles.to_numpy()})
    cross_day = (
        daily["initial_soc_kwh"].to_numpy(dtype=float)[1:]
        - daily["terminal_soc_kwh"].to_numpy(dtype=float)[:-1]
    )
    fee_residual = (
        daily["actual_cost_yuan"].to_numpy(dtype=float)
        - daily["contract_cost_yuan"].to_numpy(dtype=float)
        - daily["emergency_cost_yuan"].to_numpy(dtype=float)
    )
    next_day_planner_state = (
        daily["planner_initial_soc_kwh"].to_numpy(dtype=float)[1:]
        - daily["terminal_soc_kwh"].to_numpy(dtype=float)[:-1]
    )

    metrics = {
        "strategy_id": config.strategy_id,
        "role": config.role,
        "alpha": config.alpha,
        "gamma": config.gamma,
        "weight_mode": config.weight_mode,
        "execution_mode": "intraday_feedback_daily_replan_from_actual_soc",
        "evaluation_start": evaluation_date,
        "evaluation_end": data.dates[-1].isoformat(),
        "evaluation_days": int(len(eval_daily)),
        "contract_cost_yuan": float(eval_daily["contract_cost_yuan"].sum()),
        "emergency_cost_yuan": float(eval_daily["emergency_cost_yuan"].sum()),
        "actual_total_cost_yuan": float(eval_daily["actual_cost_yuan"].sum()),
        "planned_purchase_kwh": float(eval_daily["planned_purchase_kwh"].sum()),
        "emergency_purchase_kwh": emergency_total,
        "total_grid_purchase_kwh": float(
            eval_daily["planned_purchase_kwh"].sum() + emergency_total
        ),
        "actual_charge_kwh": float(eval_daily["charge_kwh"].sum()),
        "actual_discharge_kwh": float(eval_daily["discharge_kwh"].sum()),
        "planned_charge_kwh": float(eval_daily["planned_charge_kwh"].sum()),
        "planned_discharge_kwh": float(eval_daily["planned_discharge_kwh"].sum()),
        "planned_unused_kwh": float(eval_daily["planned_unused_kwh"].sum()),
        "actual_unused_kwh": float(eval_daily["actual_unused_kwh"].sum()),
        "soc_2025_02_01_start_kwh": float(eval_daily.iloc[0]["initial_soc_kwh"]),
        "soc_2025_12_31_end_kwh": float(eval_daily.iloc[-1]["terminal_soc_kwh"]),
        "forecast_mae_kwh": float(eval_daily["forecast_mae_kwh"].mean()),
        "emergency_interval_count": int(
            np.count_nonzero(emergency_values > NONZERO_TOLERANCE)
        ),
        "emergency_event_count": int(len(emergency_events(eval_ledger))),
        "emergency_weighted_base_price_yuan_per_kwh": emergency_weighted_price,
        "emergency_top_1pct_volume_share": float(
            sorted_emergency[:top_count].sum() / emergency_total
        )
        if emergency_total > 0.0
        else 0.0,
        "unique_daily_purchase_profiles_rounded_6dp": int(unique_profiles),
        "nonzero_actual_charge_interval_count": int(
            np.count_nonzero(
                eval_ledger["actual_charge_kwh"].to_numpy(dtype=float)
```

```
                > NONZERO_TOLERANCE
            )
        ),
        "nonzero_actual_discharge_interval_count": int(
            np.count_nonzero(
                eval_ledger["actual_discharge_kwh"].to_numpy(dtype=float)
                > NONZERO_TOLERANCE
            )
        ),
        "soc_at_lower_bound_interval_count": int(
            np.count_nonzero(
                np.isclose(
                    eval_ledger["soc_end_kwh"].to_numpy(dtype=float),
                    SOC_MIN_KWH,
                    atol=CHECK_TOLERANCE,
                )
            )
        ),
        "soc_at_upper_bound_interval_count": int(
            np.count_nonzero(
                np.isclose(
                    eval_ledger["soc_end_kwh"].to_numpy(dtype=float),
                    SOC_MAX_KWH,
                    atol=CHECK_TOLERANCE,
                )
            )
        ),
        "fallback_day_count": int(len(fallback_days)),
        "fallback_days": fallback_days,
        "january_actual_cost_yuan": float(
            daily[daily["date"] < evaluation_date]["actual_cost_yuan"].sum()
        ),
    }
    validation = {
        "strategy_id": config.strategy_id,
        "tolerance": CHECK_TOLERANCE,
        "max_residuals_or_violations": max_checks,
        "cross_day_actual_soc_continuity_max_abs": float(np.max(np.abs(cross_day))),
        "next_day_planner_uses_previous_actual_soc_max_abs": float(
            np.max(np.abs(next_day_planner_state))
        ),
        "daily_fee_reconciliation_max_abs": float(np.max(np.abs(fee_residual))),
        "daily_rows": int(len(daily)),
        "interval_rows": int(len(ledger)),
        "all_pass": False,
    }
    validation["all_pass"] = bool(
        not fallback_days
        and all(value <= CHECK_TOLERANCE for value in max_checks.values())
        and validation["cross_day_actual_soc_continuity_max_abs"] <= CHECK_TOLERANCE
        and validation["next_day_planner_uses_previous_actual_soc_max_abs"]
        <= CHECK_TOLERANCE
        and validation["daily_fee_reconciliation_max_abs"] <= CHECK_TOLERANCE
        and len(daily) == N_DAYS
        and len(ledger) == N_DAYS * N_INTERVALS
    )
    return StrategyRun(
        config=config,
        runtime_seconds=time.perf_counter() - started,
        ledger=ledger,
        daily=daily,
        metrics=metrics,
        validation=validation,
    )


def load_json_if_present(path: Path) -> dict[str, Any] | None:
    if not path.is_file():
```

```
        return None
    return json.loads(path.read_text(encoding="utf-8"))


def comparison_rows(main_run: StrategyRun, baseline_run: StrategyRun) -> pd.DataFrame:
    records: list[dict[str, Any]] = []
    old = load_json_if_present(
        PROJECT_ROOT
        / "results/Q2/experiments/round1/metrics/main_metrics.json"
    )
    replay = load_json_if_present(
        PROJECT_ROOT
        / "results/Q2/experiments/round2_same_contract_feedback/metrics/feedback_replay_metrics.json"
    )
    if old is not None:
        records.append(
            {
                "strategy": "round1_old_fixed_execution",
                "role": "legacy_comparator",
                "contract_cost_yuan": old["contract_cost_yuan"],
                "emergency_cost_yuan": old["emergency_cost_yuan"],
                "actual_total_cost_yuan": old["actual_total_cost_yuan"],
                "emergency_purchase_kwh": old["emergency_purchase_kwh"],
                "initial_soc_kwh": old["soc_2025_02_01_start_kwh"],
                "terminal_soc_kwh": old["soc_2025_12_31_end_kwh"],
            }
        )
    if replay is not None:
        records.append(
            {
                "strategy": "round2_same_contract_feedback",
                "role": "diagnostic",
                "contract_cost_yuan": replay["feedback_contract_cost_yuan"],
                "emergency_cost_yuan": replay["feedback_emergency_cost_yuan"],
                "actual_total_cost_yuan": replay["feedback_total_cost_yuan"],
                "emergency_purchase_kwh": replay["feedback_emergency_purchase_kwh"],
                "initial_soc_kwh": replay["initial_soc_kwh"],
                "terminal_soc_kwh": replay["feedback_terminal_soc_kwh"],
            }
        )
    for run in (main_run, baseline_run):
        records.append(
            {
                "strategy": run.config.strategy_id,
                "role": run.config.role,
                "contract_cost_yuan": run.metrics["contract_cost_yuan"],
                "emergency_cost_yuan": run.metrics["emergency_cost_yuan"],
                "actual_total_cost_yuan": run.metrics["actual_total_cost_yuan"],
                "emergency_purchase_kwh": run.metrics["emergency_purchase_kwh"],
                "initial_soc_kwh": run.metrics["soc_2025_02_01_start_kwh"],
                "terminal_soc_kwh": run.metrics["soc_2025_12_31_end_kwh"],
            }
        )
    comparison = pd.DataFrame(records)
    baseline_cost = float(baseline_run.metrics["actual_total_cost_yuan"])
    comparison["saving_vs_B2_yuan"] = baseline_cost - comparison["actual_total_cost_yuan"]
    comparison["saving_vs_B2_pct"] = (
        comparison["saving_vs_B2_yuan"] / baseline_cost * 100.0
    )
    return comparison


def main() -> None:
    started = time.perf_counter()
    TABLE_DIR.mkdir(parents=True, exist_ok=True)
    METRIC_DIR.mkdir(parents=True, exist_ok=True)
    data = load_q2_data()
    forecasts = build_forecasts(data, "exponential")
```

```
    margins = build_risk_margins(forecasts, 0.8)

    main_run = run_full_feedback_strategy(data, forecasts, margins, MAIN_CONFIG)
    baseline_run = run_full_feedback_strategy(data, forecasts, margins, BASELINE_CONFIG)
    comparison = comparison_rows(main_run, baseline_run)

    main_ledger_path = TABLE_DIR / "q2_M2_interval_ledger.csv"
    baseline_ledger_path = TABLE_DIR / "q2_B2_interval_ledger.csv"
    main_daily_path = TABLE_DIR / "q2_M2_daily_summary.csv"
    baseline_daily_path = TABLE_DIR / "q2_B2_daily_summary.csv"
    comparison_path = TABLE_DIR / "q2_full_feedback_comparison.csv"
    save_frame(main_run.ledger, main_ledger_path)
    save_frame(baseline_run.ledger, baseline_ledger_path)
    save_frame(main_run.daily, main_daily_path)
    save_frame(baseline_run.daily, baseline_daily_path)
    save_frame(comparison, comparison_path)

    evaluation_ledger = main_run.ledger[main_run.ledger["date"] >= "2025-02-01"]
    events_path = TABLE_DIR / "q2_M2_emergency_events.csv"
    storage_path = TABLE_DIR / "q2_M2_storage_4h.csv"
    save_frame(emergency_events(evaluation_ledger), events_path)
    save_frame(storage_four_hour_summary(evaluation_ledger), storage_path)
    table1, table2, table3 = specified_date_tables(main_run)
    table1_path = TABLE_DIR / "q2_M2_specified_date_table1.csv"
    table2_path = TABLE_DIR / "q2_M2_specified_date_table2.csv"
    table3_path = TABLE_DIR / "q2_M2_specified_date_table3.csv"
    save_frame(table1, table1_path)
    save_frame(table2, table2_path)
    save_frame(table3, table3_path)

    main_metrics_path = METRIC_DIR / "M2_main_metrics.json"
    baseline_metrics_path = METRIC_DIR / "B2_baseline_metrics.json"
    validation_path = METRIC_DIR / "validation_checks.json"
    write_json(main_metrics_path, main_run.metrics)
    write_json(baseline_metrics_path, baseline_run.metrics)
    validation = {
        "schema_version": 1,
        "main": main_run.validation,
        "baseline": baseline_run.validation,
        "direct_comparison": {
            "rules_differing": ["risk_margin_enabled"],
            "main_total_cost_yuan": main_run.metrics["actual_total_cost_yuan"],
            "baseline_total_cost_yuan": baseline_run.metrics["actual_total_cost_yuan"],
            "main_saving_yuan": (
                baseline_run.metrics["actual_total_cost_yuan"]
                - main_run.metrics["actual_total_cost_yuan"]
            ),
        },
        "all_pass": bool(
            main_run.validation["all_pass"] and baseline_run.validation["all_pass"]
        ),
    }
    write_json(validation_path, validation)

    output_paths = [
        main_ledger_path,
        baseline_ledger_path,
        main_daily_path,
        baseline_daily_path,
        comparison_path,
        events_path,
        storage_path,
        table1_path,
        table2_path,
        table3_path,
        main_metrics_path,
        baseline_metrics_path,
        validation_path,
```

```
]
run_summary = {
    "schema_version": 1,
    "question": "Q2",
    "round": "round3_full_feedback",
    "approved_decision_id": "q2_method_choice_daily_replan_actual_soc_feedback",
    "methods": [
        {
            "method_id": "M2",
            "role": "main",
            "strategy_id": MAIN_CONFIG.strategy_id,
            "script": "code/Q2/q2_full_feedback.py",
            "status": "success" if main_run.validation["all_pass"] else "failed_validation",
            "execution_time_seconds": main_run.runtime_seconds,
            "metrics_summary": main_run.metrics,
        },
        {
            "method_id": "B2",
            "role": "usable_baseline",
            "strategy_id": BASELINE_CONFIG.strategy_id,
            "script": "code/Q2/q2_full_feedback.py",
            "status": "success" if baseline_run.validation["all_pass"] else "failed_validation",
            "execution_time_seconds": baseline_run.runtime_seconds,
            "metrics_summary": baseline_run.metrics,
        },
    ],
    "input_files": [
        {"path": str(PRICE_PATH), "sha256": sha256(PRICE_PATH)},
        {"path": str(ACTUAL_PATH), "sha256": sha256(ACTUAL_PATH)},
    ],
    "output_files": [
        str(path.relative_to(PROJECT_ROOT)).replace("\\", "/")
        for path in output_paths
    ],
    "comparison": validation["direct_comparison"],
    "output_degeneracy": {
        "M2_unique_daily_purchase_profiles": main_run.metrics[
            "unique_daily_purchase_profiles_rounded_6dp"
        ],
        "M2_emergency_intervals": main_run.metrics["emergency_interval_count"],
        "B2_unique_daily_purchase_profiles": baseline_run.metrics[
            "unique_daily_purchase_profiles_rounded_6dp"
        ],
        "B2_emergency_intervals": baseline_run.metrics["emergency_interval_count"],
    },
    "fallback_trigger": {
        "fallback_id": "F1",
        "condition": "unresolved daily stage-1 or stage-2 LP failure",
        "observed": bool(
            main_run.metrics["fallback_day_count"]
            or baseline_run.metrics["fallback_day_count"]
        ),
        "main_days": main_run.metrics["fallback_days"],
        "baseline_days": baseline_run.metrics["fallback_days"],
    },
    "warnings": [
        "M2 uses causal within-interval feedback but does not jointly optimize the feedback policy with the
          day-ahead LP.",
        "Round-1 formal workbook is not overwritten; a human result decision is required before replacement.",
    ],
    "errors": [],
    "environment": environment_info(),
    "random_seed": RANDOM_SEED,
    "validation_pass": validation["all_pass"],
    "execution_time_seconds": time.perf_counter() - started,
}
write_json(ROUND_DIR / "run_summary.json", run_summary)

```

```
    print(
        json.dumps(
            {
                "status": "success" if validation["all_pass"] else "failed_validation",
                "M2_contract_cost_yuan": main_run.metrics["contract_cost_yuan"],
                "M2_emergency_cost_yuan": main_run.metrics["emergency_cost_yuan"],
                "M2_total_cost_yuan": main_run.metrics["actual_total_cost_yuan"],
                "M2_emergency_purchase_kwh": main_run.metrics["emergency_purchase_kwh"],
                "M2_feb1_initial_soc_kwh": main_run.metrics["soc_2025_02_01_start_kwh"],
                "M2_year_end_soc_kwh": main_run.metrics["soc_2025_12_31_end_kwh"],
                "B2_total_cost_yuan": baseline_run.metrics["actual_total_cost_yuan"],
                "M2_saving_vs_B2_yuan": (
                    baseline_run.metrics["actual_total_cost_yuan"]
                    - main_run.metrics["actual_total_cost_yuan"]
                ),
                "validation_pass": validation["all_pass"],
                "runtime_seconds": run_summary["execution_time_seconds"],
            },
            ensure_ascii=False,
            indent=2,
        )
    )


if __name__ == "__main__":
    raise SystemExit(" 请从辅助材料根目录运行 run.py, 选择对应问题。")

```

D.6 问题三程序

D.6.1 code/q3_model.py

```
from __future__ import annotations

import argparse
import hashlib
import json
import math
import platform
import sys
import time as time_module
from dataclasses import dataclass
from datetime import date, datetime, time, timedelta
from pathlib import Path
from typing import Any

PROJECT_ROOT = Path(__file__).resolve().parents[1]

import numpy as np
import pandas as pd
import scipy
from openpyxl import load_workbook
from scipy.optimize import linprog
from scipy.sparse import csr_matrix, lil_matrix


N_DAYS = 365
N_INTERVALS = 144
RELEASE_HOURS = (0, 6, 12, 18)
RELEASE_STARTS = tuple(hour * 6 for hour in RELEASE_HOURS)
DELTA_HOURS = 1.0 / 6.0
ETA_CHARGE = 0.9
ETA_DISCHARGE = 0.9
SOC_INITIAL_KWH = 6000.0
SOC_MIN_KWH = 1200.0
SOC_MAX_KWH = 10800.0
```

```
INTERVAL_LIMIT_KWH = 5000.0 * DELTA_HOURS
EVALUATION_START_INDEX = 31
FINAL_DAY_INDEX = 364
PRIMARY_TOLERANCE = 1e-6
CHECK_TOLERANCE = 5e-5
EVENT_TOLERANCE = 1e-6
RANDOM_SEED = 2026

SOURCE_ROOT = PROJECT_ROOT / "data"
PRICE_PATH = SOURCE_ROOT / " 附件 1.xlsx"
ACTUAL_PATH = SOURCE_ROOT / " 附件 2.xlsx"
FORECAST_PATH = SOURCE_ROOT / " 附件 3.xlsx"
ROUND_DIR = PROJECT_ROOT / "results" / "Q3" / "experiments" / "round4"
TABLE_DIR = ROUND_DIR / "tables"
METRIC_DIR = ROUND_DIR / "metrics"


@dataclass(frozen=True)
class Q3Data:
    dates: list[date]
    price: np.ndarray
    load_kw: np.ndarray
    pv_kw: np.ndarray
    net_load_kwh: np.ndarray
    official_hourly_kw: np.ndarray
    interval_starts: list[str]
    interval_ends: list[str]
    interval_labels: list[str]
    hashes: dict[str, str]


@dataclass(frozen=True)
class ForecastArchive:
    archive_id: str
    load_kw: np.ndarray
    pv_kw: np.ndarray
    net_kwh: np.ndarray
    errors_kwh: np.ndarray
    valid: np.ndarray


@dataclass(frozen=True)
class PolicyConfig:
    strategy_id: str
    role: str
    alpha: float | None = 0.8
    gamma: float = 0.5
    selected_release_hours: tuple[int, ...] = RELEASE_HOURS
    allow_adjustment: bool = True
    fixed_da_storage: bool = False
    feedback_execution: bool = True
    forecast_source: str = "official"
    collect_ledger: bool = False


@dataclass
class LPSolution:
    purchase: np.ndarray
    charge: np.ndarray
    discharge: np.ndarray
    planned_emergency: np.ndarray
    planned_unused: np.ndarray
    soc_end: np.ndarray
    stage1_objective: float
    stage2_objective: float
    throughput: float
    max_balance_residual: float
    max_storage_residual: float
```

```
    max_adjustment_decomposition_residual: float
    primary_tolerance: float
    stage1_status: str
    stage2_status: str


@dataclass
class StrategyRun:
    config: PolicyConfig
    runtime_seconds: float
    metrics: dict[str, Any]
    validation: dict[str, Any]
    daily: pd.DataFrame
    ledger: pd.DataFrame | None
    adjustments: pd.DataFrame | None


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def write_json(path: Path, payload: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")


def save_frame(frame: pd.DataFrame, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    frame.to_csv(path, index=False, encoding="utf-8-sig", float_format="%.10f")


def _clock(value: Any) -> str:
    if isinstance(value, time):
        return f"{value.hour:02d}:{value.minute:02d}"
    return str(value).strip()


def _fmt_minutes(minutes: int) -> str:
    if minutes == 1440:
        return "24:00"
    hour, minute = divmod(minutes, 60)
    return f"{hour:02d}:{minute:02d}"


def _expected_end_labels(index: int) -> set[str]:
    minutes = (index + 1) * 10
    if minutes == 1440:
        return {"0:00+1", "00:00+1", "24:00"}
    value = _fmt_minutes(minutes)
    return {value, value.lstrip("0")}


def _to_date(value: Any) -> date:
    if isinstance(value, datetime):
        return value.date()
    if isinstance(value, date):
        return value
    return datetime.strptime(str(value).strip(), "%Y-%m-%d").date()


def load_data() -> Q3Data:
    for path in (PRICE_PATH, ACTUAL_PATH, FORECAST_PATH):
        if not path.is_file():
            raise FileNotFoundError(path)
```

```
    price_wb = load_workbook(PRICE_PATH, read_only=True, data_only=True)
    price_rows = list(price_wb.active.values)
    if tuple(price_rows[0]) != (" 时间", " 电价", " 小区负载", " 光伏发电预测功率"):
        raise ValueError(f"Unexpected attachment 1 header: {price_rows[0]!r}")
    if len(price_rows) != N_INTERVALS + 1:
        raise ValueError("Attachment 1 must contain 144 intervals")
    price = np.asarray([row[1] for row in price_rows[1:]], dtype=float)
    for index, row in enumerate(price_rows[1:]):
        if _clock(row[0]) not in _expected_end_labels(index):
            raise ValueError(f"Attachment 1 time mismatch at interval {index + 1}")
    if not np.isfinite(price).all() or np.any(price <= 0):
        raise ValueError("Prices must be finite and positive")

    actual_wb = load_workbook(ACTUAL_PATH, read_only=True, data_only=True)
    if actual_wb.sheetnames != [" 小区负载", " 光伏发电实际功率"]:
        raise ValueError(f"Unexpected attachment 2 sheets: {actual_wb.sheetnames}")
    matrices: dict[str, np.ndarray] = {}
    dates: list[date] | None = None
    labels: list[str] | None = None
    for sheet_name in actual_wb.sheetnames:
        rows = list(actual_wb[sheet_name].values)
        if len(rows) != N_DAYS + 1 or len(rows[0]) != N_INTERVALS + 1:
            raise ValueError(f"Unexpected attachment 2 shape in {sheet_name}")
        current_dates = [_to_date(row[0]) for row in rows[1:]]
        current_labels = [_clock(value) for value in rows[0][1:]]
        if dates is None:
            dates, labels = current_dates, current_labels
        elif dates != current_dates or labels != current_labels:
            raise ValueError("Attachment 2 axes differ between sheets")
        matrices[sheet_name] = np.asarray([row[1:] for row in rows[1:]], dtype=float)
    assert dates is not None and labels is not None
    expected_dates = [date(2025, 1, 1) + timedelta(days=index) for index in range(N_DAYS)]
    if dates != expected_dates:
        raise ValueError("Attachment 2 does not cover every day of 2025 in order")
    for index, label in enumerate(labels):
        if label not in _expected_end_labels(index):
            raise ValueError(f"Attachment 2 time mismatch at interval {index + 1}")
    load_kw = matrices[" 小区负载"]
    pv_kw = matrices[" 光伏发电实际功率"]
    if not np.isfinite(load_kw).all() or not np.isfinite(pv_kw).all():
        raise ValueError("Attachment 2 contains missing/non-finite values")
    if np.any(load_kw < 0) or np.any(pv_kw < 0):
        raise ValueError("Attachment 2 physical powers must be nonnegative")

    forecast_wb = load_workbook(FORECAST_PATH, read_only=True, data_only=True)
    rows = list(forecast_wb.active.values)
    expected_header = (" 日期", " 预报时刻", *[f" 预报 {lead} 小时" for lead in range(1, 25)])
    if tuple(rows[0]) != expected_header or len(rows) != N_DAYS * 4 + 1:
        raise ValueError("Unexpected attachment 3 header or row count")
    official = np.full((N_DAYS, 4, 24), np.nan, dtype=float)
    current_date: date | None = None
    seen: set[tuple[date, int]] = set()
    for row in rows[1:]:
        if row[0] not in (None, ""):
            current_date = datetime.strptime(str(row[0]).strip(), "%Y-%m-%d").date()
        if current_date is None:
            raise ValueError("Attachment 3 begins with a blank date")
        release_text = str(row[1]).strip()
        try:
            release_index = [f"{hour}:00" for hour in RELEASE_HOURS].index(release_text)
        except ValueError as exc:
            raise ValueError(f"Unexpected release time {release_text!r}") from exc
        day_index = (current_date - date(2025, 1, 1)).days
        if not 0 <= day_index < N_DAYS or (current_date, release_index) in seen:
            raise ValueError("Attachment 3 has duplicate or out-of-range release")
        values = np.asarray(row[2:], dtype=float)
        if len(values) != 24 or not np.isfinite(values).all() or np.any(values < 0):
```

```
            raise ValueError(f"Invalid attachment 3 forecast at {current_date} {release_text}")
        official[day_index, release_index] = values
        seen.add((current_date, release_index))
    if np.isnan(official).any() or len(seen) != N_DAYS * 4:
        raise ValueError("Attachment 3 release coverage is incomplete")

    starts = [_fmt_minutes(index * 10) for index in range(N_INTERVALS)]
    ends = [_fmt_minutes((index + 1) * 10) for index in range(N_INTERVALS)]
    return Q3Data(
        dates=dates,
        price=price,
        load_kw=load_kw,
        pv_kw=pv_kw,
        net_load_kwh=(load_kw - pv_kw) * DELTA_HOURS,
        official_hourly_kw=official,
        interval_starts=starts,
        interval_ends=ends,
        interval_labels=[f"{start}-{end}" for start, end in zip(starts, ends)],
        hashes={"attachment1": sha256(PRICE_PATH), "attachment2": sha256(ACTUAL_PATH), "attachment3":
          sha256(FORECAST_PATH)},
    )


def build_load_forecast(data: Q3Data) -> tuple[np.ndarray, np.ndarray]:
    predicted = np.full_like(data.load_kw, np.nan)
    valid = np.zeros(N_DAYS, dtype=bool)
    for day_index in range(1, N_DAYS):
        candidates = np.arange(max(0, day_index - 28), day_index, dtype=int)
        target_weekday = data.dates[day_index].weekday()
        same_weekday = np.asarray(
            [idx for idx in candidates if data.dates[idx].weekday() == target_weekday], dtype=int
        )
        if len(same_weekday):
            indices = same_weekday
        else:
            target_weekend = target_weekday >= 5
            same_day_type = np.asarray(
                [idx for idx in candidates if (data.dates[idx].weekday() >= 5) == target_weekend], dtype=int
            )
            indices = (
                same_day_type
                if len(same_day_type)
                else np.arange(max(0, day_index - 7), day_index, dtype=int)
            )
        weights = np.exp(-(day_index - indices) / 7.0)
        predicted[day_index] = np.average(data.load_kw[indices], axis=0, weights=weights)
        valid[day_index] = True
    return predicted, valid


def _interpolate_official_day(data: Q3Data, day_index: int, release_index: int) -> np.ndarray:
    release_hour = RELEASE_HOURS[release_index]
    start = RELEASE_STARTS[release_index]
    if release_hour == 0:
        anchor = 0.0 if day_index == 0 else float(data.pv_kw[day_index - 1, -1])
    else:
        anchor = float(data.pv_kw[day_index, start - 1])
    knots = np.r_[anchor, data.official_hourly_kw[day_index, release_index]]
    targets = (np.arange(start, N_INTERVALS, dtype=float) + 1.0) * DELTA_HOURS - release_hour
    lower = np.floor(targets + 1e-12).astype(int)
    lower = np.minimum(lower, 23)
    fraction = targets - lower
    values = (1.0 - fraction) * knots[lower] + fraction * knots[lower + 1]
    result = np.full(N_INTERVALS, np.nan, dtype=float)
    result[start:] = np.maximum(values, 0.0)
    return result


```

```
def build_forecast_archive(data: Q3Data, source: str) -> ForecastArchive:
    load_forecast, valid_days = build_load_forecast(data)
    pv_forecast = np.full((N_DAYS, 4, N_INTERVALS), np.nan, dtype=float)
    if source == "official":
        for day_index in range(N_DAYS):
            for release_index in range(4):
                pv_forecast[day_index, release_index] = _interpolate_official_day(data, day_index, release_index)
    elif source == "historical":
        for day_index in range(1, N_DAYS):
            indices = np.arange(max(0, day_index - 7), day_index, dtype=int)
            weights = np.exp(-(day_index - indices) / 7.0)
            predicted = np.maximum(0.0, np.average(data.pv_kw[indices], axis=0, weights=weights))
            for release_index, start in enumerate(RELEASE_STARTS):
                pv_forecast[day_index, release_index, start:] = predicted[start:]
    else:
        raise ValueError(f"Unknown forecast source {source!r}")

    net = np.full_like(pv_forecast, np.nan)
    errors = np.full_like(pv_forecast, np.nan)
    valid = np.zeros((N_DAYS, 4, N_INTERVALS), dtype=bool)
    for day_index in range(1, N_DAYS):
        for release_index, start in enumerate(RELEASE_STARTS):
            net[day_index, release_index, start:] = (
                load_forecast[day_index, start:] - pv_forecast[day_index, release_index, start:]
            ) * DELTA_HOURS
            errors[day_index, release_index, start:] = data.net_load_kwh[day_index, start:] - net[day_index,
              release_index, start:]
            valid[day_index, release_index, start:] = valid_days[day_index]
    return ForecastArchive(source, load_forecast, pv_forecast, net, errors, valid)


def build_risk_margins(archive: ForecastArchive, alpha: float | None) -> np.ndarray:
    margins = np.zeros_like(archive.net_kwh)
    if alpha is None:
        return margins
    for day_index in range(1, N_DAYS):
        prior = np.arange(max(1, day_index - 28), day_index, dtype=int)
        if len(prior) < 7:
            continue
        for release_index, start in enumerate(RELEASE_STARTS):
            for interval in range(start, N_INTERVALS):
                eligible = prior[archive.valid[prior, release_index, interval]]
                if len(np.unique(eligible)) < 7:
                    continue
                value = float(np.quantile(archive.errors_kwh[eligible, release_index, interval], alpha,
                  method="linear"))
                margins[day_index, release_index, interval] = max(0.0, value)
    return margins


def incremental_adjustment_fee(
    new_contract: np.ndarray,
    previous_contract: np.ndarray,
    price: np.ndarray,
) -> np.ndarray:
    increase = np.maximum(new_contract - previous_contract, 0.0)
    decrease = np.maximum(previous_contract - new_contract, 0.0)
    return 1.5 * price * increase - 0.5 * price * decrease


class RollingLP:
    def __init__(self, price: np.ndarray) -> None:
        self.price = np.asarray(price, dtype=float)
        self.da_cache: dict[int, tuple[csr_matrix, list[tuple[float | None, float | None]], np.ndarray]] = {}
        self.tr_cache: dict[int, tuple[csr_matrix, np.ndarray]] = {}

    def _da_structure(self, start: int) -> tuple[csr_matrix, list[tuple[float | None, float | None]], np.ndarray]:
        if start in self.da_cache:
```

```
            return self.da_cache[start]
        m = N_INTERVALS - start
        a_eq = lil_matrix((2 * m, 5 * m), dtype=float)
        for t in range(m):
            a_eq[t, [t, m + t, 2 * m + t, 3 * m + t]] = [1.0, -1.0, 1.0, -1.0]
            a_eq[m + t, [m + t, 2 * m + t, 4 * m + t]] = [-ETA_CHARGE, 1.0 / ETA_DISCHARGE, 1.0]
            if t:
                a_eq[m + t, 4 * m + t - 1] = -1.0
        bounds = (
            [(0.0, None)] * m
            + [(0.0, INTERVAL_LIMIT_KWH)] * m
            + [(0.0, INTERVAL_LIMIT_KWH)] * m
            + [(0.0, None)] * m
            + [(SOC_MIN_KWH, SOC_MAX_KWH)] * m
        )
        secondary = np.r_[np.zeros(m), np.ones(m), np.ones(m), np.zeros(2 * m)]
        value = (a_eq.tocsr(), bounds, secondary)
        self.da_cache[start] = value
        return value

    def solve_da(self, planned_net: np.ndarray, initial_soc: float, terminal_value: float) -> LPSolution:
        start = 0
        m = N_INTERVALS
        a_eq, bounds, secondary = self._da_structure(start)
        b_eq = np.r_[planned_net, np.zeros(m)]
        b_eq[m] = initial_soc
        primary = np.r_[self.price, np.zeros(4 * m)]
        primary[-1] = -terminal_value
        stage1 = linprog(primary, A_eq=a_eq, b_eq=b_eq, bounds=bounds, method="highs")
        if not stage1.success:
            raise RuntimeError(f"DA stage1 failed: {stage1.message}")
        stage2 = linprog(
            secondary,
            A_ub=csr_matrix(primary.reshape(1, -1)),
            b_ub=np.asarray([float(stage1.fun) + PRIMARY_TOLERANCE]),
            A_eq=a_eq,
            b_eq=b_eq,
            bounds=bounds,
            method="highs",
        )
        if not stage2.success:
            raise RuntimeError(f"DA stage2 failed: {stage2.message}")
        q, charge, discharge, unused, soc = stage2.x.reshape(5, m)
        balance = q - charge + discharge - unused - planned_net
        prior_soc = np.r_[initial_soc, soc[:-1]]
        storage = soc - prior_soc - ETA_CHARGE * charge + discharge / ETA_DISCHARGE
        return LPSolution(
            q,
            charge,
            discharge,
            np.zeros(m),
            unused,
            soc,
            float(stage1.fun),
            float(primary @ stage2.x),
            float(charge.sum() + discharge.sum()),
            float(np.max(np.abs(balance))),
            float(np.max(np.abs(storage))),
            0.0,
            PRIMARY_TOLERANCE,
            str(stage1.message),
            str(stage2.message),
        )

    def _tr_structure(self, start: int) -> tuple[csr_matrix, np.ndarray]:
        if start in self.tr_cache:
            return self.tr_cache[start]
        m = N_INTERVALS - start
```

```
        # Variables by m-block: y, increase, decrease, charge, discharge,
        # planned emergency, planned unused, SOC.
        a_eq = lil_matrix((3 * m, 8 * m), dtype=float)
        for t in range(m):
            a_eq[t, [t, 3 * m + t, 4 * m + t, 5 * m + t, 6 * m + t]] = [1.0, -1.0, 1.0, 1.0, -1.0]
            a_eq[m + t, [3 * m + t, 4 * m + t, 7 * m + t]] = [-ETA_CHARGE, 1.0 / ETA_DISCHARGE, 1.0]
            if t:
                a_eq[m + t, 7 * m + t - 1] = -1.0
            a_eq[2 * m + t, [t, m + t, 2 * m + t]] = [1.0, -1.0, 1.0]
        secondary = np.r_[np.zeros(3 * m), np.ones(m), np.ones(m), np.zeros(3 * m)]
        value = (a_eq.tocsr(), secondary)
        self.tr_cache[start] = value
        return value

    def solve_tr(
        self,
        start: int,
        planned_net: np.ndarray,
        initial_soc: float,
        previous_contract: np.ndarray,
        terminal_value: float,
        allow_adjustment: bool,
        fixed_charge: np.ndarray | None = None,
        fixed_discharge: np.ndarray | None = None,
    ) -> LPSolution:
        m = N_INTERVALS - start
        a_eq, secondary = self._tr_structure(start)
        local_price = self.price[start:]
        reference = previous_contract[start:]
        b_eq = np.r_[planned_net, np.zeros(m), reference]
        b_eq[m] = initial_soc
        primary = np.r_[
            np.zeros(m),
            1.5 * local_price,
            -0.5 * local_price,
            np.zeros(2 * m),
            5.0 * local_price,
            np.zeros(2 * m),
        ]
        primary[-1] = -terminal_value
        y_bounds = [(0.0, None)] * m if allow_adjustment else [(float(value), float(value)) for value in
          reference]
        charge_bounds = (
            [(0.0, INTERVAL_LIMIT_KWH)] * m
            if fixed_charge is None
            else [(float(value), float(value)) for value in fixed_charge]
        )
        discharge_bounds = (
            [(0.0, INTERVAL_LIMIT_KWH)] * m
            if fixed_discharge is None
            else [(float(value), float(value)) for value in fixed_discharge]
        )
        bounds = (
            y_bounds
            + [(0.0, None)] * (2 * m)
            + charge_bounds
            + discharge_bounds
            + [(0.0, None)] * m
            + [(0.0, None)] * m
            + [(SOC_MIN_KWH, SOC_MAX_KWH)] * m
        )
        stage1 = linprog(
            primary,
            A_eq=a_eq,
            b_eq=b_eq,
            bounds=bounds,
            method="highs",
        )
```

```
        if not stage1.success:
            raise RuntimeError(f"TR {RELEASE_HOURS[RELEASE_STARTS.index(start)]}:00 stage1 failed:
              {stage1.message}")
        stage2 = linprog(
            secondary,
            A_ub=csr_matrix(primary.reshape(1, -1)),
            b_ub=np.asarray([float(stage1.fun) + PRIMARY_TOLERANCE]),
            A_eq=a_eq,
            b_eq=b_eq,
            bounds=bounds,
            method="highs",
        )
        if not stage2.success:
            raise RuntimeError(f"TR {RELEASE_HOURS[RELEASE_STARTS.index(start)]}:00 stage2 failed:
              {stage2.message}")
        y, increase, decrease, charge, discharge, emergency, unused, soc = stage2.x.reshape(8, m)
        balance = y - charge + discharge + emergency - unused - planned_net
        prior_soc = np.r_[initial_soc, soc[:-1]]
        storage = soc - prior_soc - ETA_CHARGE * charge + discharge / ETA_DISCHARGE
        adjustment_decomposition = y - increase + decrease - reference
        adjustment_overlap = np.minimum(increase, decrease)
        return LPSolution(
            y,
            charge,
            discharge,
            emergency,
            unused,
            soc,
            float(stage1.fun),
            float(primary @ stage2.x),
            float(charge.sum() + discharge.sum()),
            float(np.max(np.abs(balance))),
            float(np.max(np.abs(storage))),
            float(max(np.max(np.abs(adjustment_decomposition)), np.max(adjustment_overlap))),
            PRIMARY_TOLERANCE,
            str(stage1.message),
            str(stage2.message),
        )


def _fallback_da(planned_net: np.ndarray, initial_soc: float) -> LPSolution:
    q = np.maximum(planned_net, 0.0)
    zeros = np.zeros(N_INTERVALS)
    return LPSolution(
        q,
        zeros.copy(),
        zeros.copy(),
        zeros.copy(),
        np.maximum(-planned_net, 0.0),
        np.full(N_INTERVALS, initial_soc),
        math.nan,
        math.nan,
        0.0,
        0.0,
        0.0,
        0.0,
        PRIMARY_TOLERANCE,
        "failed_fallback_activated",
        "failed_fallback_activated",
    )


def run_policy(
    data: Q3Data,
    archive: ForecastArchive,
    margins: np.ndarray,
    config: PolicyConfig,
) -> StrategyRun:
```

```
    started = time_module.perf_counter()
    solver = RollingLP(data.price)
    selected = set(config.selected_release_hours)
    daily_rows: list[dict[str, Any]] = []
    ledger_rows: list[dict[str, Any]] = []
    adjustment_event_rows: list[dict[str, Any]] = []
    current_soc = SOC_INITIAL_KWH
    max_checks: dict[str, float] = {}
    fallback_events: list[dict[str, Any]] = []
    plan_count = 0

    def update_max(name: str, value: float) -> None:
        if math.isfinite(value):
            max_checks[name] = max(max_checks.get(name, 0.0), float(value))

    for day_index, day in enumerate(data.dates):
        day_initial_soc = current_soc
        q = np.zeros(N_INTERVALS)
        final_y = np.zeros(N_INTERVALS)
        exec_charge = np.zeros(N_INTERVALS)
        exec_discharge = np.zeros(N_INTERVALS)
        exec_plan_emergency = np.zeros(N_INTERVALS)
        exec_plan_unused = np.zeros(N_INTERVALS)
        actual_emergency = np.zeros(N_INTERVALS)
        actual_unused = np.zeros(N_INTERVALS)
        adjustment_fee_by_interval = np.zeros(N_INTERVALS)
        adjustment_increase_by_interval = np.zeros(N_INTERVALS)
        adjustment_decrease_by_interval = np.zeros(N_INTERVALS)
        adjustment_count_by_interval = np.zeros(N_INTERVALS, dtype=int)
        soc_end = np.zeros(N_INTERVALS)
        plan_release = np.zeros(N_INTERVALS, dtype=int)
        terminal_value = 0.0 if day_index == FINAL_DAY_INDEX else config.gamma * ETA_DISCHARGE *
          float(np.median(data.price))

        if day_index == 0:
            da = _fallback_da(np.zeros(N_INTERVALS), current_soc)
            da.stage1_status = "cold_start_no_load_history"
            da.stage2_status = "cold_start_no_load_history"
        else:
            planned_da = archive.net_kwh[day_index, 0] + margins[day_index, 0]
            try:
                da = solver.solve_da(planned_da, current_soc, terminal_value)
                plan_count += 1
            except RuntimeError as exc:
                da = _fallback_da(planned_da, current_soc)
                fallback_events.append({"date": day.isoformat(), "release_hour": 0, "error": str(exc)})
        q[:] = da.purchase
        final_y[:] = q
        current_charge = da.charge.copy()
        current_discharge = da.discharge.copy()
        current_plan_emergency = da.planned_emergency.copy()
        current_plan_unused = da.planned_unused.copy()
        da_charge = da.charge.copy()
        da_discharge = da.discharge.copy()
        update_max("planning_balance_max_abs", da.max_balance_residual)
        update_max("storage_recurrence_max_abs", da.max_storage_residual)
        update_max("stage2_objective_excess", max(0.0, da.stage2_objective - da.stage1_objective) if day_index
          else 0.0)

        for release_index, (release_hour, start) in enumerate(zip(RELEASE_HOURS, RELEASE_STARTS)):
            if day_index > 0 and release_hour > 0 and release_hour in selected:
                planned = archive.net_kwh[day_index, release_index, start:] + margins[day_index, release_index,
                  start:]
                try:
                    tr = solver.solve_tr(
                        start,
                        planned,
                        current_soc,
```

```
                        final_y,
                        terminal_value,
                        config.allow_adjustment,
                        da_charge[start:] if config.fixed_da_storage else None,
                        da_discharge[start:] if config.fixed_da_storage else None,
                    )
                    plan_count += 1
                    previous_y = final_y[start:].copy()
                    new_y = tr.purchase
                    increase = np.maximum(new_y - previous_y, 0.0)
                    decrease = np.maximum(previous_y - new_y, 0.0)
                    incremental_fee = incremental_adjustment_fee(new_y, previous_y, data.price[start:])
                    changed = np.abs(new_y - previous_y) > EVENT_TOLERANCE
                    adjustment_fee_by_interval[start:] += incremental_fee
                    adjustment_increase_by_interval[start:] += increase
                    adjustment_decrease_by_interval[start:] += decrease
                    adjustment_count_by_interval[start:] += changed.astype(int)
                    if config.collect_ledger:
                        for local_index, interval in enumerate(range(start, N_INTERVALS)):
                            adjustment_event_rows.append(
                                {
                                    "strategy_id": config.strategy_id,
                                    "date": day.isoformat(),
                                    "release_hour": release_hour,
                                    "interval_index": interval + 1,
                                    "interval_label": data.interval_labels[interval],
                                    "price_yuan_per_kwh": data.price[interval],
                                    "previous_contract_kwh": previous_y[local_index],
                                    "new_contract_kwh": new_y[local_index],
                                    "increase_kwh": increase[local_index],
                                    "decrease_kwh": decrease[local_index],
                                    "incremental_adjustment_fee_yuan": incremental_fee[local_index],
                                    "changed": bool(changed[local_index]),
                                }
                            )
                    final_y[start:] = new_y
                    current_charge[start:] = tr.charge
                    current_discharge[start:] = tr.discharge
                    current_plan_emergency[start:] = tr.planned_emergency
                    current_plan_unused[start:] = tr.planned_unused
                    plan_release[start:] = release_hour
                    update_max("planning_balance_max_abs", tr.max_balance_residual)
                    update_max("storage_recurrence_max_abs", tr.max_storage_residual)
                    update_max("adjustment_decomposition_violation", tr.max_adjustment_decomposition_residual)
                    update_max("stage2_objective_excess", max(0.0, tr.stage2_objective - tr.stage1_objective))
                    update_max("planned_emergency_main_max", float(np.max(tr.planned_emergency)) if
                      config.allow_adjustment else 0.0)
                except RuntimeError as exc:
                    fallback_events.append({"date": day.isoformat(), "release_hour": release_hour, "error":
                      str(exc)})

            end = RELEASE_STARTS[release_index + 1] if release_index < 3 else N_INTERVALS
            for interval in range(start, end):
                # The contract is frozen between forecast releases, but the problem
                # does not freeze storage.  From Jan 2 onward, storage reacts only
                # to the current interval's observed net load and current SOC.  No
                # future actual value is visible to this rule.
                net_gap = data.net_load_kwh[day_index, interval] - final_y[interval]
                if day_index > 0 and config.feedback_execution:
                    charge = min(
                        max(-net_gap, 0.0),
                        INTERVAL_LIMIT_KWH,
                        max(0.0, (SOC_MAX_KWH - current_soc) / ETA_CHARGE),
                    )
                    discharge = min(
                        max(net_gap, 0.0),
                        INTERVAL_LIMIT_KWH,
                        max(0.0, ETA_DISCHARGE * (current_soc - SOC_MIN_KWH)),
```

```
                    )
                else:
                    charge = current_charge[interval]
                    discharge = current_discharge[interval]
                residual = net_gap + charge - discharge
                emergency = max(residual, 0.0)
                unused = max(-residual, 0.0)
                next_soc = current_soc + ETA_CHARGE * charge - discharge / ETA_DISCHARGE
                exec_charge[interval] = charge
                exec_discharge[interval] = discharge
                exec_plan_emergency[interval] = current_plan_emergency[interval]
                exec_plan_unused[interval] = current_plan_unused[interval]
                actual_emergency[interval] = emergency
                actual_unused[interval] = unused
                soc_end[interval] = next_soc
                update_max("soc_lower_violation", max(0.0, SOC_MIN_KWH - next_soc))
                update_max("soc_upper_violation", max(0.0, next_soc - SOC_MAX_KWH))
                update_max("charge_limit_violation", max(0.0, charge - INTERVAL_LIMIT_KWH))
                update_max("discharge_limit_violation", max(0.0, discharge - INTERVAL_LIMIT_KWH))
                update_max("simultaneous_charge_discharge_max", min(charge, discharge))
                update_max("actual_emergency_unused_overlap_max", min(emergency, unused))
                balance = (
                    final_y[interval]
                    + emergency
                    + data.pv_kw[day_index, interval] * DELTA_HOURS
                    + discharge
                    - data.load_kw[day_index, interval] * DELTA_HOURS
                    - charge
                    - unused
                )
                update_max("actual_balance_max_abs", abs(balance))
                recurrence = next_soc - current_soc - ETA_CHARGE * charge + discharge / ETA_DISCHARGE
                update_max("actual_storage_recurrence_max_abs", abs(recurrence))
                current_soc = next_soc

        original_cost_by_interval = data.price * q
        original_cost = float(original_cost_by_interval.sum())
        contract_fee_by_interval = original_cost_by_interval + adjustment_fee_by_interval
        adjustment_net = float(adjustment_fee_by_interval.sum())
        contract_cost = float(contract_fee_by_interval.sum())
        emergency_cost = float(5.0 * data.price @ actual_emergency)
        total_cost = contract_cost + emergency_cost
        daily_rows.append(
            {
                "strategy_id": config.strategy_id,
                "date": day.isoformat(),
                "initial_soc_kwh": day_initial_soc,
                "terminal_soc_kwh": current_soc,
                "original_contract_kwh": float(q.sum()),
                "final_contract_kwh": float(final_y.sum()),
                "adjustment_increase_kwh": float(adjustment_increase_by_interval.sum()),
                "adjustment_decrease_kwh": float(adjustment_decrease_by_interval.sum()),
                "adjustment_transaction_count": int(adjustment_count_by_interval.sum()),
                "actual_total_purchase_kwh": float(final_y.sum() + actual_emergency.sum()),
                "charge_kwh": float(exec_charge.sum()),
                "discharge_kwh": float(exec_discharge.sum()),
                "planned_emergency_kwh": float(exec_plan_emergency.sum()),
                "planned_unused_kwh": float(exec_plan_unused.sum()),
                "emergency_purchase_kwh": float(actual_emergency.sum()),
                "actual_unused_kwh": float(actual_unused.sum()),
                "original_contract_cost_yuan": original_cost,
                "adjustment_net_cost_yuan": adjustment_net,
                "contract_total_cost_yuan": contract_cost,
                "emergency_cost_yuan": emergency_cost,
                "actual_total_cost_yuan": total_cost,
                "risk_margin_kwh": float(
                    sum(
                        margins[
```

```
                            day_index,
                            RELEASE_HOURS.index(int(plan_release[interval])),
                            interval,
                        ]
                        for interval in range(N_INTERVALS)
                    )
                ),
                "fallback_used": any(item["date"] == day.isoformat() for item in fallback_events),
            }
        )

        if config.collect_ledger:
            prior_soc = np.r_[day_initial_soc, soc_end[:-1]]
            for interval in range(N_INTERVALS):
                executing_release_index = RELEASE_HOURS.index(int(plan_release[interval]))
                ledger_rows.append(
                    {
                        "strategy_id": config.strategy_id,
                        "date": day.isoformat(),
                        "interval_index": interval + 1,
                        "interval_start": data.interval_starts[interval],
                        "interval_end": data.interval_ends[interval],
                        "interval_label": data.interval_labels[interval],
                        "executing_release_hour": int(plan_release[interval]),
                        "price_yuan_per_kwh": data.price[interval],
                        "actual_load_kw": data.load_kw[day_index, interval],
                        "actual_pv_kw": data.pv_kw[day_index, interval],
                        "actual_net_load_kwh": data.net_load_kwh[day_index, interval],
                        "forecast_load_kw": archive.load_kw[day_index, interval] if day_index else np.nan,
                        "forecast_pv_kw": (
                            archive.pv_kw[day_index, executing_release_index, interval]
                            if day_index
                            else np.nan
                        ),
                        "forecast_net_load_kwh": (
                            archive.net_kwh[day_index, executing_release_index, interval]
                            if day_index
                            else np.nan
                        ),
                        "risk_margin_kwh": (
                            margins[day_index, executing_release_index, interval]
                            if day_index
                            else 0.0
                        ),
                        "original_contract_kwh": q[interval],
                        "final_contract_kwh": final_y[interval],
                        "adjustment_increase_kwh": adjustment_increase_by_interval[interval],
                        "adjustment_decrease_kwh": adjustment_decrease_by_interval[interval],
                        "adjustment_transaction_count": int(adjustment_count_by_interval[interval]),
                        "actual_charge_kwh": exec_charge[interval],
                        "actual_discharge_kwh": exec_discharge[interval],
                        "planned_emergency_kwh": exec_plan_emergency[interval],
                        "planned_unused_kwh": exec_plan_unused[interval],
                        "actual_emergency_kwh": actual_emergency[interval],
                        "actual_unused_kwh": actual_unused[interval],
                        "soc_start_kwh": prior_soc[interval],
                        "soc_end_kwh": soc_end[interval],
                        "original_contract_cost_yuan": original_cost_by_interval[interval],
                        "cumulative_adjustment_fee_yuan": adjustment_fee_by_interval[interval],
                        "contract_total_cost_yuan": contract_fee_by_interval[interval],
                        "adjustment_net_cost_yuan": adjustment_fee_by_interval[interval],
                        "emergency_cost_yuan": 5.0 * data.price[interval] * actual_emergency[interval],
                        "actual_total_cost_yuan": contract_fee_by_interval[interval] + 5.0 * data.price[interval]
                          * actual_emergency[interval],
                    }
                )

    daily = pd.DataFrame(daily_rows)
```

```
    evaluation = daily.iloc[EVALUATION_START_INDEX:]
    metrics = {
        "strategy_id": config.strategy_id,
        "role": config.role,
        "evaluation_start": data.dates[EVALUATION_START_INDEX].isoformat(),
        "evaluation_end": data.dates[-1].isoformat(),
        "evaluation_days": len(evaluation),
        "alpha": config.alpha,
        "gamma": config.gamma,
        "selected_release_hours": list(config.selected_release_hours),
        "allow_adjustment": config.allow_adjustment,
        "fixed_da_storage": config.fixed_da_storage,
        "feedback_execution": config.feedback_execution,
        "execution_mode": "current_interval_actual_feedback" if config.feedback_execution else
          "fixed_planned_dispatch",
        "forecast_source": config.forecast_source,
        "initial_soc_2025_02_01_kwh": float(daily.iloc[EVALUATION_START_INDEX]["initial_soc_kwh"]),
        "terminal_soc_2025_12_31_kwh": float(evaluation.iloc[-1]["terminal_soc_kwh"]),
        "original_contract_kwh": float(evaluation["original_contract_kwh"].sum()),
        "final_contract_kwh": float(evaluation["final_contract_kwh"].sum()),
        "adjustment_increase_kwh": float(evaluation["adjustment_increase_kwh"].sum()),
        "adjustment_decrease_kwh": float(evaluation["adjustment_decrease_kwh"].sum()),
        "adjustment_transaction_count": int(evaluation["adjustment_transaction_count"].sum()),
        "emergency_purchase_kwh": float(evaluation["emergency_purchase_kwh"].sum()),
        "actual_total_purchase_kwh": float(evaluation["actual_total_purchase_kwh"].sum()),
        "charge_kwh": float(evaluation["charge_kwh"].sum()),
        "discharge_kwh": float(evaluation["discharge_kwh"].sum()),
        "original_contract_cost_yuan": float(evaluation["original_contract_cost_yuan"].sum()),
        "adjustment_net_cost_yuan": float(evaluation["adjustment_net_cost_yuan"].sum()),
        "contract_total_cost_yuan": float(evaluation["contract_total_cost_yuan"].sum()),
        "emergency_cost_yuan": float(evaluation["emergency_cost_yuan"].sum()),
        "actual_total_cost_yuan": float(evaluation["actual_total_cost_yuan"].sum()),
        "risk_margin_kwh": float(evaluation["risk_margin_kwh"].sum()),
        "fallback_event_count": len(fallback_events),
        "planning_calls": plan_count,
    }
    fee_reconciliation = abs(metrics["contract_total_cost_yuan"] - metrics["original_contract_cost_yuan"] -
      metrics["adjustment_net_cost_yuan"])
    total_reconciliation = abs(metrics["actual_total_cost_yuan"] - metrics["contract_total_cost_yuan"] -
      metrics["emergency_cost_yuan"])
    cross_day = np.abs(daily["initial_soc_kwh"].to_numpy()[1:] - daily["terminal_soc_kwh"].to_numpy()[:-1])
    validation = {
        **max_checks,
        "fee_reconciliation_abs_yuan": float(fee_reconciliation),
        "total_cost_reconciliation_abs_yuan": float(total_reconciliation),
        "cross_day_soc_max_abs_kwh": float(np.max(cross_day)),
        "incremental_example_100_to_80": float(
            100.0 + incremental_adjustment_fee(np.array([80.0]), np.array([100.0]), np.array([1.0]))[0]
        ),
        "incremental_example_100_to_80_to_100": float(
            100.0
            + incremental_adjustment_fee(np.array([80.0]), np.array([100.0]), np.array([1.0]))[0]
            + incremental_adjustment_fee(np.array([100.0]), np.array([80.0]), np.array([1.0]))[0]
        ),
        "incremental_example_100_to_120": float(
            100.0 + incremental_adjustment_fee(np.array([120.0]), np.array([100.0]), np.array([1.0]))[0]
        ),
        "fallback_events": fallback_events,
    }
    hard_values = [
        validation.get("planning_balance_max_abs", 0.0),
        validation.get("storage_recurrence_max_abs", 0.0),
        validation.get("actual_balance_max_abs", 0.0),
        validation.get("actual_storage_recurrence_max_abs", 0.0),
        validation.get("soc_lower_violation", 0.0),
        validation.get("soc_upper_violation", 0.0),
        validation.get("charge_limit_violation", 0.0),
        validation.get("discharge_limit_violation", 0.0),
```

```
        validation.get("simultaneous_charge_discharge_max", 0.0),
        validation.get("adjustment_decomposition_violation", 0.0),
        validation["fee_reconciliation_abs_yuan"],
        validation["total_cost_reconciliation_abs_yuan"],
        validation["cross_day_soc_max_abs_kwh"],
    ]
    validation["hard_checks_pass"] = bool(max(hard_values) <= CHECK_TOLERANCE and len(fallback_events) == 0)
    return StrategyRun(
        config=config,
        runtime_seconds=time_module.perf_counter() - started,
        metrics=metrics,
        validation=validation,
        daily=daily,
        ledger=pd.DataFrame(ledger_rows) if config.collect_ledger else None,
        adjustments=pd.DataFrame(adjustment_event_rows) if config.collect_ledger else None,
    )


def emergency_events(ledger: pd.DataFrame) -> pd.DataFrame:
    events: list[dict[str, Any]] = []
    for day, group in ledger.groupby("date", sort=True):
        ordered = group.sort_values("interval_index").reset_index(drop=True)
        amounts = ordered["actual_emergency_kwh"].to_numpy(dtype=float)
        active = amounts > EVENT_TOLERANCE
        cursor = 0
        event_index = 0
        while cursor < N_INTERVALS:
            if not active[cursor]:
                cursor += 1
                continue
            start = cursor
            while cursor + 1 < N_INTERVALS and active[cursor + 1]:
                cursor += 1
            end = cursor
            event_index += 1
            events.append(
                {
                    "date": day,
                    "event_index": event_index,
                    "start_interval_index": start + 1,
                    "end_interval_index": end + 1,
                    "emergency_time_range": f"{ordered.loc[start, 'interval_start']}-{ordered.loc[end,
                      'interval_end']}",
                    "emergency_purchase_kwh": float(amounts[start : end + 1].sum()),
                }
            )
            cursor += 1
    return pd.DataFrame(events)


def storage_summary(ledger: pd.DataFrame) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    for day, group in ledger.groupby("date", sort=True):
        ordered = group.sort_values("interval_index").reset_index(drop=True)
        initial_soc = float(ordered.iloc[0]["soc_start_kwh"])
        terminal_soc = float(ordered.iloc[-1]["soc_end_kwh"])
        for block in range(6):
            section = ordered.iloc[block * 24 : (block + 1) * 24]
            rows.append(
                {
                    "date": day,
                    "block_index": block + 1,
                    "time_range": f"{block * 4:02d}:00-{(block + 1) * 4:02d}:00" if block < 5 else "20:00-24:00",
                    "charge_kwh": float(section["actual_charge_kwh"].sum()),
                    "discharge_kwh": float(section["actual_discharge_kwh"].sum()),
                    "soc_00_kwh": initial_soc if block == 0 else np.nan,
                    "soc_24_kwh": terminal_soc if block == 1 else np.nan,
                }
```

```
            )
    return pd.DataFrame(rows)


def specified_tables(run: StrategyRun, events: pd.DataFrame, storage: pd.DataFrame) -> tuple[pd.DataFrame,
    pd.DataFrame, pd.DataFrame]:
    if run.ledger is None:
        raise ValueError("Specified tables require a collected ledger")
    specified = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
    indices = [61, 73, 85, 97, 109, 121]
    rows: list[dict[str, Any]] = []
    for day in specified:
        detail = run.ledger[run.ledger["date"] == day].set_index("interval_index")
        daily = run.daily[run.daily["date"] == day].iloc[0]
        row: dict[str, Any] = {"date": day}
        for index in indices:
            row[f"{detail.loc[index, 'interval_label']}_final_contract_kwh"] = float(detail.loc[index,
              "final_contract_kwh"])
        row.update(
            {
                "daily_original_contract_kwh": float(daily["original_contract_kwh"]),
                "daily_final_contract_kwh": float(daily["final_contract_kwh"]),
                "daily_emergency_purchase_kwh": float(daily["emergency_purchase_kwh"]),
                "daily_actual_total_purchase_kwh": float(daily["actual_total_purchase_kwh"]),
                "daily_original_contract_cost_yuan": float(daily["original_contract_cost_yuan"]),
                "daily_adjustment_net_cost_yuan": float(daily["adjustment_net_cost_yuan"]),
                "daily_contract_total_cost_yuan": float(daily["contract_total_cost_yuan"]),
                "daily_emergency_cost_yuan": float(daily["emergency_cost_yuan"]),
                "daily_actual_total_cost_yuan": float(daily["actual_total_cost_yuan"]),
            }
        )
        rows.append(row)
    return (
        pd.DataFrame(rows),
        storage[storage["date"].isin(specified)].copy(),
        events[events["date"].isin(specified)].copy(),
    )


def _comparison_frame(runs: list[StrategyRun]) -> pd.DataFrame:
    rows = []
    for run in runs:
        row = {"strategy_id": run.config.strategy_id, "role": run.config.role, **run.metrics}
        row["runtime_seconds"] = run.runtime_seconds
        row["hard_checks_pass"] = run.validation["hard_checks_pass"]
        rows.append(row)
    frame = pd.DataFrame(rows)
    reference = float(frame.iloc[0]["actual_total_cost_yuan"])
    frame["cost_difference_vs_first_yuan"] = frame["actual_total_cost_yuan"] - reference
    frame["cost_change_vs_first_pct"] = frame["cost_difference_vs_first_yuan"] / reference * 100.0
    return frame


def forecast_diagnostics(data: Q3Data, archive: ForecastArchive) -> dict[str, Any]:
    eval_errors = archive.errors_kwh[EVALUATION_START_INDEX:]
    values = eval_errors[np.isfinite(eval_errors)]
    release_metrics = {}
    for index, hour in enumerate(RELEASE_HOURS):
        current = eval_errors[:, index]
        current = current[np.isfinite(current)]
        release_metrics[str(hour)] = {
            "n": int(len(current)),
            "mae_kwh": float(np.mean(np.abs(current))),
            "rmse_kwh": float(np.sqrt(np.mean(current**2))),
            "bias_actual_minus_forecast_kwh": float(np.mean(current)),
            "q80_kwh": float(np.quantile(current, 0.8, method="linear")),
        }
    integer_hour_checks = []
```

```
    for day_index in (31, 171, 364):
        for release_index, start in enumerate(RELEASE_STARTS):
            for lead in (1, 6, 12, 24):
                target = start + lead * 6 - 1
                if target < N_INTERVALS:
                    integer_hour_checks.append(
                        abs(archive.pv_kw[day_index, release_index, target] - data.official_hourly_kw[day_index,
                           release_index, lead - 1])
                    )
    return {
        "archive_id": archive.archive_id,
        "all_release_error_n": int(len(values)),
        "all_release_mae_kwh": float(np.mean(np.abs(values))),
        "release_metrics": release_metrics,
        "official_integer_hour_mapping_max_abs_kw": float(max(integer_hour_checks)) if archive.archive_id ==
          "official" else None,
    }


def environment_info() -> dict[str, Any]:
    return {
        "python": sys.version,
        "platform": platform.platform(),
        "numpy": np.__version__,
        "pandas": pd.__version__,
        "scipy": scipy.__version__,
        "solver": "scipy.optimize.linprog(method='highs')",
        "random_seed": RANDOM_SEED,
    }


def run_suite(suite: str) -> None:
    TABLE_DIR.mkdir(parents=True, exist_ok=True)
    METRIC_DIR.mkdir(parents=True, exist_ok=True)
    data = load_data()
    official = build_forecast_archive(data, "official")
    historical = build_forecast_archive(data, "historical")
    margin_cache: dict[tuple[str, float | None], np.ndarray] = {}

    def get_margins(archive: ForecastArchive, alpha: float | None) -> np.ndarray:
        key = (archive.archive_id, alpha)
        if key not in margin_cache:
            margin_cache[key] = build_risk_margins(archive, alpha)
        return margin_cache[key]

    core_configs = [
        PolicyConfig("M1I_official_S3_alpha0.8_gamma0.5", "main", collect_ledger=True),
        PolicyConfig("B1I_official_S3_b0_gamma0.5", "usable_baseline", alpha=None),
        PolicyConfig("D_I_frequency_S0", "diagnostic_frequency", selected_release_hours=(0,)),
        PolicyConfig("D_I_frequency_S1", "diagnostic_frequency", selected_release_hours=(0, 6)),
        PolicyConfig("D_I_frequency_S2", "diagnostic_frequency", selected_release_hours=(0, 6, 12)),
        PolicyConfig("D_I_no_adjustment", "diagnostic_mechanism", allow_adjustment=False),
        PolicyConfig("D_I_historical_pv", "diagnostic_mechanism", forecast_source="historical"),
        PolicyConfig("D_I_fixed_plan_execution", "diagnostic_mechanism", feedback_execution=False),
    ]
    sensitivity_configs = [
        PolicyConfig("S_I_alpha0.5", "sensitivity", alpha=0.5),
        PolicyConfig("S_I_alpha0.95", "sensitivity", alpha=0.95),
        PolicyConfig("S_I_gamma0", "sensitivity", gamma=0.0),
        PolicyConfig("S_I_gamma1", "sensitivity", gamma=1.0),
    ]
    configs = sensitivity_configs if suite == "sensitivity" else core_configs + (sensitivity_configs if suite ==
      "full" else [])

    runs: list[StrategyRun] = []
    for config in configs:
        archive = official if config.forecast_source == "official" else historical
        print(f"START {config.strategy_id}", flush=True)
```

```
        run = run_policy(data, archive, get_margins(archive, config.alpha), config)
        print(
            f"DONE {config.strategy_id} cost={run.metrics['actual_total_cost_yuan']:.2f} "
            f"emergency={run.metrics['emergency_purchase_kwh']:.3f} runtime={run.runtime_seconds:.2f}s",
            flush=True,
        )
        if not run.validation["hard_checks_pass"]:
            raise RuntimeError(f"Hard checks failed for {config.strategy_id}: {run.validation}")
        runs.append(run)

    if suite == "sensitivity":
        comparison = _comparison_frame(runs)
        save_frame(comparison, TABLE_DIR / "sensitivity_comparison.csv")
        write_json(METRIC_DIR / "sensitivity_validation.json", {run.config.strategy_id: run.validation for run in
          runs})
        summary_path = ROUND_DIR / "run_summary.json"
        if summary_path.is_file():
            prior = json.loads(summary_path.read_text(encoding="utf-8"))
            for run in runs:
                prior["methods"].append(
                    {
                        "method_id": run.config.strategy_id,
                        "role": run.config.role,
                        "script": "code/Q3/q3_model.py",
                        "status": "success",
                        "execution_time_seconds": run.runtime_seconds,
                        "input_files": [],
                        "output_files": ["tables/sensitivity_comparison.csv"],
                        "figure_files": [],
                        "metrics_summary": run.metrics,
                        "warnings": [],
                        "errors": [],
                    }
                )
                prior["validation"][run.config.strategy_id] = run.validation
            prior["suite"] = "core_plus_sensitivity"
            prior["output_files"].extend(["tables/sensitivity_comparison.csv",
               "metrics/sensitivity_validation.json"])
            write_json(summary_path, prior)
        return

    main_run = runs[0]
    baseline_run = runs[1]
    assert main_run.ledger is not None
    assert main_run.adjustments is not None
    evaluation_ledger = main_run.ledger[main_run.ledger["date"] >= "2025-02-01"].copy()
    evaluation_adjustments = main_run.adjustments[main_run.adjustments["date"] >= "2025-02-01"].copy()
    evaluation_daily = main_run.daily[main_run.daily["date"] >= "2025-02-01"].copy()
    events = emergency_events(evaluation_ledger)
    storage = storage_summary(evaluation_ledger)
    table1, table2, table3 = specified_tables(main_run, events, storage)

    save_frame(evaluation_ledger, TABLE_DIR / "q3_main_interval_ledger.csv")
    save_frame(evaluation_adjustments, TABLE_DIR / "q3_adjustment_events.csv")
    save_frame(evaluation_daily, TABLE_DIR / "q3_main_daily_summary.csv")
    save_frame(baseline_run.daily[baseline_run.daily["date"] >= "2025-02-01"], TABLE_DIR /
      "q3_baseline_daily_summary.csv")
    save_frame(events, TABLE_DIR / "q3_emergency_events.csv")
    save_frame(storage, TABLE_DIR / "q3_storage_4h.csv")
    save_frame(table1, TABLE_DIR / "q3_specified_date_table1.csv")
    save_frame(table2, TABLE_DIR / "q3_specified_date_table2.csv")
    save_frame(table3, TABLE_DIR / "q3_specified_date_table3.csv")

    comparison = _comparison_frame(runs)
    save_frame(comparison, TABLE_DIR / "strategy_comparison.csv")
    save_frame(comparison[comparison["role"] == "diagnostic_frequency"], TABLE_DIR / "frequency_comparison.csv")
    save_frame(comparison[comparison["role"] == "diagnostic_mechanism"], TABLE_DIR / "mechanism_comparison.csv")
    if suite == "full":
```

```
        save_frame(comparison[comparison["role"] == "sensitivity"], TABLE_DIR / "sensitivity_comparison.csv")

    write_json(METRIC_DIR / "main_metrics.json", main_run.metrics)
    write_json(METRIC_DIR / "baseline_metrics.json", baseline_run.metrics)
    write_json(METRIC_DIR / "validation_checks.json", main_run.validation)
    write_json(METRIC_DIR / "forecast_diagnostics.json", forecast_diagnostics(data, official))

    method_entries = []
    for run in runs:
        method_entries.append(
            {
                "method_id": run.config.strategy_id,
                "role": run.config.role,
                "script": "code/Q3/q3_model.py",
                "status": "success",
                "execution_time_seconds": run.runtime_seconds,
                "input_files": [str(PRICE_PATH.relative_to(PROJECT_ROOT.parent)),
                  str(ACTUAL_PATH.relative_to(PROJECT_ROOT.parent)),
                  str(FORECAST_PATH.relative_to(PROJECT_ROOT.parent))],
                "output_files": [],
                "figure_files": [],
                "metrics_summary": run.metrics,
                "warnings": [" 减购退款及逐次相对上一有效合同结算为 Q3 修订稿采用的解释性假设"],
                "errors": [],
            }
        )
    q_profiles = evaluation_ledger.pivot(index="date", columns="interval_index", values="original_contract_kwh")
    y_profiles = evaluation_ledger.pivot(index="date", columns="interval_index", values="final_contract_kwh")
    run_summary = {
        "schema_version": 1,
        "question": "Q3",
        "round": "round4",
        "implementation_target": "python",
        "random_seed": RANDOM_SEED,
        "approved_decision_id": "q3_method_choice_incremental_settlement_v4",
        "suite": suite,
        "methods": method_entries,
        "comparison": {
            "main_minus_baseline_cost_yuan": main_run.metrics["actual_total_cost_yuan"] -
              baseline_run.metrics["actual_total_cost_yuan"],
            "main_cost_change_vs_baseline_pct": (
                main_run.metrics["actual_total_cost_yuan"] / baseline_run.metrics["actual_total_cost_yuan"] - 1.0
            ) * 100.0,
        },
        "fallback_trigger": {
            "fallback_id": "F1",
            "condition": "any DA/TR stage failure or hard constraint violation",
            "observed": any(run.metrics["fallback_event_count"] for run in runs),
            "evidence": {run.config.strategy_id: run.validation["fallback_events"] for run in runs if
              run.validation["fallback_events"]},
        },
        "input_hashes": data.hashes,
        "environment": environment_info(),
        "validation": {run.config.strategy_id: run.validation for run in runs},
        "output_degeneracy": {
            "unique_original_contract_profiles_rounded_6dp": int(
                len(np.unique(np.round(q_profiles.to_numpy(), 6), axis=0))
            ),
            "unique_final_contract_profiles_rounded_6dp": int(
                len(np.unique(np.round(y_profiles.to_numpy(), 6), axis=0))
            ),
            "nonzero_adjustment_interval_count": int(
                np.count_nonzero(
                    np.abs(
                        evaluation_ledger["final_contract_kwh"].to_numpy()
                        - evaluation_ledger["original_contract_kwh"].to_numpy()
                    )
```

```
                    > 1e-6
                )
            ),
            "emergency_event_count": int(len(events)),
            "adjustment_release_interval_rows": int(len(evaluation_adjustments)),
            "adjustment_changed_rows": int(evaluation_adjustments["changed"].sum()),
        },
        "output_files": [
            "tables/q3_main_interval_ledger.csv",
            "tables/q3_adjustment_events.csv",
            "tables/q3_main_daily_summary.csv",
            "tables/q3_baseline_daily_summary.csv",
            "tables/q3_emergency_events.csv",
            "tables/q3_storage_4h.csv",
            "tables/q3_specified_date_table1.csv",
            "tables/q3_specified_date_table2.csv",
            "tables/q3_specified_date_table3.csv",
            "tables/strategy_comparison.csv",
            "metrics/main_metrics.json",
            "metrics/baseline_metrics.json",
            "metrics/validation_checks.json",
            "metrics/forecast_diagnostics.json",
        ],
    }
    write_json(ROUND_DIR / "run_summary.json", run_summary)


def main() -> None:
    parser = argparse.ArgumentParser(description="Solve Q3 rolling purchase and storage strategy")
    parser.add_argument("--suite", choices=("core", "sensitivity", "full"), default="core")
    args = parser.parse_args()
    run_suite(args.suite)


if __name__ == "__main__":
    raise SystemExit(" 请从辅助材料根目录运行 run.py, 选择对应问题。")

```

D.7 问题四及对照检验程序

D.7.1 code/q4_model.py

```
from __future__ import annotations

import argparse
import hashlib
import json
import math
import platform
import sys
import time as time_module
from dataclasses import dataclass
from datetime import date, datetime, time, timedelta
from pathlib import Path
from typing import Any


PROJECT_ROOT = Path(__file__).resolve().parents[1]

import numpy as np
import pandas as pd
import scipy
from openpyxl import load_workbook
from scipy.optimize import linprog
from scipy.sparse import csr_matrix, lil_matrix

```

```
25
26 N_DAYS = 365
27 N_INTERVALS = 144
28 RELEASE_HOURS = (0, 6, 12, 18)
29 RELEASE_STARTS = tuple(hour * 6 for hour in RELEASE_HOURS)
30 DELTA_HOURS = 1.0 / 6.0
31 ETA_CHARGE = 0.9
32 ETA_DISCHARGE = 0.9
33 SOC_INITIAL_KWH = 6000.0
34 SOC_MIN_KWH = 1200.0
35 SOC_MAX_KWH = 10800.0
36 INTERVAL_LIMIT_KWH = 5000.0 * DELTA_HOURS
37 EVALUATION_START_INDEX = 31
38 FINAL_DAY_INDEX = 364
39 SCENARIO_WINDOW_DAYS = 28
40 MIN_SCENARIO_DAYS = 7
41 GAMMA = 0.5
42 PRIMARY_TOLERANCE = 1e-6
43 PLANNING_CHECK_TOLERANCE = 2e-3
44 PHYSICAL_CHECK_TOLERANCE = 5e-5
45 EVENT_TOLERANCE = 1e-6
46 RANDOM_SEED = 2026
47 FORMAL_PRICE_INFORMATION = (
48     " 为完成题目给定数据下的离线重算，将附件 4 目标日完整价格路径视为规划输入；"
49     " 这不代表现实运行可提前获得未来实时价格。"
50 )
51
52 SOURCE_ROOT = PROJECT_ROOT / "data"
53 ACTUAL_PATH = SOURCE_ROOT / " 附件 2.xlsx"
54 FORECAST_PATH = SOURCE_ROOT / " 附件 3.xlsx"
55 FLOATING_PRICE_PATH = SOURCE_ROOT / " 附件 4.xlsx"
56
57
58 @dataclass(frozen=True)
59 class Q4Data:
60     dates: list[date]
61     load_kw: np.ndarray
62     pv_kw: np.ndarray
63     net_load_kwh: np.ndarray
64     actual_price: np.ndarray
65     official_hourly_kw: np.ndarray
66     interval_starts: list[str]
67     interval_ends: list[str]
68     interval_labels: list[str]
69     hashes: dict[str, str]
70
71
72 @dataclass(frozen=True)
73 class ForecastArchive:
74     archive_id: str
75     load_kw: np.ndarray
76     price_hat: np.ndarray
77     price_ratio: np.ndarray
78     pv_kw: np.ndarray
79     net_kwh: np.ndarray
80     errors_kwh: np.ndarray
81     valid: np.ndarray
82
83
84 @dataclass(frozen=True)
85 class ScenarioSet:
86     net_kwh: np.ndarray
87     price: np.ndarray
88     probabilities: np.ndarray
89     historical_day_indices: tuple[int, ...]
90     degenerate: bool
91     reason: str | None
92
```

```

@dataclass(frozen=True)
class PolicyConfig:
    strategy_id: str
    role: str
    question_variant: str
    forecast_source: str
    scenario_mode: str
    allow_adjustment: bool
    alpha: float | None = 0.8
    collect_evidence: bool = False


@dataclass
class LPSolution:
    contract: np.ndarray
    charge: np.ndarray
    discharge: np.ndarray
    soc_end: np.ndarray
    emergency: np.ndarray
    unused: np.ndarray
    stage1_objective: float
    stage2_primary_objective: float
    throughput: float
    max_balance_residual: float
    max_storage_residual: float
    max_emergency_formula_residual: float
    max_adjustment_residual: float
    max_charge_discharge_overlap: float
    stage1_status: str
    stage2_status: str


@dataclass
class StrategyRun:
    config: PolicyConfig
    runtime_seconds: float
    metrics: dict[str, Any]
    validation: dict[str, Any]
    daily: pd.DataFrame
    ledger: pd.DataFrame | None
    adjustments: pd.DataFrame | None
    plan_calls: pd.DataFrame | None
    plan_trajectory: pd.DataFrame | None


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def write_json(path: Path, payload: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")


def save_frame(frame: pd.DataFrame, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    frame.to_csv(path, index=False, encoding="utf-8-sig", float_format="%.10f")


def _clock(value: Any) -> str:
    if isinstance(value, time):
        return f"{value.hour:02d}:{value.minute:02d}"
    return str(value).strip()
```

```
161
162
163 def _fmt_minutes(minutes: int) -> str:
164     if minutes == 1440:
165         return "24:00"
166     hour, minute = divmod(minutes, 60)
167     return f"{hour:02d}:{minute:02d}"
168
169
170 def _expected_end_labels(index: int) -> set[str]:
171     minutes = (index + 1) * 10
172     if minutes == 1440:
173         return {"0:00+1", "00:00+1", "24:00"}
174     value = _fmt_minutes(minutes)
175     return {value, value.lstrip("0")}
176
177
178 def _to_date(value: Any) -> date:
179     if isinstance(value, datetime):
180         return value.date()
181     if isinstance(value, date):
182         return value
183     return datetime.strptime(str(value).strip(), "%Y-%m-%d").date()
184
185
186 def load_data() -> Q4Data:
187     for path in (ACTUAL_PATH, FORECAST_PATH, FLOATING_PRICE_PATH):
188         if not path.is_file():
189             raise FileNotFoundError(path)
190
191     actual_book = load_workbook(ACTUAL_PATH, read_only=True, data_only=True)
192     if actual_book.sheetnames != [" 小区负载", " 光伏发电实际功率"]:
193         raise ValueError(f"Unexpected attachment 2 sheets: {actual_book.sheetnames}")
194     matrices: dict[str, np.ndarray] = {}
195     dates: list[date] | None = None
196     labels: list[str] | None = None
197     for sheet_name in actual_book.sheetnames:
198         rows = list(actual_book[sheet_name].values)
199         if len(rows) != N_DAYS + 1 or len(rows[0]) != N_INTERVALS + 1:
200             raise ValueError(f"Unexpected attachment 2 shape in {sheet_name}")
201         current_dates = [_to_date(row[0]) for row in rows[1:]]
202         current_labels = [_clock(value) for value in rows[0][1:]]
203         if dates is None:
204             dates, labels = current_dates, current_labels
205         elif dates != current_dates or labels != current_labels:
206             raise ValueError("Attachment 2 axes differ between sheets")
207         matrices[sheet_name] = np.asarray([row[1:] for row in rows[1:]], dtype=float)
208     assert dates is not None and labels is not None
209     expected_dates = [date(2025, 1, 1) + timedelta(days=index) for index in range(N_DAYS)]
210     if dates != expected_dates:
211         raise ValueError("Attachment 2 does not cover every day of 2025 in order")
212     for index, label in enumerate(labels):
213         if label not in _expected_end_labels(index):
214             raise ValueError(f"Attachment 2 time mismatch at interval {index + 1}")
215     load_kw = matrices[" 小区负载"]
216     pv_kw = matrices[" 光伏发电实际功率"]
217     if not np.isfinite(load_kw).all() or not np.isfinite(pv_kw).all():
218         raise ValueError("Attachment 2 contains missing/non-finite values")
219     if np.any(load_kw < 0) or np.any(pv_kw < 0):
220         raise ValueError("Attachment 2 physical powers must be nonnegative")
221
222     price_book = load_workbook(FLOATING_PRICE_PATH, read_only=True, data_only=True)
223     price_rows = list(price_book.active.values)
224     if len(price_rows) != N_DAYS + 1 or len(price_rows[0]) != N_INTERVALS + 1:
225         raise ValueError("Unexpected attachment 4 shape")
226     price_dates = [_to_date(row[0]) for row in price_rows[1:]]
227     price_labels = [_clock(value) for value in price_rows[0][1:]]
228     if price_dates != dates or price_labels != labels:
```

```
        raise ValueError("Attachment 4 axes do not match attachment 2")
    actual_price = np.asarray([row[1:] for row in price_rows[1:]], dtype=float)
    if not np.isfinite(actual_price).all() or np.any(actual_price <= 0):
        raise ValueError("Attachment 4 prices must be finite and strictly positive")

    forecast_book = load_workbook(FORECAST_PATH, read_only=True, data_only=True)
    forecast_rows = list(forecast_book.active.values)
    expected_header = (" 日期", " 预报时刻", *[f" 预报 {lead} 小时" for lead in range(1, 25)])
    if tuple(forecast_rows[0]) != expected_header or len(forecast_rows) != N_DAYS * 4 + 1:
        raise ValueError("Unexpected attachment 3 header or row count")
    official = np.full((N_DAYS, 4, 24), np.nan, dtype=float)
    current_date: date | None = None
    seen: set[tuple[date, int]] = set()
    release_labels = [f"{hour}:00" for hour in RELEASE_HOURS]
    for row in forecast_rows[1:]:
        if row[0] not in (None, ""):
            current_date = datetime.strptime(str(row[0]).strip(), "%Y-%m-%d").date()
        if current_date is None:
            raise ValueError("Attachment 3 begins with a blank date")
        try:
            release_index = release_labels.index(str(row[1]).strip())
        except ValueError as exc:
            raise ValueError(f"Unexpected release time {row[1]!r}") from exc
        day_index = (current_date - date(2025, 1, 1)).days
        if not 0 <= day_index < N_DAYS or (current_date, release_index) in seen:
            raise ValueError("Attachment 3 has duplicate or out-of-range release")
        values = np.asarray(row[2:], dtype=float)
        if len(values) != 24 or not np.isfinite(values).all() or np.any(values < 0):
            raise ValueError(f"Invalid attachment 3 forecast at {current_date} {row[1]}")
        official[day_index, release_index] = values
        seen.add((current_date, release_index))
    if np.isnan(official).any() or len(seen) != N_DAYS * 4:
        raise ValueError("Attachment 3 release coverage is incomplete")

    starts = [_fmt_minutes(index * 10) for index in range(N_INTERVALS)]
    ends = [_fmt_minutes((index + 1) * 10) for index in range(N_INTERVALS)]
    return Q4Data(
        dates=dates,
        load_kw=load_kw,
        pv_kw=pv_kw,
        net_load_kwh=(load_kw - pv_kw) * DELTA_HOURS,
        actual_price=actual_price,
        official_hourly_kw=official,
        interval_starts=starts,
        interval_ends=ends,
        interval_labels=[f"{start}-{end}" for start, end in zip(starts, ends)],
        hashes={
            "attachment2": sha256(ACTUAL_PATH),
            "attachment3": sha256(FORECAST_PATH),
            "attachment4": sha256(FLOATING_PRICE_PATH),
        },
    )


def _historical_indices(dates: list[date], day_index: int) -> np.ndarray:
    candidates = np.arange(max(0, day_index - 28), day_index, dtype=int)
    target_weekday = dates[day_index].weekday()
    same_weekday = np.asarray([idx for idx in candidates if dates[idx].weekday() == target_weekday], dtype=int)
    if len(same_weekday):
        return same_weekday
    target_weekend = target_weekday >= 5
    same_type = np.asarray(
        [idx for idx in candidates if (dates[idx].weekday() >= 5) == target_weekend], dtype=int
    )
    if len(same_type):
        return same_type
    return np.arange(max(0, day_index - 7), day_index, dtype=int)

```

```
297
298 def build_load_and_price_forecasts(data: Q4Data) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
299     load_hat = np.full_like(data.load_kw, np.nan)
300     price_hat = np.full_like(data.actual_price, np.nan)
301     valid_days = np.zeros(N_DAYS, dtype=bool)
302     for day_index in range(1, N_DAYS):
303         indices = _historical_indices(data.dates, day_index)
304         if not len(indices):
305             continue
306         weights = np.exp(-(day_index - indices) / 7.0)
307         load_hat[day_index] = np.average(data.load_kw[indices], axis=0, weights=weights)
308         price_hat[day_index] = np.average(data.actual_price[indices], axis=0, weights=weights)
309         valid_days[day_index] = True
310     if np.any(price_hat[valid_days] <= 0):
311         raise ValueError("Causal price forecasts must remain strictly positive")
312     return load_hat, price_hat, valid_days
313
314
315 def _interpolate_official_day(data: Q4Data, day_index: int, release_index: int) -> np.ndarray:
316     release_hour = RELEASE_HOURS[release_index]
317     start = RELEASE_STARTS[release_index]
318     if release_hour == 0:
319         anchor = 0.0 if day_index == 0 else float(data.pv_kw[day_index - 1, -1])
320     else:
321         anchor = float(data.pv_kw[day_index, start - 1])
322     knots = np.r_[anchor, data.official_hourly_kw[day_index, release_index]]
323     targets = (np.arange(start, N_INTERVALS, dtype=float) + 1.0) * DELTA_HOURS - release_hour
324     lower = np.minimum(np.floor(targets + 1e-12).astype(int), 23)
325     fraction = targets - lower
326     values = (1.0 - fraction) * knots[lower] + fraction * knots[lower + 1]
327     result = np.full(N_INTERVALS, np.nan, dtype=float)
328     result[start:] = np.maximum(values, 0.0)
329     return result
330
331
332 def build_forecast_archive(data: Q4Data, source: str) -> ForecastArchive:
333     load_hat, price_hat, valid_days = build_load_and_price_forecasts(data)
334     pv_hat = np.full((N_DAYS, 4, N_INTERVALS), np.nan, dtype=float)
335     if source == "official":
336         for day_index in range(N_DAYS):
337             for release_index in range(4):
338                 pv_hat[day_index, release_index] = _interpolate_official_day(data, day_index, release_index)
339     elif source == "historical":
340         for day_index in range(1, N_DAYS):
341             indices = np.arange(max(0, day_index - 7), day_index, dtype=int)
342             weights = np.exp(-(day_index - indices) / 7.0)
343             value = np.maximum(0.0, np.average(data.pv_kw[indices], axis=0, weights=weights))
344             for release_index, start in enumerate(RELEASE_STARTS):
345                 pv_hat[day_index, release_index, start:] = value[start:]
346     else:
347         raise ValueError(f"Unknown forecast source {source!r}")
348
349     net_hat = np.full_like(pv_hat, np.nan)
350     errors = np.full_like(pv_hat, np.nan)
351     valid = np.zeros_like(pv_hat, dtype=bool)
352     for day_index in range(1, N_DAYS):
353         for release_index, start in enumerate(RELEASE_STARTS):
354             net_hat[day_index, release_index, start:] = (
355                 load_hat[day_index, start:] - pv_hat[day_index, release_index, start:]
356             ) * DELTA_HOURS
357             errors[day_index, release_index, start:] = (
358                 data.net_load_kwh[day_index, start:] - net_hat[day_index, release_index, start:]
359             )
360             valid[day_index, release_index, start:] = valid_days[day_index]
361     price_ratio = np.full_like(data.actual_price, np.nan)
362     price_ratio[valid_days] = data.actual_price[valid_days] / price_hat[valid_days]
363     return ForecastArchive(source, load_hat, price_hat, price_ratio, pv_hat, net_hat, errors, valid)
364
```

```
365
366 def build_scenarios(
367     archive: ForecastArchive,
368     day_index: int,
369     release_index: int,
370     start: int,
371     mode: str,
372 ) -> ScenarioSet:
373     point_net = archive.net_kwh[day_index, release_index, start:]
374     point_price = archive.price_hat[day_index, start:]
375     if not np.isfinite(point_net).all() or not np.isfinite(point_price).all():
376         raise ValueError(f"Point forecast unavailable on day index {day_index}, release
              {RELEASE_HOURS[release_index]}")
377     if mode == "point":
378         return ScenarioSet(point_net[None, :], point_price[None, :], np.ones(1), tuple(), True, "point_baseline")
379     if mode != "paired":
380         raise ValueError(f"Unknown scenario mode {mode!r}")
381
382     candidates = range(max(1, day_index - SCENARIO_WINDOW_DAYS), day_index)
383     eligible = [
384         idx
385         for idx in candidates
386         if archive.valid[idx, release_index, start:].all()
387         and np.isfinite(archive.price_ratio[idx, start:]).all()
388     ]
389     if len(eligible) < MIN_SCENARIO_DAYS:
390         return ScenarioSet(
391             point_net[None, :],
392             point_price[None, :],
393             np.ones(1),
394             tuple(eligible),
395             True,
396             "insufficient_history",
397         )
398     index = np.asarray(eligible, dtype=int)
399     scenario_net = point_net[None, :] + archive.errors_kwh[index, release_index, start:]
400     scenario_price = point_price[None, :] * archive.price_ratio[index, start:]
401     probabilities = np.full(len(index), 1.0 / len(index))
402     if not np.isfinite(scenario_net).all() or not np.isfinite(scenario_price).all() or np.any(scenario_price <=
          0):
403         raise ValueError("Constructed scenarios contain invalid values")
404     return ScenarioSet(scenario_net, scenario_price, probabilities, tuple(eligible), False, None)
405
406
407 def build_inherited_risk_margins(
408     archive: ForecastArchive,
409     question_variant: str,
410     alpha: float | None,
411 ) -> np.ndarray:
412     """Reproduce the approved Q2/Q3 risk-margin rules without adding price uncertainty."""
413     margins = np.zeros_like(archive.net_kwh)
414     if alpha is None:
415         return margins
416     if question_variant == "Q4-2":
417         for day_index in range(1, N_DAYS):
418             prior = np.arange(max(1, day_index - 28), day_index, dtype=int)
419             eligible = prior[np.all(archive.valid[prior, 0], axis=1)]
420             if len(np.unique(eligible)) < 7:
421                 continue
422             for group in range(6):
423                 section = slice(group * 24, (group + 1) * 24)
424                 samples = archive.errors_kwh[eligible, 0, section].reshape(-1)
425                 value = float(np.quantile(samples, alpha, method="linear"))
426                 margins[day_index, 0, section] = max(0.0, value)
427         return margins
428     if question_variant == "Q4-3":
429         for day_index in range(1, N_DAYS):
430             prior = np.arange(max(1, day_index - 28), day_index, dtype=int)
```

```
            if len(prior) < 7:
                continue
            for release_index, start in enumerate(RELEASE_STARTS):
                for interval in range(start, N_INTERVALS):
                    eligible = prior[archive.valid[prior, release_index, interval]]
                    if len(np.unique(eligible)) < 7:
                        continue
                    value = float(
                        np.quantile(
                            archive.errors_kwh[eligible, release_index, interval],
                            alpha,
                            method="linear",
                        )
                    )
                    margins[day_index, release_index, interval] = max(0.0, value)
        return margins
    raise ValueError(f"Unknown question variant {question_variant!r}")


def build_inherited_planning_case(
    data: Q4Data,
    archive: ForecastArchive,
    margins: np.ndarray,
    day_index: int,
    release_index: int,
    start: int,
) -> ScenarioSet:
    """ 为题给数据离线重算使用附件 4 目标日完整价格路径; 不主张现实运行可预知未来实时价格。"""
    planned_net = archive.net_kwh[day_index, release_index, start:] + margins[
        day_index, release_index, start:
    ]
    price = data.actual_price[day_index, start:]
    if not np.isfinite(planned_net).all() or not np.isfinite(price).all() or np.any(price <= 0):
        raise ValueError(
            f"Inherited planning inputs unavailable on day {day_index}, release {RELEASE_HOURS[release_index]}"
        )
    return ScenarioSet(
        planned_net[None, :],
        price[None, :],
        np.ones(1),
        tuple(),
        False,
        "known_price_schedule_inherited_from_q2_q3",
    )


class ScenarioLP:
    def __init__(self) -> None:
        self.da_cache: dict[tuple[int, int], tuple[csr_matrix, list[tuple[float | None, float | None]],
          np.ndarray]] = {}
        self.tr_cache: dict[tuple[int, int], tuple[csr_matrix, list[tuple[float | None, float | None]],
          np.ndarray]] = {}

    def _da_structure(
        self, length: int, scenario_count: int
    ) -> tuple[csr_matrix, list[tuple[float | None, float | None]], np.ndarray]:
        key = (length, scenario_count)
        if key in self.da_cache:
            return self.da_cache[key]
        t_count = length
        j_count = scenario_count
        jt = j_count * t_count
        x0, c0, d0, s0, e0, w0 = 0, t_count, 2 * t_count, 3 * t_count, 4 * t_count, 4 * t_count + jt
        columns = 4 * t_count + 2 * jt
        matrix = lil_matrix((jt + t_count, columns), dtype=float)
        for j in range(j_count):
            for t in range(t_count):
                row = j * t_count + t
```

```
                matrix[row, [x0 + t, c0 + t, d0 + t, e0 + row, w0 + row]] = [1.0, -1.0, 1.0, 1.0, -1.0]
        for t in range(t_count):
            row = jt + t
            matrix[row, [c0 + t, d0 + t, s0 + t]] = [-ETA_CHARGE, 1.0 / ETA_DISCHARGE, 1.0]
            if t:
                matrix[row, s0 + t - 1] = -1.0
        bounds = (
            [(0.0, None)] * t_count
            + [(0.0, INTERVAL_LIMIT_KWH)] * t_count
            + [(0.0, INTERVAL_LIMIT_KWH)] * t_count
            + [(SOC_MIN_KWH, SOC_MAX_KWH)] * t_count
            + [(0.0, None)] * (2 * jt)
        )
        secondary = np.zeros(columns)
        secondary[c0 : c0 + t_count] = 1.0
        secondary[d0 : d0 + t_count] = 1.0
        value = (matrix.tocsr(), bounds, secondary)
        self.da_cache[key] = value
        return value

    def solve_da(self, scenarios: ScenarioSet, initial_soc: float, terminal_value: float) -> LPSolution:
        j_count, t_count = scenarios.net_kwh.shape
        jt = j_count * t_count
        matrix, bounds, secondary = self._da_structure(t_count, j_count)
        rhs = np.r_[scenarios.net_kwh.reshape(-1), np.zeros(t_count)]
        rhs[jt] = initial_soc
        x0, c0, d0, s0, e0, w0 = 0, t_count, 2 * t_count, 3 * t_count, 4 * t_count, 4 * t_count + jt
        mean_price = scenarios.probabilities @ scenarios.price
        primary = np.zeros(4 * t_count + 2 * jt)
        primary[x0 : x0 + t_count] = mean_price
        primary[s0 + t_count - 1] = -terminal_value
        primary[e0 : e0 + jt] = (5.0 * scenarios.probabilities[:, None] * scenarios.price).reshape(-1)
        return self._solve(
            "DA", matrix, rhs, bounds, primary, secondary, scenarios, initial_soc,
            (x0, c0, d0, s0, e0, w0), None,
        )

    def _tr_structure(
        self, length: int, scenario_count: int
    ) -> tuple[csr_matrix, list[tuple[float | None, float | None]], np.ndarray]:
        key = (length, scenario_count)
        if key in self.tr_cache:
            return self.tr_cache[key]
        t_count = length
        j_count = scenario_count
        jt = j_count * t_count
        y0, ap0, am0, c0, d0, s0, e0, w0 = (
            0, t_count, 2 * t_count, 3 * t_count, 4 * t_count, 5 * t_count, 6 * t_count, 6 * t_count + jt
        )
        columns = 6 * t_count + 2 * jt
        matrix = lil_matrix((jt + 2 * t_count, columns), dtype=float)
        for j in range(j_count):
            for t in range(t_count):
                row = j * t_count + t
                matrix[row, [y0 + t, c0 + t, d0 + t, e0 + row, w0 + row]] = [1.0, -1.0, 1.0, 1.0, -1.0]
        for t in range(t_count):
            row = jt + t
            matrix[row, [c0 + t, d0 + t, s0 + t]] = [-ETA_CHARGE, 1.0 / ETA_DISCHARGE, 1.0]
            if t:
                matrix[row, s0 + t - 1] = -1.0
            matrix[jt + t_count + t, [y0 + t, ap0 + t, am0 + t]] = [1.0, -1.0, 1.0]
        bounds = (
            [(0.0, None)] * (3 * t_count)
            + [(0.0, INTERVAL_LIMIT_KWH)] * t_count
            + [(0.0, INTERVAL_LIMIT_KWH)] * t_count
            + [(SOC_MIN_KWH, SOC_MAX_KWH)] * t_count
            + [(0.0, None)] * (2 * jt)
        )
```

```
        secondary = np.zeros(columns)
        secondary[c0 : c0 + t_count] = 1.0
        secondary[d0 : d0 + t_count] = 1.0
        value = (matrix.tocsr(), bounds, secondary)
        self.tr_cache[key] = value
        return value

    def solve_tr(
        self,
        scenarios: ScenarioSet,
        initial_soc: float,
        previous_contract: np.ndarray,
        terminal_value: float,
    ) -> LPSolution:
        j_count, t_count = scenarios.net_kwh.shape
        jt = j_count * t_count
        matrix, bounds, secondary = self._tr_structure(t_count, j_count)
        rhs = np.r_[scenarios.net_kwh.reshape(-1), np.zeros(t_count), previous_contract]
        rhs[jt] = initial_soc
        y0, ap0, am0, c0, d0, s0, e0, w0 = (
            0, t_count, 2 * t_count, 3 * t_count, 4 * t_count, 5 * t_count, 6 * t_count, 6 * t_count + jt
        )
        mean_price = scenarios.probabilities @ scenarios.price
        primary = np.zeros(6 * t_count + 2 * jt)
        primary[ap0 : ap0 + t_count] = 1.5 * mean_price
        primary[am0 : am0 + t_count] = -0.5 * mean_price
        primary[s0 + t_count - 1] = -terminal_value
        primary[e0 : e0 + jt] = (5.0 * scenarios.probabilities[:, None] * scenarios.price).reshape(-1)
        return self._solve(
            "TR", matrix, rhs, bounds, primary, secondary, scenarios, initial_soc,
            (y0, c0, d0, s0, e0, w0), previous_contract,
            adjustment_blocks=(ap0, am0),
        )

    @staticmethod
    def _solve(
        label: str,
        matrix: csr_matrix,
        rhs: np.ndarray,
        bounds: list[tuple[float | None, float | None]],
        primary: np.ndarray,
        secondary: np.ndarray,
        scenarios: ScenarioSet,
        initial_soc: float,
        block_starts: tuple[int, int, int, int, int, int],
        previous_contract: np.ndarray | None,
        adjustment_blocks: tuple[int, int] | None = None,
    ) -> LPSolution:
        stage1 = linprog(primary, A_eq=matrix, b_eq=rhs, bounds=bounds, method="highs")
        if not stage1.success:
            raise RuntimeError(f"{label} stage1 failed: {stage1.message}")
        stage2 = linprog(
            secondary,
            A_ub=csr_matrix(primary.reshape(1, -1)),
            b_ub=np.asarray([float(stage1.fun) + PRIMARY_TOLERANCE]),
            A_eq=matrix,
            b_eq=rhs,
            bounds=bounds,
            method="highs",
        )
        if not stage2.success:
            raise RuntimeError(f"{label} stage2 failed: {stage2.message}")
        t_count = scenarios.net_kwh.shape[1]
        j_count = scenarios.net_kwh.shape[0]
        x0, c0, d0, s0, e0, w0 = block_starts
        values = stage2.x
        contract = values[x0 : x0 + t_count]
        charge = values[c0 : c0 + t_count]
```

```
        discharge = values[d0 : d0 + t_count]
        soc = values[s0 : s0 + t_count]
        emergency = values[e0 : e0 + j_count * t_count].reshape(j_count, t_count)
        unused = values[w0 : w0 + j_count * t_count].reshape(j_count, t_count)
        balance = contract[None, :] + emergency + discharge[None, :] - scenarios.net_kwh - charge[None, :] -
          unused
        prior_soc = np.r_[initial_soc, soc[:-1]]
        storage = soc - prior_soc - ETA_CHARGE * charge + discharge / ETA_DISCHARGE
        gap = scenarios.net_kwh + charge[None, :] - discharge[None, :] - contract[None, :]
        emergency_formula = np.maximum(gap, 0.0)
        unused_formula = np.maximum(-gap, 0.0)
        emergency_residual = max(
            float(np.max(np.abs(emergency - emergency_formula))),
            float(np.max(np.abs(unused - unused_formula))),
            float(np.max(np.minimum(emergency, unused))),
        )
        adjustment_residual = 0.0
        if adjustment_blocks is not None and previous_contract is not None:
            ap0, am0 = adjustment_blocks
            increase = values[ap0 : ap0 + t_count]
            decrease = values[am0 : am0 + t_count]
            adjustment_residual = max(
                float(np.max(np.abs(contract - increase + decrease - previous_contract))),
                float(np.max(np.minimum(increase, decrease))),
            )
        return LPSolution(
            contract=contract,
            charge=charge,
            discharge=discharge,
            soc_end=soc,
            emergency=emergency,
            unused=unused,
            stage1_objective=float(stage1.fun),
            stage2_primary_objective=float(primary @ stage2.x),
            throughput=float(charge.sum() + discharge.sum()),
            max_balance_residual=float(np.max(np.abs(balance))),
            max_storage_residual=float(np.max(np.abs(storage))),
            max_emergency_formula_residual=emergency_residual,
            max_adjustment_residual=adjustment_residual,
            max_charge_discharge_overlap=float(np.max(np.minimum(charge, discharge))),
            stage1_status=str(stage1.message),
            stage2_status=str(stage2.message),
        )


def _fallback_da(point_net: np.ndarray, scenario_count: int, initial_soc: float) -> LPSolution:
    contract = np.maximum(point_net, 0.0)
    zeros = np.zeros_like(contract)
    gap = point_net - contract
    emergency = np.tile(np.maximum(gap, 0.0), (scenario_count, 1))
    unused = np.tile(np.maximum(-gap, 0.0), (scenario_count, 1))
    return LPSolution(
        contract, zeros.copy(), zeros.copy(), np.full_like(contract, initial_soc), emergency, unused,
        math.nan, math.nan, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
        "failed_fallback_activated", "failed_fallback_activated",
    )


def incremental_adjustment_fee(
    new_contract: np.ndarray, previous_contract: np.ndarray, actual_price: np.ndarray
) -> np.ndarray:
    increase = np.maximum(new_contract - previous_contract, 0.0)
    decrease = np.maximum(previous_contract - new_contract, 0.0)
    return actual_price * (1.5 * increase - 0.5 * decrease)


def run_policy(
    data: Q4Data,
```

```
    archive: ForecastArchive,
    config: PolicyConfig,
    margins: np.ndarray | None = None,
    max_days: int = N_DAYS,
    start_day_index: int = 0,
    initial_soc_override: float | None = None,
) -> StrategyRun:
    started = time_module.perf_counter()
    solver = ScenarioLP()
    day_limit = min(max_days, N_DAYS)
    if not 0 <= start_day_index < day_limit:
        raise ValueError("start_day_index must identify a day before max_days")
    release_indices = (0,) if config.question_variant == "Q4-2" else (0, 1, 2, 3)
    current_soc = SOC_INITIAL_KWH if initial_soc_override is None else float(initial_soc_override)
    if not SOC_MIN_KWH <= current_soc <= SOC_MAX_KWH:
        raise ValueError("Initial SOC must satisfy the storage energy bounds")
    run_initial_soc = current_soc
    daily_rows: list[dict[str, Any]] = []
    ledger_rows: list[dict[str, Any]] = []
    adjustment_rows: list[dict[str, Any]] = []
    plan_call_rows: list[dict[str, Any]] = []
    plan_trajectory_rows: list[dict[str, Any]] = []
    fallback_events: list[dict[str, Any]] = []
    max_checks: dict[str, float] = {}
    total_charge_full = 0.0
    total_discharge_full = 0.0
    plan_count = 0
    degenerate_count = 0

    if config.scenario_mode == "inherited" and margins is None:
        raise ValueError("Inherited Q2/Q3 execution requires an explicit risk-margin array")

    def planning_case(day_index: int, release_index: int, start: int) -> ScenarioSet:
        if config.scenario_mode == "inherited":
            assert margins is not None
            return build_inherited_planning_case(
                data, archive, margins, day_index, release_index, start
            )
        return build_scenarios(archive, day_index, release_index, start, config.scenario_mode)

    def update_max(name: str, value: float) -> None:
        if math.isfinite(value):
            max_checks[name] = max(max_checks.get(name, 0.0), float(value))

    def record_solution(
        day_index: int,
        release_index: int,
        start: int,
        scenarios: ScenarioSet,
        solution: LPSolution,
        call_id: str,
        terminal_value: float,
    ) -> None:
        nonlocal plan_count, degenerate_count
        plan_count += 1
        degenerate_count += int(scenarios.degenerate)
        update_max("planning_balance_max_abs", solution.max_balance_residual)
        update_max("planning_storage_recurrence_max_abs", solution.max_storage_residual)
        update_max("scenario_emergency_formula_max_abs", solution.max_emergency_formula_residual)
        update_max("planning_adjustment_max_abs", solution.max_adjustment_residual)
        update_max("planning_charge_discharge_overlap_max", solution.max_charge_discharge_overlap)
        update_max("stage2_objective_excess_yuan", max(0.0, solution.stage2_primary_objective -
          solution.stage1_objective))
        if not config.collect_evidence:
            return
        historical_dates = [data.dates[index].isoformat() for index in scenarios.historical_day_indices]
        plan_call_rows.append(
            {
```

```
                "strategy_id": config.strategy_id,
                "call_id": call_id,
                "date": data.dates[day_index].isoformat(),
                "release_hour": RELEASE_HOURS[release_index],
                "start_interval_index": start + 1,
                "remaining_interval_count": len(solution.contract),
                "scenario_count": len(scenarios.probabilities),
                "scenario_mode": config.scenario_mode,
                "degenerate": scenarios.degenerate,
                "degenerate_reason": scenarios.reason,
                "historical_scenario_dates": ";".join(historical_dates),
                "stage1_objective_yuan": solution.stage1_objective,
                "stage2_primary_objective_yuan": solution.stage2_primary_objective,
                "throughput_kwh": solution.throughput,
                "terminal_value_yuan_per_kwh": terminal_value,
                "stage1_status": solution.stage1_status,
                "stage2_status": solution.stage2_status,
            }
        )
        expected_emergency = scenarios.probabilities @ solution.emergency
        expected_unused = scenarios.probabilities @ solution.unused
        for local, interval in enumerate(range(start, N_INTERVALS)):
            plan_trajectory_rows.append(
                {
                    "strategy_id": config.strategy_id,
                    "call_id": call_id,
                    "date": data.dates[day_index].isoformat(),
                    "release_hour": RELEASE_HOURS[release_index],
                    "interval_index": interval + 1,
                    "interval_label": data.interval_labels[interval],
                    "contract_kwh": solution.contract[local],
                    "reference_charge_kwh": solution.charge[local],
                    "reference_discharge_kwh": solution.discharge[local],
                    "reference_soc_end_kwh": solution.soc_end[local],
                    "expected_emergency_kwh": expected_emergency[local],
                    "expected_unused_kwh": expected_unused[local],
                }
            )

    for day_index in range(start_day_index, day_limit):
        day = data.dates[day_index]
        day_initial_soc = current_soc
        q = np.zeros(N_INTERVALS)
        final_contract = np.zeros(N_INTERVALS)
        current_plan_charge = np.zeros(N_INTERVALS)
        current_plan_discharge = np.zeros(N_INTERVALS)
        current_plan_emergency = np.zeros(N_INTERVALS)
        current_plan_unused = np.zeros(N_INTERVALS)
        actual_charge = np.zeros(N_INTERVALS)
        actual_discharge = np.zeros(N_INTERVALS)
        actual_emergency = np.zeros(N_INTERVALS)
        actual_unused = np.zeros(N_INTERVALS)
        soc_end = np.zeros(N_INTERVALS)
        adjustment_fee_by_interval = np.zeros(N_INTERVALS)
        adjustment_increase_by_interval = np.zeros(N_INTERVALS)
        adjustment_decrease_by_interval = np.zeros(N_INTERVALS)
        adjustment_count_by_interval = np.zeros(N_INTERVALS, dtype=int)
        executing_release = np.zeros(N_INTERVALS, dtype=int)
        day_scenario_counts: list[int] = []

        if day_index == 0:
            terminal_value = 0.0
        else:
            da_scenarios = planning_case(day_index, 0, 0)
            terminal_value = (
                0.0
                if day_index == FINAL_DAY_INDEX
                else GAMMA * ETA_DISCHARGE * float(np.median(da_scenarios.probabilities @ da_scenarios.price))
```

```
            )
            try:
                da = solver.solve_da(da_scenarios, current_soc, terminal_value)
            except RuntimeError as exc:
                da = _fallback_da(archive.net_kwh[day_index, 0], len(da_scenarios.probabilities), current_soc)
                fallback_events.append({"date": day.isoformat(), "release_hour": 0, "error": str(exc)})
            q[:] = da.contract
            final_contract[:] = q
            current_plan_charge[:] = da.charge
            current_plan_discharge[:] = da.discharge
            current_plan_emergency[:] = da_scenarios.probabilities @ da.emergency
            current_plan_unused[:] = da_scenarios.probabilities @ da.unused
            day_scenario_counts.append(len(da_scenarios.probabilities))
            record_solution(day_index, 0, 0, da_scenarios, da, f"{day.isoformat()}_00", terminal_value)

        for position, release_index in enumerate(release_indices):
            release_hour = RELEASE_HOURS[release_index]
            start = RELEASE_STARTS[release_index]
            if day_index > 0 and release_index > 0:
                scenarios = planning_case(day_index, release_index, start)
                previous = final_contract[start:].copy()
                try:
                    solution = solver.solve_tr(scenarios, current_soc, previous, terminal_value)
                    new_contract = solution.contract
                except RuntimeError as exc:
                    fallback_events.append({"date": day.isoformat(), "release_hour": release_hour, "error":
                      str(exc)})
                    new_contract = previous.copy()
                    point_gap = archive.net_kwh[day_index, release_index, start:] - new_contract
                    solution = LPSolution(
                        new_contract,
                        np.zeros_like(new_contract),
                        np.zeros_like(new_contract),
                        np.full_like(new_contract, current_soc),
                        np.tile(np.maximum(point_gap, 0.0), (len(scenarios.probabilities), 1)),
                        np.tile(np.maximum(-point_gap, 0.0), (len(scenarios.probabilities), 1)),
                        math.nan,
                        math.nan,
                        0.0,
                        0.0,
                        0.0,
                        0.0,
                        0.0,
                        0.0,
                        "failed_hold_previous",
                        "failed_hold_previous",
                    )
                increase = np.maximum(new_contract - previous, 0.0)
                decrease = np.maximum(previous - new_contract, 0.0)
                incremental_fee = incremental_adjustment_fee(new_contract, previous, data.actual_price[day_index,
                  start:])
                changed = np.abs(new_contract - previous) > EVENT_TOLERANCE
                adjustment_fee_by_interval[start:] += incremental_fee
                adjustment_increase_by_interval[start:] += increase
                adjustment_decrease_by_interval[start:] += decrease
                adjustment_count_by_interval[start:] += changed.astype(int)
                final_contract[start:] = new_contract
                current_plan_charge[start:] = solution.charge
                current_plan_discharge[start:] = solution.discharge
                current_plan_emergency[start:] = scenarios.probabilities @ solution.emergency
                current_plan_unused[start:] = scenarios.probabilities @ solution.unused
                executing_release[start:] = release_hour
                day_scenario_counts.append(len(scenarios.probabilities))
                call_id = f"{day.isoformat()}_{release_hour:02d}"
                record_solution(day_index, release_index, start, scenarios, solution, call_id, terminal_value)
                if config.collect_evidence:
                    for local, interval in enumerate(range(start, N_INTERVALS)):
                        adjustment_rows.append(
```

```
                        {
                            "strategy_id": config.strategy_id,
                            "date": day.isoformat(),
                            "release_hour": release_hour,
                            "interval_index": interval + 1,
                            "interval_label": data.interval_labels[interval],
                            "actual_price_yuan_per_kwh": data.actual_price[day_index, interval],
                            "previous_contract_kwh": previous[local],
                            "new_contract_kwh": new_contract[local],
                            "increase_kwh": increase[local],
                            "decrease_kwh": decrease[local],
                            "incremental_adjustment_fee_yuan": incremental_fee[local],
                            "changed": bool(changed[local]),
                        }
                    )

        end = RELEASE_STARTS[release_indices[position + 1]] if position + 1 < len(release_indices) else
          N_INTERVALS
        for interval in range(start, end):
            net_gap = data.net_load_kwh[day_index, interval] - final_contract[interval]
            if day_index == 0:
                charge = 0.0
                discharge = 0.0
            else:
                charge = min(
                    max(-net_gap, 0.0),
                    INTERVAL_LIMIT_KWH,
                    max(0.0, (SOC_MAX_KWH - current_soc) / ETA_CHARGE),
                )
                discharge = min(
                    max(net_gap, 0.0),
                    INTERVAL_LIMIT_KWH,
                    max(0.0, ETA_DISCHARGE * (current_soc - SOC_MIN_KWH)),
                )
            residual = net_gap + charge - discharge
            emergency = max(residual, 0.0)
            unused = max(-residual, 0.0)
            next_soc = current_soc + ETA_CHARGE * charge - discharge / ETA_DISCHARGE
            actual_charge[interval] = charge
            actual_discharge[interval] = discharge
            actual_emergency[interval] = emergency
            actual_unused[interval] = unused
            soc_end[interval] = next_soc
            total_charge_full += charge
            total_discharge_full += discharge
            update_max("actual_soc_lower_violation", max(0.0, SOC_MIN_KWH - next_soc))
            update_max("actual_soc_upper_violation", max(0.0, next_soc - SOC_MAX_KWH))
            update_max("actual_charge_limit_violation", max(0.0, charge - INTERVAL_LIMIT_KWH))
            update_max("actual_discharge_limit_violation", max(0.0, discharge - INTERVAL_LIMIT_KWH))
            update_max("actual_charge_discharge_overlap", min(charge, discharge))
            update_max("actual_emergency_unused_overlap", min(emergency, unused))
            balance = (
                final_contract[interval]
                + emergency
                + data.pv_kw[day_index, interval] * DELTA_HOURS
                + discharge
                - data.load_kw[day_index, interval] * DELTA_HOURS
                - charge
                - unused
            )
            recurrence = next_soc - current_soc - ETA_CHARGE * charge + discharge / ETA_DISCHARGE
            update_max("actual_balance_max_abs", abs(balance))
            update_max("actual_storage_recurrence_max_abs", abs(recurrence))
            current_soc = next_soc

    original_cost_by_interval = data.actual_price[day_index] * q
    contract_fee_by_interval = original_cost_by_interval + adjustment_fee_by_interval
    path_identity = contract_fee_by_interval - data.actual_price[day_index] * (
```

```
            final_contract + 0.5 * (adjustment_increase_by_interval + adjustment_decrease_by_interval)
        )
        update_max("contract_path_identity_max_abs", float(np.max(np.abs(path_identity))))
        emergency_cost_by_interval = 5.0 * data.actual_price[day_index] * actual_emergency
        total_cost_by_interval = contract_fee_by_interval + emergency_cost_by_interval
        daily_rows.append(
            {
                "strategy_id": config.strategy_id,
                "question_variant": config.question_variant,
                "date": day.isoformat(),
                "initial_soc_kwh": day_initial_soc,
                "terminal_soc_kwh": current_soc,
                "original_contract_kwh": float(q.sum()),
                "final_contract_kwh": float(final_contract.sum()),
                "adjustment_increase_kwh": float(adjustment_increase_by_interval.sum()),
                "adjustment_decrease_kwh": float(adjustment_decrease_by_interval.sum()),
                "adjustment_transaction_count": int(adjustment_count_by_interval.sum()),
                "emergency_purchase_kwh": float(actual_emergency.sum()),
                "actual_total_purchase_kwh": float(final_contract.sum() + actual_emergency.sum()),
                "charge_kwh": float(actual_charge.sum()),
                "discharge_kwh": float(actual_discharge.sum()),
                "unused_kwh": float(actual_unused.sum()),
                "original_contract_cost_yuan": float(original_cost_by_interval.sum()),
                "adjustment_net_cost_yuan": float(adjustment_fee_by_interval.sum()),
                "contract_total_cost_yuan": float(contract_fee_by_interval.sum()),
                "emergency_cost_yuan": float(emergency_cost_by_interval.sum()),
                "actual_total_cost_yuan": float(total_cost_by_interval.sum()),
                "planning_call_count": len(day_scenario_counts),
                "min_scenario_count": min(day_scenario_counts) if day_scenario_counts else 0,
                "max_scenario_count": max(day_scenario_counts) if day_scenario_counts else 0,
            }
        )
        if config.collect_evidence:
            prior_soc = np.r_[day_initial_soc, soc_end[:-1]]
            for interval in range(N_INTERVALS):
                release_index = RELEASE_HOURS.index(int(executing_release[interval]))
                ledger_rows.append(
                    {
                        "strategy_id": config.strategy_id,
                        "question_variant": config.question_variant,
                        "date": day.isoformat(),
                        "interval_index": interval + 1,
                        "interval_start": data.interval_starts[interval],
                        "interval_end": data.interval_ends[interval],
                        "interval_label": data.interval_labels[interval],
                        "executing_release_hour": int(executing_release[interval]),
                        "actual_price_yuan_per_kwh": data.actual_price[day_index, interval],
                        "actual_load_kw": data.load_kw[day_index, interval],
                        "actual_pv_kw": data.pv_kw[day_index, interval],
                        "actual_net_load_kwh": data.net_load_kwh[day_index, interval],
                        "forecast_load_kw": archive.load_kw[day_index, interval] if day_index else np.nan,
                        "forecast_pv_kw": archive.pv_kw[day_index, release_index, interval] if day_index else
                          np.nan,
                        "forecast_net_load_kwh": archive.net_kwh[day_index, release_index, interval] if day_index
                          else np.nan,
                        "forecast_price_yuan_per_kwh": (
                            data.actual_price[day_index, interval]
                            if day_index and config.scenario_mode == "inherited"
                            else archive.price_hat[day_index, interval] if day_index else np.nan
                        ),
                        "original_contract_kwh": q[interval],
                        "final_contract_kwh": final_contract[interval],
                        "adjustment_increase_kwh": adjustment_increase_by_interval[interval],
                        "adjustment_decrease_kwh": adjustment_decrease_by_interval[interval],
                        "adjustment_transaction_count": int(adjustment_count_by_interval[interval]),
                        "reference_charge_kwh": current_plan_charge[interval],
                        "reference_discharge_kwh": current_plan_discharge[interval],
                        "reference_expected_emergency_kwh": current_plan_emergency[interval],
```

```
                        "reference_expected_unused_kwh": current_plan_unused[interval],
                        "actual_charge_kwh": actual_charge[interval],
                        "actual_discharge_kwh": actual_discharge[interval],
                        "actual_emergency_kwh": actual_emergency[interval],
                        "actual_unused_kwh": actual_unused[interval],
                        "soc_start_kwh": prior_soc[interval],
                        "soc_end_kwh": soc_end[interval],
                        "original_contract_cost_yuan": original_cost_by_interval[interval],
                        "cumulative_adjustment_fee_yuan": adjustment_fee_by_interval[interval],
                        "contract_total_cost_yuan": contract_fee_by_interval[interval],
                        "emergency_cost_yuan": emergency_cost_by_interval[interval],
                        "actual_total_cost_yuan": total_cost_by_interval[interval],
                    }
                )

    daily = pd.DataFrame(daily_rows)
    evaluation = daily[daily["date"] >= "2025-02-01"].copy()
    if evaluation.empty:
        evaluation = daily.copy()
    metrics = {
        "strategy_id": config.strategy_id,
        "role": config.role,
        "question_variant": config.question_variant,
        "forecast_source": config.forecast_source,
        "scenario_mode": config.scenario_mode,
        "alpha": config.alpha,
        "price_information": (
            FORMAL_PRICE_INFORMATION
            if config.scenario_mode == "inherited"
            else "extension_price_forecast_or_joint_historical_scenarios_not_formal_q4_round2"
        ),
        "simulation_start": daily.iloc[0]["date"],
        "simulation_days": len(daily),
        "run_initial_soc_kwh": run_initial_soc,
        "evaluation_start": evaluation.iloc[0]["date"],
        "evaluation_end": evaluation.iloc[-1]["date"],
        "evaluation_days": len(evaluation),
        "scenario_window_days": SCENARIO_WINDOW_DAYS,
        "minimum_scenario_days": MIN_SCENARIO_DAYS,
        "gamma": GAMMA,
        "initial_soc_evaluation_kwh": float(evaluation.iloc[0]["initial_soc_kwh"]),
        "terminal_soc_evaluation_kwh": float(evaluation.iloc[-1]["terminal_soc_kwh"]),
        "original_contract_kwh": float(evaluation["original_contract_kwh"].sum()),
        "final_contract_kwh": float(evaluation["final_contract_kwh"].sum()),
        "adjustment_increase_kwh": float(evaluation["adjustment_increase_kwh"].sum()),
        "adjustment_decrease_kwh": float(evaluation["adjustment_decrease_kwh"].sum()),
        "adjustment_transaction_count": int(evaluation["adjustment_transaction_count"].sum()),
        "emergency_purchase_kwh": float(evaluation["emergency_purchase_kwh"].sum()),
        "actual_total_purchase_kwh": float(evaluation["actual_total_purchase_kwh"].sum()),
        "charge_kwh": float(evaluation["charge_kwh"].sum()),
        "discharge_kwh": float(evaluation["discharge_kwh"].sum()),
        "original_contract_cost_yuan": float(evaluation["original_contract_cost_yuan"].sum()),
        "adjustment_net_cost_yuan": float(evaluation["adjustment_net_cost_yuan"].sum()),
        "contract_total_cost_yuan": float(evaluation["contract_total_cost_yuan"].sum()),
        "emergency_cost_yuan": float(evaluation["emergency_cost_yuan"].sum()),
        "actual_total_cost_yuan": float(evaluation["actual_total_cost_yuan"].sum()),
        "planning_calls_full_run": plan_count,
        "degenerate_scenario_calls_full_run": degenerate_count,
        "fallback_event_count": len(fallback_events),
    }
    fee_reconciliation = abs(
        metrics["contract_total_cost_yuan"]
        - metrics["original_contract_cost_yuan"]
        - metrics["adjustment_net_cost_yuan"]
    )
    total_reconciliation = abs(
        metrics["actual_total_cost_yuan"]
        - metrics["contract_total_cost_yuan"]
```

```
        - metrics["emergency_cost_yuan"]
    )
    cross_day = np.abs(daily["initial_soc_kwh"].to_numpy()[1:] - daily["terminal_soc_kwh"].to_numpy()[:-1])
    energy_identity = total_discharge_full - (
        ETA_CHARGE * ETA_DISCHARGE * total_charge_full
        - ETA_DISCHARGE * (current_soc - run_initial_soc)
    )
    validation = {
        **max_checks,
        "fee_reconciliation_abs_yuan": float(fee_reconciliation),
        "total_cost_reconciliation_abs_yuan": float(total_reconciliation),
        "cross_day_soc_max_abs_kwh": float(np.max(cross_day)) if len(cross_day) else 0.0,
        "full_run_storage_energy_identity_abs_kwh": abs(float(energy_identity)),
        "incremental_example_100_to_80_to_100_yuan": float(
            100.0
            + incremental_adjustment_fee(np.array([80.0]), np.array([100.0]), np.array([1.0]))[0]
            + incremental_adjustment_fee(np.array([100.0]), np.array([80.0]), np.array([1.0]))[0]
        ),
        "fallback_events": fallback_events,
    }
    planning_values = [
        validation.get("planning_balance_max_abs", 0.0),
        validation.get("planning_storage_recurrence_max_abs", 0.0),
        validation.get("scenario_emergency_formula_max_abs", 0.0),
        validation.get("planning_adjustment_max_abs", 0.0),
        validation.get("planning_charge_discharge_overlap_max", 0.0),
    ]
    physical_values = [
        validation.get("actual_soc_lower_violation", 0.0),
        validation.get("actual_soc_upper_violation", 0.0),
        validation.get("actual_charge_limit_violation", 0.0),
        validation.get("actual_discharge_limit_violation", 0.0),
        validation.get("actual_charge_discharge_overlap", 0.0),
        validation.get("actual_emergency_unused_overlap", 0.0),
        validation.get("actual_balance_max_abs", 0.0),
        validation.get("actual_storage_recurrence_max_abs", 0.0),
        validation.get("contract_path_identity_max_abs", 0.0),
        validation["fee_reconciliation_abs_yuan"],
        validation["total_cost_reconciliation_abs_yuan"],
        validation["cross_day_soc_max_abs_kwh"],
        validation["full_run_storage_energy_identity_abs_kwh"],
    ]
    validation["planning_checks_pass"] = bool(max(planning_values, default=0.0) <= PLANNING_CHECK_TOLERANCE)
    validation["physical_and_cost_checks_pass"] = bool(max(physical_values, default=0.0) <=
      PHYSICAL_CHECK_TOLERANCE)
    validation["hard_checks_pass"] = bool(
        validation["planning_checks_pass"]
        and validation["physical_and_cost_checks_pass"]
        and validation["incremental_example_100_to_80_to_100_yuan"] == 120.0
        and not fallback_events
    )
    return StrategyRun(
        config=config,
        runtime_seconds=time_module.perf_counter() - started,
        metrics=metrics,
        validation=validation,
        daily=daily,
        ledger=pd.DataFrame(ledger_rows) if config.collect_evidence else None,
        adjustments=(
            pd.DataFrame(adjustment_rows)
            if adjustment_rows
            else pd.DataFrame(
                columns=[
                    "strategy_id", "date", "release_hour", "interval_index", "interval_label",
                    "actual_price_yuan_per_kwh", "previous_contract_kwh", "new_contract_kwh",
                    "increase_kwh", "decrease_kwh", "incremental_adjustment_fee_yuan", "changed",
                ]
            )
```

```
        ) if config.collect_evidence else None,
        plan_calls=pd.DataFrame(plan_call_rows) if config.collect_evidence else None,
        plan_trajectory=pd.DataFrame(plan_trajectory_rows) if config.collect_evidence else None,
    )


def emergency_events(ledger: pd.DataFrame) -> pd.DataFrame:
    events: list[dict[str, Any]] = []
    for day, group in ledger.groupby("date", sort=True):
        ordered = group.sort_values("interval_index").reset_index(drop=True)
        amounts = ordered["actual_emergency_kwh"].to_numpy(dtype=float)
        active = amounts > EVENT_TOLERANCE
        cursor = 0
        event_index = 0
        while cursor < N_INTERVALS:
            if not active[cursor]:
                cursor += 1
                continue
            start = cursor
            while cursor + 1 < N_INTERVALS and active[cursor + 1]:
                cursor += 1
            end = cursor
            event_index += 1
            events.append(
                {
                    "date": day,
                    "event_index": event_index,
                    "start_interval_index": start + 1,
                    "end_interval_index": end + 1,
                    "emergency_time_range": f"{ordered.loc[start, 'interval_start']}-{ordered.loc[end,
                      'interval_end']}",
                    "emergency_purchase_kwh": float(amounts[start : end + 1].sum()),
                }
            )
            cursor += 1
    return pd.DataFrame(events)


def storage_summary(ledger: pd.DataFrame) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    for day, group in ledger.groupby("date", sort=True):
        ordered = group.sort_values("interval_index").reset_index(drop=True)
        for block in range(6):
            start = block * 24
            end = start + 24
            rows.append(
                {
                    "date": day,
                    "block_index": block + 1,
                    "time_range": f"{_fmt_minutes(block * 240)}-{_fmt_minutes((block + 1) * 240)}",
                    "charge_kwh": float(ordered.loc[start : end - 1, "actual_charge_kwh"].sum()),
                    "discharge_kwh": float(ordered.loc[start : end - 1, "actual_discharge_kwh"].sum()),
                    "soc_00_kwh": float(ordered.loc[0, "soc_start_kwh"]) if block == 0 else np.nan,
                    "soc_24_kwh": float(ordered.loc[N_INTERVALS - 1, "soc_end_kwh"]) if block == 5 else np.nan,
                }
            )
    return pd.DataFrame(rows)


def specified_tables(
    ledger: pd.DataFrame, storage: pd.DataFrame, events: pd.DataFrame
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    selected_dates = {"2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"}
    table1 = ledger[ledger["date"].isin(selected_dates)][
        [
            "date", "interval_index", "interval_label", "original_contract_kwh", "final_contract_kwh",
            "actual_emergency_kwh", "actual_total_cost_yuan",
        ]
```

```
    ].copy()
    table2 = storage[storage["date"].isin(selected_dates)].copy()
    table3 = events[events["date"].isin(selected_dates)].copy()
    return table1, table2, table3


def forecast_diagnostics(data: Q4Data, archive: ForecastArchive) -> dict[str, Any]:
    price_mask = np.isfinite(archive.price_hat)
    diagnostics: dict[str, Any] = {
        "archive_id": archive.archive_id,
        "price_mae_yuan_per_kwh": float(np.mean(np.abs(data.actual_price[price_mask] -
          archive.price_hat[price_mask]))),
        "release_metrics": {},
    }
    for release_index, (hour, start) in enumerate(zip(RELEASE_HOURS, RELEASE_STARTS)):
        mask = archive.valid[:, release_index, start:]
        errors = archive.errors_kwh[:, release_index, start:][mask]
        price_residual = (archive.price_ratio[:, start:] - 1.0)[mask]
        correlation = float(np.corrcoef(errors, price_residual)[0, 1]) if len(errors) > 1 else None
        diagnostics["release_metrics"][str(hour)] = {
            "observations": int(len(errors)),
            "net_load_mae_kwh": float(np.mean(np.abs(errors))),
            "net_error_price_ratio_residual_correlation": correlation,
        }
    return diagnostics


def environment_info() -> dict[str, Any]:
    return {
        "python": sys.version,
        "platform": platform.platform(),
        "numpy": np.__version__,
        "pandas": pd.__version__,
        "scipy": scipy.__version__,
        "solver": "scipy.optimize.linprog(method='highs')",
        "random_seed": RANDOM_SEED,
    }


def _configurations(suite: str) -> list[PolicyConfig]:
    if suite == "extension":
        return [
            PolicyConfig("M4J_Q42_paired", "extension", "Q4-2", "historical", "paired", False, alpha=None,
              collect_evidence=True),
            PolicyConfig("B4P_Q42_point", "extension_baseline", "Q4-2", "historical", "point", False, alpha=None),
            PolicyConfig("M4J_Q43_paired", "extension", "Q4-3", "official", "paired", True, alpha=None,
              collect_evidence=True),
            PolicyConfig("B4P_Q43_point", "extension_baseline", "Q4-3", "official", "point", True, alpha=None),
        ]
    return [
        PolicyConfig("M4I_Q42_inherit_M3_alpha08", "main", "Q4-2", "historical", "inherited", False, alpha=0.8,
          collect_evidence=True),
        PolicyConfig("B4I_Q42_inherit_M3_b0", "usable_baseline", "Q4-2", "historical", "inherited", False,
          alpha=None),
        PolicyConfig("M4I_Q43_inherit_M1I_alpha08", "main", "Q4-3", "official", "inherited", True, alpha=0.8,
          collect_evidence=True),
        PolicyConfig("B4I_Q43_inherit_M1I_b0", "usable_baseline", "Q4-3", "official", "inherited", True,
          alpha=None),
    ]


def run_suite(suite: str, max_days: int | None = None) -> None:
    data = load_data()
    archives = {
        "historical": build_forecast_archive(data, "historical"),
        "official": build_forecast_archive(data, "official"),
    }
```

```
    effective_days = max_days if max_days is not None else (60 if suite == "pilot" else N_DAYS)
    output_round = (
        "pilot_inherited_price"
        if suite == "pilot"
        else "extension_rerun" if suite == "extension" else "round2_inherited_price"
    )
    round_dir = PROJECT_ROOT / "results" / "Q4" / "experiments" / output_round
    table_dir = round_dir / "tables"
    metric_dir = round_dir / "metrics"
    table_dir.mkdir(parents=True, exist_ok=True)
    metric_dir.mkdir(parents=True, exist_ok=True)

    runs: list[StrategyRun] = []
    margin_cache: dict[tuple[str, float | None], np.ndarray] = {}
    for config in _configurations(suite):
        print(f"START {config.strategy_id} days={effective_days}", flush=True)
        margins = None
        if config.scenario_mode == "inherited":
            key = (config.question_variant, config.alpha)
            if key not in margin_cache:
                margin_cache[key] = build_inherited_risk_margins(
                    archives[config.forecast_source], config.question_variant, config.alpha
                )
            margins = margin_cache[key]
        run = run_policy(
            data,
            archives[config.forecast_source],
            config,
            margins=margins,
            max_days=effective_days,
        )
        print(
            f"DONE {config.strategy_id} cost={run.metrics['actual_total_cost_yuan']:.2f} "
            f"emergency={run.metrics['emergency_purchase_kwh']:.3f} "
            f"plans={run.metrics['planning_calls_full_run']} runtime={run.runtime_seconds:.2f}s",
            flush=True,
        )
        if not run.validation["hard_checks_pass"]:
            raise RuntimeError(f"Hard checks failed for {config.strategy_id}: {run.validation}")
        runs.append(run)

    by_id = {run.config.strategy_id: run for run in runs}
    comparison_rows: list[dict[str, Any]] = []
    comparison_ids = (
        [("Q4-2", "M4J_Q42_paired", "B4P_Q42_point"), ("Q4-3", "M4J_Q43_paired", "B4P_Q43_point")]
        if suite == "extension"
        else [
            ("Q4-2", "M4I_Q42_inherit_M3_alpha08", "B4I_Q42_inherit_M3_b0"),
            ("Q4-3", "M4I_Q43_inherit_M1I_alpha08", "B4I_Q43_inherit_M1I_b0"),
        ]
    )
    for variant, main_id, baseline_id in comparison_ids:
        main_run = by_id[main_id]
        baseline_run = by_id[baseline_id]
        for run in (main_run, baseline_run):
            comparison_rows.append(
                {
                    "question_variant": variant,
                    "strategy_id": run.config.strategy_id,
                    "role": run.config.role,
                    "scenario_mode": run.config.scenario_mode,
                    "actual_total_cost_yuan": run.metrics["actual_total_cost_yuan"],
                    "contract_total_cost_yuan": run.metrics["contract_total_cost_yuan"],
                    "emergency_cost_yuan": run.metrics["emergency_cost_yuan"],
                    "emergency_purchase_kwh": run.metrics["emergency_purchase_kwh"],
                    "final_contract_kwh": run.metrics["final_contract_kwh"],
                    "initial_soc_evaluation_kwh": run.metrics["initial_soc_evaluation_kwh"],
                    "terminal_soc_evaluation_kwh": run.metrics["terminal_soc_evaluation_kwh"],
```

```
                    "runtime_seconds": run.runtime_seconds,
                    "cost_change_vs_baseline_pct": (
                        run.metrics["actual_total_cost_yuan"] / baseline_run.metrics["actual_total_cost_yuan"] -
                          1.0
                    ) * 100.0,
                }
            )
    save_frame(pd.DataFrame(comparison_rows), table_dir / "strategy_comparison.csv")

    output_files = ["tables/strategy_comparison.csv"]
    for run in runs:
        prefix = run.config.question_variant.lower().replace("-", "") + ("_main" if run.config.role == "main" else
          "_baseline")
        evaluation_daily = run.daily[run.daily["date"] >= "2025-02-01"].copy()
        save_frame(evaluation_daily, table_dir / f"{prefix}_daily_summary.csv")
        output_files.append(f"tables/{prefix}_daily_summary.csv")
        write_json(metric_dir / f"{prefix}_metrics.json", run.metrics)
        write_json(metric_dir / f"{prefix}_validation.json", run.validation)
        output_files.extend([f"metrics/{prefix}_metrics.json", f"metrics/{prefix}_validation.json"])
        if run.config.collect_evidence:
            assert run.ledger is not None and run.adjustments is not None
            assert run.plan_calls is not None and run.plan_trajectory is not None
            evaluation_ledger = run.ledger[run.ledger["date"] >= "2025-02-01"].copy()
            evaluation_adjustments = run.adjustments[run.adjustments["date"] >= "2025-02-01"].copy()
            events = emergency_events(evaluation_ledger)
            storage = storage_summary(evaluation_ledger)
            table1, table2, table3 = specified_tables(evaluation_ledger, storage, events)
            frames = {
                f"{prefix}_interval_ledger.csv": evaluation_ledger,
                f"{prefix}_adjustment_events.csv": evaluation_adjustments,
                f"{prefix}_planning_calls.csv": run.plan_calls,
                f"{prefix}_planning_trajectory.csv": run.plan_trajectory,
                f"{prefix}_emergency_events.csv": events,
                f"{prefix}_storage_4h.csv": storage,
                f"{prefix}_specified_date_table1.csv": table1,
                f"{prefix}_specified_date_table2.csv": table2,
                f"{prefix}_specified_date_table3.csv": table3,
            }
            for name, frame in frames.items():
                save_frame(frame, table_dir / name)
                output_files.append(f"tables/{name}")

    write_json(metric_dir / "historical_forecast_diagnostics.json", forecast_diagnostics(data,
      archives["historical"]))
    write_json(metric_dir / "official_forecast_diagnostics.json", forecast_diagnostics(data,
      archives["official"]))
    output_files.extend(["metrics/historical_forecast_diagnostics.json",
      "metrics/official_forecast_diagnostics.json"])

    q42_main = by_id[comparison_ids[0][1]]
    q42_base = by_id[comparison_ids[0][2]]
    q43_main = by_id[comparison_ids[1][1]]
    q43_base = by_id[comparison_ids[1][2]]
    degeneracy: dict[str, Any] = {}
    for key, run in [("q42", q42_main), ("q43", q43_main)]:
        assert run.ledger is not None
        eval_ledger = run.ledger[run.ledger["date"] >= "2025-02-01"]
        q_profiles = eval_ledger.pivot(index="date", columns="interval_index", values="original_contract_kwh")
        y_profiles = eval_ledger.pivot(index="date", columns="interval_index", values="final_contract_kwh")
        events = emergency_events(eval_ledger)
        degeneracy[key] = {
            "unique_original_contract_profiles_rounded_6dp": int(len(np.unique(np.round(q_profiles.to_numpy(), 6),
              axis=0))),
            "unique_final_contract_profiles_rounded_6dp": int(len(np.unique(np.round(y_profiles.to_numpy(), 6),
              axis=0))),
            "nonzero_final_vs_original_intervals": int(np.count_nonzero(np.abs(eval_ledger["final_contract_kwh"] -
              eval_ledger["original_contract_kwh"]) > EVENT_TOLERANCE)),
```

```
            "emergency_event_count": int(len(events)),
        }
    run_summary = {
        "schema_version": 1,
        "question": "Q4",
        "round": output_round,
        "implementation_target": "python",
        "random_seed": RANDOM_SEED,
        "approved_decision_id": (
            "q4_method_choice_joint_scenario_v1"
            if suite == "extension"
            else "q4_method_choice_inherit_q2_q3_variable_price_v2"
        ),
        "suite": suite,
        "days_executed": effective_days,
        "methods": [
            {
                "method_id": run.config.strategy_id,
                "role": run.config.role,
                "script": "code/Q4/q4_model.py",
                "status": "success",
                "execution_time_seconds": run.runtime_seconds,
                "input_files": [
                    str(ACTUAL_PATH.relative_to(PROJECT_ROOT.parent)),
                    *(
                        [str(FORECAST_PATH.relative_to(PROJECT_ROOT.parent))]
                        if run.config.question_variant == "Q4-3"
                        else []
                    ),
                    str(FLOATING_PRICE_PATH.relative_to(PROJECT_ROOT.parent)),
                ],
                "output_files": [],
                "figure_files": [],
                "metrics_summary": run.metrics,
                "warnings": [
                    *(
                        [
                            "future prices are unknown and settlements use actual delivery-interval prices",
                            "shared planning storage trajectory is a tractable proxy for the real-time feedback
                              policy",
                        ]
                        if suite == "extension"
                        else [
                            FORMAL_PRICE_INFORMATION,
                            "正式 Q4 不另建价格预测或价荷联合情景；上述完整价格路径仅用于题给数据下的离线重算。
                              ",
                        ]
                    ),
                ],
                "errors": [],
            }
            for run in runs
        ],
        "comparison": {
            "q42_main_minus_baseline_cost_yuan": q42_main.metrics["actual_total_cost_yuan"] -
              q42_base.metrics["actual_total_cost_yuan"],
            "q42_main_cost_change_vs_baseline_pct": (q42_main.metrics["actual_total_cost_yuan"] /
              q42_base.metrics["actual_total_cost_yuan"] - 1.0) * 100.0,
            "q43_main_minus_baseline_cost_yuan": q43_main.metrics["actual_total_cost_yuan"] -
              q43_base.metrics["actual_total_cost_yuan"],
            "q43_main_cost_change_vs_baseline_pct": (q43_main.metrics["actual_total_cost_yuan"] /
              q43_base.metrics["actual_total_cost_yuan"] - 1.0) * 100.0,
            "q43_minus_q42_main_cost_yuan": q43_main.metrics["actual_total_cost_yuan"] -
              q42_main.metrics["actual_total_cost_yuan"],
            "q43_cost_change_vs_q42_main_pct": (q43_main.metrics["actual_total_cost_yuan"] /
              q42_main.metrics["actual_total_cost_yuan"] - 1.0) * 100.0,
        },
```

```
        "fallback_trigger": {
            "fallback_id": "F4",
            "condition": "any DA/TR stage failure or hard constraint violation",
            "observed": any(run.metrics["fallback_event_count"] for run in runs),
            "evidence": {run.config.strategy_id: run.validation["fallback_events"] for run in runs if
              run.validation["fallback_events"]},
        },
        "input_hashes": data.hashes,
        "environment": environment_info(),
        "validation": {run.config.strategy_id: run.validation for run in runs},
        "output_degeneracy": degeneracy,
        "output_files": output_files,
    }
    write_json(round_dir / "run_summary.json", run_summary)


def main() -> None:
    parser = argparse.ArgumentParser(description="Solve Q4 by inheriting Q2/Q3 and replacing only the price
      matrix")
    parser.add_argument("--suite", choices=("pilot", "core", "extension"), default="core")
    parser.add_argument("--max-days", type=int, default=None)
    args = parser.parse_args()
    run_suite(args.suite, args.max_days)


if __name__ == "__main__":
    raise SystemExit(" 请从辅助材料根目录运行 run.py, 选择对应问题。")
```

D.7.2 code/verify_inheritance.py

```
from __future__ import annotations

import json
from dataclasses import replace
from pathlib import Path

import numpy as np
from openpyxl import load_workbook

from q4_model import (
    N_DAYS,
    PolicyConfig,
    build_forecast_archive,
    build_inherited_risk_margins,
    load_data,
    run_policy,
    write_json,
)


PROJECT_ROOT = Path(__file__).resolve().parents[1]
FIXED_PRICE_PATH = PROJECT_ROOT / "data" / " 附件 1.xlsx"
OUTPUT_PATH = (
    PROJECT_ROOT
    / "results"
    / "Q4"
    / "experiments"
    / "round2_inherited_price"
    / "metrics"
    / "inheritance_regression.json"
)


EXPECTED = {
    "q42": {
```

```
        "actual_total_cost_yuan": 13923903.386969224,
        "original_contract_kwh": 21605749.445191227,
        "emergency_purchase_kwh": 111928.37713393185,
        "initial_soc_evaluation_kwh": 2474.99712581418,
        "terminal_soc_evaluation_kwh": 1718.6737161078236,
    },
    "q43": {
        "actual_total_cost_yuan": 13833843.322638258,
        "original_contract_kwh": 21848677.8903264,
        "final_contract_kwh": 21057849.899175413,
        "adjustment_increase_kwh": 502453.2704979111,
        "adjustment_decrease_kwh": 1293281.261648901,
        "emergency_purchase_kwh": 51525.69845274206,
        "initial_soc_evaluation_kwh": 2492.6295222483786,
        "terminal_soc_evaluation_kwh": 1661.099380902042,
    },
}


def load_fixed_prices() -> np.ndarray:
    rows = list(load_workbook(FIXED_PRICE_PATH, read_only=True, data_only=True).active.values)
    price = np.asarray([row[1] for row in rows[1:]], dtype=float)
    if price.shape != (144,) or not np.isfinite(price).all() or np.any(price <= 0):
        raise ValueError(" 附件 1 固定电价向量无效")
    return price


def compare(actual: dict[str, float], expected: dict[str, float]) -> dict[str, float]:
    return {key: float(actual[key] - value) for key, value in expected.items()}


def main() -> None:
    data = load_data()
    fixed_data = replace(data, actual_price=np.tile(load_fixed_prices(), (N_DAYS, 1)))

    historical = build_forecast_archive(fixed_data, "historical")
    q42_config = PolicyConfig(
        "Q4_inheritance_regression_Q2_M3",
        "regression",
        "Q4-2",
        "historical",
        "inherited",
        False,
        alpha=0.8,
    )
    q42 = run_policy(
        fixed_data,
        historical,
        q42_config,
        margins=build_inherited_risk_margins(historical, "Q4-2", 0.8),
    )

    official = build_forecast_archive(fixed_data, "official")
    q43_config = PolicyConfig(
        "Q4_inheritance_regression_Q3_M1I",
        "regression",
        "Q4-3",
        "official",
        "inherited",
        True,
        alpha=0.8,
    )
    q43 = run_policy(
        fixed_data,
        official,
        q43_config,
        margins=build_inherited_risk_margins(official, "Q4-3", 0.8),
    )
```

```
104
105     q42_residuals = compare(q42.metrics, EXPECTED["q42"])
106     q43_residuals = compare(q43.metrics, EXPECTED["q43"])
107     max_abs = max(abs(value) for value in [*q42_residuals.values(), *q43_residuals.values()])
108     payload = {
109         "schema_version": 1,
110         "question_id": "Q4",
111         "purpose": "prove that the corrected implementation reproduces the approved Q2/Q3 results when
              attachment-4 prices are replaced by the original fixed price vector",
112         "fixed_price_source": FIXED_PRICE_PATH.relative_to(PROJECT_ROOT).as_posix(),
113         "q42_residuals": q42_residuals,
114         "q43_residuals": q43_residuals,
115         "maximum_absolute_residual": max_abs,
116         "tolerance": 5e-5,
117         "q42_hard_checks_pass": q42.validation["hard_checks_pass"],
118         "q43_hard_checks_pass": q43.validation["hard_checks_pass"],
119         "status": "PASS" if max_abs <= 5e-5 and q42.validation["hard_checks_pass"] and
              q43.validation["hard_checks_pass"] else "FAIL",
120     }
121     write_json(OUTPUT_PATH, payload)
122     print(json.dumps(payload, ensure_ascii=False, indent=2))
123     if payload["status"] != "PASS":
124         raise RuntimeError(payload)
125
126
127 if __name__ == "__main__":
128     raise SystemExit(" 请运行根目录 run.py --task full, 以先生成本次主方案和基准结果。")
129
```

D.7.3 code/run_common_soc_robustness.py

```
1  from __future__ import annotations
2
3  import json
4  from datetime import datetime
5  from pathlib import Path
6  from typing import Any
7
8  from q4_model import (
9      EVALUATION_START_INDEX,
10     FORMAL_PRICE_INFORMATION,
11     N_DAYS,
12     PROJECT_ROOT,
13     PolicyConfig,
14     build_forecast_archive,
15     build_inherited_risk_margins,
16     load_data,
17     run_policy,
18     write_json,
19 )
20
21
22 ROUND_DIR = PROJECT_ROOT / "results" / "Q4" / "experiments" / "round2_inherited_price"
23 OUTPUT_PATH = PROJECT_ROOT / "robustness" / "Q4" / "q4_robustness_summary.json"
24
25
26 def load_formal_metrics(name: str) -> dict[str, Any]:
27     path = ROUND_DIR / "metrics" / name
28     with path.open("r", encoding="utf-8") as handle:
29         return json.load(handle)
30
31
32 def comparison(main: dict[str, Any], baseline: dict[str, Any]) -> dict[str, float]:
33     main_cost = float(main["actual_total_cost_yuan"])
34     baseline_cost = float(baseline["actual_total_cost_yuan"])
35     main_emergency = float(main["emergency_purchase_kwh"])
```

```
    baseline_emergency = float(baseline["emergency_purchase_kwh"])
    return {
        "main_minus_baseline_cost_yuan": main_cost - baseline_cost,
        "main_cost_change_vs_baseline_pct": (main_cost / baseline_cost - 1.0) * 100.0,
        "cost_saving_vs_baseline_yuan": baseline_cost - main_cost,
        "cost_saving_vs_baseline_pct": (1.0 - main_cost / baseline_cost) * 100.0,
        "main_minus_baseline_emergency_kwh": main_emergency - baseline_emergency,
        "emergency_reduction_vs_baseline_kwh": baseline_emergency - main_emergency,
        "emergency_reduction_vs_baseline_pct": (1.0 - main_emergency / baseline_emergency) * 100.0,
    }


def selected_metrics(run: Any) -> dict[str, Any]:
    keys = (
        "strategy_id",
        "role",
        "question_variant",
        "alpha",
        "simulation_start",
        "simulation_days",
        "run_initial_soc_kwh",
        "terminal_soc_evaluation_kwh",
        "original_contract_kwh",
        "final_contract_kwh",
        "adjustment_increase_kwh",
        "adjustment_decrease_kwh",
        "adjustment_transaction_count",
        "contract_total_cost_yuan",
        "emergency_cost_yuan",
        "emergency_purchase_kwh",
        "actual_total_cost_yuan",
        "planning_calls_full_run",
        "degenerate_scenario_calls_full_run",
        "fallback_event_count",
    )
    return {
        **{key: run.metrics[key] for key in keys},
        "runtime_seconds": run.runtime_seconds,
        "hard_checks_pass": run.validation["hard_checks_pass"],
        "maximum_reported_validation_residual": max(
            abs(float(value))
            for key, value in run.validation.items()
            if isinstance(value, (int, float)) and not isinstance(value, bool) and key !=
              "incremental_example_100_to_80_to_100_yuan"
        ),
    }


def main() -> None:
    data = load_data()
    archives = {
        "Q4-2": build_forecast_archive(data, "historical"),
        "Q4-3": build_forecast_archive(data, "official"),
    }
    formal = {
        "Q4-2": {
            "main": load_formal_metrics("q42_main_metrics.json"),
            "baseline": load_formal_metrics("q42_baseline_metrics.json"),
        },
        "Q4-3": {
            "main": load_formal_metrics("q43_main_metrics.json"),
            "baseline": load_formal_metrics("q43_baseline_metrics.json"),
        },
    }
    configs = {
        "Q4-2": (
            PolicyConfig("M4I_Q42_common_soc_alpha08", "main", "Q4-2", "historical", "inherited", False,
              alpha=0.8),
```

```
            PolicyConfig("B4I_Q42_common_soc_b0", "usable_baseline", "Q4-2", "historical", "inherited", False,
              alpha=None),
        ),
        "Q4-3": (
            PolicyConfig("M4I_Q43_common_soc_alpha08", "main", "Q4-3", "official", "inherited", True, alpha=0.8),
            PolicyConfig("B4I_Q43_common_soc_b0", "usable_baseline", "Q4-3", "official", "inherited", True,
              alpha=None),
        ),
    }

    variants: dict[str, Any] = {}
    all_checks_pass = True
    for variant in ("Q4-2", "Q4-3"):
        common_soc = float(formal[variant]["main"]["initial_soc_evaluation_kwh"])
        runs = []
        for config in configs[variant]:
            print(f"START {config.strategy_id} common_soc={common_soc:.12f}", flush=True)
            margins = build_inherited_risk_margins(archives[variant], variant, config.alpha)
            run = run_policy(
                data,
                archives[variant],
                config,
                margins=margins,
                max_days=N_DAYS,
                start_day_index=EVALUATION_START_INDEX,
                initial_soc_override=common_soc,
            )
            print(
                f"DONE {config.strategy_id} cost={run.metrics['actual_total_cost_yuan']:.2f} "
                f"emergency={run.metrics['emergency_purchase_kwh']:.3f} "
                f"terminal_soc={run.metrics['terminal_soc_evaluation_kwh']:.3f} "
                f"hard_checks={run.validation['hard_checks_pass']}",
                flush=True,
            )
            all_checks_pass = all_checks_pass and bool(run.validation["hard_checks_pass"])
            runs.append(run)

        paired = comparison(runs[0].metrics, runs[1].metrics)
        formal_comparison = comparison(formal[variant]["main"], formal[variant]["baseline"])
        variants[variant] = {
            "common_feb01_soc_kwh": common_soc,
            "common_soc_selection_rule": "use the formal main policy's February 1 initial SOC for this
              subquestion",
            "main": selected_metrics(runs[0]),
            "baseline": selected_metrics(runs[1]),
            "paired_common_soc_comparison": paired,
            "formal_end_to_end_whole_policy_comparison": formal_comparison,
            "interpretation": (
                "The paired replay starts both policies from the same February 1 SOC and therefore isolates the "
                "evaluation-period risk-margin difference conditional on that selected SOC. The formal comparison
                  also "
                "contains the policies' different January state trajectories and is not an alpha-only estimate."
            ),
        }

    payload = {
        "schema_version": 1,
        "question": "Q4",
        "test_id": "q4_round2_common_feb01_soc_paired_replay_v1",
        "generated_at": datetime.now().astimezone().isoformat(timespec="seconds"),
        "status": "PASS" if all_checks_pass else "FAIL",
        "claim_scope": "exploratory paired replay; formal runs and workbooks are unchanged",
        "price_information": FORMAL_PRICE_INFORMATION,
        "fixed_elements": [
            "pre-existing historical forecast calibration and official forecast archive",
            "attachment 4 target-day full 144-interval price path",
            "forecast, feedback, storage, settlement, and all other Q4 round2 rules",
```

```
        ],
        "changed_element": "both main and corresponding zero-margin baseline start at the same February 1 SOC",
        "common_soc_choice_limit": (
            "Each subquestion uses its formal main policy's February 1 SOC as the common state. "
            "This is a transparent conditional comparison, not proof that the result is invariant to every
              feasible common SOC."
        ),
        "variants": variants,
        "input_files": ["data/附件 2.xlsx", "data/附件 3.xlsx", "data/附件 4.xlsx"],
        "source_files": [f"metrics/{name}" for name in (
            "q42_main_metrics.json", "q42_baseline_metrics.json",
            "q43_main_metrics.json", "q43_baseline_metrics.json")],
        "source_files_base": " 本 JSON 文件所在的本次 Q4 计算输出目录",
    }
    write_json(OUTPUT_PATH, payload)
    print(f"WROTE {OUTPUT_PATH}", flush=True)


if __name__ == "__main__":
    raise SystemExit(" 请运行根目录 run.py --task full, 以先生成本次主方案和基准结果。")
```

D.8 Excel 读写、导出与核对程序

D.8.1 code/export_results.py

```
""" 将本次计算结果写入 Excel; 只使用 Python 标准库, 不依赖 Node.js 或 Excel 软件。

根目录的既有结果文件仅提供工作表名称、表头与样式。导出时清空所有数据行,
用本次模型结果重新填入; 写入后逐格读取核对。不会覆盖根目录的提交结果。
"""
from __future__ import annotations

import math
import posixpath
import re
import zipfile
from datetime import date, datetime, timedelta
from pathlib import Path
import xml.etree.ElementTree as ET
from xml.sax.saxutils import escape

NS = {"s": "http://schemas.openxmlformats.org/spreadsheetml/2006/main"}
REL = "http://schemas.openxmlformats.org/officeDocument/2006/relationships"
START = "2025-02-01"
END = "2025-12-31"


def column_name(index: int) -> str:
    text = ""
    while index:
        index, digit = divmod(index - 1, 26)
        text = chr(65 + digit) + text
    return text


def column_index(address: str) -> int:
    value = 0
    for ch in re.match(r"[A-Z]+", address).group():
        value = value * 26 + ord(ch) - 64
    return value


def excel_date(value) -> int:
    if isinstance(value, datetime):
        value = value.date()
```

```
    elif not isinstance(value, date):
        value = date.fromisoformat(str(value)[:10])
    return (value - date(1899, 12, 30)).days


def sheet_paths(archive: zipfile.ZipFile) -> dict[str, str]:
    book = ET.fromstring(archive.read("xl/workbook.xml"))
    rels = ET.fromstring(archive.read("xl/_rels/workbook.xml.rels"))
    targets = {r.attrib["Id"]: r.attrib["Target"] for r in rels}
    result = {}
    for node in book.find("s:sheets", NS):
        target = targets[node.attrib[f"{{{REL}}}id"]]
        result[node.attrib["name"]] = target.lstrip("/") if target.startswith("/") else posixpath.normpath("xl/" +
          target)
    return result


def read_workbook(path: Path) -> dict[str, dict[str, object]]:
    """Read literal/cached cell values without launching spreadsheet software."""
    result = {}
    with zipfile.ZipFile(path) as archive:
        bad = archive.testzip()
        if bad:
            raise ValueError(f" 工作簿压缩结构损坏: {path.name}/{bad}")
        strings = []
        if "xl/sharedStrings.xml" in archive.namelist():
            strings = ["".join(n.itertext()) for n in ET.fromstring(archive.read("xl/sharedStrings.xml"))]
        for title, member in sheet_paths(archive).items():
            cells = {}
            root = ET.fromstring(archive.read(member))
            for cell in root.findall("s:sheetData/s:row/s:c", NS):
                address = cell.attrib["r"]
                value = cell.find("s:v", NS)
                kind = cell.get("t")
                if kind == "e":
                    raise ValueError(f"{path.name}/{title}/{address}: 单元格错误")
                if kind == "inlineStr":
                    content = cell.find("s:is", NS)
                    parsed = "".join(content.itertext()) if content is not None else ""
                elif value is None or value.text is None:
                    parsed = None
                elif kind == "s":
                    parsed = strings[int(value.text)]
                elif kind in ("str", "d"):
                    parsed = value.text
                elif kind == "b":
                    parsed = bool(int(value.text))
                else:
                    parsed = float(value.text)
                    if not math.isfinite(parsed):
                        raise ValueError(f"{path.name}/{title}/{address}: 非有限数值")
                if parsed is not None:
                    cells[address] = parsed
            result[title] = cells
    return result


def _sheet_xml(original: bytes, rows: list[list[object]]) -> bytes:
    text = original.decode("utf-8")
    match = re.search(r"<(?P<p>(?:[\w]+:)?)sheetData\b[^>]*>.*?</(?P=p)sheetData>", text, re.S)
    if match is None:
        raise ValueError(" 缺少 sheetData, 不能使用该工作表作为格式基准")
    prefix = match.group("p")
    root = ET.fromstring(original)
    original_rows = root.findall("s:sheetData/s:row", NS)
    header = original_rows[0]
    header_end = re.search(r"<" + prefix + r"row\b.*?</" + prefix + r"row>", match.group(), re.S)
    styles = {}
```

```
    for row in original_rows[1:]:
        for cell in row:
            styles.setdefault(column_index(cell.attrib["r"]), cell.get("s", "0"))
    chunks = [f"<{prefix}sheetData>", header_end.group()]
    width = max(column_index(c.attrib["r"]) for c in header)
    for row_index, values in enumerate(rows, 2):
        if len(values) != width:
            raise ValueError(f" 输出第 {row_index} 行列数应为 {width}, 实际为 {len(values)}")
        chunks.append(f'<{prefix}row r="{row_index}" ht="18" customHeight="1">')
        for col, value in enumerate(values, 1):
            address = column_name(col) + str(row_index)
            attrs = f'r="{address}" s="{styles.get(col, "0")}"'
            if value is None:
                chunks.append(f"<{prefix}c {attrs} />")
            elif isinstance(value, str):
                chunks.append(f'<{prefix}c {attrs}
                  t="inlineStr"><{prefix}is><{prefix}t>{escape(value)}</{prefix}t></{prefix}is></{prefix}c>')
            else:
                number = float(value)
                if not math.isfinite(number):
                    raise ValueError(f"{address} 不能输出非有限值")
                # 仅去除浮点噪声; 不把显示精度应用于模型计算。
                number = 0.0 if abs(number) < 1e-9 else number
                chunks.append(f'<{prefix}c {attrs} t="n"><{prefix}v>{number:.17g}</{prefix}v></{prefix}c>')
        chunks.append(f"</{prefix}row>")
    chunks.append(f"</{prefix}sheetData>")
    text = text[:match.start()] + "".join(chunks) + text[match.end():]
    # 原表没有数据公式、表对象或合并数据单元格; 保留页边距、列宽和冻结设置。
    if root.find("s:mergeCells", NS) is not None or root.find("s:tableParts", NS) is not None:
        raise ValueError(" 格式基准含额外合并/表对象, 需单独核对")
    dimension = f'<{prefix}dimension ref="A1:{column_name(width)}{len(rows)+1}" />'
    if re.search(r"<" + prefix + r"dimension\b[^>]*/>", text):
        text = re.sub(r"<" + prefix + r"dimension\b[^>]*/>", dimension, text, count=1)
    return text.encode("utf-8")


def write_workbook(template: Path, output: Path, tables: dict[str, list[list[object]]]) -> dict:
    if template.resolve() == output.resolve():
        raise ValueError(" 不能覆盖根目录正式结果; 请选择独立输出目录")
    output.parent.mkdir(parents=True, exist_ok=True)
    pending = output.with_suffix(".xlsx.tmp")
    try:
        with zipfile.ZipFile(template) as archive:
            paths = sheet_paths(archive)
            if set(paths) != set(tables):
                raise ValueError(f"{output.name} 工作表集合不匹配")
            changed = {paths[name]: _sheet_xml(archive.read(paths[name]), rows) for name, rows in tables.items()}
            with zipfile.ZipFile(pending, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=6) as target:
                for info in archive.infolist():
                    target.writestr(info, changed.get(info.filename, archive.read(info.filename)))
        pending.replace(output)
    finally:
        if pending.exists():
            pending.unlink()
    # 核对写入后的值与本次计算值, 而非仅检验文件能打开。
    actual = read_workbook(output)
    checked = 0
    for name, rows in tables.items():
        expected_addresses = {column_name(j)+str(i) for i,row in enumerate(rows,2) for j,v in enumerate(row,1) if
          v is not None}
        for i, row in enumerate(rows, 2):
            for j, value in enumerate(row, 1):
                address = column_name(j) + str(i)
                parsed = actual[name].get(address)
                if value is None:
                    if parsed is not None:
                        raise ValueError(f"{name}!{address} 应为空")
                elif isinstance(value, str):
```

```
                    if parsed != value:
                        raise ValueError(f"{name}!{address} 文本写入不一致")
                else:
                    if parsed is None or abs(float(parsed) - float(value)) > 2e-8:
                        raise ValueError(f"{name}!{address} 数值写入不一致")
                    checked += 1
        stale = [a for a in actual[name] if int(re.search(r"\d+",a).group())>1 and a not in expected_addresses]
        if stale:
            raise ValueError(f"{name} 含旧数据残留: {stale[:3]}")
    return {"numeric_cells_checked": checked, "status": "PASS"}


def export_q1(root: Path, output: Path, data, dispatch) -> dict:
    plan = [[label, float(amount)] for label, amount in zip(data.interval_labels, dispatch.purchase)]
    storage = []
    for block in range(6):
        start = block*24
        label = f"{block*4:02d}:00-{(block+1)*4:02d}:00"
        storage.append([label, float(dispatch.charge[start:start+24].sum()),
          float(dispatch.discharge[start:start+24].sum()),
                        "0:00" if block == 0 else "24:00" if block == 1 else None,
                        6000.0 if block == 0 else float(dispatch.soc_end[-1]) if block == 1 else None])
    return write_workbook(root/"result1.xlsx", output/"result1.xlsx", {" 计划购电量":plan," 充放电量":storage})


def export_run(root: Path, output: Path, name: str, run, model, is_q2: bool = False, rolling: bool = False) ->
  dict:
    ledger = run.ledger
    if ledger is None:
        raise ValueError(" 缺少本次运行的逐时段结果, 不能导出")
    ledger = ledger[(ledger["date"]>=START)&(ledger["date"]<=END)].copy().sort_values(["date","interval_index"])
    daily = run.daily[(run.daily["date"]>=START)&(run.daily["date"]<=END)].copy()
    expected_dates=[(date(2025,2,1)+timedelta(days=i)).isoformat() for i in range(334)]
    if len(ledger)!=48096 or daily["date"].tolist()!=expected_dates:
        raise ValueError(" 导出范围必须为 2025 年 2—12 月的 334 天")
    by_day=daily.set_index("date")
    purchase=[];adjusted=[];storage=[]
    for day,group in ledger.groupby("date",sort=True):
        ordered=group.sort_values("interval_index")
        if ordered["interval_index"].tolist()!=list(range(1,145)):
            raise ValueError(f"{day} 的时段不完整")
        summary=by_day.loc[day]
        qfield="planned_purchase_kwh" if is_q2 else "original_contract_kwh"
        cfield="contract_cost_yuan" if is_q2 else "original_contract_cost_yuan"
        purchase.append([excel_date(day),*map(float,ordered[qfield]),float(summary[qfield]),float(summary[cfield]」
          )])
        if rolling:
            adjusted.append([excel_date(day),*map(float,ordered["final_contract_kwh"]),float(summary["final_contr」
              act_kwh"]),float(summary["actual_total_cost_yuan"])])
        initial=float(ordered.iloc[0]["soc_start_kwh"])
        terminal=float(ordered.iloc[-1]["soc_end_kwh"])
        for block in range(6):
            part=ordered.iloc[block*24:(block+1)*24]
            storage.append([excel_date(day) if block==0 else None,f"{block*4:02d}:00-{(block+1)*4:02d}:00",
                            float(part["actual_charge_kwh"].sum()),float(part["actual_discharge_kwh"].sum()),
                            "0:00" if block==0 else "24:00" if block==1 else None,
                            initial if block==0 else terminal if block==1 else None])
    events=model.emergency_events(ledger)
    emergency=[];prev=None
    for record in events.to_dict("records"):
        day=record["date"]
        emergency.append([excel_date(day) if day!=prev else
          None,record["emergency_time_range"],float(record["emergency_purchase_kwh"])])
        prev=day
    tables={" 计划购电量":purchase}
    if rolling:tables[" 调整购电量"]=adjusted
    tables.update({" 充放电量":storage," 紧急购电量":emergency})
```

```
    return write_workbook(root/name,output/name,tables)


def compare_workbooks(reference: Path, computed: Path, tolerance: float = 5e-5) -> dict:
    """Compare all data cells; header and style differences are not numeric evidence."""
    old=read_workbook(reference);new=read_workbook(computed)
    differences=[];count=0;maximum=0.0
    if set(old)!=set(new):raise ValueError(" 工作表集合不同")
    for name in old:
        a={k:v for k,v in old[name].items() if int(re.search(r"\d+",k).group())>1}
        b={k:v for k,v in new[name].items() if int(re.search(r"\d+",k).group())>1}
        for key in a.keys()|b.keys():
            x=a.get(key);y=b.get(key)
            if isinstance(x,(int,float)) and isinstance(y,(int,float)):
                delta=abs(float(x)-float(y));maximum=max(maximum,delta);count+=1
                if delta>tolerance:differences.append((name,key,x,y))
            elif x!=y:differences.append((name,key,x,y))
    return {"file":reference.name,"numeric_cells_compared":count,"maximum_absolute_difference":maximum,
            "tolerance":tolerance,"different_cells":len(differences),"examples":differences[:8],
            "status":"PASS" if not differences else "DIFFERENT"}
```

D.8.2 code/verify_results.py

```
""" 从原始输入及现有结果重算费用、储能反馈与逐次调整；仅使用标准库。

python run.py --task verify
本程序不是重新求解优化模型；重算模型使用 --task all 或 --task full。
"""
from __future__ import annotations
import csv
import gzip
import math
from datetime import date,timedelta
from pathlib import Path
from export_results import read_workbook, column_name, excel_date

TOL = 5e-5
EXPECTED_COST = {"result1.xlsx":35126.94859028963,"result2.xlsx":13923903.386969224,
                 "result3.xlsx":13833843.322638258,"result4-2.xlsx":14571629.312229294,
                 "result4-3.xlsx":14487237.236657604}


def verify_package(root: Path) -> dict:
    checks=0;maximum=0.0
    def near(actual,expected,label,tol=TOL):
        nonlocal checks,maximum
        checks+=1
        delta=abs(float(actual)-float(expected));maximum=max(maximum,delta)
        if not math.isfinite(delta) or delta>tol:
            raise ValueError(f" 核对失败 {label}: {actual} != {expected}; 差值 {delta}")
    fixed=next(iter(read_workbook(root/'data'/'附件 1.xlsx').values()))
    actual=read_workbook(root/'data'/'附件 2.xlsx')
    floating=next(iter(read_workbook(root/'data'/'附件 4.xlsx').values()))
    price0=[fixed[f'B{i+2}'] for i in range(144)]
    names=list(EXPECTED_COST);report={}
    for name in names:
        wb=read_workbook(root/name)
        plan=wb['计划购电量'];store=wb['充放电量']
        if name=='result1.xlsx':
            q=[plan[f'B{i+2}'] for i in range(144)]
            cost=math.fsum(p*v for p,v in zip(price0,q))
            near(cost,EXPECTED_COST[name],name+'总费')
            near(store['E2'],6000,name+'期初');near(store['E3'],6000,name+'期末')
            near(0.9*math.fsum(store[f'B{i}'] for i in range(2,8)),math.fsum(store[f'C{i}'] for i in
              range(2,8))/0.9,name+'首末能量')
```

```
        report[name]={'total_cost_yuan':cost,'intervals':144,'scope':'购电费及四小时储能汇总核对'}
        print(f'[PASS] {name}: {cost:,.2f} 元',flush=True)
        continue
    rolling='调整购电量' in wb
    variable=name.startswith('result4')
    final=wb['调整购电量'] if rolling else plan
    record_by_day={}
    if rolling:
        fname='q3_adjustment_events.csv.gz' if name=='result3.xlsx' else 'q43_main_adjustment_events.csv.gz'
        with gzip.open(root/'comparison'/fname,'rt',encoding='utf-8-sig',newline='') as handle:
            for row in csv.DictReader(handle):record_by_day.setdefault(row['date'],[]).append(row)
    all_expected_events=[];total=0.0;emergency_total=0.0;previous_end=None;contract_cost=0.0;adjustment_cost=0.0
    for d in range(334):
        day=date(2025,2,1)+timedelta(days=d);serial=excel_date(day);r=d+2;rawrow=d+33
        near(plan[f'A{r}'],serial,name+'日期')
        if rolling:near(final[f'A{r}'],serial,name+'调整页日期')
        p=[floating[f'{column_name(t+2)}{rawrow}'] if variable else price0[t] for t in range(144)]
        q=[plan[f'{column_name(t+2)}{r}'] for t in range(144)]
        y=[final[f'{column_name(t+2)}{r}'] for t in range(144)]
        near(plan[f'EP{r}'],math.fsum(q),name+'日初总电量')
        basecost=math.fsum(p[t]*q[t] for t in range(144));near(plan[f'EQ{r}'],basecost,name+'日初合同费')
        fees=[]
        if rolling:
            current=q.copy();events=record_by_day.get(day.isoformat(),[])
            expected_keys=[(hour,t) for hour in (6,12,18) for t in range(hour*6+1,145)]
            found_keys=[(int(row['release_hour']),int(row['interval_index'])) for row in events]
            if found_keys!=expected_keys:raise ValueError(f'{day} 的合同调整记录顺序或范围不完整')
            for row in events:
                t=int(row['interval_index'])-1;old=float(row['previous_contract_kwh']);new=float(row['new_contract_kwh'])
                near(old,current[t],name+'上一合同');inc=max(new-old,0);dec=max(old-new,0)
                near(float(row['increase_kwh']),inc,name+'增购量');near(float(row['decrease_kwh']),dec,name+'减购量')
                fee=p[t]*(1.5*inc-.5*dec);near(float(row['incremental_adjustment_fee_yuan']),fee,name+'调整费')
                current[t]=new;fees.append(fee)
            for t in range(144):near(current[t],y[t],name+'最终合同')
            near(final[f'EP{r}'],math.fsum(y),name+'最终总电量')
        startrow=2+d*6;S=float(store[f'F{startrow}']);initial=S
        near(store[f'A{startrow}'],serial,name+'储能页日期')
        if previous_end is not None:near(S,previous_end,name+'跨日状态')
        charges=[];discharges=[];emergencies=[]
        for t in range(144):
            col=column_name(t+2)
            n=(float(actual['小区负载'][f'{col}{rawrow}'])-float(actual['光伏发电实际功率'][f'{col}{rawrow}']))/6
            gap=n-y[t]
            charge=min(max(-gap,0.0),5000/6,max(0.0,(10800-S)/.9))
            discharge=min(max(gap,0.0),5000/6,max(0.0,.9*(S-1200)))
            emergency=max(gap+charge-discharge,0.0);S+=.9*charge-discharge/.9
            if not 1200-TOL<=S<=10800+TOL:raise ValueError(f'{name} 储量越界')
            charges.append(charge);discharges.append(discharge);emergencies.append(emergency)
        for b in range(6):
            near(store[f'C{startrow+b}'],math.fsum(charges[b*24:(b+1)*24]),name+'四小时充电')
            near(store[f'D{startrow+b}'],math.fsum(discharges[b*24:(b+1)*24]),name+'四小时放电')
        near(store[f'F{startrow+1}'],S,name+'日末状态');previous_end=S
        dayemcost=math.fsum(5*p[t]*emergencies[t] for t in range(144))
        adj=math.fsum(fees);daycost=basecost+adj+dayemcost
        if rolling:near(final[f'EQ{r}'],daycost,name+'含紧急购电总费用')
        contract_cost+=basecost;adjustment_cost+=adj;total+=daycost;emergency_total+=math.fsum(emergencies)
        t=0
        while t<144:
            if emergencies[t]<=1e-6:t+=1;continue
            first=t
            while t+1<144 and emergencies[t+1]>1e-6:t+=1
            end=t+1
```

```
                interval=f'{first//6:02d}:{first%6*10:02d}-{end//6:02d}:{end%6*10:02d}'
                all_expected_events.append((serial,interval,math.fsum(emergencies[first:end])))
                t+=1
        # 确认只包含 334 天，而不是把 1 月合计混入结果。
        if f'A336' in plan:raise ValueError(f'{name} 含评价期以外日期')
        ev=wb['紧急购电量'];actual_events=[];current_date=None
        for r in range(2,2+len(all_expected_events)):
            current_date=ev.get(f'A{r}',current_date)
            actual_events.append((current_date,ev.get(f'B{r}'),ev.get(f'C{r}')))
        if f'B{2+len(all_expected_events)}' in ev:raise ValueError(name+'紧急购电事件数量不符')
        for a,b in zip(actual_events,all_expected_events):
            near(a[0],b[0],name+'补购日期')
            if a[1]!=b[1]:raise ValueError(f'{name} 补购区间不符: {a[1]} != {b[1]}')
            near(a[2],b[2],name+'补购事件电量')
        near(total,EXPECTED_COST[name],name+'评价期费用')
        report[name]={'intervals':48096,'total_cost_yuan':total,'original_contract_cost_yuan':contract_cost,
                      'adjustment_net_cost_yuan':adjustment_cost,'emergency_purchase_kwh':emergency_total,
                      'emergency_events':len(all_expected_events),'status':'PASS'}
        print(f'[PASS] {name}: {total:,.2f} 元; 334 天; 紧急购电 {emergency_total:,.2f} kWh',flush=True)
    return {'status':'PASS','numeric_checks':checks,'maximum_absolute_difference':maximum,'workbooks':report}

if __name__=='__main__':
    print(verify_package(Path(__file__).resolve().parents[1]))
```

D.9 补充实验运行入口

补充实验分别调用第六、七章采用的预测与调度函数，独立保存运行记录，不覆盖五份主结果工作簿。

D.9.1 code/run_supplement.py

```
""" 两项补充实验的运行与独立核验；调用与原实验相同的模型函数。"""
from __future__ import annotations
from pathlib import Path
import subprocess
import sys

ROOT = Path(__file__).resolve().parents[1]


def run(task: str, output: Path) -> None:
    output.mkdir(parents=True, exist_ok=False)
    jobs = [("forecast", "run_validation.py", "verify_validation.py", "results"),
            ("storage", "run_experiment.py", "verify_experiment.py", "full_run")]
    for name, runner, verifier, saved in jobs:
        folder = ROOT / "experiments" / name
        if task == "supplement":
            results = output / name
            subprocess.run([sys.executable, str(folder / runner),
                            "--output", str(results)], cwd=ROOT, check=True)
        elif task == "supplement_verify":
            results = folder / saved
        else:
            raise ValueError(f"Unknown supplementary task: {task}")
        report = output / f"{name}_verification.json"
        subprocess.run([sys.executable, str(folder / verifier),
                        "--output", str(results), "--report", str(report)],
                       cwd=ROOT, check=True)
    print(" 两项补充实验核验完成，输出目录：", output, flush=True)
```

D.10 预测、风险裕量与日末估值补充实验

本组程序实现同期延续与等权均值预测，按各自历史误差重新构造裕量，并比较日末价值系数。核验程序从保存的预测数组和逐时段记录重新计算误差、覆盖率、费用及储电量变化。

D.10.1 experiments/forecast/run_validation.py

```
"""Causal forecast baselines, empirical margin coverage, and inherited gamma checks."""
from __future__ import annotations
import argparse
import importlib.util
import json
import sys
import time
from dataclasses import replace
from pathlib import Path

sys.dont_write_bytecode=True
HERE=Path(__file__).resolve().parent
PROJECT=HERE.parents[1]
sys.path.insert(0,str(PROJECT/'code'))
import numpy as np
import pandas as pd
import q2_common as c
from q2_forecast_experiment import forecast, audit
from q2_full_feedback import run_full_feedback_strategy, MAIN_CONFIG


def save_json(path,value):
    Path(path).write_text(json.dumps(value,ensure_ascii=False,indent=2),encoding='utf-8')


def save_csv(path,frame):
    frame.to_csv(path,index=False,encoding='utf-8-sig',float_format='%.10f')


def baseline_forecast(data, mode, main):
    if mode=='adopted':return main
    load=np.full_like(data.load_kw,np.nan);pv=load.copy()
    for d in range(1,365):
        if mode=='seasonal_persistence':
            load[d]=data.load_kw[d-7 if d>=7 else d-1]
            pv[d]=data.pv_kw[d-1]
        elif mode=='equal_window_mean':
            same=[i for i in range(max(0,d-28),d) if data.dates[i].weekday()==data.dates[d].weekday()]
            if not same:
                same=[i for i in range(max(0,d-28),d) if
                  (data.dates[i].weekday()>=5)==(data.dates[d].weekday()>=5)]
            if not same:same=list(range(max(0,d-7),d))
            load[d]=data.load_kw[same].mean(axis=0)
            pv[d]=data.pv_kw[max(0,d-7):d].mean(axis=0)
        else:raise ValueError(mode)
    net=(load-pv)/6
    return replace(main,load_kw=load,pv_kw=pv,net_load_kwh=net,
                   errors_kwh=data.net_load_kwh-net,weight_mode=mode)


def forecast_metrics(data,f,margin):
    le=data.load_kw[31:]-f.load_kw[31:];pe=data.pv_kw[31:]-f.pv_kw[31:]
    e=f.errors_kwh[31:];b=margin[31:]
    return dict(n=int(e.size),load_mae_kw=float(abs(le).mean()),pv_mae_kw=float(abs(pe).mean()),
      net_mae_kwh=float(abs(e).mean()),net_rmse_kwh=float(np.sqrt((e**2).mean())),
      net_bias_kwh=float(e.mean()),net_error_q80_kwh=float(np.quantile(e,.8)),
      coverage_percent=float(100*np.mean(e<=b+1e-9)),
      excess_mean_kwh=float(np.maximum(e-b,0).mean()))


```

```
def coverage_row(question,label,e,b,**extra):
    e=np.asarray(e).ravel();b=np.asarray(b).ravel()
    covered=e<=b+1e-9
    return dict(question=question,group=label,n=int(len(e)),covered_n=int(covered.sum()),
       coverage_percent=float(100*covered.mean()),exceedance_percent=float(100*(~covered).mean()),
       mae_kwh=float(abs(e).mean()),mean_margin_kwh=float(b.mean()),
       excess_mean_kwh=float(np.maximum(e-b,0).mean()),**extra)


def independent_q2_margin(f):
    b=np.zeros((365,144))
    for d in range(8,365):
        ix=np.arange(max(1,d-28),d);ix=ix[f.valid[ix]]
        if len(ix)<7:continue
        for g in range(6):
            # Scalar independent call on explicitly enumerated historic errors.
            values=[f.errors_kwh[i,j] for i in ix for j in range(24*g,24*g+24)]
            b[d,g*24:g*24+24]=max(0.,float(np.quantile(values,.8)))
    return b


def main():
    ap=argparse.ArgumentParser();ap.add_argument('--output',type=Path,required=True)
    args=ap.parse_args();out=args.output;out.mkdir(parents=True,exist_ok=False)
    started=time.perf_counter()
    p2=PROJECT/'comparison'
    p3=PROJECT/'comparison'
    q3_path=PROJECT/'code/q3_model.py'
    spec=importlib.util.spec_from_file_location('q3_parameter_validation',q3_path)
    m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
    paths=[Path(__file__),Path(c.__file__),PROJECT/'code/q2_full_feedback.py',
      PROJECT/'code/q2_forecast_experiment.py',q3_path,c.PRICE_PATH,c.ACTUAL_PATH,m.FORECAST_PATH,
      p2/'q2_main_interval_ledger.csv.gz',p3/'q3_main_interval_ledger.csv.gz',
      p3/'sensitivity_comparison.csv']
    hashes={p.resolve().relative_to(PROJECT).as_posix():c.sha256(p) for p in paths}
    protocol=dict(status='DIAGNOSTIC_ONLY_NO_OFFICIAL_REPLACEMENT',evaluation=['2025-02-01','2025-12-31'],
      forecasting='one-day-at-a-time causal replay; every target uses only earlier days, Q3 uses released
        forecasts',
      baselines={'seasonal_persistence':'load previous week same slot; PV previous day same slot; before day7 load
        yesterday',
                 'equal_window_mean':'same weekdays within preceding28d load mean, preceding7d PV mean; no
                   exponential weights'},
      adopted='same weekdays preceding28d load and preceding7d PV, exp(-age/7) weights',
      margin='alpha=.8; each predictor reconstructs prior forecast errors using unchanged Q2 four-hour pooling',
      gamma_values=[0.,.5,1.],gamma_default=.5,
      initialization='all policy runs start Jan1 at6000kWh and carry own realized SOC; costs evaluated Feb-Dec',
      gamma_scope='Q2 and Q3 full policy reruns; forecasts and margins fixed within each question',
      coverage='fraction of actual net-load energy <= issued point forecast + nonnegative historical margin; not
        supply reliability',
      q3_coverage_scope='both latest forecast actually used in each6h block, and each release on all remaining
        same-day slots',
      no_new_parameter_selection=True,
      limitations=['Historical dataset has already informed method choices; this is causal retrospective
        validation, not a pristine holdout.',
                   'Jan1 state is common; different Feb1 states are consequences of January policy operation and
                     must be reported.'],
      source_hashes=hashes)
    save_json(out/'protocol.json',protocol)
    print('Loading Q2 and computing predeclared forecast baselines...',flush=True)
    data=c.load_q2_data();main_f=forecast(data,'weekday')
    modes=['adopted','seasonal_persistence','equal_window_mean']
    fs={mode:baseline_forecast(data,mode,main_f) for mode in modes}
    bs={mode:c.build_risk_margins(fs[mode],.8) for mode in modes}
    old2=pd.read_csv(p2/'q2_main_interval_ledger.csv.gz')
    reg={}
    for col,arr in [('forecast_load_kw',main_f.load_kw),('forecast_pv_kw',main_f.pv_kw),
                    ('point_forecast_net_kwh',main_f.net_load_kwh),('risk_margin_kwh',bs['adopted'])]:
```

```
    reg[col]=float(np.nanmax(abs(old2[col].to_numpy()-arr.ravel())))
assert max(reg.values())<2e-5,reg
reg['independent_margin']=float(abs(independent_q2_margin(main_f)-bs['adopted']).max())
assert reg['independent_margin']<1e-9
forecast_rows=[];runs={};validations={};gamma_rows=[]
for mode in modes:
    print('Q2 forecast policy '+mode,flush=True)
    result=run_full_feedback_strategy(data,fs[mode],bs[mode],replace(MAIN_CONFIG,strategy_id='forecast_'+mode))
    checks=audit(result.ledger,result.daily,data)
    assert checks['all_pass'] and result.validation['all_pass']
    validations['Q2_'+mode]={'engine':result.validation,'independent':checks}
    runs[mode]=result
    row=forecast_metrics(data,fs[mode],bs[mode]);row.update(mode=mode,
       actual_total_cost_yuan=result.metrics['actual_total_cost_yuan'],
       emergency_kwh=result.metrics['emergency_purchase_kwh'],
       initial_soc_feb1_kwh=float(result.daily.loc[result.daily.date=='2025-02-01','initial_soc_kwh'].iloc[0]))
    forecast_rows.append(row)
    save_csv(out/f'q2_{mode}_daily.csv',result.daily)
    ev=result.ledger[result.ledger.date>='2025-02-01']
    ev.to_csv(out/f'q2_{mode}_interval.csv.gz',index=False,encoding='utf-8-sig',float_format='%.10f',compression='gzip')
    print(json.dumps(row,ensure_ascii=False),flush=True)
# Only gamma changes here; the selected forecast and its margins are reused.
for gamma in [0.,.5,1.]:
    print(f'Q2 gamma {gamma}',flush=True)
    r=runs['adopted'] if gamma==.5 else run_full_feedback_strategy(data,main_f,bs['adopted'],replace(MAIN_CONFIG,gamma=gamma,strategy_id=f'gamma_{gamma}'))
    check=audit(r.ledger,r.daily,data);assert check['all_pass'] and r.validation['all_pass']
    validations[f'Q2_gamma_{gamma}']={'engine':r.validation,'independent':check}
    ev=r.ledger[r.ledger.date>='2025-02-01'];daily=r.daily[r.daily.date>='2025-02-01']
    gamma_rows.append(dict(question='Q2',gamma=gamma,actual_total_cost_yuan=r.metrics['actual_total_cost_yuan'],
      emergency_kwh=r.metrics['emergency_purchase_kwh'],initial_soc_feb1_kwh=float(ev.soc_start_kwh.iloc[0]),
      terminal_soc_kwh=float(ev.soc_end_kwh.iloc[-1]),mean_carryover_soc_kwh=float(daily.terminal_soc_kwh.iloc[:-1].mean()),
      upper_carryover_days=int((daily.terminal_soc_kwh.iloc[:-1]>=10800-1e-4).sum())))
    save_csv(out/f'q2_gamma_{gamma}_daily.csv',r.daily)
    if gamma!=.5:ev.to_csv(out/f'q2_gamma_{gamma}_interval.csv.gz',index=False,encoding='utf-8-sig',float_format='%.10f',compression='gzip')
    print(json.dumps(gamma_rows[-1]),flush=True)
cols=['planned_purchase_kwh','actual_charge_kwh','actual_discharge_kwh','emergency_purchase_kwh','soc_end_kwh','contract_cost_yuan','emergency_cost_yuan']
reg['q2_baseline_ledger']=float(np.max(abs(runs['adopted'].ledger[cols].to_numpy()-old2[cols].to_numpy())))
assert reg['q2_baseline_ledger']<2e-5
save_csv(out/'forecast_comparison.csv',pd.DataFrame(forecast_rows))

print('Rebuilding Q3 released forecasts and risk archive...',flush=True)
d3=m.load_data();a3=m.build_forecast_archive(d3,'official');b3=m.build_risk_margins(a3,.8)
old3=pd.read_csv(p3/'q3_main_interval_ledger.csv.gz')
er3=np.empty((334,144));mr3=er3.copy();fr3=er3.copy()
cov=[]
for k,hour in enumerate([0,6,12,18]):
    sl=slice(k*36,(k+1)*36)
    er3[:,sl]=a3.errors_kwh[31:,k,sl];mr3[:,sl]=b3[31:,k,sl];fr3[:,sl]=a3.net_kwh[31:,k,sl]
    cov.append(coverage_row('Q3',f'release_{hour}_remaining',a3.errors_kwh[31:,k,hour*6:],b3[31:,k,hour*6:],scope='all_remaining'))
    cov.append(coverage_row('Q3',f'release_{hour}_executed',a3.errors_kwh[31:,k,sl],b3[31:,k,sl],scope='executed_6h'))
reg['q3_forecast']=float(np.max(abs(fr3.ravel()-old3.forecast_net_load_kwh)))
reg['q3_margin']=float(np.max(abs(mr3.ravel()-old3.risk_margin_kwh)))
assert max(reg.values())<2e-5
e2=main_f.errors_kwh[31:];b2=bs['adopted'][31:]
cov.append(coverage_row('Q2','overall',e2,b2,scope='unique_intervals'))
cov.append(coverage_row('Q3','overall',er3,mr3,scope='unique_intervals'))
for g in range(6):
```

```
    cov.append(coverage_row('Q2',f'{4*g:02d}-{4*g+4:02d}',e2[:,g*24:g*24+24],b2[:,g*24:g*24+24],scope='4h_group'))
month=np.array([d.isoformat()[:7] for d in data.dates[31:]])
for key in sorted(set(month)):
    ix=month==key
    cov.append(coverage_row('Q2',key,e2[ix],b2[ix],scope='month'))
    cov.append(coverage_row('Q3',key,er3[ix],mr3[ix],scope='month'))
save_csv(out/'coverage.csv',pd.DataFrame(cov))
np.savez_compressed(out/'forecast_evidence.npz',q2_actual_net=data.net_load_kwh[31:],
   q2_actual_load=data.load_kw[31:],q2_actual_pv=data.pv_kw[31:],
   **{f'{mode}_load':fs[mode].load_kw[31:] for mode in modes},
   **{f'{mode}_pv':fs[mode].pv_kw[31:] for mode in modes},
   **{f'{mode}_margin':bs[mode][31:] for mode in modes},
   q3_forecast_latest=fr3,q3_error_latest=er3,q3_margin_latest=mr3,
   q3_all_errors=a3.errors_kwh[31:],q3_all_margins=b3[31:])

for gamma in [0.,.5,1.]:
    print(f'Q3 gamma {gamma}',flush=True)
    r=m.run_policy(d3,a3,b3,m.PolicyConfig(f'gamma_{gamma}','diagnostic',gamma=gamma,collect_ledger=True))
    assert r.validation['hard_checks_pass'],r.validation
    ev=r.ledger[r.ledger.date>='2025-02-01'];daily=r.daily[r.daily.date>='2025-02-01']
    gamma_rows.append(dict(question='Q3',gamma=gamma,actual_total_cost_yuan=r.metrics['actual_total_cost_yuan'],
     emergency_kwh=r.metrics['emergency_purchase_kwh'],initial_soc_feb1_kwh=float(ev.soc_start_kwh.iloc[0]),
     terminal_soc_kwh=float(ev.soc_end_kwh.iloc[-1]),mean_carryover_soc_kwh=float(ev.groupby('date').soc_end_kwh.last().iloc[:-1].mean()),
     upper_carryover_days=int((ev.groupby('date').soc_end_kwh.last().iloc[:-1]>=10800-1e-4).sum())))
    validations[f'Q3_gamma_{gamma}']=r.validation
    ev.to_csv(out/f'q3_gamma_{gamma}_interval.csv.gz',index=False,encoding='utf-8-sig',float_format='%.10f',compression='gzip')
    save_csv(out/f'q3_gamma_{gamma}_daily.csv',r.daily)
    if gamma==.5:
        cs=['original_contract_kwh','final_contract_kwh','actual_charge_kwh','actual_discharge_kwh','actual_emergency_kwh','soc_end_kwh','actual_total_cost_yuan']
        reg['q3_baseline_ledger']=float(np.max(abs(ev[cs].to_numpy()-old3[cs].to_numpy())))
        assert reg['q3_baseline_ledger']<5e-5
    print(json.dumps(gamma_rows[-1]),flush=True)
gamma_df=pd.DataFrame(gamma_rows)
gamma_df['cost_change_vs_gamma05_yuan']=gamma_df.apply(lambda row:row.actual_total_cost_yuan-gamma_df[(gamma_df.question==row.question)&(gamma_df.gamma==.5)].actual_total_cost_yuan.iloc[0],axis=1)
save_csv(out/'gamma_comparison.csv',gamma_df)
save_json(out/'engine_verification.json',{'status':'PASS','regression':reg,'strategy_checks':validations})

print('Checking future-observation perturbations of all three Q2 predictors...',flush=True)
causal=[]
for cut in [31,180,300]:
    load=data.load_kw.copy();pv=data.pv_kw.copy();load[cut:]+=1234.;pv[cut:]*=.3
    alt=replace(data,load_kw=load,pv_kw=pv,net_load_kwh=(load-pv)/6)
    af=forecast(alt,'weekday')
    for mode in modes:
        altered=baseline_forecast(alt,mode,af);mb=c.build_risk_margins(altered,.8)
        diff=float(max(np.nanmax(abs(fs[mode].net_load_kwh[1:cut+1]-altered.net_load_kwh[1:cut+1])),
                       np.max(abs(bs[mode][:cut+1]-mb[:cut+1]))))
        assert diff<1e-9,(cut,mode,diff)
        causal.append(dict(cut_day_index=cut,mode=mode,max_difference=diff))
assert all(c.sha256(PROJECT/p)==h for p,h in hashes.items())
summary=dict(status='CONDITIONAL',verification_status='PASS',official_strategy_changed=False,
   forecast_comparison=forecast_rows,coverage_overall=[r for r in cov if r['group']=='overall'],
   gamma_comparison=gamma_df.to_dict('records'),causality_checks=causal,
   source_hashes_unchanged=True,runtime_seconds=time.perf_counter()-started,
   limitations=protocol['limitations']+['Empirical coverage is not a formal chance constraint or guaranteed reliability.',
    'Gamma grid is a sensitivity diagnostic, not an optimizer; policy evaluation includes different January-induced February starting SOC.'])
save_json(out/'summary.json',summary)
print('COMPLETE '+str(out/'summary.json'),flush=True)
```

```

if __name__=='__main__':main()
```

D.10.2 experiments/forecast/verify_validation.py

```
"""Independent re-read of saved forecast evidence and policy ledgers."""
from pathlib import Path
import argparse
import json
import sys
sys.dont_write_bytecode=True
HERE=Path(__file__).resolve().parent
PROJECT=HERE.parents[1]
sys.path.insert(0,str(PROJECT/'code'))
import numpy as np
import pandas as pd


def main():
    ap=argparse.ArgumentParser()
    ap.add_argument('--output',type=Path,default=HERE/'results')
    ap.add_argument('--report',type=Path)
    args=ap.parse_args()
    out=args.output;s=json.loads((out/'summary.json').read_text(encoding='utf-8'))
    a=np.load(out/'forecast_evidence.npz')
    fc=pd.read_csv(out/'forecast_comparison.csv');coverage=pd.read_csv(out/'coverage.csv')
    gamma=pd.read_csv(out/'gamma_comparison.csv');checks={}
    for mode in ['adopted','seasonal_persistence','equal_window_mean']:
        row=fc[fc['mode']==mode].iloc[0]
        le=a['q2_actual_load']-a[mode+'_load'];pe=a['q2_actual_pv']-a[mode+'_pv']
        net=(a[mode+'_load']-a[mode+'_pv'])/6;e=a['q2_actual_net']-net;b=a[mode+'_margin']
        expected=[abs(le).mean(),abs(pe).mean(),abs(e).mean(),np.sqrt((e**2).mean()),e.mean(),100*(e<=b+1e-9).mea⌋
          n()]
        actual=row[['load_mae_kw','pv_mae_kw','net_mae_kwh','net_rmse_kwh','net_bias_kwh','coverage_percent']].to⌋
          _numpy(float)
        checks['forecast_'+mode]=float(np.max(abs(actual-expected)))
        assert checks['forecast_'+mode]<1e-8
    e2=a['q2_actual_net']-(a['adopted_load']-a['adopted_pv'])/6;b2=a['adopted_margin']
    e3=a['q3_error_latest'];b3=a['q3_margin_latest']
    assert np.max(abs(a['q2_actual_net']-a['q3_forecast_latest']-e3))<1e-9
    months=np.array([str(x)[:7] for x in pd.date_range('2025-02-01','2025-12-31')])
    for _,row in coverage.iterrows():
        question=row.question;label=row['group'];scope=row.scope
        error,margin=(e2,b2) if question=='Q2' else (e3,b3)
        if scope=='month':error,margin=error[months==label],margin[months==label]
        elif scope=='4h_group':
            start=int(label[:2])*6;error,margin=error[:,start:start+24],margin[:,start:start+24]
        elif scope in ['all_remaining','executed_6h']:
            hour=int(label.split('_')[1]);k=hour//6
            section=slice(hour*6,144 if scope=='all_remaining' else hour*6+36)
            error=a['q3_all_errors'][:,k,section];margin=a['q3_all_margins'][:,k,section]
        assert error.size==row.n and np.isfinite(error).all()
        covered=int((error<=margin+1e-9).sum());assert covered==row.covered_n
        checks['coverage_'+question+'_'+label]=abs(100*covered/error.size-row.coverage_percent)
        assert checks['coverage_'+question+'_'+label]<1e-8
    ledger_checks={}
    for path in sorted(out.glob('*_interval.csv.gz')):
        f=pd.read_csv(path);assert len(f)==334*144
        assert not f[['date','interval_index']].duplicated().any()
        q3=path.name.startswith('q3_')
        q=f.final_contract_kwh if q3 else f.planned_purchase_kwh
        e=f.actual_emergency_kwh if q3 else f.emergency_purchase_kwh
        ch=f.actual_charge_kwh;dis=f.actual_discharge_kwh;unused=f.actual_unused_kwh
        price=f.price_yuan_per_kwh;net=f.actual_net_load_kwh
        contract=f.contract_total_cost_yuan if q3 else f.contract_cost_yuan
        fees=contract+5*price*e
```

```
        errors={'balance':float(abs(q+e+dis-net-ch-unused).max()),
          'soc':float(abs(f.soc_end_kwh-f.soc_start_kwh-.9*ch+dis/.9).max()),
          'chain':float(abs(f.soc_start_kwh.to_numpy()[1:]-f.soc_end_kwh.to_numpy()[:-1]).max()),
          'bounds':float(max(0,1200-f.soc_end_kwh.min(),f.soc_end_kwh.max()-10800,ch.max()-5000/6,dis.max()-5000/6)),
          'overlap':float(np.minimum(ch,dis).max()),
          'emergency_fee':float(abs(f.emergency_cost_yuan-5*price*e).max())}
        if not q3:errors['normal_fee']=float(abs(contract-price*q).max())
        else:errors['total_fee']=float(abs(f.actual_total_cost_yuan-fees).max())
        assert max(errors.values())<5e-5,(path,errors)
        if 'gamma' in path.name:
            g=float(path.name.split('gamma_')[1].split('_interval')[0]);question='Q3' if q3 else 'Q2'
            row=gamma[(gamma.question==question)&(gamma.gamma==g)].iloc[0]
        else:
            mode=path.name.removeprefix('q2_').removesuffix('_interval.csv.gz')
            row=fc[fc['mode']==mode].iloc[0]
        errors['aggregate_fee']=abs(float(fees.sum())-row.actual_total_cost_yuan)
        errors['aggregate_emergency']=abs(float(e.sum())-row.emergency_kwh)
        assert max(errors.values())<5e-5,(path,errors)
        ledger_checks[path.name]=errors
    verified={'status':'PASS','forecast_and_coverage_checks':checks,'ledger_arithmetic':ledger_checks,
       'causality_source':'summary.json','boundary':'Causal historical replay; no claim of untouched test-set validation or global parameter optimality.'}
    (args.report or out/'independent_verification.json').write_text(json.dumps(verified,ensure_ascii=False,indent=2),encoding='utf-8')
    print('PASS independent forecast metrics, coverage counts, and saved ledger arithmetic.',flush=True)


if __name__=='__main__':main()
```

D.11 给定合同下的储能预留补充实验

本组程序保留问题三各次合同安排与结算，分别执行原实时反馈和价格感知储能预留策略，并核对两组运行记录。当前时段优先采用实测净负荷，后续时段采用当时已发布的预报与风险裕量。

D.11.1 experiments/storage/run_experiment.py

```
"""Same-contract causal storage-control diagnostic; never changes official Q3 outputs."""
from __future__ import annotations

import argparse
import importlib.util
import json
import sys
import time
from functools import lru_cache
from pathlib import Path

sys.dont_write_bytecode = True
ROOT = Path(__file__).resolve().parents[2]
DEFAULT_PROJECT = ROOT
sys.path.insert(0, str(DEFAULT_PROJECT / 'code'))
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix, csr_matrix

ETA = 0.9
LOW, HIGH, U = 1200., 10800., 5000 / 6
TOL = 5e-5
PRIMARY_TOL = 1e-7


def write_json(path, value):
    Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2), encoding='utf-8')


@lru_cache(None)
def recurrence(n):
    a = lil_matrix((n, 3 * n), dtype=float)
    for i in range(n):
        a[i, i] = -ETA
        a[i, n+i] = 1 / ETA
        a[i, 2*n+i] = 1
        if i: a[i, 2*n+i-1] = -1
    return a.tocsr()


def choose_discharge(gaps, prices, soc, terminal_value, force_lp=False):
    """gaps[0] is observed current gap; all later gaps are causal forecasts.

    Only surplus contract/PV can charge. Emergency energy cannot charge storage.
    Primary: forecast emergency expense minus inherited terminal value.
    Tie-break: maximize current discharge among primary-optimal plans.
    Returns current discharge and whether an LP pair was used.
    """
    available = max(0., ETA * (soc-LOW))
    greedy = min(max(0., gaps[0]), U, available)
    if greedy <= 1e-9: return 0., False
    if not force_lp and (len(gaps) == 1 or prices[0] >= np.max(prices[1:]) - 1e-12):
        return greedy, False
    n = len(gaps)
    bounds = [(0., min(U, max(0., -float(z)))) for z in gaps]
    bounds += [(0., min(U, max(0., float(z)))) for z in gaps]
    bounds += [(LOW, HIGH)] * n
    rhs = np.zeros(n); rhs[0] = soc
    objective = np.r_[np.zeros(n), -5 * prices, np.zeros(n)]
```

```
    objective[-1] = -terminal_value
    options = {'primal_feasibility_tolerance': 1e-8, 'dual_feasibility_tolerance': 1e-8}
    first = linprog(objective, A_eq=recurrence(n), b_eq=rhs, bounds=bounds,
                    method='highs', options=options)
    if not first.success: raise RuntimeError(f'storage LP failed: {first.message}')
    secondary = np.zeros(3*n); secondary[n] = -1
    second = linprog(secondary, A_eq=recurrence(n), b_eq=rhs, bounds=bounds,
                     A_ub=csr_matrix(objective.reshape(1,-1)),
                     b_ub=[first.fun+PRIMARY_TOL], method='highs', options=options)
    if not second.success: raise RuntimeError(f'tie-break LP failed: {second.message}')
    if objective @ second.x > first.fun + 2e-6: raise AssertionError('objective tolerance')
    return float(np.clip(second.x[n], 0., greedy)), True


def unit_checks():
    # One deliverable kWh. Saving it for the second, dearer shortage costs 5 vs 10.
    d, _ = choose_discharge(np.array([1.,1.]), np.array([1.,2.]), LOW+1/ETA, 0.)
    assert d < 1e-6, d
    flat, _ = choose_discharge(np.array([1.,1.]), np.array([1.,1.]), LOW+1/ETA, 0., True)
    assert abs(flat-1) < 1e-6, flat
    # Free replenishment before an expensive shortage removes the need to reserve.
    refill, _ = choose_discharge(np.array([1.,-2.,1.]), np.array([1.,1.,2.]), LOW+1/ETA, 0.)
    assert abs(refill-1) < 1e-6, refill
    return {'two_period_reserve': d, 'flat_price_tie_discharge': flat,
            'replenishment_before_peak_discharge': refill, 'status': 'PASS'}


def prepare(project, out):
    print('Loading original Q3 inputs and reconstructing causal forecast/margin archive...', flush=True)
    source_code = project / 'code/q3_model.py'
    spec = importlib.util.spec_from_file_location('q3_diagnostic_source', source_code)
    m = importlib.util.module_from_spec(spec); sys.modules[spec.name] = m; spec.loader.exec_module(m)
    pack = project / 'comparison'
    ledger_path = pack / 'q3_main_interval_ledger.csv.gz'
    events_path = pack / 'q3_adjustment_events.csv.gz'
    old = pd.read_csv(ledger_path).sort_values(['date','interval_index']).reset_index(drop=True)
    events = pd.read_csv(events_path)
    dates = old.date.unique()
    assert len(old) == 334 * 144 and len(dates) == 334 and len(events) == 72144
    data = m.load_data()
    archive = m.build_forecast_archive(data, 'official')
    margins = m.build_risk_margins(archive, .8)
    print('Forecasts ready; checking 72,144 contract revision records...', flush=True)
    snapshots = np.full((334,4,144), np.nan)
    max_prev = 0.; max_fee = 0.; max_forecast = 0.; max_margin = 0.
    for d, day in enumerate(dates):
        rows = old.iloc[d*144:(d+1)*144]
        q = rows.original_contract_kwh.to_numpy().copy()
        snapshots[d,0] = q
        fee = np.zeros(144)
        for k, hour in enumerate([6,12,18], start=1):
            cur = events[(events.date == day) & (events.release_hour == hour)].sort_values('interval_index')
            idx = cur.interval_index.to_numpy(int)-1
            assert np.array_equal(idx, np.arange(hour*6,144))
            max_prev = max(max_prev, float(np.max(abs(q[idx]-cur.previous_contract_kwh))))
            new = cur.new_contract_kwh.to_numpy()
            cost = 1.5*data.price[idx]*np.maximum(new-q[idx],0)-.5*data.price[idx]*np.maximum(q[idx]-new,0)
            max_fee = max(max_fee, float(np.max(abs(cost-cur.incremental_adjustment_fee_yuan))))
            fee[idx] += cost; q[idx] = new; snapshots[d,k] = q
        assert np.max(abs(q-rows.final_contract_kwh.to_numpy())) < TOL
        assert np.max(abs(fee-rows.cumulative_adjustment_fee_yuan.to_numpy())) < TOL
        for k in range(4):
            sl = slice(k*36,(k+1)*36)
            max_forecast = max(max_forecast,
              float(np.max(abs(archive.net_kwh[d+31,k,sl]-rows.forecast_net_load_kwh.to_numpy()[sl]))))
            max_margin = max(max_margin,
              float(np.max(abs(margins[d+31,k,sl]-rows.risk_margin_kwh.to_numpy()[sl]))))
    assert max(max_prev,max_fee,max_forecast,max_margin) < TOL
```

```
    assert np.max(abs(data.net_load_kwh[31:].ravel()-old.actual_net_load_kwh.to_numpy())) < TOL
    hashes = {p.resolve().relative_to(project).as_posix(): m.sha256(p) for p in [
              source_code, ledger_path, events_path, m.PRICE_PATH,m.ACTUAL_PATH,m.FORECAST_PATH,Path(__file__)]}
    protocol = {
      'claim': 'Does price-aware forecast-based storage reservation improve greedy feedback under the same Q3
        contracts?',
      'status': 'EXPERIMENT_ONLY_NOT_OFFICIAL_REPLACEMENT',
      'evaluation': ['2025-02-01','2025-12-31'], 'days':334,
      'arms': ['greedy_replay','price_aware_mpc'], 'alpha': .8, 'gamma': .5,
      'horizon': 'current 10-minute interval to current midnight; replan each interval with a deficit and higher
        future price',
      'current_information': 'same idealized current-interval net-load observation as paper Eq.9',
      'future_information': 'most recent released official forecast plus inherited risk margin; only
        then-effective contract snapshot; no anticipated revisions',
      'contracts': 'exogenous replay of original schedule, unveiled at 00/06/12/18; no endogenous next-day or
        intraday procurement changes',
      'primary': 'minimize forecast emergency cost minus gamma*eta*median(price)*terminal_SOC; last day terminal
        value zero',
      'tie_break': 'maximize current discharge within 1e-7 yuan primary tolerance',
      'actions': 'charge only actual surplus, discharge only deficit; no emergency charging; surplus charging
        stays greedy',
      'initial_soc_kwh':float(old.soc_start_kwh.iloc[0]), 'cross_day':'each arm carries its own actual SOC; no
        daily reset',
      'cost': 'identical original contract plus all original adjustment costs, plus new actual emergency expenses;
        no fictitious terminal credit',
      'predeclared_comparison': 'report signed cost difference without selecting parameters based on evaluation
        results',
      'verification_tolerance_kwh':TOL, 'input_hashes':hashes,
      'source_checks':{'contract_previous_max_abs':max_prev,'fee_max_abs':max_fee,
                       'forecast_max_abs':max_forecast,'margin_max_abs':max_margin},
      'unit_checks':unit_checks(),
      'limitation':'fixed-contract counterfactual isolates execution; does not measure fully reoptimized
        end-to-end policy or global optimality'
    }
    write_json(out/'protocol.json',protocol)
    return m, data, archive, margins, old, snapshots, protocol


def replay(old, snapshots, archive, margins, prices, arm, ndays, out):
    soc = float(old.soc_start_kwh.iloc[0]); result=[]; lp_count=0
    started=time.perf_counter()
    for d in range(ndays):
        day = str(old.date.iloc[d*144]); terminal=0. if d+31==364 else .5*ETA*float(np.median(prices))
        block=old.iloc[d*144:(d+1)*144]
        actual=block.actual_net_load_kwh.to_numpy()
        for t in range(144):
            k=t//36; known=snapshots[d,k]; gap=float(actual[t]-known[t]); s0=soc
            c=min(max(-gap,0.),U,max((HIGH-soc)/ETA,0.))
            greedy=min(max(gap,0.),U,max(ETA*(soc-LOW),0.))
            r=greedy
            if arm=='price_aware_mpc' and greedy>1e-9:
                predicted=archive.net_kwh[d+31,k,t:] + margins[d+31,k,t:] - known[t:]
                predicted=predicted.copy(); predicted[0]=gap
                r, used=choose_discharge(predicted,prices[t:],soc,terminal)
                lp_count+=int(used)
            e=max(gap-r,0.); w=max(-gap-c,0.)
            soc += ETA*c-r/ETA
            if not LOW-TOL<=soc<=HIGH+TOL: raise AssertionError(('SOC',day,t,soc))
            result.append((day,t+1,known[t],c,r,e,w,s0,soc,greedy-r,k*6))
        if (d+1)%20==0 or d+1==ndays:
            print(f'{arm}: {d+1}/{ndays} days, {lp_count} LP pairs,
              {time.perf_counter()-started:.1f}s',flush=True)
    r=pd.DataFrame(result,columns=['date','interval_index','contract_kwh','charge_kwh','discharge_kwh',
                     'emergency_kwh','unused_kwh','soc_start_kwh','soc_end_kwh','withheld_vs_greedy_kwh','availabl
                       e_release_hour'])
    ref=old.iloc[:len(r)]
    r['actual_net_kwh']=ref.actual_net_load_kwh.to_numpy()
```

```
    r['price_yuan_per_kwh']=ref.price_yuan_per_kwh.to_numpy()
    r['contract_cost_yuan']=ref.contract_total_cost_yuan.to_numpy()
    r['emergency_cost_yuan']=5*r.price_yuan_per_kwh*r.emergency_kwh
    r['actual_total_cost_yuan']=r.contract_cost_yuan+r.emergency_cost_yuan
    # Independent vectorized arithmetic verification of executed, not planned, values.
    checks={
      'balance':float(np.max(abs(r.contract_kwh+r.emergency_kwh+r.discharge_kwh-r.actual_net_kwh-r.charge_kwh-r.unused_kwh))),
      'state':float(np.max(abs(r.soc_end_kwh-r.soc_start_kwh-ETA*r.charge_kwh+r.discharge_kwh/ETA))),
      'chain':float(np.max(abs(r.soc_start_kwh.to_numpy()[1:]-r.soc_end_kwh.to_numpy()[:-1]))),
      'bounds':float(max(0,LOW-r.soc_end_kwh.min(),r.soc_end_kwh.max()-HIGH,r.charge_kwh.max()-U,r.discharge_kwh.max()-U)),
      'overlap':float(np.minimum(r.charge_kwh,r.discharge_kwh).max()),
      'contract':float(np.max(abs(r.contract_kwh-ref.final_contract_kwh.to_numpy())))
    }
    if arm=='greedy_replay':
        checks['baseline_reproduction']=float(np.max(abs(r[['charge_kwh','discharge_kwh','emergency_kwh','unused_kwh','soc_start_kwh','soc_end_kwh']].to_numpy()-ref[['actual_charge_kwh','actual_discharge_kwh','actual_emergency_kwh','actual_unused_kwh','soc_start_kwh','soc_end_kwh']].to_numpy())))
    assert max(checks.values()) < TOL, checks
    cols=['contract_cost_yuan','emergency_cost_yuan','actual_total_cost_yuan','emergency_kwh','unused_kwh','charge_kwh','discharge_kwh','withheld_vs_greedy_kwh']
    daily=r.groupby('date')[cols].sum()
    daily['soc_start_kwh']=r.groupby('date').soc_start_kwh.first()
    daily['soc_end_kwh']=r.groupby('date').soc_end_kwh.last()
    r.to_csv(out/f'{arm}_interval.csv.gz',index=False,encoding='utf-8-sig',float_format='%.10f',compression='gzip')
    daily.to_csv(out/f'{arm}_daily.csv',encoding='utf-8-sig',float_format='%.10f')
    summary={x:float(r[x].sum()) for x in cols}
    summary.update(arm=arm,days=ndays,initial_soc_kwh=float(r.soc_start_kwh.iloc[0]),
        terminal_soc_kwh=float(r.soc_end_kwh.iloc[-1]),emergency_intervals=int((r.emergency_kwh>1e-6).sum()),
        withheld_intervals=int((r.withheld_vs_greedy_kwh>1e-6).sum()),lp_pairs=lp_count,
        runtime_seconds=time.perf_counter()-started,verification=checks)
    print(json.dumps(summary,ensure_ascii=False),flush=True)
    return summary,daily,r


def main():
    ap=argparse.ArgumentParser();ap.add_argument('--project',type=Path,default=DEFAULT_PROJECT)
    ap.add_argument('--output',type=Path,required=True);ap.add_argument('--days',type=int,default=334)
    args=ap.parse_args()
    args.project=args.project.resolve()
    if not 1 <= args.days <= 334:ap.error('--days must be between 1 and 334')
    args.output.mkdir(parents=True,exist_ok=False)
    m,data,archive,margins,old,snapshots,protocol=prepare(args.project,args.output)
    summaries=[];daily_results=[]
    for arm in ['greedy_replay','price_aware_mpc']:
        sm,dr,ir=replay(old,snapshots,archive,margins,data.price,arm,args.days,args.output)
        summaries.append(sm);daily_results.append(dr)
    base,new=summaries
    saving=base['actual_total_cost_yuan']-new['actual_total_cost_yuan']
    comparison=pd.DataFrame(summaries).drop(columns='verification')
    comparison['saving_vs_greedy_yuan']=base['actual_total_cost_yuan']-comparison.actual_total_cost_yuan
    comparison.to_csv(args.output/'comparison.csv',index=False,encoding='utf-8-sig',float_format='%.10f')
    paired=pd.DataFrame({'greedy_cost_yuan':daily_results[0].actual_total_cost_yuan,'mpc_cost_yuan':daily_results[1].actual_total_cost_yuan})
    paired['saving_yuan']=paired.greedy_cost_yuan-paired.mpc_cost_yuan
    paired.to_csv(args.output/'paired_daily.csv',encoding='utf-8-sig',float_format='%.10f')
    paired.groupby(paired.index.str[:7]).sum().to_csv(args.output/'monthly.csv',encoding='utf-8-sig',float_format='%.10f')
    unchanged=all(m.sha256(args.project/path)==h for path,h in protocol['input_hashes'].items())
    assert unchanged
    summary={'status':'CONDITIONAL','verification_status':'PASS','official_strategy_changed':False,
      'claim':protocol['claim'],'arms':summaries,'saving_yuan':saving,
      'saving_percent':100*saving/base['actual_total_cost_yuan'],
      'days_mpc_cheaper':int((paired.saving_yuan>1e-4).sum()),
      'days_mpc_dearer':int((paired.saving_yuan< -1e-4).sum()),
```

```
      'max_daily_loss_yuan':float(max(0,-paired.saving_yuan.min())),
      'terminal_soc_difference_kwh':new['terminal_soc_kwh']-base['terminal_soc_kwh'],
      'source_hashes_unchanged':unchanged,'limitations':[protocol['limitation'],
        'risk level and terminal value inherited, not newly tuned; the test cannot establish superiority over all causal controllers',
        'current interval observation follows the same idealization as the original paper']}
    write_json(args.output/'summary.json',summary)
    print('COMPLETE '+str(args.output/'summary.json'),flush=True)


if __name__=='__main__': main()
```

D.11.2 experiments/storage/verify_experiment.py

```
"""Reopen saved results; independently check arithmetic and information boundaries."""
from __future__ import annotations
import argparse
import importlib.util
import json
import sys
from dataclasses import replace
from pathlib import Path

sys.dont_write_bytecode=True
HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(HERE))
import run_experiment as run
import numpy as np
import pandas as pd


def main():
    ap=argparse.ArgumentParser();ap.add_argument('--output',type=Path,default=HERE/'full_run')
    ap.add_argument('--report',type=Path)
    args=ap.parse_args(); out=args.output
    protocol=json.loads((out/'protocol.json').read_text(encoding='utf-8'))
    summary=json.loads((out/'summary.json').read_text(encoding='utf-8'))
    project=run.DEFAULT_PROJECT
    code=project/'code/q3_model.py'
    spec=importlib.util.spec_from_file_location('q3_verifier_source',code)
    m=importlib.util.module_from_spec(spec);sys.modules[spec.name]=m;spec.loader.exec_module(m)
    pack=project/'comparison'
    old=pd.read_csv(pack/'q3_main_interval_ledger.csv.gz')
    checks={}
    frames={}
    for arm in ['greedy_replay','price_aware_mpc']:
        f=pd.read_csv(out/f'{arm}_interval.csv.gz');frames[arm]=f
        assert len(f)==334*144
        assert not f[['date','interval_index']].duplicated().any()
        np.testing.assert_array_equal(f[['date','interval_index']],old[['date','interval_index']])
        flow=f.contract_kwh+f.emergency_kwh+f.discharge_kwh-f.actual_net_kwh-f.charge_kwh-f.unused_kwh
        soc=f.soc_start_kwh+.9*f.charge_kwh-f.discharge_kwh/.9
        gap=f.actual_net_kwh-f.contract_kwh
        limits=max(0.,1200-f.soc_end_kwh.min(),f.soc_end_kwh.max()-10800,
                   f.charge_kwh.max()-5000/6,f.discharge_kwh.max()-5000/6,
                   float((f.discharge_kwh-np.maximum(gap,0)).max()))
        report={'balance_max_kwh':float(abs(flow).max()),
          'state_max_kwh':float(abs(soc-f.soc_end_kwh).max()),
          'state_chain_max_kwh':float(abs(f.soc_start_kwh.to_numpy()[1:]-f.soc_end_kwh.to_numpy()[:-1]).max()),
          'constraint_violation_kwh':float(limits),
          'overlap_kwh':float(np.minimum(f.charge_kwh,f.discharge_kwh).max()),
          'emergency_cost_error_yuan':float(abs(5*f.price_yuan_per_kwh*f.emergency_kwh-f.emergency_cost_yuan).max()),
          'fixed_cost_error_yuan':float(abs(f.contract_cost_yuan-old.contract_total_cost_yuan).max()),
          'delivered_contract_error_kwh':float(abs(f.contract_kwh-old.final_contract_kwh).max()),
          'greedy_charge_error_kwh':float(abs(f.charge_kwh-np.minimum(np.minimum(np.maximum(-gap,0),5000/6),np.maximum((10800-f.soc_start_kwh)/.9,0))).max())}
```

```
        record=next(x for x in summary['arms'] if x['arm']==arm)
        for field in ['actual_total_cost_yuan','emergency_cost_yuan','emergency_kwh','unused_kwh']:
            report[f'{field}_summary_error']=abs(float(f[field].sum())-record[field])
        assert max(report.values())<5e-5,report
        assert (f[['charge_kwh','discharge_kwh','emergency_kwh','unused_kwh']].min()>=-1e-8).all()
        checks[arm]=report
    assert frames['greedy_replay'].soc_start_kwh.iloc[0]==frames['price_aware_mpc'].soc_start_kwh.iloc[0]
    saved_diff=float(frames['greedy_replay'].actual_total_cost_yuan.sum()-frames['price_aware_mpc'].actual_total_
      cost_yuan.sum())
    assert abs(saved_diff-summary['saving_yuan'])<5e-5
    print('Saved ledgers and summary independently checked.',flush=True)

    # Rebuild three causal decision inputs after strongly perturbing information
    # unavailable at those instants. Past errors and current observation are held.
    data=m.load_data(); archive=m.build_forecast_archive(data,'official')
    events=pd.read_csv(pack/'q3_adjustment_events.csv.gz')
    causal=[]
    for day,t in [('2025-03-20',65),('2025-06-21',65),('2025-09-23',65)]:
        d=[x.isoformat() for x in data.dates].index(day);k=t//36
        load=data.load_kw.copy();pv=data.pv_kw.copy();official=data.official_hourly_kw.copy()
        load[d,t+1:]+=10000.;load[d+1:]+=10000.
        pv[d,t+1:]*=1.7;pv[d+1:]*=1.7
        official[d,k+1:]+=9999.;official[d+1:]+=9999.
        altered=replace(data,load_kw=load,pv_kw=pv,net_load_kwh=(load-pv)/6,official_hourly_kw=official)
        altarchive=m.build_forecast_archive(altered,'official')
        prior=np.arange(max(1,d-28),d)
        def margin(a):
            return np.maximum(0,np.quantile(a.errors_kwh[prior,k,t:],.8,axis=0,method='linear'))
        def known_contract(ev):
            q=old[old.date==day].original_contract_kwh.to_numpy().copy()
            for hour in [6,12,18]:
                if hour>k*6: break
                rev=ev[(ev.date==day)&(ev.release_hour==hour)]
                q[rev.interval_index.to_numpy(int)-1]=rev.new_contract_kwh
            return q
        modified_events=events.copy()
        modified_events.loc[(events.date>day)|((events.date==day)&(events.release_hour>k*6)),'new_contract_kwh']+
          =1234.
        q=known_contract(events);qa=known_contract(modified_events)
        baseline_input=archive.net_kwh[d,k,t:]+margin(archive)-q[t:]
        altered_input=altarchive.net_kwh[d,k,t:]+margin(altarchive)-qa[t:]
        baseline_input[0]=data.net_load_kwh[d,t]-q[t]
        altered_input[0]=altered.net_load_kwh[d,t]-qa[t]
        maximum=float(np.max(abs(baseline_input-altered_input)))
        assert maximum<1e-10, {'date':day,'max_difference':maximum}
        assert np.max(abs(altered.net_load_kwh[d,t+1:]-data.net_load_kwh[d,t+1:]))>100.
        assert np.nanmax(abs(archive.net_kwh[d,k+1]-altarchive.net_kwh[d,k+1]))>100.
        v=.5*.9*np.median(data.price)
        action,_=run.choose_discharge(baseline_input,data.price[t:],6000.,v,True)
        changed,_=run.choose_discharge(altered_input,data.price[t:],6000.,v,True)
        assert abs(action-changed)<1e-10
        causal.append({'date':day,'interval_index':t+1,'available_release_hour':k*6,
                       'future_information_perturbed':True,'controller_input_max_difference':maximum,
                       'action_difference_kwh':abs(action-changed)})
    hashes=protocol.get('packaged_input_hashes',protocol.get('input_hashes',{}))
    if not hashes:raise RuntimeError('Input hash record is missing')
    unchanged=all(m.sha256(project/p)==h for p,h in hashes.items())
    assert unchanged
    run.write_json(args.report or out/'independent_verification.json',{'status':'PASS','arithmetic_checks':checks,
                   'causality_checks':causal,'original_inputs_unchanged':True,
                   'verifier_sha256':m.sha256(Path(__file__)),
                   'boundary':'Checks fixed-contract execution; does not certify a different endogenous
                     procurement policy.'})
    print('PASS arithmetic, physical constraints, and future-information perturbation checks.',flush=True)


if __name__=='__main__':main()
```

附录 E　补充实验结果

本附录汇总预测基准、风险裕量覆盖、日末估值与储能预留实验。各项费用均统计 2025 年 2 月 1 日至 12 月 31 日的 334 天；费用与电量按未舍入结果计算后保留两位小数。逐日及分月汇总、比较指标和原实验核验结果见附录 A 所列随包文件；补充实验完整逐时段记录与预测证据数组由附录 D.1 所述命令重新生成，并由附录 D 所列程序独立核验。

E.1　预测基准比较

三种预测分别按各自历史误差构造风险裕量，均取 $\alpha = 0.8$、$\gamma = 0.5$，购电与储能控制规则相同。各方案从 1 月 1 日 6,000 kWh 开始连续运行，进入评价期时采用各自实际储电量。表 E-1 补列对应紧急购电量与评价期初始储电量。

表 E-1　预测基准的误差、费用与评价期起点

预测方案	负荷 MAE（kW）	光伏 MAE（kW）	净负荷 MAE（kWh/时段）	总购电费用（元）	紧急购电量（kWh）	期初储电量（kWh）
上周同期负荷＋昨日同期光伏	176.80	185.35	50.64	14,101,205.81	102,731.15	2,325.20
同窗口等权均值	188.88	153.40	47.48	13,913,353.02	104,955.68	2,468.10
指数加权预测	166.03	150.61	43.97	13,923,903.39	111,928.38	2,475.00

E.2　日末价值系数比较

固定各问的预测与风险裕量规则，分别取 $\gamma = 0$、0.5 和 1，比较运行费用与跨日留存电量。费用差额以同问 $\gamma = 0.5$ 方案为基准；平均跨日储电量与达到上限的天数按评价期前 333 个日末状态统计，不含 12 月 31 日。

不同日末价值系数下的费用与紧急购电量见表 E-2。

表 E-2　不同日末价值系数下的费用与紧急购电量

问题	γ	总购电费用（元）	相对 $\gamma = 0.5$ 的费用变化（元）	紧急购电量（kWh）
Q2	0.0	13,923,903.39	+0.00	111,928.38
Q2	0.5	13,923,903.39	+0.00	111,928.38
Q2	1.0	14,102,566.39	+178,663.00	101,980.16
Q3	0.0	13,833,843.32	+0.00	51,525.70
Q3	0.5	13,833,843.32	+0.00	51,525.70
Q3	1.0	14,005,735.03	+171,891.71	39,980.76

对应的评价期初、期末及跨日储电量统计见表 E-3。

表 E-3　不同日末价值系数下的储电量

问题	γ	评价期初储电量（kWh）	评价期末储电量（kWh）	平均跨日储电量（kWh）	日末达到上限天数
Q2	0.0	2,475.00	1,718.67	2,232.60	0
Q2	0.5	2,475.00	1,718.67	2,232.60	0
Q2	1.0	10,800.00	1,718.67	10,767.66	303
Q3	0.0	2,492.63	1,661.10	2,207.64	0
Q3	0.5	2,492.63	1,661.10	2,207.64	0
Q3	1.0	10,800.00	1,661.10	10,766.81	307

E.3 风险裕量覆盖结果

按实测净负荷不超过预测值与风险裕量之和统计覆盖时段，问题二采用日初预测，问题三采用当时最近一次已发布的预测。总体及分月结果见表 E-4。

表 E-4 风险裕量的总体及分月覆盖率

区间	评价时段数	Q2 覆盖时段数	Q2 覆盖率	Q3 覆盖时段数	Q3 覆盖率
全评价期	48,096	37,907	78.82%	37,202	77.35%
2025-02	4,032	3,329	82.56%	3,142	77.93%
2025-03	4,464	3,434	76.93%	3,394	76.03%
2025-04	4,320	3,339	77.29%	3,303	76.46%
2025-05	4,464	3,394	76.03%	3,467	77.67%
2025-06	4,320	3,329	77.06%	3,288	76.11%
2025-07	4,464	3,480	77.96%	3,375	75.60%
2025-08	4,464	3,675	82.33%	3,652	81.81%
2025-09	4,320	3,413	79.00%	3,414	79.03%
2025-10	4,464	3,506	78.54%	3,389	75.92%
2025-11	4,320	3,439	79.61%	3,381	78.26%
2025-12	4,464	3,569	79.95%	3,397	76.10%

问题三各发布时间对剩余时段及其后六小时的覆盖情况见表 E-5。

表 E-5 问题三分发布时间的风险裕量覆盖率

发布时间	统计范围	评价时段数	覆盖时段数	覆盖率
00:00	当日剩余时段	48,096	37,320	77.59%
00:00	发布后六小时	12,024	9,283	77.20%
06:00	当日剩余时段	36,072	27,920	77.40%
06:00	发布后六小时	12,024	9,311	77.44%
12:00	当日剩余时段	24,048	18,608	77.38%
12:00	发布后六小时	12,024	9,322	77.53%
18:00	当日剩余时段	12,024	9,286	77.23%
18:00	发布后六小时	12,024	9,286	77.23%

E.4 储能预留对照结果

两种储能控制使用相同的合同安排、逐次调整费用与评价期初始储电量，各自连续传递实际日末储电量。表 E-6 列示费用及运行指标。

表 E-6 给定合同下两种储能控制的完整汇总

指标	原实时反馈策略	价格感知储能预留策略
合同购电与调整费用（元）	13,586,618.43	13,586,618.43
紧急购电费用（元）	247,224.89	351,752.97
总购电费用（元）	13,833,843.32	13,938,371.41
紧急购电量（kWh）	51,525.70	88,689.61
未利用电量（kWh）	1,961,159.10	2,007,040.48
累计充电量（kWh）	6,367,238.83	6,321,357.45
累计放电量（kWh）	5,158,211.83	5,121,047.91
评价期初储电量（kWh）	2,492.63	2,492.63
评价期末储电量（kWh）	1,661.10	1,661.10
发生紧急购电的时段数	720	2,655
相对当期最大可放电量预留的时段数	0	2,201

两种储能控制的分月费用比较见表 E-7，费用变化为预留策略减去原实时反馈策略。

表 E-7　两种储能控制的分月购电费用（单位：元）

月份	原实时反馈策略	储能预留策略	费用变化
2025-02	1,215,639.59	1,229,353.99	+13,714.41
2025-03	1,131,809.78	1,145,784.77	+13,974.99
2025-04	937,791.41	951,361.59	+13,570.18
2025-05	1,006,408.78	1,007,176.06	+767.28
2025-06	1,365,938.97	1,383,209.55	+17,270.58
2025-07	1,595,984.14	1,596,978.95	+994.81
2025-08	1,302,273.86	1,313,864.90	+11,591.04
2025-09	1,151,960.17	1,165,141.34	+13,181.17
2025-10	1,126,798.12	1,142,508.74	+15,710.62
2025-11	1,281,846.15	1,292,797.68	+10,951.52
2025-12	1,717,392.36	1,710,193.84	-7,198.52
合计	13,833,843.32	13,938,371.41	+104,528.08

储能预留策略有 19 天费用较低、277 天费用较高，其余日期的费用差异在核验容差内；评价期总费用增加 104,528.08 元。两种控制的期末储电量相同，补充结果与正文表 7-7 的比较一致。

E.5　实验结果核对

逐时段复算八组预测与参数实验及两组储能控制实验，每组均含 48,096 个评价时段。误差、覆盖率与费用汇总一致，主要核验量的最大差异见表 E-8。

表 E-8　补充实验的数值核验

核验量	预测与参数实验	原实时反馈策略	储能预留策略
电量平衡（kWh）	1.00×10^{-10}	2.42×10^{-13}	9.99×10^{-11}
储电量递推（kWh）	1.46×10^{-10}	1.30×10^{-10}	1.35×10^{-10}
相邻时段状态衔接（kWh）	0	0	0
设备边界（kWh）	0	1.92×10^{-13}	1.14×10^{-13}
同时充放电量（kWh）	0	0	0
紧急购电费复算（元）	3.83×10^{-10}	3.38×10^{-10}	3.62×10^{-10}
总费用汇总（元）	3.73×10^{-8}	1.86×10^{-9}	0

另对预测程序实施未来观测扰动检验，并在三个指定决策时点改变尚未发布的预报、未来实测和后续合同调整。已形成的预测与裕量差异小于 10^{-9} kWh，储能预留策略在受检时点的当前放电决策未发生变化。